



Introduction à STRUTS 2 par l'exemple

Serge Tahé, janvier 2012

[Ce site a été créé avec le convertisseur \[Word ou ODT - > HTML\] créé par les IA Gemini 3 et ChatGPT 5.2 en janvier 2026.](#)

Avec le convertisseur Gemini / ChatGPT, plusieurs sites web ont été produits à partir de documents ODT de LibreOffice, et de documents Word :

Java

- [\[Introduction au langage Java et à l'écosystème Spring au travers d'une étude de cas \(2015\)\]](#) ;
- [\[Introduction à Spring MVC et Thymeleaf par l'exemple \(2015\)\]](#)
- [\[Exploiter une base de données relationnelle avec l'écosystème Spring \(2015\)\]](#)

.NET

- [\[Apprentissage du langage C# Version 3.0 avec le Framework .NET 3.5 \(2008\)\]](#) ;
- [\[Introduction par l'exemple à Entity Framework 5 Code First \(2012\)\]](#) ;
- [\[Introduction par l'exemple au framework ASP.NET MVC par l'exemple \(2013\)\]](#) ;

Javascript

- [\[Introduction au langage ECMAScript6 par l'exemple \(2019\)\]](#) ;
- [\[Introduction au framework VUE.js par l'exemple \(2019\)\]](#) ;
- [\[Introduction au framework Nuxt.js par l'exemple \(2019\)\]](#) ;

PHP

- [[Introduction au langage PHP7 par l'exemple \(2019\)](#)] ;

Python

- [[Introduction au langage Python et au framework web Flask par l'exemple \(2020\)](#)] ;
- [[Générer un script Python avec des outils d'IA \(2025\)](#)] ;

VBScript

- [[Introduction au langage VBScript \(2002\)](#)] ;

Ce sont tous d'anciens cours de plusieurs centaines de pages. La taille du document ODT ou DOCX n'importe pas au convertisseur Gemini / ChatGPT. Ce qui lui importe ce sont les structures que l'on y trouve (cf. chapitre : [Les exemples de ce document](#)).

1 Introduction

Nous nous proposons ici d'introduire, à l'aide d'exemples les notions importantes de Struts 2. Struts 2 est un framework web qui fournit :

- un certain nombre de bibliothèques sous forme de jars
- un cadre de développement : Struts 2 influe sur la manière de développer une application web.

Les pré-requis nécessaires à la compréhension des exemples sont les suivants :

- connaissances de base du langage Java
- connaissances de base du développement web, le langage Html en particulier.

On trouvera sur le site [<http://developpez.com>] toutes les ressources nécessaires à ces pré-requis. J'en ai écrit quelques-unes qu'on trouvera sur le site [<http://tahe.developpez.com>].

Les exemples de ce document sont disponibles à l'URL [<http://tahe.developpez.com/java/struts2>].

Pour approfondir Struts 2, on pourra utiliser les références suivantes :

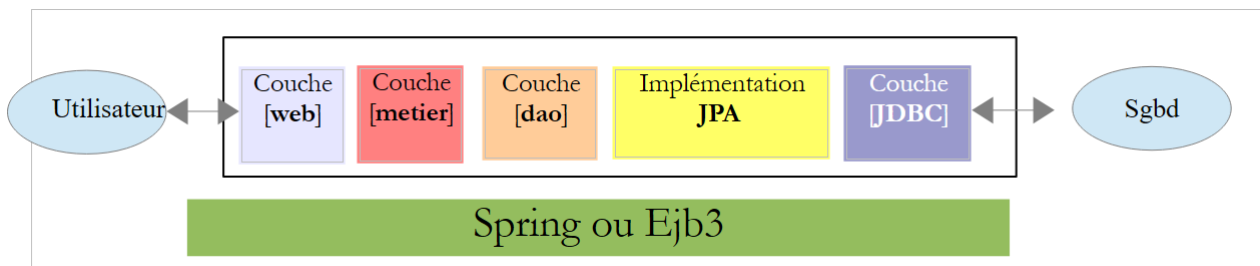
- [ref1] : la documentation de Struts 2 qu'on trouvera sur le site de Struts
- [ref2] : le livre " Struts 2 in Action " écrit par Donald Brown - Chad Michael Davis – Scott Stanlick aux éditions Manning. Ce livre est particulièrement pédagogique.

Nous ferons parfois référence à [ref2] pour indiquer au lecteur qu'il peut approfondir un domaine avec ce livre.

Le document a été écrit de telle façon qu'il puisse être lu sans ordinateur sous la main. Aussi, donne-t-on beaucoup de copies d'écran.

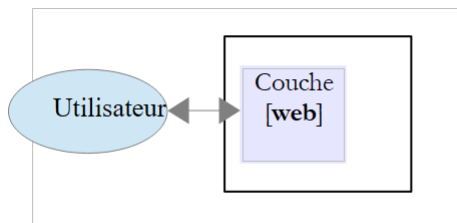
1.1 La place de Struts 2 dans une application web

Tout d'abord, situons Struts 2 dans le développement d'une application web. Le plus souvent, celle-ci sera bâtie sur une architectures multi-couches telle que la suivante :

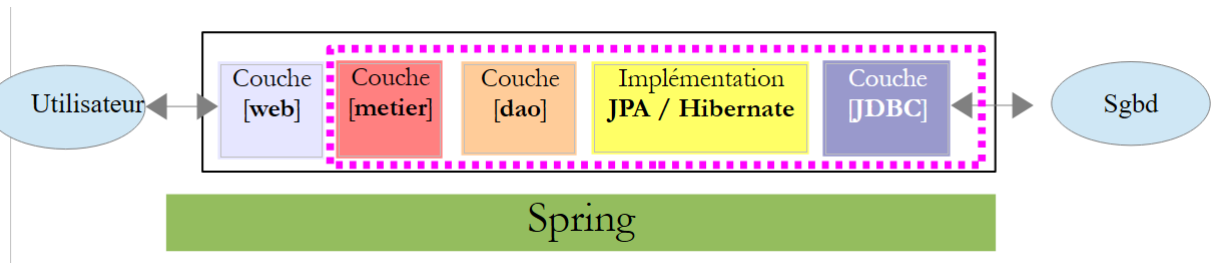


- la couche [web] est la couche en contact avec l'utilisateur de l'application web. Celui-ci interagit avec l'application web au travers de pages web visualisées par un navigateur. **C'est dans cette couche que se situe Struts 2 et uniquement dans cette couche.**
- la couche [metier] implémente les règles de gestion de l'application, tels que le calcul d'un salaire ou d'une facture. Cette couche utilise des données provenant de l'utilisateur via la couche [web] et du Sgbd via la couche [dao].
- la couche [dao] (Data Access Objects), la couche [jpa] (Java Persistence Api) et le pilote Jdbc gèrent l'accès aux données du Sgbd. Le couche [jpa] sert d'Orm (Object Relational Mapper). Elle fait un pont entre les objets manipulés par la couche [dao] et les lignes et les colonnes des données d'une base de données relationnelle.
- l'intégration des couches peut être réalisée par un conteneur Spring ou Ejb3 (Enterprise Java Bean).

La plupart des exemples donnés dans la suite, n'utiliseront qu'une seule couche, la couche [web] :



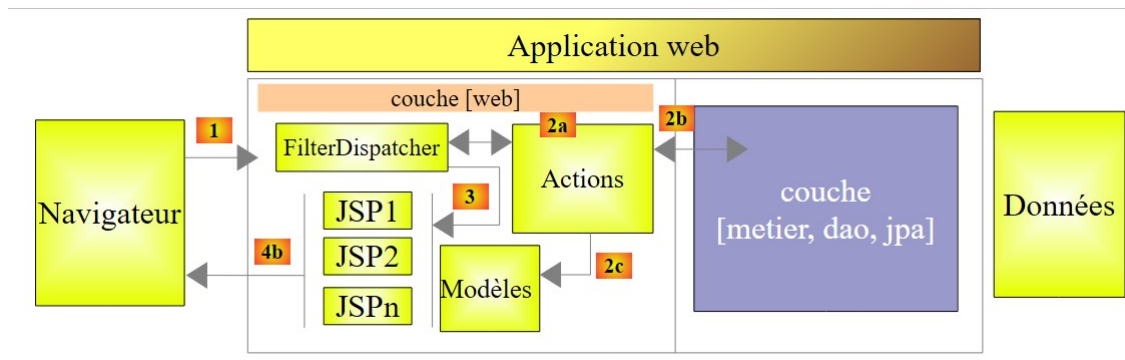
Ce document se terminera cependant par la construction d'une application web multicouches :



L'ensemble des couches [metier], [dao], [jpa/hibernate] nous sera fourni sous la forme d'une archive jar afin que de nouveau, nous n'ayons que la couche [web] à construire.

1.2 Le modèle de développement MVC de Struts 2

Struts 2 implémente le modèle d'architecture dit MVC (Modèle – Vue – Contrôleur) de la façon suivante :



Le traitement d'une demande d'un client se déroule de la façon suivante :

Les URL demandées sont de la forme *http://machine:port/contexte/rep1/rep2/.../Action*. Le chemin *[/rep1/rep2/.../Action]* doit correspondre à une action définie dans un fichier de configuration de Struts 2, sinon elle est refusée. Une action est définie dans un fichier Xml sous la forme suivante :

```

1. <package name="actions" namespace="/actions" extends="struts-default">
2.   <action name="Action1" class="actions.Action1">
3.     <result name="page1">/vues/Page1.JSP</result>
4.     <result name="page2">/vues/Page2.JSP</result>
5.   </action>
6. </package>
  
```

Sur l'exemple précédent, supposons que l'URL *[http://machine:port/contexte/actions/Action1]* soit demandée. Les étapes suivantes sont alors exécutées :

1. **demande** - le client navigateur fait une demande au contrôleur **[FilterDispatcher]**. Celui-ci voit passer toutes les demandes des clients. C'est la porte d'entrée de l'application. C'est le **C** de MVC.

2. Traitement

- le contrôleur **C** consulte son fichier de configuration et découvre que l'action *actions/Action1* existe. Le *namespace* (ligne 1) concaténé avec le nom de l'action (attribut *name* de ligne 2) définit l'action *actions/Action1*.
- le contrôleur **C** instancie [2a] une classe de type *[actions.Action1]* (attribut *class* de la ligne 2). Le nom et le package de cette classe peuvent être quelconques.
- si l'URL demandée est accompagnée de paramètres de type *[param1=val1¶m2=val2&...]*, alors le contrôleur **C** affecte ces paramètres à la classe *[actions.Action1]* de la façon suivante :

```
[actions.Action1].setParami(valeuri) ;
```

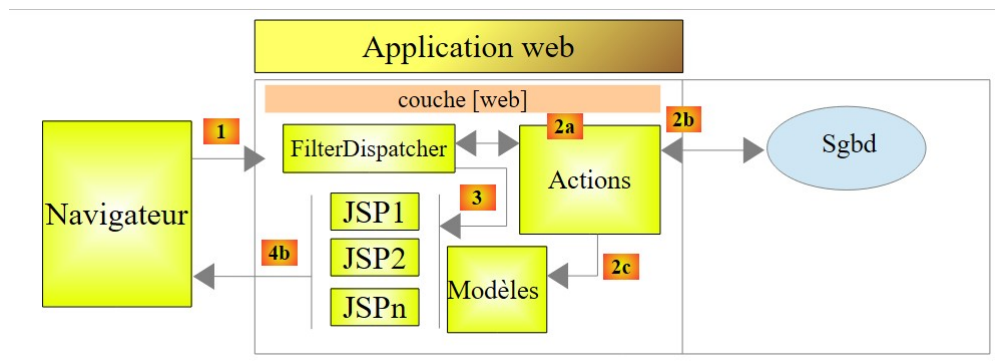
Il faut donc que la classe *[actions.Action1]* ait ces méthodes *setParami* pour chacun des paramètres *parami* attendus.

- (suite)
 - le contrôleur **C** demande à la méthode de signature *[String execute()]* de la classe *[actions.Action1]* de s'exécuter. Celle-ci peut alors exploiter les paramètres *parami* que la classe a récupérés. Dans le traitement de la demande de l'utilisateur, elle peut avoir besoin de la couche [metier] [2b]. Une fois la demande du client traitée, celle-ci peut appeler diverses réponses. Un exemple classique est :
 - une page d'erreurs si la demande n'a pu être traitée correctement
 - une page de confirmation sinon

La méthode *execute* rend au contrôleur **C** un résultat de type chaîne de caractères appelée **clé de navigation**. Dans l'exemple ci-dessus, *[actions.Action1].execute* peut produire deux clés de navigation " *page1* " (ligne 3) et " *page2* " (ligne 4). La méthode *[actions.Action1].execute* va également mettre à jour le modèle **M** [2c] que va exploiter la page JSP qui va être envoyée en réponse à l'utilisateur. Ce modèle peut comporter des éléments de :

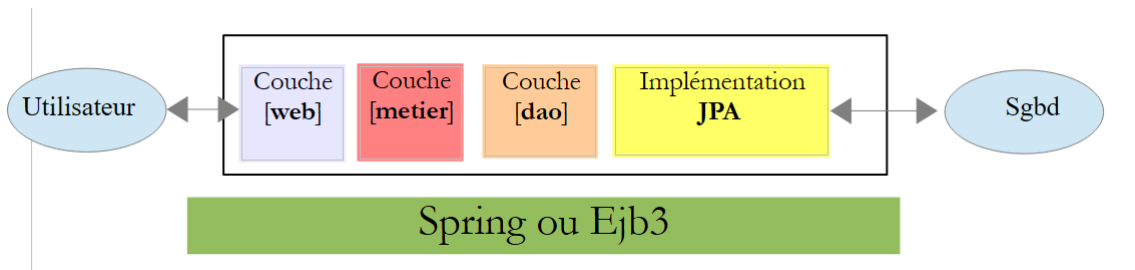
- (suite)
 - la classe *[actions.Action1]* instanciée
 - la session de l'utilisateur
 - données de portée Application
 - ...
- 3. **réponse** - le contrôleur **C** demande à la page JSP correspondant à la clé de navigation de s'afficher [3]. C'est la vue, le **V** de MVC. La page JSP utilise un modèle **M** pour initialiser les parties dynamiques de la réponse qu'elle doit envoyer au client.

Maintenant, précisons le lien entre architecture web MVC et architecture en couches. En fait, ce sont deux concepts différents qui sont parfois confondus. Prenons une application web Struts 2 à une couche :



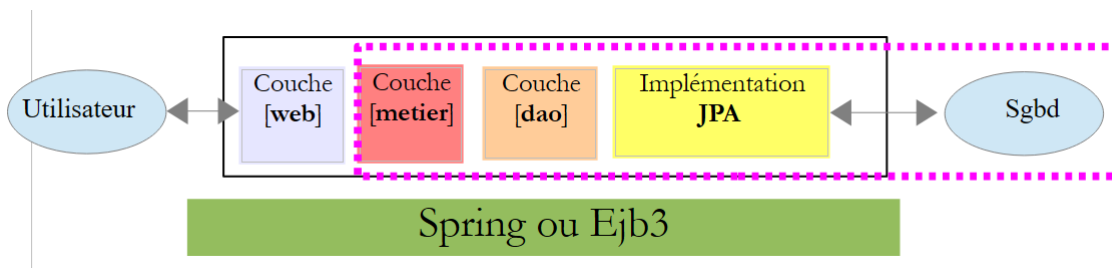
Si nous implémentons la couche [web] avec Struts 2, nous aurons bien une architecture web MVC mais pas une architecture multi-couches. Ici, la couche [web] s'occupera de tout : présentation, métier, accès aux données. Avec Struts 2, ce sont les classes de type *[Action]* qui feront ce travail.

Maintenant, considérons une architecture web multi-couches :



La couche [web] peut être implémentée sans framework et sans suivre le modèle MVC. On a bien alors une architecture multi-couches mais la couche web n'implémente pas le modèle MVC.

Dans MVC, nous avons dit que le modèle M était celui de la vue V, c.a.d. l'ensemble des données affichées par la vue V. Parfois (souvent), une autre définition du modèle M de MVC est donnée :



Beaucoup d'auteurs considèrent que ce qui est à droite de la couche [web] forme le modèle M du MVC. Pour éviter les ambiguïtés on peut parler :

- du modèle du domaine lorsqu'on désigne tout ce qui est à droite de la couche [présentation]
- du modèle de la vue lorsqu'on désigne les données affichées par une vue V

Dans la suite, le terme " modèle M " désignera exclusivement le modèle d'une vue V.

1.3 Les outils utilisés

Dans la suite, nous utilisons (décembre 2011)

- l'IDE Netbeans 7.01 disponible à l'URL [<http://www.netbeans.org>]
- le plugin Struts 2 pour Netbeans 7.01 disponible à l'URL [<http://plugins.netbeans.org/plugin/39218>]
- la version 2.2.3 de Struts 2 disponible à l'URL [<http://struts.apache.org/>]

On notera que seules les bibliothèques de Struts 2 obtenues à l'URL [<http://struts.apache.org/>] sont indispensables pour développer les exemples qui vont suivre. Netbeans peut être remplacé par un autre IDE (Eclipse, Jdeveloper, IntelliJ, ...) et le plugin Struts 2 n'est là que pour faciliter la vie du développeur. Il n'est pas indispensable lui non plus.

1.3.1 IDE Netbeans

Sur le site de téléchargement de Netbeans, nous choisissons la version Java EE :

Packs de téléchargement de l'EDI NetBeans dans des langues contribuées par la communauté¹

Technologies supportées *	Java SE	Java EE	C/C++	PHP	Tous
① SDK de la plateforme NetBeans	•	•			•
① Java SE	•	•			•
① Java EE		•			•
① Java ME					•
① Java Card™ 3 Connected					•
① C/C++			•		•
① Groovy					•
① PHP				•	•
Serveurs inclus					
① GlassFish Server Open Source Edition 3.1.1		•			•
① Apache Tomcat 7.0.14		•			•
	Télécharger	Télécharger	Télécharger	Télécharger	Télécharger
	75 Mo disponibles	166 Mo disponibles	49 Mo disponibles	47 Mo disponibles	251 Mo disponibles

1.3.2 Plugin Struts 2

Selon les versions de Netbeans, ce plugin n'a pas toujours été disponible. En décembre 2011, on le trouve à l'URL <http://plugins.netbeans.org>. Cette URL permet de connaître les différents plugins disponibles pour Netbeans. On peut filtrer ce que l'on cherche. En [1], on demande les plugins qui contiennent le mot " struts " dans leur nom.

Plugins catalogue

Live Filters - select or type to filter the catalogue, filters are applied all together. There is also multi column sorting ac

1

Name:
Category:
NetBeans version:

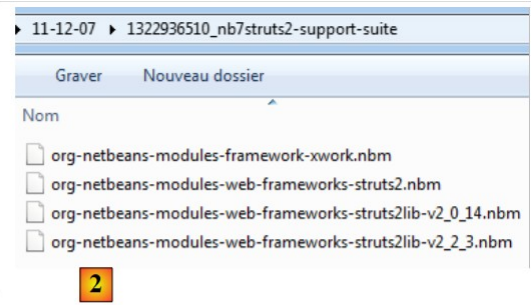
2

Show entries

Plugin name	Category	NetBeans Version
➡ Struts2 Support Repack for NB 7.0 + XWork	Web Applications , Enterprise Applications	7.0, 7.1

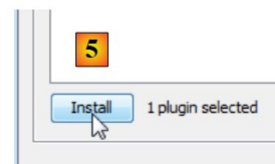
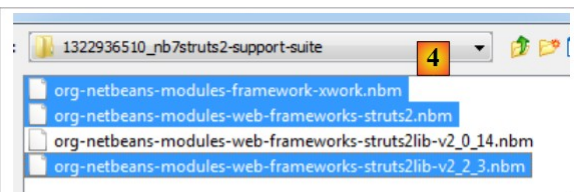
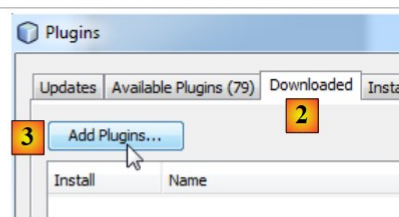
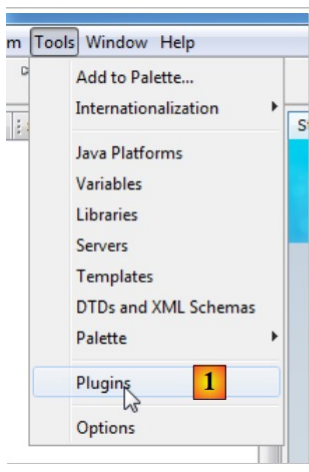
Showing 1 to 1 of 1 entries (filtered from 738 total entries)

- on suit le lien [2]



On télécharge le plugin et on le dézippe [2]. Pour l'intégrer à Netbeans, on pourra procéder comme suit :

- on lance Netbeans



- en [1], choisir le menu Tools/Plugins
- dans l'onglet [2], utiliser le bouton [3]
- en [4], choisir les fichiers .nbm des plugins téléchargés. Ici, nous choisissons les bibliothèques de Struts version 2.2.3 plutôt que celles de Struts 2.0.14
- revenu dans l'onglet [2], on installe les plugins sélectionnés avec le bouton [5].

L'installation des plugins nécessite souvent le redémarrage de Netbeans.

1.3.3 Les bibliothèques Struts 2

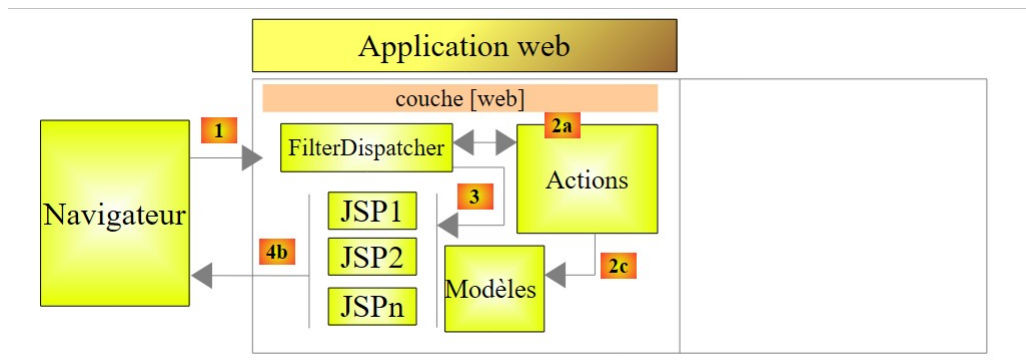
Si on a téléchargé le plugin Struts 2 pour Netbeans, on dispose des principales bibliothèques de Struts mais pas toutes. Dans la suite, nous aurons besoin de certaines bibliothèques disponibles sur le site de Struts 2 [<http://struts.apache.org/>].

The screenshot shows the Apache Struts 2.2.3.1 download page. The page title is "The Apache Software Foundation" and the URL is "http://www.apache.org/". The page is dated "Published: 2011-12-05". The main heading is "Recent Releases". Below it, there is a section for "7 September 2011 - Struts 2.2.3.1 General". The text states: "The latest production release of Struts 2 is Struts 2.2.3.1, v... Struts 2.2.3.1 includes important security fixes. All d...". A yellow box with the number "1" is placed over the word "All". To the right, there is a sidebar with a list of links: "Release Notes", "Full Distribution:", "Example Application:", "struts-2.2.3.1-all.zip (76mb) [PGP] [MD5]", "struts-2.2.3.1-app.zip (29mb) [PGP] [MD5]". A yellow box with the number "2" is placed over the link "struts-2.2.3.1-app.zip (29mb) [PGP] [MD5]". Below the sidebar, there is a file explorer window showing the directory structure: "downloads > struts > 2.2.3.1 > struts-2.2.3.1-all > struts-2.2.3.1 > lib". The file explorer shows a list of files: "antlr-2.7.2.jar", "aopalliance-1.0.jar", "asm-3.1.jar", "asm-commons-3.1.jar", "asm-tree-3.1.jar", "classworlds-1.1.jar". A yellow box with the number "3" is placed over the file "aopalliance-1.0.jar".

Nous suivons le lien [1] puis le lien [2] pour télécharger le fichier zip de la distribution 2.2.3.1. Une fois la distribution dézippée, les bibliothèques nécessaires à Struts sont trouvées dans le dossier [lib] [3] de la distribution. On en trouve plusieurs dizaines et se pose alors la question de savoir lesquelles sont indispensables. C'est là que le plugin de Struts nous aidera.

2 Un premier exemple

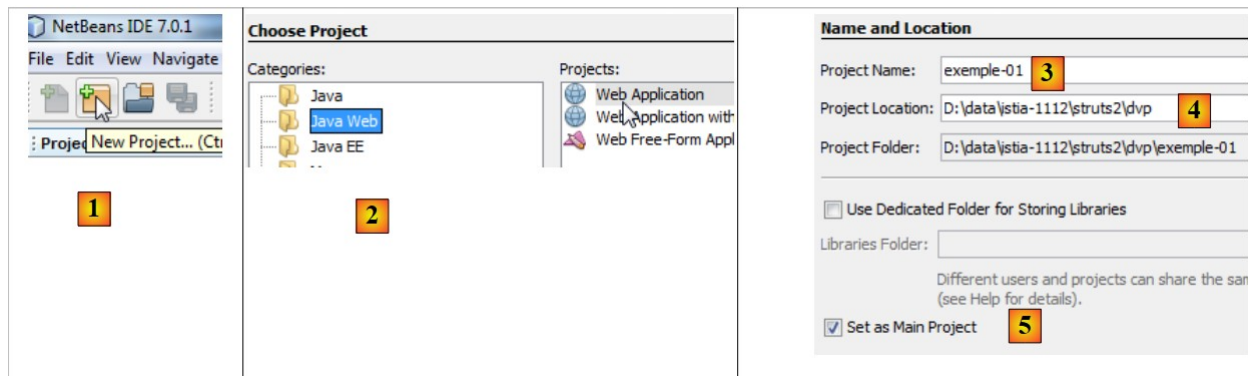
La plupart de nos exemples seront réduits à la seule couche web implémentée avec Struts 2 :



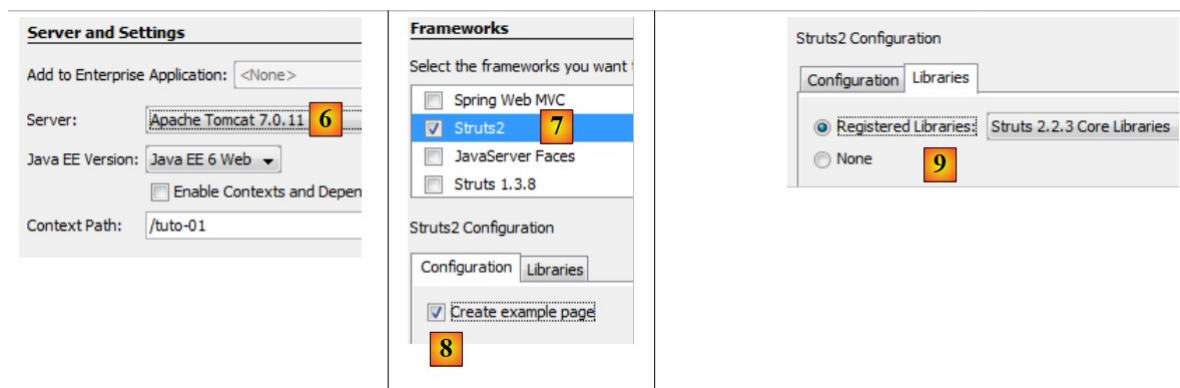
Lorsque les bases seront acquises, nous étudierons un exemple plus complexe avec une architecture multi-couches.

2.1 Génération de l'exemple

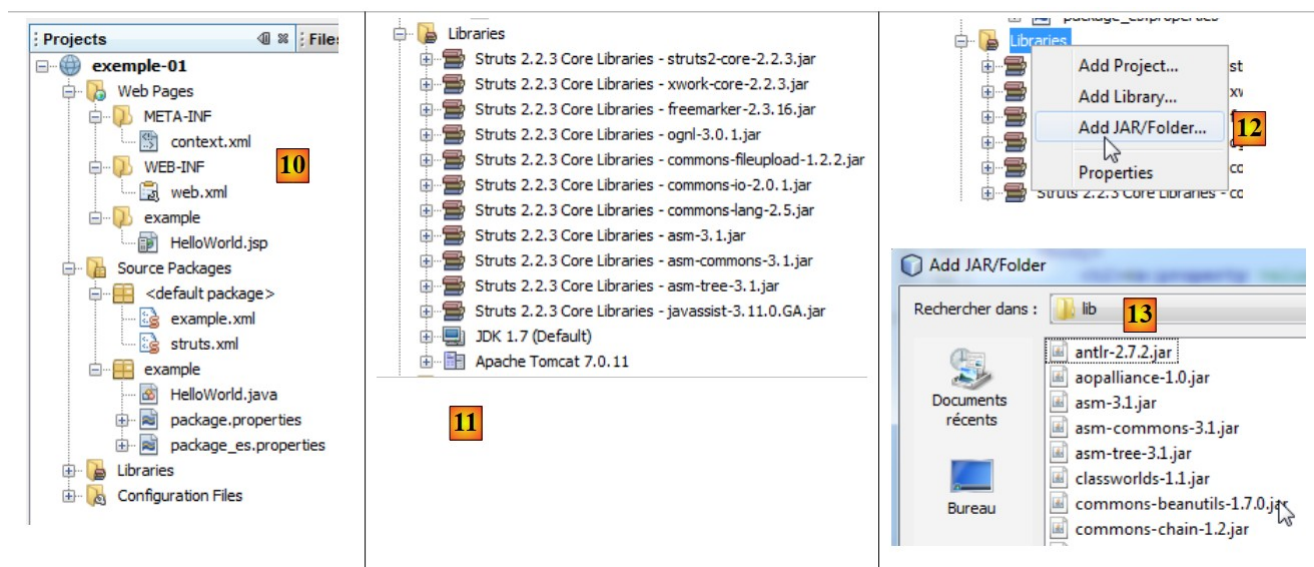
Nous construisons notre première application.



- en [1], on crée un nouveau projet
- en [2], on choisit le type Java Web / Web Application
- en [3], on donne un nom au projet
- en [4], on indique l'emplacement du projet.
- en [5], on fait du nouveau projet, le projet principal.

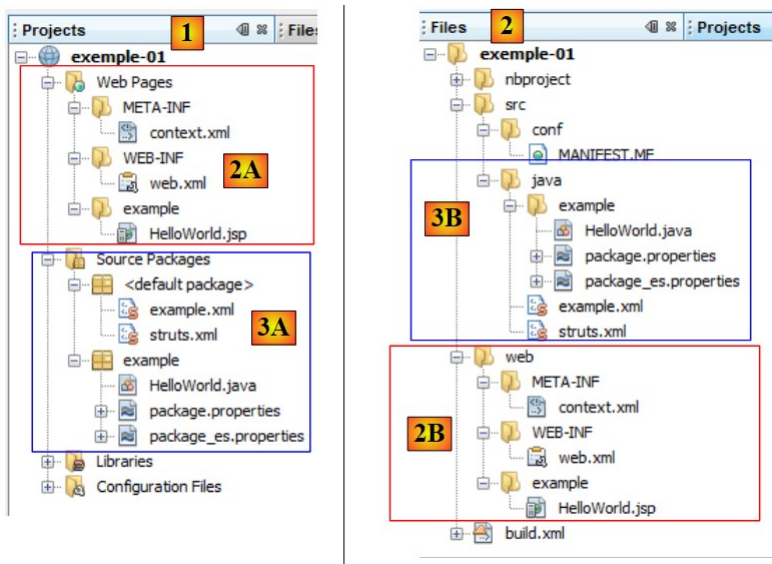


- en [6], on choisit le serveur Tomcat. A l'installation de Netbeans 7.01, deux serveurs ont été installés, Apache Tomcat et Glassfish 3.1.
- en [7], on indique qu'on va travailler avec le framework Struts 2. C'est l'installation du plugin pour Struts 2 qui nous donne cette possibilité. Sans le plugin, le framework Struts 2 ne nous est pas proposé.
- en [8], on demande la création du projet d'exemple que nous allons étudier.
- en [9], on peut vérifier quelles bibliothèques Struts 2 vont être utilisées.



- en [10], le projet généré. Nous allons y revenir.
- en [11], les bibliothèques du projet. Elles ont été intégrées par le plugin Struts 2. Si on ne dispose pas du plugin, on pourra trouver ces bibliothèques dans le dossier [lib] de la distribution Struts 2 téléchargée. On suivra alors les étapes 12 et 13.

2.2 Le projet généré dans le système de fichiers



- en [1], l'onglet [Projects] présente une vue "développeur" du projet
- en [2], l'onglet [Files] présente le dossier du projet dans le système de fichiers
- en [2A], la branche [Web Pages] est représentée en [2] par le dossier [web] [2B]
- en [3A], la branche [Source Packages] est représentée en [2] par le dossier [java] [3B]

2.3 Le fichier de configuration [META-INF/context.xml]

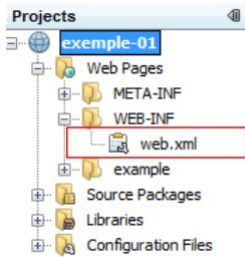


Ce fichier est le suivant :

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <Context antiJARLocking="true" path="/exemple-01"/>
```

La ligne 2 indique que le contexte de l'application web est `/exemple-01`. Toutes les URL du type `[http://machine:port/exemple-01/...]` seront traitées par cette application. On peut retrouver ce contexte dans les propriétés du projet [2] : *clic droit sur le projet / Properties / Run*.

2.4 Le fichier de configuration [WEB-INF/web.xml]

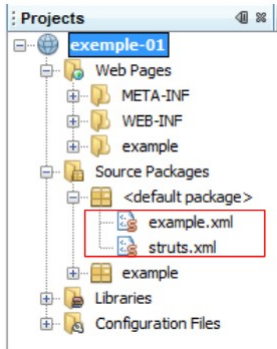


Toute application web est configurée par le fichier [web.xml] du dossier [WEB-INF] de l'application. Celui qui a été généré est le suivant :

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <web-app version="3.0" xmlns="http://java.sun.com/xml/ns/javaee"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://java.sun.com/xml/ns/javaee/web-
   app_3_0.xsd">
3.     <filter>
4.         <filter-name>struts2</filter-name>
5.         <filter-class>org.apache.struts2.dispatcher.FilterDispatcher</filter-class>
6.     </filter>
7.     <filter-mapping>
8.         <filter-name>struts2</filter-name>
9.         <URL-pattern>/*</URL-pattern>
10.    </filter-mapping>
11.    <session-config>
12.        <session-timeout>
13.            30
14.        </session-timeout>
15.    </session-config>
16.    <welcome-file-list>
17.        <welcome-file>example/HelloWorld.JSP</welcome-file>
18.    </welcome-file-list>
19. </web-app>
```

- les lignes 3-6 définissent un filtre implémenté par la classe [org.apache.struts2.dispatcher.FilterDispatcher] de Struts 2. C'est cette classe qui jouera le rôle du contrôleur C du modèle MVC.
- lignes 7-10 : définissent une liaison entre un modèle d'URL et le filtre qui doit traiter les URL qui suivent ce modèle. Ici, il est indiqué que toute URL (modèle /*) doit être traité par le filtre nommé *struts2*. C'est le filtre défini lignes 3-6. Donc ici, toutes les URL passeront par le contrôleur Struts 2.
- lignes 11-15 : définissent la durée d'une session utilisateur, ici 30 minutes. Lors de des différentes requêtes, un utilisateur est suivi par un jeton de session qui lui a été attribué à sa première requête et qu'il envoie ensuite systématiquement à chaque nouvelle requête. Cela permet au serveur web de le reconnaître et de gérer une " mémoire " pour l'utilisateur qu'on appelle la session. Si entre deux requêtes s'écoulent plus de 30 mn, un nouveau jeton de session est généré pour l'utilisateur qui perd ainsi sa " mémoire " et en recommence une nouvelle.
- lignes 16-18 : définissent le fichier à afficher lorsque l'utilisateur interroge l'application web sans demander de document. Ainsi lorsque l'URL demandée sera [http://machine:port/exemple-01], l'URL servie sera [http://machine:port/exemple-01/exemple/HelloWord.JSP].

2.5 Le fichier de configuration [struts.xml]



Le fichier [struts.xml] est le fichier de configuration de Struts 2. Il peut être n'importe où dans le *ClassPath* du projet. Dans le projet Netbeans ci-dessus, le *ClassPath* du projet est formé des deux branches :

- Source Packages
- Libraries

Tout dossier ou bibliothèque situé dans ces deux branches fait donc partie du *ClassPath* du projet. Il est usuel de mettre [struts.xml] dans <default package>. Ici, son contenu est le suivant :

```

1. <!DOCTYPE struts PUBLIC
2. "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
3. "http://struts.apache.org/dtds/struts-2.0.dtd">
4.
5. <struts>
6.     <include file="example.xml"/>
7.     <!-- Configuration for the default package. -->
8.     <package name="default" extends="struts-default">
9.     </package>
10. </struts>

```

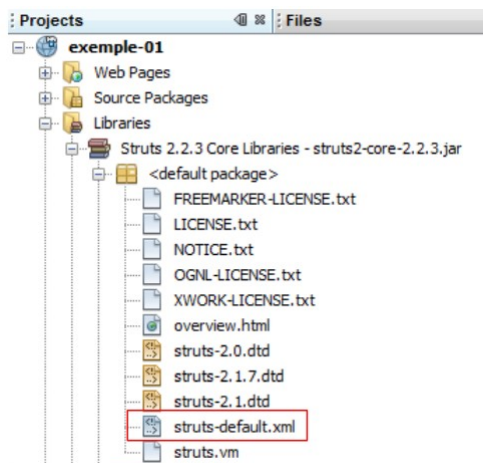
- lignes 5 et 10 : la balise racine du document est la balise <struts>
- ligne 6 : le fichier [example.xml] vient s'insérer ici. Il amène donc sa propre configuration. Nous y reviendrons.
- lignes 8-9 : définissent un package nommé ici "default". Un package permet de configurer un groupe d'actions Struts 2 ayant la même URL. Par exemple [/chemin/Action1] et [/chemin/Action2]. On peut alors définir un package pour ces actions :

```

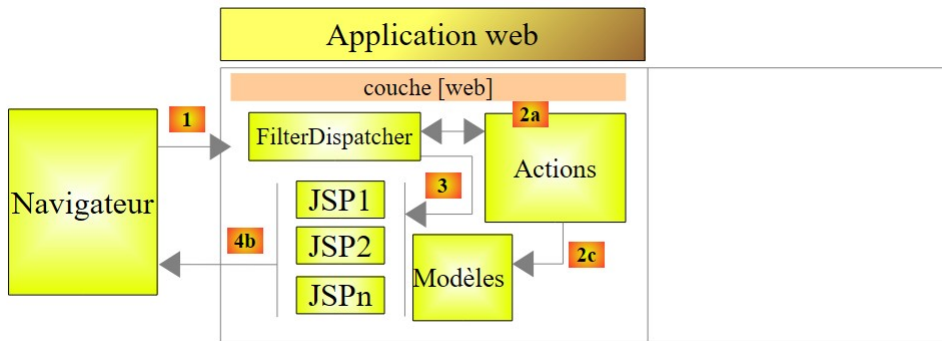
<package name="employes" namespace="/employes" extends="struts-default">
... configuration
</package>

```

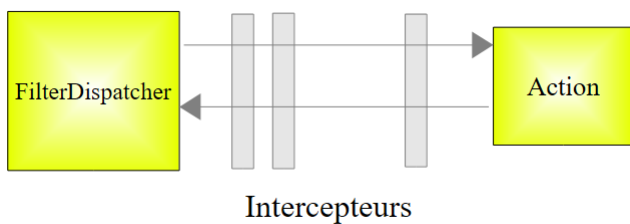
Le package ci-dessus s'appelle "employes" et configure les actions d'URL */employes/Action*. Le package peut hériter d'un autre package avec le mot clé "extends". Ci-dessus, le package *employes* hérite du package *struts-default*. Ce package se trouve dans le fichier [struts-default.xml] de la bibliothèque *struts2-core.jar* :



Le package "struts-default" défini dans le fichier [struts-default.xml] configure diverses choses dont une liste d'intercepteurs exécutés lors de l'appel à une action. Revenons à la structure MVC d'une application Struts 2 :



Pour traiter une URL de la forme [http://machine:port/.../Action], le contrôleur [FilterDispatcher] va instancier la classe qui implémente l'action demandée et va exécuter l'une de ses méthodes, par défaut une méthode appelée *execute*. L'appel à cette méthode *execute* va traverser une série d'intercepteurs :



Les intercepteurs et l'action traitent tous la même requête. La liste des intercepteurs définie dans le package *struts-default* est suffisante la plupart du temps. Aussi nos packages Struts étendent-ils toujours le package *struts-default*. Pour savoir ce que font les différents intercepteurs, on lira le chapitre 4 de [ref2].

Revenons au fichier de configuration *struts.xml* :

```
1. <!DOCTYPE struts PUBLIC
2. "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
3. "http://struts.apache.org/dtds/struts-2.0.dtd">
4.
5. <struts>
6.   <include file="example.xml"/>
7.   <!-- Configuration for the default package. -->
8.   <package name="default" extends="struts-default">
9.   </package>
10. </struts>
```

- ligne 8 : le package nommé *default* a un rôle particulier. Il traite les actions qui n'ont pas été configurées dans les autres packages. Ici, dans les lignes 8-9, aucune configuration n'est faite pour le package *default*. On pourrait donc supprimer la définition de ce package.

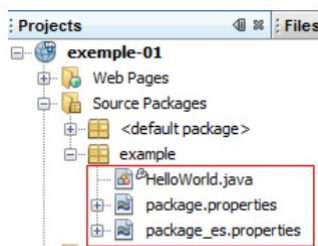
Voyons maintenant le fichier [example.xml] inclus en ligne 6 du fichier [struts.xml] :

```
1. <?xml version="1.0" encoding="UTF-8" ?>
2.
3. <!DOCTYPE struts PUBLIC
4.   "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
5.   "http://struts.apache.org/dtds/struts-2.0.dtd">
6.
7. <struts>
8.   <package name="example" namespace="/example" extends="struts-default">
9.     <action name="HelloWorld" class="example.HelloWorld">
10.      <result>/example/HelloWorld.JSP</result>
11.    </action>
12.  </package>
13. </struts>
```

- ligne 8 : définit un package nommé *example* qui étend le package *struts-default*. Ce package gère les URL du type */example/Action* (namespace */example*).
- lignes 9-11 : définissent une action nommée *HelloWorld* (attribut *name*) qui correspond donc à l'URL */example/HelloWorld*. Cette action est traitée par une instance de la classe *example>HelloWorld* (attribut *class*).
 - le contrôleur [FilterDispatcher] fera exécuter la méthode *execute* de cette classe.
 - cette méthode rendra une chaîne de caractères appelée **clé de navigation**.
 - les différentes clés de navigation doivent être définies par des balises *<result name="clé"/>* (ligne 10). En l'absence d'attribut *name*, c'est la clé *success* qui est utilisée par défaut. C'est le cas ci-dessus. Donc la méthode *execute* de la classe *example>HelloWorld* doit rendre au contrôleur [FilterDispatcher] la clé *success*.
 - le contrôleur [FilterDispatcher] fait alors afficher la page */example/HelloWorld.jsp* (ligne 10).

Si nous fusionnons les deux fichiers [struts.xml] et [example.xml], supprimons le package *default* qui semble inutile, tout se passe comme si on avait le fichier [struts.xml] réduit au seul fichier [example.xml].

2.6 L'action HelloWorld



D'après le fichier [struts.xml] étudié, l'action *HelloWorld* est déclenchée lorsque l'URL demandée par le client est */example/HelloWorld*. Sa méthode *execute* est alors exécutée. Elle doit rendre une clé de navigation. Nous avons vu qu'il n'y en avait qu'une : *success* et qu'alors c'était la page */example/HelloWorld.jsp* qui était envoyée en réponse à l'utilisateur.

Le code de l'action *HelloWorld* est le suivant :

```

1. package example;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4.
5. public class HelloWorld extends ActionSupport {
6.
7.     public String execute() throws Exception {
8.         setMessage(getText(MESSAGE));
9.         return SUCCESS;
10.    }
11.
12.    public static final String MESSAGE = "HelloWorld.message";
13.
14.    private String message;
15.
16.    public String getMessage() {
17.        return message;
18.    }
19.
20.    public void setMessage(String message) {
21.        this.message = message;
22.    }
23. }

```

- ligne 5 : la classe *HelloWorld* dérive de la classe *ActionSupport* de Struts 2. C'est pratiquement toujours le cas. Cela permet de bénéficier de certaines méthodes comme la méthode *getText* de la ligne 8.
- lignes 7-10 : la méthode *execute* qui est exécutée sur invocation du contrôleur Struts. On sait qu'elle doit rendre une chaîne de caractères d'où sa signature ligne 7. Ici elle va rendre la constante *SUCCESS* définie elle aussi dans *ActionSupport*. D'autres constantes sont ainsi définies pour le résultat de la méthode *execute* :

Constante	Valeur
SUCCESS	"success"

ERROR	"error"
INPUT	"input"
LOGIN	"login"

Donc ici, la méthode *execute* rend la chaîne *success*. Si on se réfère au fichier [struts.xml], c'est donc la page */example/HelloWorld.JSP* qui sera renvoyée en réponse à l'utilisateur.

- ligne 8 : la méthode *execute* initialise le champ *message* de la ligne 14 avec la valeur de *getText(" HelloWorld.message ")*. La méthode *getText* appartient à la classe parent *ActionSupport*. Elle permet de récupérer un texte dans un fichier selon la langue utilisée. Par défaut, le fichier *package.properties* situé dans le même package que l'action va être ici utilisé. Celui-ci vient en deux versions :

Le fichier [package.properties] est le suivant :

```
|HelloWorld.message= Struts is up and running ...
```

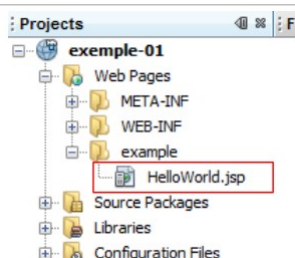
C'est une suite de lignes de texte de la forme *clé=valeur*. Si ce fichier est utilisé, *getText("HelloWorld.message")* aura pour valeur *Struts is up and running ...*

Le fichier [package_es.properties] est lui le suivant :

```
|HelloWorld.message= ¡Struts está bien! ...
```

Si la langue utilisée par le navigateur client est l'espagnol (attribut **es**, dans *package_es.properties*), *getText("HelloWorld.message")* aura pour valeur *¡Struts está bien! ...* Dans tous les autres cas, c'est le fichier [package.properties] qui sera utilisé.

2.7 La vue HelloWorld.JSP



C'est le dernier élément du puzzle Struts. C'est la vue qui est affichée, lorsque l'URL */example/HelloWorld* est demandée. Nous avons vu par quel dédale, la requête initiale est passée pour enfin afficher cette réponse. Le code de la page est le suivant :

```
1. <%@ page contentType="text/html; charset=UTF-8" %>
2.
3. <%@ taglib prefix="s" uri="/struts-tags" %>
4.
5. <html>
6.   <head>
7.     <title><s:text name="HelloWorld.message"/></title>
8.   </head>
9.
10.  <body>
11.    <h2><s:property value="message"/></h2>
12.
13.    <h3>Languages</h3>
14.    <ul>
15.      <li>
16.        <s:url id="URL" action="HelloWorld">
17.          <s:param name="request_locale">en</s:param>
18.        </s:url>
19.        <s:a href="%{URL}">English</s:a>
20.      </li>
21.
```

```

22.         <li>
23.             <s:url id="URL" action="HelloWorld">
24.                 <s:param name="request_locale">es</s:param>
25.             </s:url>
26.
27.             <s:a href="%{URL}">Espanol</s:a>
28.
29.         </li>
30.     </ul>
31. </body>
32. </html>

```

- la page utilise des balises Html (lignes 5, 6, ...) et des balises d'une bibliothèque définie par la ligne 3. Toutes les balises **<s:xx>** appartiennent à cette bibliothèque.
- ligne 7 : la balise **<s:text>** permet d'afficher un texte différent selon la langue du navigateur client. L'attribut **name** indique la clé à chercher dans les fichiers des messages. Ici, également, ce sont les fichiers *package_xx.properties* qui vont être exploités. On se rappelle qu'ils ne contiennent qu'un unique message de clé *HelloWorld.message*.
- ligne 11 : la balise **<s:property name="propriété">** permet d'écrire la valeur d'une propriété d'un objet appelé *ActionContext*. On trouve dans cet objet :
 - les propriétés de l'action qui a été exécutée. **name="message"** va afficher la valeur du champ **message** de l'action courante. Ce sont les méthodes *get* et *set* associées au champ qui sont utilisées pour en obtenir la valeur ou l'initialiser. Ces méthodes doivent donc exister.
 - les attributs de la requête courante notés **<s:property name="#request['clé']">**
 - les attributs de la session de l'utilisateur notés **<s:property name="#session['clé']">**
 - les attributs de l'application elle-même notés **<s:property name="#application['clé']">**
 - les paramètres envoyés par le navigateur client notés **<s:property name="#parameters['clé']">**
 - la notation **<s:property name="#attr['clé']">** affiche la valeur d'un objet cherché dans la page, la requête, la session, l'application dans cet ordre.
- lignes 16-18 : la balise **<s:url >** sert à définir une URL. L'attribut **id** donne un nom à l'URL qui va être créée. Ce nom est ensuite utilisé ligne 19. L'attribut **action** indique vers quelle action doit pointer l'URL.
- ligne 17 : la balise **<s:param ..>** permet d'ajouter des paramètres à l'URL sous la forme *?param1=valeur1¶m2=valeur2&...* Ici, le paramètre ajouté sera *?request_locale=es*.

Au final, l'URL générée sera la suivante :

```
/exemple-01/example/HelloWorld.action?request_locale=en
```

Pour comprendre cette URL, il faut se rappeler que la page [HelloWorld.JSP] est affichée dans deux cas :

- (suite)
 - sur demande directe de l'URL [/exemple-01/example/HelloWorld.JSP]
 - sur demande de l'action [/exemple-01/example/HelloWorld.action]

Dans les deux cas, le chemin des URL est */exemple-01/example*. La balise **<s:url action= "... ">** ajoute l'action définie par l'attribut **action** à ce chemin. Ainsi on obtient */exemple-01/example/HelloWorld*. Elle ajoute ensuite le suffixe *.action* à l'URL précédente ainsi que les paramètres de l'URL s'il y en a. L'URL obtenue */exemple-01/example/HelloWorld.action?request_locale=en* va appeler l'action *HelloWorld* définie dans le fichier [struts.xml] tout en passant le paramètre *request_locale=en*. Ce dernier ne sera pas traité par l'action *HelloWorld* mais par l'un des intercepteurs de Struts, celui qui gère l'internationalisation des pages. Le paramètre *request_locale* sera reconnu et traité. La langue des pages deviendra l'anglais (*en*).

- ligne 19 : définit un lien Html. L'attribut **href** de la balise **<a>** attend une chaîne de caractères. Ici, on veut utiliser la valeur de l'URL définie en ligne 16 et d'id **URL**. On écrit pour cela, *href="%{URL}"*. La variable *URL* est évaluée et sa valeur affectée à l'attribut *href*. Dans la plupart des cas, l'évaluation des variables est implicite. Par exemple, lorsqu'on écrit **<s:property name="message"/>** c'est la valeur de la propriété *message* qui est affichée et non la chaîne " *message* ". Mais dans d'autres cas, il faut forcer l'évaluation des variables ou propriétés. Si on avait écrit *href="URL"*, c'est la chaîne *URL* qui aurait été affectée à l'attribut *href*.
- lignes 23-27 : créent un lien Html pour changer la langue des pages en espagnol.

2.8 Exécution de l'application

Nous lançons l'exécution du projet :



- en [1], on lance l'exécution du projet [exemple-01]. Le serveur web Tomcat est alors lancé automatiquement s'il ne l'était pas déjà. L'URL [/exemple-01] est demandée [3]. Le fichier [web.xml] est alors utilisé :

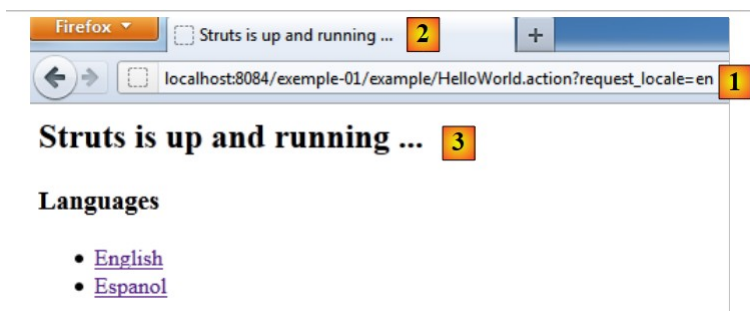
```

1. <?xml version="1.0" encoding="UTF-8"?>
2. <web-app version="3.0" xmlns="http://java.sun.com/xml/ns/javaee"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://java.sun.com/xml/ns/javaee/web-
   app_3_0.xsd">
3.   <filter>
4.     <filter-name>struts2</filter-name>
5.     <filter-class>org.apache.struts2.dispatcher.FilterDispatcher</filter-class>
6.   </filter>
7.   <filter-mapping>
8.     <filter-name>struts2</filter-name>
9.     <URL-pattern>/*</URL-pattern>
10.  </filter-mapping>
11.  <session-config>
12.    <session-timeout>
13.      30
14.    </session-timeout>
15.  </session-config>
16.  <welcome-file-list>
17.    <welcome-file>example/HelloWorld.JSP</welcome-file>
18.  </welcome-file-list>
19. </web-app>

```

- parce que dans l'URL [/exemple-01] demandée, aucune page n'est précisée, Tomcat va utiliser la balise `<welcome-file-list>` des lignes 16 et 18. C'est donc l'URL `/exemple-01/example/HelloWorld.JSP` qui va être servie.
- parce que Struts 2 traite toutes les URL (ligne 8 et 9), cette URL va être filtrée par Struts. Comme elle ne correspond pas à une action mais à une page JSP, cette dernière va être affichée.
- ce que nous voyons en [2] est donc la page `HelloWorld.JSP` que nous avons étudiée.
- en [4], on voit que la balise `<title><s:text name="HelloWorld.message"/></title>` n'a pas produit son effet ni la balise `<h2><s:property value="message"/></h2>`. La raison en est qu'aucune action n'a été appelée. La liste des intercepteurs qui s'exécutent avant l'action n'a donc pas été exécutée, notamment celui qui gère l'internationalisation. La balise d'internationalisation `<s:text ...>` n'a pu être traitée correctement. Par ailleurs, la propriété `message` qui référence le champ `message` de la classe `Action1` n'existe pas. D'où l'absence d'affichage.

Maintenant, suivons le lien [English]. Nous obtenons la page suivante :



- en [1], l'URL demandée. Nous avons expliqué la formation de cette URL. Cette fois-ci une action Struts est demandée : l'action *HelloWorld* défini dans [example.xml].

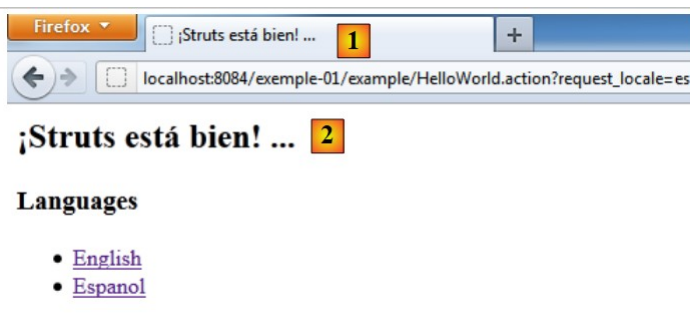
```

1. <struts>
2.   <package name="example" namespace="/example" extends="struts-default">
3.     <action name="HelloWorld" class="example.HelloWorld">
4.       <result>/example/HelloWorld.JSP</result>
5.     </action>
6.   </package>
7. </struts>

```

- la méthode *execute* de cette action a été exécutée. Nous avons vu qu'elle rendait la clé *success*. De la ligne 4 ci-dessus, on en déduit que la page */example/HelloWorld.JSP* est renvoyée en réponse au client. C'est ce que nous voyons en [3].
- en [1], nous voyons que l'URL demandée est paramétrée par le paramètre *request_locale=en*. La langue des pages sera désormais l'anglais. En effet, ce choix de langue est stocké dans la session de l'utilisateur qui conservera ce choix jusqu'à ce qu'il en change.
- en [2] et [3], on voit l'internationalisation à l'oeuvre. Les balises `<title><s:text name="HelloWorld.message"/></title>` et `<h2><s:property value="message"/></h2>` ont cette fois-ci produit leur effet.

Si nous choisissons maintenant le lien [Español], nous obtenons alors la page en espagnol :



2.9 Conclusion

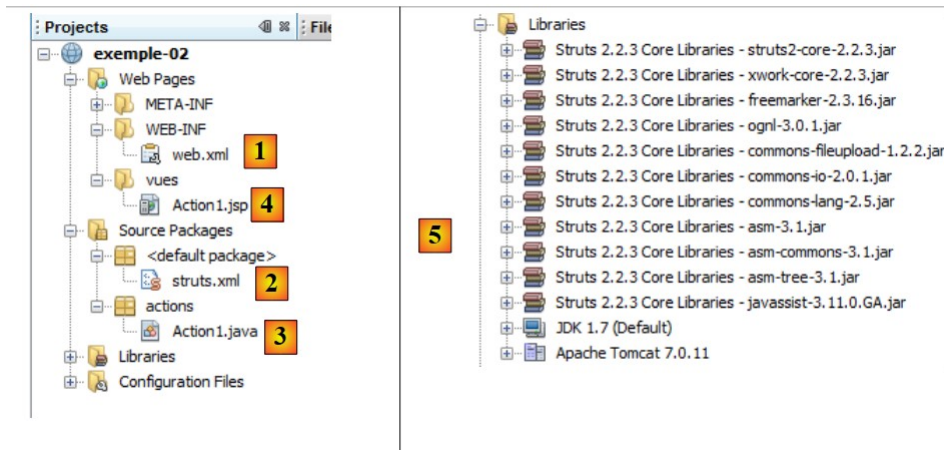
Nous avons étudié un exemple généré automatiquement par le plugin Struts 2 pour Netbeans. Nous avons constaté qu'il était assez difficile de suivre le traitement d'une requête. Les éléments suivants interviennent :

- la configuration [web.xml], [struts.xml]
- l'action exécutée : intercepteurs et méthode *execute*.

Au début, la mécanique de Struts 2 peut paraître complexe. Il faut un peu de temps pour s'y habituer. Nous allons maintenant présenter une série d'exemples qui éclairent chacun un point particulier de Struts.

3 Exemple 02 – Injection de paramètres dans l'action

3.1 Le projet Netbeans



- en [1,2], les fichiers de configuration
- en [3], l'action
- en [4], la vue
- en [5], les bibliothèques du projet

3.2 Les fichiers de configuration

Le fichier [web.xml] est le suivant :

```

1. <?xml version="1.0" encoding="UTF-8"?>
2.
3. <web-app xmlns="http://java.sun.com/xml/ns/javaee"
4.   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5.   xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://java.sun.com/xml/ns/javaee/web-
   app_3_0.xsd"
6.   version="3.0">
7.
8.   <display-name>Struts tuto-001</display-name>
9.   <filter>
10.    <filter-name>struts2</filter-name>
11.    <filter-class>org.apache.struts2.dispatcher.FilterDispatcher</filter-class>
12.  </filter>
13.  <filter-mapping>
14.    <filter-name>struts2</filter-name>
15.    <URL-pattern>/*</URL-pattern>
16.  </filter-mapping>
17.  <session-config>
18.    <session-timeout>
19.      30
20.    </session-timeout>
21.  </session-config>
22. </web-app>

```

Nous avons déjà commenté ce fichier. Nous ne le ferons plus. Il faut simplement se rappeler qu'il fait en sorte que toutes les URL (ligne 15) sont passées au filtre de Struts (ligne 10).

Le fichier [struts.xml] est le suivant :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.   "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.   "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.   <package name="default" namespace="/" extends="struts-default">
8.     <default-action-ref name="index" />
9.     <action name="index">
10.      <result type="redirectAction">
11.        <param name="actionName">Action1</param>
12.        <param name="namespace">/actions</param>
13.      </result>
14.    </action>

```

```

15. </package>
16. <package name="actions" namespace="/actions" extends="struts-default">
17.   <action name="Action1" class="actions.Action1">
18.     <result name="success">/vues/Action1.JSP</result>
19.   </action>
20. </package>
21. </struts>

```

- lignes 7-15 : définissent le package [default], celui qui est utilisé lorsqu'une action n'a pu être trouvée dans un autre package.
- ligne 8 : définit une action par défaut pour ce package nommé *index*.
- lignes 9-14 : configurent l'action nommée *index*. On voit qu'elle n'est pas associée à une classe (absence de l'attribut *class* ligne 9).
- ligne 10 : le résultat est pour la clé *success* (absence de l'attribut *name*). Il est de type *redirect:Action* (attribut *type*). Ce type permet de rediriger une action vers une autre action. Ici lorsque le client demandera l'action */index*, il sera redirigé vers l'action */actions/Action1* (lignes 11-12).
- lignes 16-20 : définissent le package *actions* (*name*) associé aux actions d'URL */actions/Action* (*class*).
- lignes 17-19 : configurent l'action */actions/Action1* (*name*). Sur demande de cette action, Strutsinstanciera la classe *actions.Action1* (*class*), puis la méthode *execute* de cette classe sera exécutée. Celle-ci devra rendre la clé *success*, car c'est la seule clé définie ligne 18.
- ligne 18 : pour la clé *success*, la vue à afficher sera la page JSP [vues/Action1.JSP].

Au final, on retiendra que le projet Struts ne sait exécuter que l'action d'URL [actions/Action1].

3.3 L'action

L'action *Action1* est représentée par la classe suivante :

```

1. package actions;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4.
5. public class Action1 extends ActionSupport{
6.
7.   // modèle de l'action
8.   private String param1="valeur1";
9.   private String param2="valeur2";
10.
11.   @Override
12.   public String execute(){
13.     return SUCCESS;
14.   }
15.
16.   // getters et setters
17.
18.   public String getParam1() {
19.     return param1;
20.   }
21.
22.   public void setParam1(String param1) {
23.     this.param1 = param1;
24.   }
25.
26.   public String getParam2() {
27.     return param2;
28.   }
29.
30.   public void setParam2(String param2) {
31.     this.param2 = param2;
32.   }
33.
34. }

```

- lignes 8-9 : deux champs accessibles via des get / set (lignes 18-32)
- lignes 12-14 : la méthode *execute* rend la clé *success* comme il était attendu. Elle ne fait rien d'autre.

3.4 La vue JSP

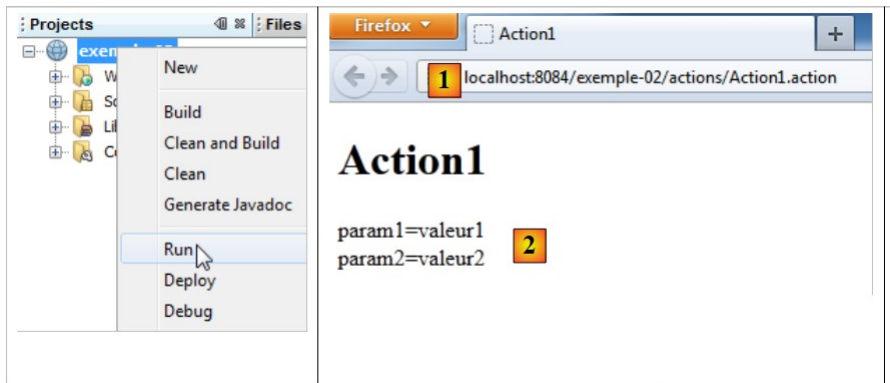
La vue *Action1.JSP* est la suivante :

```
1. <%@page contentType="text/html" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <!DOCTYPE html>
4. <html>
5.   <head>
6.     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
7.     <title>Action1</title>
8.   </head>
9.   <body>
10.    <h1>Action1</h1>
11.    param1=<s:property value="param1"/><br/>
12.    param2=<s:property value="param2"/><br/>
13.  </body>
14. </html>
```

Cette vue est affichée après l'exécution de la méthode *[Action1].execute*. Elle affiche les valeurs des champs *param1* (ligne 11) et *param2* (ligne 19) de la classe *[Action1]*.

3.5 Les tests

Exécutons le projet [exemple-02] :



- en [1], l'URL demandée. A noter que l'URL initiale demandée était [/exemple-02] sans action. Les fichiers [web.xml] [struts.xml] ont alors été exploités. On a vu que le fichier [web.xml] confiait le traitement de toute requête à Struts 2. Le fichier [struts.xml] a alors été exploité :

```
1. <struts>
2.   <package name="default" namespace="/" extends="struts-default">
3.     <default-action-ref name="index" />
4.     <action name="index">
5.       <result type="redirectAction">
6.         <param name="actionName">Action1</param>
7.         <param name="namespace">/actions</param>
8.       </result>
9.     </action>
10.  </package>
11.  <package name="actions" namespace="/actions" extends="struts-default">
12.    <action name="Action1" class="actions.Action1">
13.      <result name="success">/vues/Action1.JSP</result>
14.    </action>
15.  </package>
16. </struts>
```

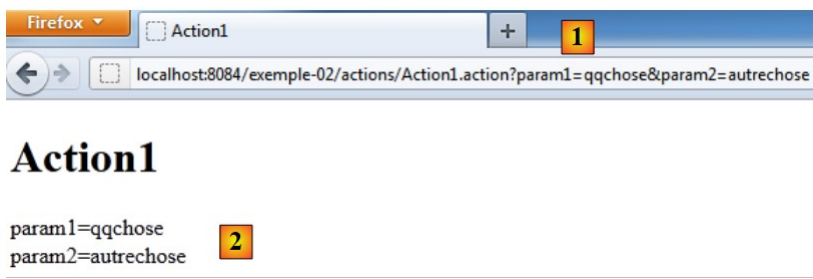
L'URL [/exemple-02] sans action a été traitée par le package *default* des lignes 2-10. En l'absence d'action dans l'URL, l'action est devenue *index* à cause de la ligne 3 (action par défaut). Les lignes 4-8 ont fait que Struts a renvoyé au client une URL de redirection [/exemple-02/actions/Action1] comme il est affiché en [1].

L'action [Action1] s'est alors exécutée comme configurée lignes 12-14. Sa méthode *execute* a été exécutée. On a vu qu'elle renvoyait la cé *success*. La ligne 13 de [struts.xml] a fait que la page [/vues/Action1.JSP] a été renvoyée au navigateur :

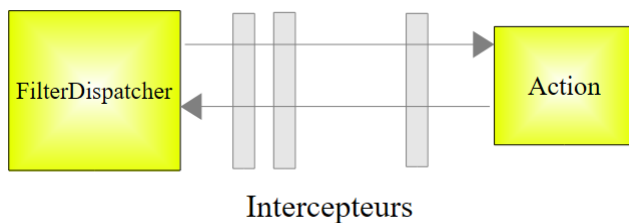
```
1. ...
2. <html>
3.   <head>
4.     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
5.     <title>Action1</title>
6.   </head>
7.   <body>
8.     <h1>Action1</h1>
9.     param1=<s:property value="param1"/><br/>
10.    param2=<s:property value="param2"/><br/>
11.   </body>
12. </html>
```

Les lignes 9 et 10 ont affiché les valeurs des champs *param1* et *param2* de [Action1]. C'est ce que montre [2].

Demandons une autre URL :



En [1], l'action [Action1] est demandée avec des paramètres *param1* et *param2*. Revenons au schéma d'exécution d'une action Struts :



Le contrôleur [FilterDispatcher] fait exécuter la méthode *execute* de l'action. Le flux d'exécution passe au-travers d'intercepteurs. L'un d'entre-eux traite la chaîne de paramètres *param1=qqchose¶m2=autrechose*. Il utilise alors les méthodes *setParam1* et *setParam2* de l'action *Action1* :

```
1. package actions;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4.
5. public class Action1 extends ActionSupport{
6.
7.   // modèle de l'action
8.   private String param1="valeur1";
9.   private String param2="valeur2";
10.
11.   @Override
12.   public String execute(){
13.     return SUCCESS;
14.   }
15.
16.   // getters et setters
17.
18.   public String getParam1() {
19.     return param1;
20.   }
21.
22.   public void setParam1(String param1) {
```

```

23.     this.param1 = param1;
24. }
25.
26. public String getParam2() {
27.     return param2;
28. }
29.
30. public void setParam2(String param2) {
31.     this.param2 = param2;
32. }
33.
34. }

```

L'intercepteur *params* exécute les méthodes suivantes :

```

[Action1].setParam1("qqchose") ;
[Action1].setParam2("autrechose") ;

```

Il faut donc qu'elles existent. On retiendra la chose suivante : pour récupérer les paramètres *parami* d'une requête Http, il faut que l'action Struts appelée ait des champs portant les mêmes noms que les paramètres et des méthodes get / set associées.

Lorsque la méthode *execute* de [Action1] s'exécute, les champs *param1* et *param2* ont été initialisés par l'intercepteur *params* :

```

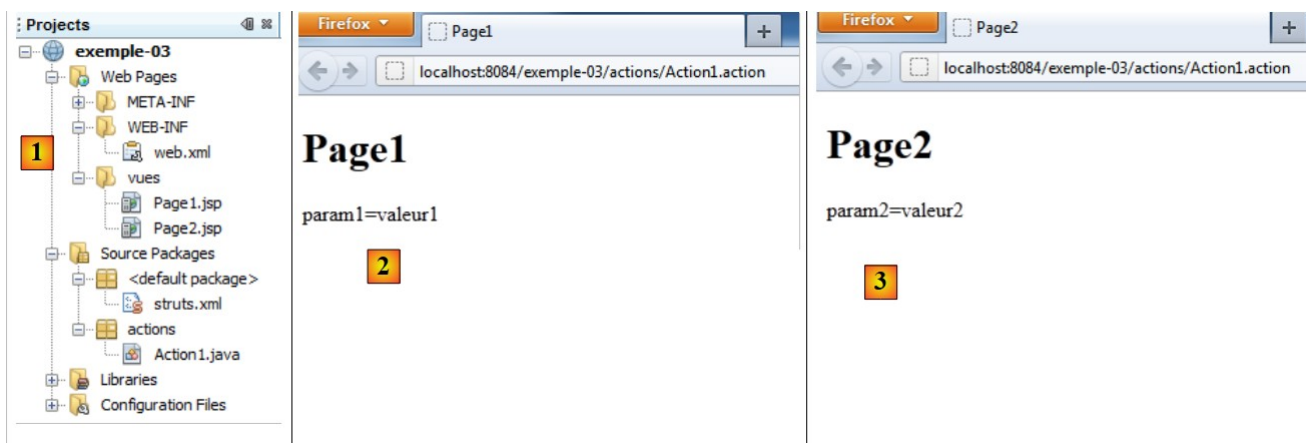
param1="qqchose"
param2="autrechose"

```

La méthode *execute* rend la clé *success* et la page [Action1.JSP] s'affiche avec les nouvelles valeurs de *param1* et *param2* [2].

4 Exemple 03 – Les clés de navigation

4.1 Le projet Netbeans



- en [1] :
 - [web.xml] : le fichier de configuration de l'application web
 - [struts.xml] : le fichier de configuration de Struts
 - [Action1.java] : l'unique action de l'application
 - [Page1.JSP, Page2.JSP] : les deux vues de l'application
- en [2] : affichage de [Page1.JSP]
- en [3] : affichage de [Page2.JSP]

4.2 Le fichier [struts.xml]

C'est le suivant :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">

```

```

5.
6. <struts>
7.   <package name="default" namespace="/" extends="struts-default">
8.     <default-action-ref name="index" />
9.     <action name="index">
10.      <result type="redirectAction">
11.        <param name="actionName">Action1</param>
12.        <param name="namespace">/actions</param>
13.      </result>
14.    </action>
15.  </package>
16.  <package name="actions" namespace="/actions" extends="struts-default">
17.    <action name="Action1" class="actions.Action1">
18.      <result name="page1">/vues/Page1.JSP</result>
19.      <result name="page2">/vues/Page2.JSP</result>
20.    </action>
21.  </package>
22. </struts>

```

- lignes 17-19 : la méthode *execute* de [Action1] va rendre deux clés de navigation :
 - *page1* qui fera afficher la vue */vues/Page1.JSP*
 - *page2* qui fera afficher la vue */vues/Page2.JSP*

4.3 L'action [Action1]

Elle est analogue à celle de l'exemple précédent :

```

1. package actions;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4.
5. public class Action1 extends ActionSupport{
6.
7.   // modèle de l'action
8.   private String param1="valeur1";
9.   private String param2="valeur2";
10.
11.   @Override
12.   public String execute(){
13.     // choix aléatoire entre deux vues
14.     int i=(int) (Math.random()*2);
15.     if(i==0){
16.       return "page1";
17.     }else{
18.       return "page2";
19.     }
20.   }
21.
22.   // getters et setters
23.   ...
24. }

```

- lignes 12-14 : la méthode *execute* rend de façon aléatoire les clés *page1* et *page2* attendues.

4.4 Les vues JSP

Page1.JSP

```

1. <%@page contentType="text/html" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <!DOCTYPE html>
4. <html>
5.   <head>
6.     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
7.     <title>Page1</title>
8.   </head>
9.   <body>
10.    <h1>Page1</h1>
11.    param1=<s:property value="param1"/><br/>
12.   </body>

```

```
13. </html>
```

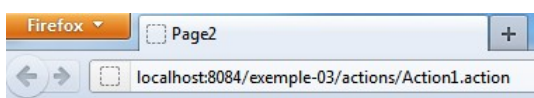
Ligne 11, la page affiche la valeur du champ *param1* de [Action1].

Page2.JSP

```
1. <%@page contentType="text/html" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <!DOCTYPE html>
4. <html>
5.   <head>
6.     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
7.     <title>Page2</title>
8.   </head>
9.   <body>
10.    <h1>Page2</h1>
11.    param2=<s:property value="param2"/><br/>
12.  </body>
13. </html>
```

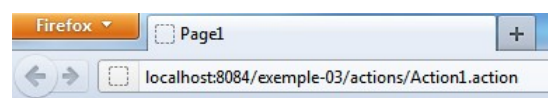
Ligne 11, la page affiche la valeur du champ *param2* de [Action1].

4.5 Les tests



Page2

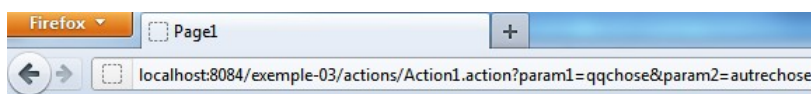
param2=valeur2



Page1

param1=valeur1

Une fois l'une des pages dans le navigateur, il faut la rafraîchir (F5) pour exécuter de façon répétée l'action [Action1] jusqu'à avoir l'autre page. On peut également passer des paramètres :



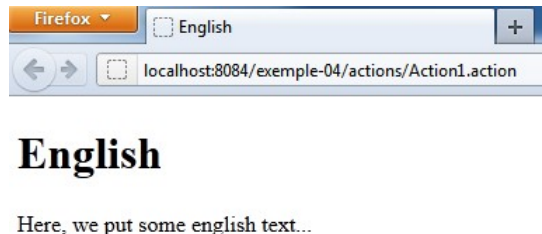
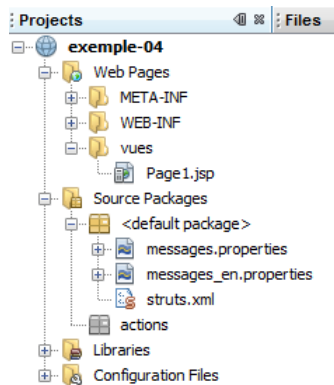
Page1

param1=qqchose

5 Exemple 04 – Internationalisation

Cet exemple revient sur l'internationalisation étudiée dans le premier exemple.

5.1 Le projet Netbeans



On y trouve :

- une vue [Page1.JSP]
- deux fichiers de messages [messages*.properties]
- pas d'actions

5.2 Configuration du projet

Le fichier [struts.xml] est le suivant :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.     <!-- internationalisation -->
8.     <constant name="struts.custom.i18n.resources" value="messages" />
9.     <!-- package default -->
10.    <package name="default" namespace="/" extends="struts-default">
11.        <default-action-ref name="index" />
12.        <action name="index">
13.            <result type="redirectAction">
14.                <param name="actionName">Action1</param>
15.                <param name="namespace">/actions</param>
16.            </result>
17.        </action>
18.    </package>
19.    <!-- package actions -->
20.    <package name="actions" namespace="/actions" extends="struts-default">
21.        <action name="Action1">
22.            <result name="success">/vues/Page1.JSP</result>
23.        </action>
24.    </package>
25. </struts>

```

- la ligne 8 définit une constante Struts, celle qui fixe le nom du fichier des messages internationalisés, ici *messages*. Les fichiers [messages_xx.properties] seront cherchés dans le *ClassPath* du projet. Pour cette raison, ils ont été placés ici à la racine de [Source Packages] [1].
- lignes 21-23 : définissent une action [Action1] sans classe associée. Une classe par défaut de Struts sera alors utilisée dont la méthode *execute* rend la clé *success*. La vue [/vues/Page1.JSP] sera alors renvoyée au client. Il est important de comprendre que cela ne revient pas au même que d'appeler directement la vue [/vues/Page1.JSP]. En effet, l'appel d'une action déclenche l'exécution des intercepteurs ce que ne fait pas l'appel direct à une vue. L'un des intercepteurs gère l'internationalisation.

5.3 Les fichiers de messages

messages.properties

```
page.texte=Ici, on met un texte fran\u00e7ais...
page.titre1=Fran\u00e7ais
page.titre2=Fran\u00e7ais
```

messages_en.properties

```
page.texte=Here, we put some english text...
page.titre1=English
page.titre2=English
```

Ils définissent trois clés *page.texte*, *page.titre1*, *page.titre2*. Le fichier [messages_**en**.properties] sera utilisé si la langue de la page est l'anglais (**en**). Le fichier [messages.properties] sera utilisé pour toutes les autres langues. C'est le fichier de messages par défaut. La langue utilisée est soit :

- celle du navigateur client dans ses préférences envoyées au serveur
- celle demandée par une requête du client avec le paramètre *request_locale=xx*.

5.4 La vue [Page1.JSP]

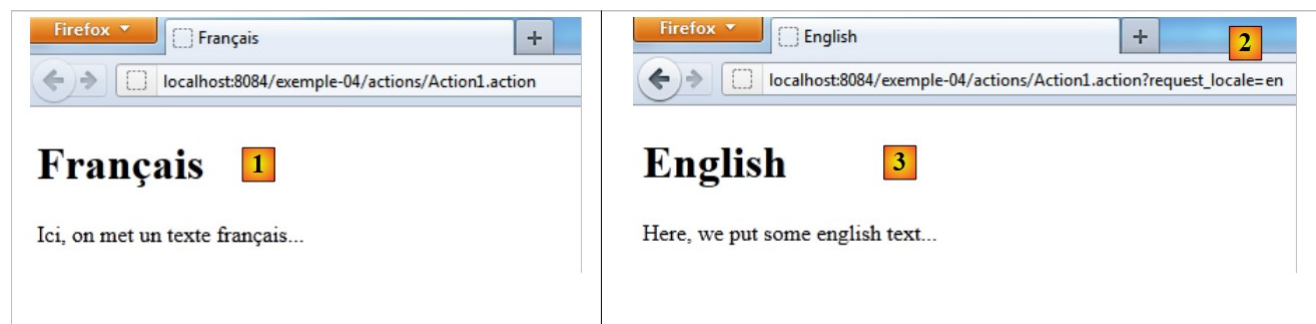
La vue est la suivante :

```
1. <%@page contentType="text/html" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <!DOCTYPE html>
4. <html>
5.   <head>
6.     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
7.     <title><s:text name="page.titre1"/></title>
8.   </head>
9.   <body>
10.    <h1><s:text name="page.titre2"/></h1>
11.    <s:text name="page.texte"/>
12.   </body>
13. </html>
```

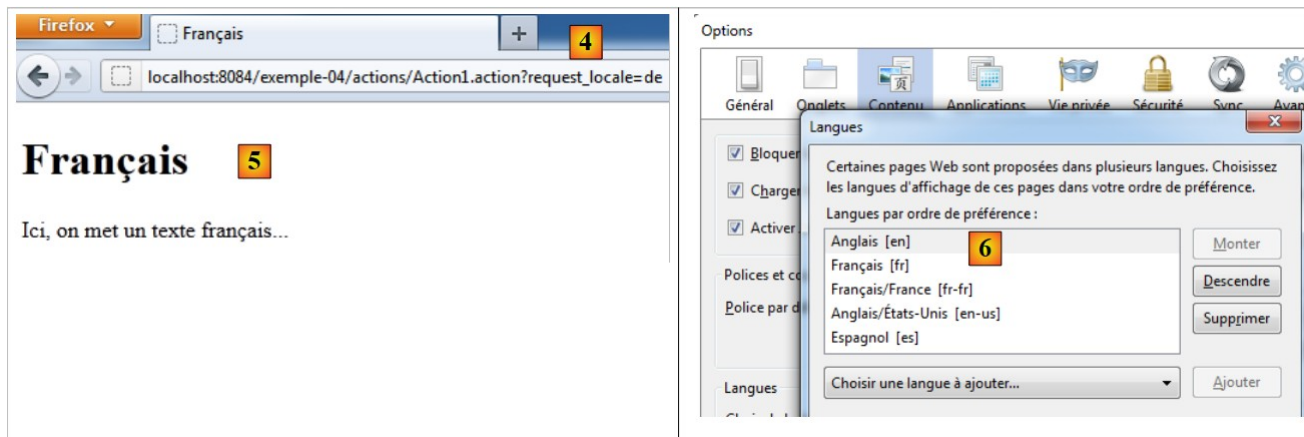
- ligne 7 : affiche le message de clé *page.titre1*
- ligne 10 : affiche le message de clé *page.titre2*
- ligne 11 : affiche le message de clé *page.texte*

5.5 Les tests

Exécutons le projet :



- en [1], la page s'est affichée en français. C'est le fichier [messages.properties] qui a été utilisé parce que le navigateur utilisé avait le Français comme langue préférée.
- en [3], la page s'est affichée en anglais. C'est le fichier [messages_**en**.properties] qui a été utilisé parce que l'URL [2] a utilisé le paramètre *request_locale=en*.



En [4], c'est une page en allemand (de) qui est demandée. Comme il n'existe pas de fichier [messages_de.properties], c'est le fichier par défaut [messages.properties] qui est utilisé [5].

En [6], on met l'anglais comme langue préférée du navigateur.



En [7], l'action [Action1] est demandée sans paramètres de langue. La requête vient d'un navigateur dont la langue préférée est l'anglais (en). C'est donc le fichier [messages_en.properties] qui est utilisé [8].

A partir de maintenant, les projets auront tous un fichier [messages.properties] unique qui sera utilisé pour toutes les langues. Cela nous obligera à écrire des vues qui utilisent les clés de ce fichier. Internationaliser en anglais reviendra à créer le fichier [messages_en.properties] des messages anglais. Il n'y a pas d'autre modification à faire.

6 Exemple 05 – Le formulaire de saisie

Nous introduisons sur un exemple simple la notion de formulaire de saisie.

6.1 Le projet Netbeans



En [1], le projet :

- les vues [Saisie.JSP], [Confirmation.JSP]
- le fichier de configuration [struts.xml]
- le fichier des messages [messages.properties]
- l'action [Confirmer.java]

En [2], la page de saisie

6.2 Configuration

Le fichier [struts.xml] est le suivant :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.     <!-- internationalisation -->
8.     <constant name="struts.custom.i18n.resources" value="messages" />
9.     <!-- package default -->
10.    <package name="default" namespace="/" extends="struts-default">
11.        <default-action-ref name="index" />
12.        <action name="index">
13.            <result type="redirectAction">
14.                <param name="actionName">Saisir</param>
15.                <param name="namespace">/actions</param>
16.            </result>
17.        </action>
18.    </package>
19.    <!-- package actions -->
20.    <package name="actions" namespace="/actions" extends="struts-default">
21.        <action name="Saisir">
22.            <result name="success">/vues/Saisie.JSP</result>
23.        </action>
24.        <action name="Confirmer" class="actions.Confirmer">
25.            <result name="success">/vues/Confirmation.JSP</result>
26.        </action>
27.    </package>
28. </struts>

```

- ligne 20 : le package des actions d'URL /actions/Action.
- lignes 21-23 : définissent l'action [Saisir]. Aucune classe ne lui est associée. La réponse est toujours la vue [/vues/Saisie.JSP]
- lignes 24-26 : définissent l'action [Confirmer]. La classe [actions.Confirmer] lui est associée. La réponse est toujours la vue [/vues/Confirmation.JSP]

6.3 L'action [Confirmer]

Son code est le suivant :

```
1. package actions;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4.
5. public class Confirmer extends ActionSupport{
6.
7.     // modèle
8.     private String nom;
9.
10.    // getters et setters
11.
12.    public String getNom() {
13.        return nom;
14.    }
15.
16.    public void setNom(String nom) {
17.        this.nom = nom;
18.    }
19.
20. }
```

La classe n'a qu'un champ, celui de la ligne 8 avec ses get / set.

6.4 Le fichier des messages

Le contenu de [messages.properties] est le suivant :

```
1. saisie.texte=Formulaire de saisie
2. saisie.titre1=Saisie
3. saisie.titre2=Saisie
4. saisie.libelle=Tapez votre nom
5. saisie.valider=Valider
6. confirm.titre1=Confirmation
7. confirm.titre2=Confirmation
8. confirm.texte=Bonjour
9. confirm. retour=Retour au formulaire de saisie
```

Ces clés de messages sont utilisées dans les deux vues [Saisie.JSP] et [Confirmation.JSP].

6.5 Les vues

6.5.1 La vue [Saisie.JSP]

C'est visuellement la suivante :



Son code source est le suivant :

```
1. <%@page contentType="text/html" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
```

```

3. <!DOCTYPE html>
4. <html>
5.   <head>
6.     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
7.     <title><s:text name="saisie.titre1"/></title>
8.   </head>
9.   <body>
10.    <h1><s:text name="saisie.titre2"/></h1>
11.    <s:form action="Confirmer">
12.      <s:textfield key="saisie.libelle" name="nom"/>
13.      <s:submit key="saisie.valider" action="Confirmer"/>
14.    </s:form>
15.  </body>
16. </html>

```

- ligne 7 : affiche le message de clé *saisie.titre1* (Saisie) [1].
- ligne 10 : affiche le message de clé *saisie.titre2* (Saisie) [2].
- lignes 11, 14 : la balise `<s:form>` introduit un formulaire Html. Toutes les saisies à l'intérieur de ce formulaire seront envoyées au serveur web. On dit qu'elles sont *postées*, car l'opération Http qui intervient dans cet envoi de données au serveur s'appelle POST. Les données envoyées le sont sous forme de chaîne de caractères *param1=valeur1¶m2=valeur2&...* Nous avons déjà rencontré cette chaîne paragraphe 3.5 dans l'exemple 02. Dans les deux cas, la chaîne des paramètres est traitée de la même façon par Struts 2. Les valeurs *valeuri* des paramètres *parami* sont injectées dans l'action exécutée, dans des champs portant les mêmes noms que les paramètres *parami*. A qui sont envoyées les données postées ? Par défaut, à l'action qui a affiché le formulaire, ici l'action *Saisir* [5]. On peut également utiliser l'attribut **action** pour préciser une autre action, ici l'action [Confirmer].
- ligne 12 : la balise `<s:textfield ...>` affiche le champ de saisie [3]. Son attribut *key* désigne la clé du libellé à afficher à gauche du champ de saisie. Le libellé est donc cherché dans le fichier [messages.properties]. L'attribut *name* est le nom du paramètre qui sera posté. La valeur associée à ce paramètre sera le texte saisi dans le champ de saisie.
- ligne 13 : la balise `<s:submit ...>` affiche le bouton [4]. Son attribut *key* désigne la clé du libellé à afficher sur le bouton. Le libellé est donc cherché dans le fichier [messages.properties]. Un bouton de type *submit* provoque le POST du formulaire vers une action. Nous avons dit plus haut que celle-ci était l'action [Confirmer] à cause de l'attribut *action* de la balise `<s:form>`. L'attribut *action* de la balise `<s:submit>` permet de préciser éventuellement une autre action. Ici nous avons remis l'action [Confirmer]. Le bouton a un nom de paramètre par défaut : *action:nom_action*, ici *action:Confirmer*. La valeur associée à ce paramètre est le libellé du bouton. Au final, la chaîne de paramètres postée à l'action [Confirmer] lorsque l'utilisateur cliquera sur le bouton [Valider] sera :

```
?nom=xx&action:Confirmer=Valider.
```

où **xx** est le texte saisi par l'utilisateur dans le champ de saisie.

6.5.2 La vue [Confirmation.JSP]

C'est visuellement la suivante :



Nous entrons un nom et validons la page de saisie. Nous obtenons alors la réponse [1], celle de la vue [Confirmation.JSP]. L'URL affichée en [2] est celle de l'action [Confirmer]. Dans l'exemple ci-dessus, la chaîne de caractères suivante a été postée :

```
nom=bernard&action:Confirmer=Valider
```

Le paramètre `action:Confirmer` permet à Struts 2 d'orienter la requête vers la bonne action, ici l'action [Confirmer]. La valeur du paramètre `nom` sera alors injectée dans le champ `nom` de l'action [Confirmer].

Le code source de l'action [Confirmation.JSP] est le suivant :

```
1. <%@page contentType="text/html" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <!DOCTYPE html>
4. <html>
5.   <head>
6.     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
7.     <title><s:text name="confirm.titre1"/></title>
8.   </head>
9.   <body>
10.    <h1><s:text name="confirm.titre2"/></h1>
11.    <s:text name="confirm.texte"/>&nbsp;
12.    <s:property value="nom"/> !
13.    <br/><br/>
14.    <a href="<s:url action="Saisir"/>"><s:text name="confirm.retour"/></a>
15.  </body>
16. </html>
```

- ligne 7 : affiche le libellé de clé `confirm.titre1` trouvé dans le fichier des messages [1]
- ligne 10 : affiche le libellé de clé `confirm.titre2` trouvé dans le fichier des messages [3]
- ligne 12 : affiche la propriété `nom` de l'action [Confirmer] qui a été exécutée.
- ligne 14 : crée un lien vers l'action [Saisir]. La balise `<s:url>`, déjà rencontrée, va être générée à partir de l'URL [2] du navigateur. Le chemin sera celui de [2] `http://localhost:8084/exemple-05/actions` et l'action celle de l'attribut `action`, ici `Saisir`. L'URL du lien sera donc `http://localhost:8084/exemple-05/actions/Saisir.action`. Le texte du lien sera le libellé associé à la clé `confirm.retour` [4].

6.6 Les tests

Voici deux requêtes :



En [1], on saisit un nom et on valide. En [2], la page de confirmation.



En [3] on suit le lien de retour au formulaire. En [4] le résultat. Tout a été expliqué, sauf le passage de [3] à [4]. Pourquoi ne retrouve-t-on pas dans [4], le nom qu'on avait saisi ?

Le lien en [3] est un lien vers l'URL [<http://localhost:8084/exemple-05/actions/Saisir.action>]. L'action [Saisir] est donc exécutée. Regardons sa configuration dans [struts.xml] :

```
1. <package name="actions" namespace="/actions" extends="struts-default">
2.   <action name="Saisir">
3.     <result name="success">/vues/Saisie.JSP</result>
4.   </action>
5.   <action name="Confirmer" class="actions.Confirmer">
6.     <result name="success">/vues/Confirmation.JSP</result>
7.   </action>
8. </package>
```

L'action [Saisir] n'est pas associée à une action. Donc la vue [Saisie.JSP] est immédiatement affichée. Regardons le code de cette vue :

```
1. <%@page contentType="text/html" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <!DOCTYPE html>
4. <html>
5.   <head>
6.     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
7.     <title><s:text name="saisie.titre1"/></title>
8.   </head>
9.   <body>
10.    <h1><s:text name="saisie.titre2"/></h1>
11.    <s:form>
12.      <s:textfield key="saisie.libelle" name="nom"/>
13.      <s:submit key="saisie.valider" action="Confirmer"/>
14.    </s:form>
15.  </body>
16. </html>
```

En ligne 12, le champ de saisie est affiché. Son contenu est initialisé avec la propriété *nom* (name). Rappelons ce qui avait été écrit plus haut sur le fonctionnement de la balise `<s:property>` :

La balise `<s:property name="propriété">` permet d'écrire la valeur d'une propriété d'un objet appelé *ActionContext*. On trouve dans cet objet :

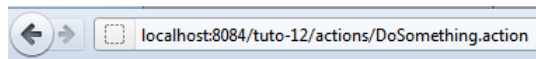
1. les propriétés de la classe associée à l'action qui a été exécutée.
2. les attributs de la requête courante notés `<s:property name="#request['clé']">`
3. les attributs de la session de l'utilisateur notés `<s:property name="#session['clé']">`
4. les attributs de l'application elle-même notés `<s:property name="#application['clé']">`
5. les paramètres envoyés par le navigateur client notés `<s:property name="#parameters['clé']">`
6. la notation `<s:property name="#attr['clé']">` affiche la valeur d'un objet cherché dans la page, la requête, la session, l'application dans cet ordre.

On est ici dans le cas 1. L'action [Saisir] n'a pas de classe associée. La propriété *nom* n'existe donc pas. Une chaîne vide est alors affichée dans le champ de saisie.

Pour afficher en [4] le nom qui a été saisi en [1], nous utiliserons la session de l'utilisateur.

7 Exemple 05B – Navigation dans un formulaire de saisie

Nous étudions un formulaire présentant plusieurs boutons pour soumettre (submit) les données saisies à une action.



Actions et Méthodes

DoSomething.execute

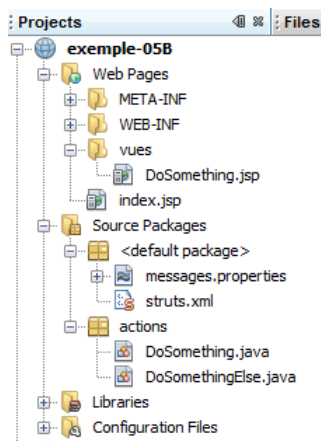
DoSomething.action1

DoSomethingElse.action2

[DoSomething.action3](#)

7.1 Le projet Netbeans

Le projet Netbeans est le suivant :



Les éléments du projet sont les suivants :

- [DoSomething.JSP] : l'unique vue de l'application qui présente les trois boutons de navigation ainsi qu'un lien.
- [DoSomething.java] et [DoSomethingElse.java] : les deux actions du projet
- [messages.properties] : le fichier des messages internationalisés
- [struts.xml] : le fichier de configuration Struts 2

7.2 Configuration

Le fichier [struts.xml] suivant configure l'application :

```
1. <!DOCTYPE struts PUBLIC
2. "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
3. "http://struts.apache.org/dtds/struts-2.0.dtd">
4.
5. <struts>
6.   <constant name="struts.custom.i18n.resources" value="messages" />
7.   <package name="actions" namespace="/actions" extends="struts-default">
8.     <action name="DoSomething" class="actions.DoSomething">
9.       <result name="success">/vues/DoSomething.JSP</result>
10.    </action>
11.    <action name="DoSomethingElse" class="actions.DoSomethingElse">
12.      <result name="success">/vues/DoSomething.JSP</result>
13.    </action>
```

```
14. </package>
15. </struts>
```

- l'action [/actions/DoSomething] provoquera l'instanciation de la classe [actions.DoSomething] et sa méthode *execute* sera exécutée par défaut. Ce défaut peut être annulé si les paramètres passés à l'action [Something] précisent une autre méthode. Quelque soit la méthode exécutée, elle devra rendre la clé de navigation *success* puisque seule cette clé a été définie. La vue [DoSomething.JSP] sera alors affichée.
- la configuration de l'action [/actions/DoSomethingElse] est identique.

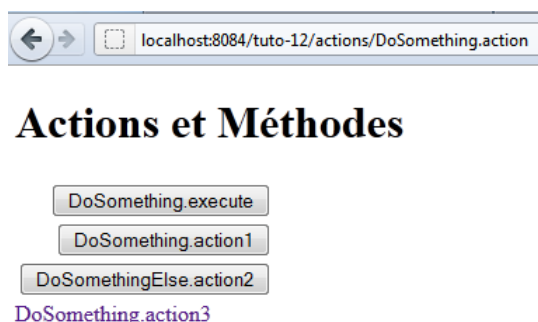
7.3 Le fichier des messages

Le fichier des messages [messages.properties] est le suivant :

```
1. formulaire.titre1=Actions et M\u00e9thodes
2. formulaire.titre2=Actions et M\u00e9thodes
3. formulaire.execute=DoSomething.execute
4. formulaire.action1=DoSomething.action1
5. formulaire.action2=DoSomethingElse.action2
6. formulaire.action3=DoSomething.action3
```

7.4 La vue [DoSomething.JSP]

La vue [DoSomething.JSP] est la suivante :



Son code est le suivant :

```
1. <%@page contentType="text/html" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <!DOCTYPE html>
4. <html>
5.   <head>
6.     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
7.     <title><s:text name="formulaire.titre1"/></title>
8.   </head>
9.   <body>
10.    <h1><s:text name="formulaire.titre2"/></h1>
11.    <s:form action="DoSomething">
12.      <s:submit key="formulaire.execute"/>
13.      <s:submit key="formulaire.action1" method="action1"/>
14.      <s:submit key="formulaire.action2" action="DoSomethingElse" method="action2"/>
15.    </s:form>
16.    <s:url id="URL" action="DoSomething" method="action3"/>
17.    <s:a href="{URL}"><s:text name="formulaire.action3"/></s:a>
18.  </body>
19. </html>
```

- ligne 11 : le formulaire sera posté à l'action [DoSomething]. Cela ne veut pas forcément dire que c'est cette action qui va être déclenchée. Cela dépend du bouton *submit* utilisé pour le POST.

- ligne 12 : le bouton *submit* ne précise ni action, ni méthode. Ce sera l'action [DoSomething] précisée par la balise <form> qui sera instanciée. La méthode exécutée sera celle définie dans le fichier [struts.xml], la méthode *execute*.
- ligne 13 : le bouton *submit* précise une méthode. Ce sera l'action [DoSomething] précisée par la balise <form> qui sera instanciée. La méthode exécutée sera la méthode *action1*.
- ligne 14 : le bouton *submit* précise une action et une méthode. Ce sera l'action [DoSomethingElse] qui sera instanciée. La méthode exécutée sera la méthode *action2*.
- lignes 16-17 : un lien vers l'action [DoSomething] et la méthode *action3*. L'action [DoSomething] sera instanciée et sa méthode *action3* exécutée. Contrairement aux boutons *submit*, le clic sur le lien ne provoque pas un POST mais un GET. Il n'y a donc aucun paramètre posté.

Le code Html généré par ce code, lorsqu'on demande l'URL [http://localhost:8084/exemple-05B/actions/DoSomething.action] est le suivant :

```

1. <!DOCTYPE html>
2. <html>
3.   <head>
4.     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
5.     <title>Actions et Méthodes</title>
6.   </head>
7.   <body>
8.     <h1>Actions et Méthodes</h1>
9.     <form ... action="/exemple-05B/actions/DoSomething.action" method="post">
10. ...
11.       <input type="submit" ... name="formulaire.execute" value="DoSomething.execute"/>
12. ...
13.       <input type="submit" ... name="method:action1" value="DoSomething.action1"/>
14. ...
15.       <input type="submit" ... name="action:DoSomethingElse!action2" value="DoSomethingElse.action2"/>
16. ...
17.     </form>
18. ...
19.     <a href="/exemple-05B/actions/DoSomething!action3.action">DoSomething.action3</a>
20.   </body>
21. </html>

```

- ligne 9 : la balise *form* du formulaire Html. L'attribut *action* indique l'URL à laquelle seront postés les paramètres du formulaire. Cette URL est celle de l'action [DoSomething].
- lignes 11, 13, 15 : les attributs *name* des boutons seront postés à l'action cible du bouton *submit*. C'est ce qui permet à Struts de déterminer l'action à instancier et la méthode à exécuter.
- ligne 13 : l'attribut *name=method:action1* indique que la méthode *DoSomething.action1* doit être exécutée.
- ligne 15 : l'attribut *name=action:DoSomethingElse!action2* indique que la méthode *DoSomethingElse.action2* doit être exécutée.
- ligne 11 : l'attribut *name= formulaire.execute* ne précise ni action, ni méthode. C'est donc la méthode *DoSomething.execute* qui sera exécutée.
- ligne 19 : le lien demande explicitement l'exécution de la méthode *DoSomething.action3*

7.5 Les actions

L'action [DoSomething] est la suivante :

```

1. package actions;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4.
5. public class DoSomething extends ActionSupport {
6.
7.     public DoSomething() {
8.         System.out.println("DoSomething");
9.     }
10.
11.     @Override
12.     public String execute() {
13.         System.out.println("DoSomething.execute");
14.         return SUCCESS;
15.     }
16.
17.     public String action1() {
18.         System.out.println("DoSomething.action1");
19.         return SUCCESS;
20.     }
21.
22.     public String action3() {
23.         System.out.println("DoSomething.action3");

```

```

24.     return SUCCESS;
25. }
26. }

```

- les méthodes *execute*, *action1* et *action3* écrivent sur la console du serveur web et rendent la clé de navigation *success*.
- ligne 8 : le constructeur écrit également sur la console du serveur web

Les différentes écritures sur la console permettent de savoir quelles méthodes sont exécutées lors d'une requête.

L'action [DoSomethingElse] est similaire :

```

1. package actions;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4.
5. public class DoSomethingElse extends ActionSupport {
6.
7.     public DoSomethingElse() {
8.         System.out.println("DoSomethingElse");
9.     }
10.
11.     @Override
12.     public String execute() {
13.         System.out.println("DoSomethingElse.execute");
14.         return SUCCESS;
15.     }
16.
17.     public String action2() {
18.         System.out.println("DoSomethingElse.action2");
19.         return SUCCESS;
20.     }
21. }

```

7.6 Les tests

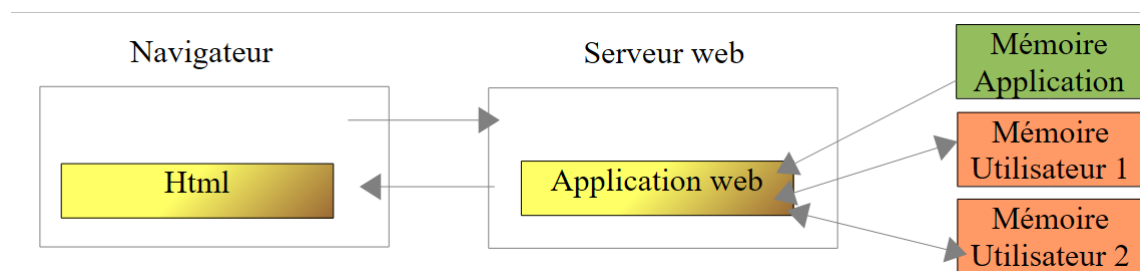
Les tests montrent les résultats suivants (sur la console du serveur web) :

- lors d'un clic sur le bouton [DoSomething.execute], l'action [DoSomething] est instanciée et sa méthode *execute* exécutée.
- lors d'un clic sur le bouton [DoSomething.action1], l'action [DoSomething] est instanciée et sa méthode *action1* exécutée.
- lors d'un clic sur le bouton [DoSomethingElse.action2], l'action [DoSomethingElse] est instanciée et sa méthode *action2* exécutée.
- lors d'un clic sur le lien [DoSomething.action3], l'action [DoSomething] est instanciée et sa méthode *action3* exécutée.

8 Exemple 06 – La session

8.1 La notion de session

Lorsqu'un navigateur client se connecte la première fois à une application web, il reçoit un jeton de session, une suite de caractères unique qu'il renvoie à chaque nouvelle requête qu'il fait à l'application web. Cela permet à celle-ci de reconnaître le navigateur client. A ce jeton de session, elle peut alors associer des données. Ces données appartiennent à un unique navigateur client. Ainsi, au fil des requêtes du navigateur client, se constitue une mémoire.



Ci-dessus, chaque utilisateur (navigateur) a sa propre mémoire qu'on appelle sa session. Cette mémoire est partagée par toutes les requêtes d'un même utilisateur. Il existe également une mémoire de plus haut niveau appelée mémoire de l'application. Cette mémoire est partagée par toutes les requêtes de tous les utilisateurs. Elle est en général en lecture seule.

8.2 Le projet Netbeans



Le projet [exemple-06] est obtenue par recopie du projet [exemple-05]. Nous allons changer quelques éléments pour

- bénéficier de la session de l'utilisateur.
- ajouter une nouvelle action [Effacer] [1] pour effacer le champ de saisie.

8.3 Configuration

Le fichier [struts.xml] évolue de la façon suivante :

```
1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.     <!-- internationalisation -->
8.     <constant name="struts.custom.i18n.resources" value="messages" />
9.     <!-- package default -->
10.    <package name="default" namespace="/" extends="struts-default">
11.        <default-action-ref name="index" />
12.        <action name="index">
13.            <result type="redirectAction">
14.                <param name="actionName">Saisir</param>
15.                <param name="namespace">/actions</param>
16.            </result>
17.        </action>
18.    </package>
19.    <!-- package actions -->
20.    <package name="actions" namespace="/actions" extends="struts-default">
21.        <action name="Saisir">
22.            <result name="success">/vues/Saisie.JSP</result>
23.        </action>
24.        <action name="Confirmer" class="actions.Confirmer">
25.            <result name="success">/vues/Confirmation.JSP</result>
26.        </action>
27.        <action name="Effacer" class="actions.Effacer">
28.            <result name="success">/vues/Saisie.JSP</result>
29.        </action>
30.    </package>
31. </struts>
```

Les lignes 27-29 définissent une nouvelle action [Effacer] associée à une classe [Effacer]. La réponse à cette action est la vue [Saisie.JSP].

8.4 L'action [Confirmer]

Elle évolue comme suit :

```
1. package actions;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4. import java.util.Map;
5. import org.apache.struts2.interceptor.SessionAware;
6.
7. public class Confirmer extends ActionSupport implements SessionAware{
8.
9.     // modèle
10.    private String nom;
11.    // session
12.    private Map<String, Object> session;
13.
14.    // getters et setters
15.
16.    public String getNom() {
17.        return nom;
18.    }
19.
20.    public void setNom(String nom) {
21.        this.nom = nom;
22.    }
23.
24.    @Override
25.    public void setSession(Map<String, Object> session) {
26.        this.session=session;
27.    }
28.
29.    @Override
30.    public String execute(){
31.        // on met le nom dans la session
32.        session.put("nom",nom);
33.        // navigation
34.        return SUCCESS;
35.    }
36.
37. }
```

- ligne 7 : la classe [Confirmer] implémente l'interface **SessionAware**. Cette interface n'a qu'une méthode, la méthode **setSession** des lignes 25-27. Avant l'appel de la méthode *execute*, l'un des intercepteurs de la requête va injecter, via la méthode *setSession*, la session de l'utilisateur sous la forme d'un dictionnaire *Map<String, Object>* (ligne 25). Nous choisissons de mémoriser ce dictionnaire dans le champ *session* de la ligne 12.
- lignes 30-34 : la méthode *execute* de l'action. Lorsqu'elle s'exécute, le champ *session* a été initialisé par l'un des intercepteurs ainsi que le champ *nom* par un autre intercepteur. On utilise ce dictionnaire *session* pour y stocker le champ *nom*. Ainsi le nom va-t-il faire partie de la mémoire de l'utilisateur et être disponible à toutes les requêtes de celui-ci.

8.5 Les vues [Confirmation.JSP] et [Saisie.JSP]

La vue [Confirmation.JSP] reste inchangée. La vue [Saisie.JSP] évolue comme suit :

```
1. <%@page contentType="text/html" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <!DOCTYPE html>
4. <html>
5.     <head>
6.         <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
7.         <title><s:text name="saisie.titre1"/></title>
8.     </head>
9.     <body>
10.        <h1><s:text name="saisie.titre2"/></h1>
11.        <s:form action="Confirmer">
12.            <s:textfield key="saisie.libelle" name="nom" value="%{#attr['nom']}" />
```



```

13.     <s:submit key="saisie.valider" action="Confirmer"/>
14.     <s:submit key="saisie.effacer" action="Effacer"/>
15. </s:form>
16. </body>
17. </html>

```

- ligne 12 : nous introduisons un attribut *value* à la balise `<s:textfield>`. Cet attribut fixe la valeur à afficher dans la zone de saisie. En l'absence de cet attribut, *value* = *name*. Ici, la valeur de l'attribut est une expression OGNL (Object-Graph Navigation Language) de la forme `%{expression_à_évaluer}`. Ici, l'expression à évaluer est `#attr['nom']`. L'attribut *nom* sera cherché dans l'action courante, la page, la requête, la session, l'application dans cet ordre. Comme l'action [Confirmer] met l'attribut *nom* dans la session, il sera trouvé là. C'est ce que montre la requête suivante :



En [1], le nom saisi a été *ST*. On sait que l'action [Confirmer] a mis ce nom dans la session. Le lien [2] nous mène à l'URL [3]. La vue [Saisie.JSP] est affichée. Pour le champ de saisie, l'attribut `%{#attr['nom']}` permet de retrouver le nom dans la session.

- ligne 14 : le bouton [Effacer] qui va déclencher l'exécution de l'action [Effacer] et l'affichage de la vue [Saisie.JSP]

```

1. <action name="Effacer" class="actions.Effacer">
2.     <result name="success">/vues/Saisie.JSP</result>
3. </action>

```

8.6 L'action [Effacer]

Le code de l'action [Effacer] est le suivant :

```

1. package actions;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4. import java.util.Map;
5. import org.apache.struts2.interceptor.SessionAware;
6.
7. public class Effacer extends ActionSupport implements SessionAware{
8.
9.     // session
10.    private Map<String, Object> session;
11.
12.    @Override
13.    public String execute(){
14.        // on récupère le nom dans la session
15.        String nom=(String)session.get("nom");
16.        // on l'enlève de la session si besoin est
17.        if(nom!=null){
18.            session.remove("nom");
19.        }
20.        // navigation
21.        return SUCCESS;
22.    }
23.
24.    @Override
25.    public void setSession(Map<String, Object> map) {
26.        this.session=map;
27.    }

```

- ligne 7 : la classe [Effacer] implémente l'interface [SessionAware] comme le faisait l'action [Confirmer].
- ligne 13 : l'action [Effacer] doit effacer le contenu du champ de saisie du nom dans la vue [Saisie.JSP]. On sait que cette vue va chercher ce nom dans la session. On doit donc enlever le nom de la session. C'est ce que fait la méthode *execute*.

Voyons ce que ça donne :



En [1], on veut effacer le champ de saisie. On clique sur le bouton [Effacer].

```
<s:submit key="saisie.effacer" action="Effacer"/>
```

L'action [Effacer] va s'exécuter. En [2], on remarque que l'URL appelée a été celle de l'action [Confirmer]. Cela vient de la balise `<s:form>` du formulaire :

```
<s:form action="Confirmer">
```

qui fait que le formulaire est posté à l'action [Confirmer]. Au clic sur le bouton [Effacer], le paramètre

action:Effacer=Effacer

a été posté à l'URL [/actions/Confirmer.action]. Struts utilise ce paramètre pour faire traiter les données postées par l'action [Effacer]. Celle-ci enlève le nom de la session. La page [Saisie.JSP] est la réponse de l'action [Effacer] :

```
1. <action name="Effacer" class="actions.Effacer">
2.   <result name="success">/vues/Saisie.JSP</result>
3. </action>
```

Celle-ci qui affiche le nom de la session affiche alors une chaîne vide [3].

Nous avons écrit plusieurs exemples simples afin d'introduire des concepts importants de Struts 2 :

- l'internationalisation des pages
- l'injection de paramètres postés dans les champs des actions
- la notion de session
- l'articulation entre Actions et Vues

Avec ces concepts acquis, nous sommes capables maintenant d'aborder des exemples plus complexes. Nous commençons par présenter les différentes balises utilisables dans un formulaire.

9 Exemple 07 – Les balises de formulaire

Nous allons construire et exploiter le formulaire suivant :

Firefox | Struts 2 - les tags de formulaire | localhost:8084/exemple-07/exemple/Form.action

Struts 2 - les tags de formulaire

langues

- Anglais
- Français

1-textfield:

2-password:

3-textarea:

ligne1
ligne2

4-select (multiple=false, size=1):

5-select (multiple=false, size=3):

un
deux
trois

6-select (multiple=true, size=3):

zéro
un
deux

7-radio: ☒ bleu ☐ blanc ☐ rouge

8-checkbox: ☐

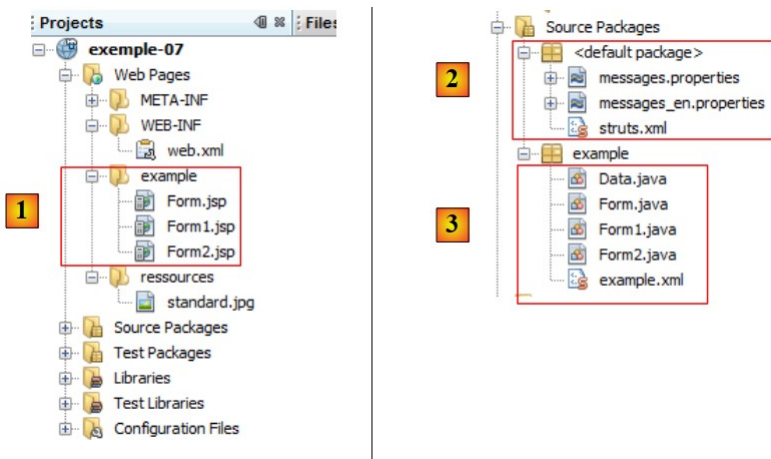
9-checkboxlist: ☐ voiture ☐ tram ☒ vélo ☒ bus ☐ métro

Confirmation des valeurs saisies

champ	valeur
1-textfield	texte
2-password	secret
3-textarea	ligne1 ligne2
4-select (multiple=false, size=1)	zéro
5-select (multiple=false, size=3)	trois
6-select (multiple=true, size=3)	zéro deux
7-radio	bleu
8-checkbox	false
9-checkboxlist	vélo bus
10-hidden	initial

Nous allons étendre le concept d'injection de paramètres vu pour un champ de saisie texte aux autres éléments Html d'un formulaire.

9.1 Le projet Netbeans



- en [1], les vues du projet [Form.x]SP
- en [2], le fichier de configuration de Struts et les messages internationalisés
- en [3], les actions [Form.x.java] et un autre fichier de configuration de Struts.

9.2 Configuration de Struts

Le fichier [struts.xml] est le suivant :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">

```

```

5.
6. <struts>
7.   <constant name="struts.custom.i18n.resources" value="messages" />
8.
9.   <include file="example/example.xml"/>
10.
11.   <package name="default" namespace="/" extends="struts-default">
12.     <default-action-ref name="index" />
13.     <action name="index">
14.       <result type="redirectAction">
15.         <param name="actionName">Form</param>
16.         <param name="namespace">/example</param>
17.       </result>
18.     </action>
19.   </package>
20. </struts>

```

- ligne 9 : inclut un autre fichier de configuration [example.xml]. C'est lui qui va configurer les actions.
- lignes 11-19 : le package *default*. Définit une adresse de redirection pour l'URL /. Elle sera redirigée vers l'URL */example/Form.action*.

Le fichier [example.xml] qui configure les actions est le suivant :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.   "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.   "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.   <package name="example" namespace="/example" extends="struts-default">
8.     <action name="Form" class="example.Form">
9.       <result name="success">/example/Form.JSP</result>
10.    </action>
11.    <action name="Form1" class="example.Form1">
12.      <result name="success">/example/Form1.JSP</result>
13.    </action>
14.    <action name="Form2" class="example.Form2">
15.      <result name="success">/example/Form2.JSP</result>
16.    </action>
17.  </package>
18. </struts>

```

- lignes 7-17 : définissent les actions */example/Form*, */example/Form1*, */example/Form2* où */example* est le namespace (ligne 7) et *Formx* les actions.
- lignes 8-10 : l'exécution de l'action *Form* produira l'affichage de la vue */example/Form.JSP*.
- lignes 11-13 : l'exécution de l'action *Form1* produira l'affichage de la vue */example/Form1.JSP*.
- lignes 14-16 : l'exécution de l'action *Form2* produira l'affichage de la vue */example/Form2.JSP*.

9.3 Les fichiers des messages

Nous ne nous attarderons pas sur l'internationalisation du projet. Nous l'avons déjà étudiée dans l'exemple 01. Nous donnons le contenu du fichier [messages.properties] afin que le lecteur puisse comprendre les vues qui vont suivre.

```

1. Form.francais=Fran ais
2. Form.anglais=Anglais
3. Form.titre=Struts 2 - les tags de formulaire
4. Form.message=Struts 2 - les tags de formulaire
5. Form.langues=langues
6. Form.textfield=1-textfield
7. Form.password=2-password
8. Form.textarea=3-textarea
9. Form.select1=4-select (multiple=false, size=1)
10. Form.select1.header=<-- select1 -->
11. Form.select2=5-select (multiple=false, size=3)
12. Form.select3=6-select (multiple=true, size=3)
13. Form.radio=7-radio
14. Form.checkbox=8-checkbox
15. Form.checkboxlist=9-checkboxlist
16. Form.hidden=10-hidden
17. Form.submitText=Valider
18. Form.buttonRazText=Raz

```

```

19. Confirmation.message=Confirmation des valeurs saisies
20. Confirmation.champ=champ
21. Confirmation.valeur=valeur
22. Confirmation.textfield=1-textfield
23. Confirmation.password=2-password
24. Confirmation.textarea=3-textarea
25. Confirmation.select1=4-select (multiple=false, size=1)
26. Confirmation.select2=5-select (multiple=false, size=3)
27. Confirmation.select3=6-select (multiple=true, size=3)
28. Confirmation.radio=7-radio
29. Confirmation.checkbox=8-checkbox
30. Confirmation.checkboxlist=9-checkboxlist
31. Confirmation.hidden=10-hidden

```

9.4 La vue [Form.JSP] – partie saisie

Son apparence visuelle est la suivante :

C'est la suivante :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
4.   <head>
5.     <title><s:text name="Form.titre"/></title>
6.     <s:head/>
7.   </head>
8.
9.   <body background="<s:url value="/ressources/standard.jpg"/>">
10.    <h2><s:text name="Form.message"/></h2>
11.    <h3><s:text name="Form.langues"/></h3>
12.    <ul>
13.      <li>
14.        <s:url id="URL" action="Form">
15.          <s:param name="request_locale">en</s:param>
16.        </s:url>
17.        <s:a href="%{URL}"><s:text name="Form.anglais"/></s:a>
18.      </li>
19.      <li>
20.        <s:url id="URL" action="Form">

```

```

21.     <s:param name="request_locale">fr</s:param>
22.     </s:url>
23.     <s:a href="%{URL}"><s:text name="Form.francais"/></s:a>
24.     </li>
25. </ul>
26. <s:form name="formulaire">
27.     <s:textfield name="textfield" key="Form.textfield" />
28.     <s:password name="password" key="Form.password"/>
29.     <s:textarea name="textarea" key="Form.textarea" cols="40" rows="5"/>
30.     <s:select name="select1" list="select1Values" size="1" key="Form.select1" headerValue="--
select 1 --" headerKey="-1" />
31.     <s:select name="select2" size="3" list="select2Values" key="Form.select2"/>
32.     <s:select name="select3" size="3" list="select3Values" key="Form.select3" multiple="true"/>
33.     <s:radio name="radio" list="radioValues" key="Form.radio"/>
34.     <s:checkbox name="checkbox" key="Form.checkbox"/>
35.     <s:checkboxlist name="checkboxlist" list="checkboxlistValues" key="Form.checkboxlist"/>
36.     <s:hidden name="hidden" key="Form.hidden"/>
37.     <s:submit key="Form.submitText" name="submitText"/>
38. </s:form>
39. <hr/>
40. ...
41. </body>
42. </html>

```

- le formulaire commence ligne 26 et se termine ligne 38. Ligne 26, l'attribut *action* est absent. Cet attribut précise à quelle action, les données du formulaire seront postées. En l'absence de l'attribut *action* elles seront postées à l'action qui a conduit à l'affichage de la vue, ici l'action [Form].
- ligne 27 : un champ de saisie. L'attribut *key* précise son libellé. L'attribut *name* est le nom du paramètre qui sera posté. La chaîne postée pour ce champ sera de la forme :

```
textfield=texte_saisi
```

Le nom du paramètre correspond à un champ de l'action [Form]. La valeur du champ de saisie *texte_saisi* sera ici injectée dans le champ *textfield* de l'action [Form]. Inversement, si la vue [Form.JSP] est affichée suite à l'action [Form], la valeur du champ de saisie sera celle du champ *textfield* de l'action [Form]. Ce champ est donc utilisé en lecture et écriture. La classe [Form] doit avoir les méthodes *getTextField* et *setTextField* pour cela.

Ce raisonnement est valable pour toutes les balises de saisie qui vont suivre. Nous ne le répéterons pas.

- ligne 28 : saisie d'un mot de passe. La chaîne postée à l'action [Form] sera de la forme

```
password=mot_de_passe_saisi
```

Un champ *password* avec ses get / set doit exister dans l'action [Form].

- ligne 29 : saisie d'un texte multi-lignes dans une zone de 5 lignes et 40 colonnes. La chaîne postée à l'action [Form] sera de la forme

```
textarea=saisie
```

Un champ *textarea* avec ses get / set doit exister dans l'action [Form].

- ligne 30 : une liste déroulante (*size=1*) à sélection unique. Une seule valeur de la liste peut être sélectionnée. La chaîne postée à l'action [Form] sera de la forme :

```
select1=valeur_sélectionnée
```

Un champ *select1* avec ses get / set doit exister dans l'action [Form].

L'attribut *key* est le libellé de la liste. L'attribut *list* définit la liste des valeurs à mettre dans le combo. Ici, la méthode *getSelect1Values* de l'action [Form] sera appelée pour remplir le combo. Cette méthode peut rendre une collection, un dictionnaire ou un tableau. Ici, elle rendra un tableau de chaînes de caractères [*chaîne1*, *chaîne2*, ...] qui générera le code Html suivant :

```

<option value="chaîne1">chaîne1</option>
<option value="chaîne2">chaîne2</option>
...

```

Le texte entre les balises *<option>...</option>* sera affiché dans le combo. L'attribut *value* désigne la valeur à poster si l'option est sélectionnée par l'utilisateur. Ici l'attribut *value* et le texte affiché dans le combo sont identiques. Le plus souvent, ce n'est pas le cas. Si la deuxième option est sélectionnée, la chaîne suivante sera postée à l'action [Form] :

```
select1=chaîne2
```

Le champ `[Form].select1` recevra alors la valeur `chaine2`. Inversement, si au moment de l'affichage du formulaire, ce champ vaut `chaine3`, alors visuellement, le 3ième élément

```
<option value="chaine3">chaine3</option>
```

apparaîtra sélectionné dans le combo.

L'attribut `headerValue` fixe le texte à afficher comme 1ère option du combo et `headerKey` la valeur à poster si cette option est choisie par l'utilisateur.

- ligne 31 : là encore une liste de taille 3 (`size=3`) : contrairement à la précédente liste où un seul élément est visible, ici 3 éléments de la liste seront visibles. Le mode de sélection reste le même. On ne peut sélectionner qu'un élément.
- ligne 32 : de nouveau une liste mais à sélection multiple (`multiple=true`). Dans ce cas, la valeur postée à l'action `[Form]` sera de la forme :

```
select3=chaine2&select3=chaine5
```

si les options `chaine2` et `chaine5` ont été sélectionnées. Lorsque plusieurs valeurs sont postées pour un même paramètre, ici `select3`, l'action doit avoir un champ ayant le nom du paramètre et être de type *tableau*. Donc par exemple, ici l'action `[Form]` pourrait avoir un champ du genre :

```
String[] select3 ;
```

et les méthodes `get / set` qui vont avec. Le tableau `select3` recevra alors le tableau de chaînes de caractères `[chaine2,chaine5]`. Inversement, à l'affichage de la vue `[Form].JSP`, les valeurs du champ `[Form].select3` déterminent les options de la liste `select3` qui apparaîtront sélectionnées. On est toujours en lecture / écriture.

- ligne 33 : un groupe de boutons radio. Le libellé du groupe est fourni par l'attribut `key`. Les libellés des boutons individuels sont fournis par l'attribut `list`. La valeur de cet attribut est le nom d'une méthode de l'action `[Form]` qui doit rendre une collection, un dictionnaire ou un tableau. Ici, la méthode `[Form].getRadioValues` rendra un tableau de chaînes de caractères `[chaine1,chaine2,...]` qui génèrera les balises Html suivantes :

```
<input type="radio" name="radio" ... value="chaine1"/>chaine1
<input type="radio" name="radio" ... value="chaine2"/>chaine2
...
```

Si le bouton radio `chaine2` est coché, la valeur postée sera

```
radio=chaine2
```

et cette valeur sera injectée dans le champ `[Form].radio`.

- ligne 34 : une case à cocher. Si elle est cochée, la chaîne de paramètres suivante sera postée à l'action `[Form]`

```
checkbox=TRUE
```

Si elle n'est pas cochée, aucune chaîne de paramètres n'est postée. Struts 2 a un mécanisme interne qui fait que tout se passe comme si la chaîne

```
checkbox=FALSE
```

avait été postée.

- ligne 35 : une liste de cases à cocher. On peut en cocher plusieurs. Le libellé du groupe est fourni par l'attribut `key`. Les libellés des cases à cocher individuelles sont fournis par l'attribut `list`. La valeur de cet attribut est le nom d'une méthode de l'action `[Form]` qui doit rendre une collection, un dictionnaire ou un tableau. Ici, la méthode `[Form].getCheckboxlistValues` rendra un tableau de chaînes de caractères `[chaine1,chaine2,...]` qui génèrera les balises Html suivantes :

```
<input type="checkbox" name="checkboxlist" ... value="chaine1"/>chaine1
<input type="checkbox" name="checkboxlist" ... value="chaine2"/>chaine2
...
```

Si l'utilisateur sélectionne les cases libellées `chaine2` et `chaine5`, la chaîne de caractère suivante sera postée à l'action `[Form]` :

```
checkboxlist=chaine2&checkboxlist=chaine5
```

Le tableau `[chaine2, chaine5]` sera affecté au champ `[Form].checkboxlist` qui doit donc être de type tableau de chaînes de caractères comme pour les listes à sélection multiple. Inversement, à l'affichage de la vue `[Form].JSP`, les valeurs du champ `[Form].checkboxlist` déterminent les cases à cocher qui seront cochées.

- ligne 36 : un champ caché. Un champ caché est un champ qui est posté sans être saisi par l'utilisateur. Sa valeur est fixée par le programmeur. Ici le champ caché prendra sa valeur du champ `[Form].hidden`. Cette même valeur sera postée sous la forme :

```
hidden=qqchose
```

et réaffecté au champ `[Form].hidden`.

- ligne 37 : un bouton de type `submit`. Un clic sur ce bouton, postera toutes les valeurs du formulaire à l'action `[Form]`. La chaîne des paramètres postés ressemblera à ceci :

```
textfield=texte&password=&textarea=ligne1%0D%0Aaligne2%0D%0A&select1=z%C3%A9ro&select2=trois&select3=z%C3%A9ro&select3=deux&__multiselect_select3=&radio=bleu&checkbox=true&__checkbox_checkbox=true&checkboxlist=v%C3%A9lo&checkboxlist=bus&__multiselect_checkboxlist=&hidden=initial&submitText=Valider
```

9.5 L'action [Form] – partie saisie

Son code est le suivant :

```
1. package example;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4.
5. public class Form extends ActionSupport {
6.
7.     // constructeur sans paramètre
8.     public Form() {
9.     }
10.    // champs du formulaire
11.    private String textfield = "texte";
12.    private String password = "secret";
13.    private String textarea = "ligne1\nligne2\n";
14.    private String select1 = "zéro";
15.    private String[] select1Values = new String[]{"zéro", "un", "deux"};
16.    private String select2 = "trois";
17.    private String[] select2Values = new String[]{"zéro", "un", "deux", "trois", "quatre"};
18.    private String[] select3 = new String[]{"zéro", "deux"};
19.    private String[] select3Values = new String[]{"zéro", "un", "deux", "trois", "quatre"};
20.    private String radio = "bleu";
21.    private String[] radioValues = new String[]{"bleu", "blanc", "rouge"};
22.    private Boolean checkbox = false;
23.    private String[] checkboxlist = new String[]{"vélo", "bus"};
24.    private String[] checkboxlistValues = new String[]{"voiture", "tram", "vélo", "bus", "métro"};
25.    private String hidden = "initial";
26.    private String submitText;
27.
28.    // valeurs sélectionnées dans champ select3
29.    public String getSelect3SelectedValues() {
30.        return getValue(select3);
31.    }
32.
33.    // valeurs sélectionnées dans champ checkboxlist
34.    public String getCheckboxlistSelectedValues() {
35.        return getValue(checkboxlist);
36.    }
37.
38.    // méthode utilitaire
39.    public String getValue(String[] values) {
40.        String result = "";
41.        for (String value : values) {
42.            result += " " + value;
43.        }
44.        return result;
45.    }
46.
47.    // getters et setters des champs
48.    ....
49. }
```

- ligne 1 : le champ associé à la balise `textfield` du formulaire. Servira à initialiser ce champ de saisie aussi bien qu'à en récupérer la valeur. Reçoit au moment du POST, la valeur saisie dans ce champ.
- ligne 2 : le champ associé à la balise `password` du formulaire. Reçoit au moment du POST, la valeur saisie dans ce champ.

- ligne 13 : associé à la balise *textarea* du formulaire. Reçoit au moment du POST, le texte saisi dans ce champ.
- ligne 14 : associé à la balise *select1* du formulaire, une liste à choix unique. Sa valeur initiale sélectionne un élément du combo. C'est l'élément ayant son attribut *value* égal à cette valeur qui sera sélectionnée. Au moment du POST, reçoit l'attribut *value* de l'option sélectionnée dans le combo.
- ligne 15 : les chaînes de caractères qui vont alimenter le combo *select1*.
- lignes 16 et 17 : idem *select1*
- ligne 18 : associé à la balise *select1* du formulaire, une liste à choix multiple. Les valeurs initiales dans le tableau sélectionnent les éléments correspondants de la liste. Ce sont les éléments ayant leur attribut *value* égal à l'un de ces éléments qui seront sélectionnés. Au moment du POST, reçoit les attributs *value* des différentes options sélectionnées dans le combo.
- ligne 19 : les chaînes de caractères qui vont alimenter la liste *select3*
- ligne 20 : associé à la balise *radio* du formulaire. Sa valeur sert à cocher l'un des boutons radio, celui qui a son attribut *value* égal à cette valeur. Reçoit au moment du POST, la valeur du bouton radio coché.
- ligne 21 : les différents libellés et valeurs des boutons radio.
- ligne 22 : associé à la balise *checkbox* du formulaire. Sa valeur initiale sert à cocher / décocher la case. Reçoit au moment du POST, la valeur *true* si la case a été cochée, *false* sinon.
- ligne 23 : associé à la balise *checkboxlist* du formulaire. Les valeurs initiales dans le tableau sélectionnent les cases à cocher correspondantes. Ce sont les cases ayant leur attribut *value* égal à l'un de ces valeurs initiales qui seront cochées. Au moment du POST, reçoit les attributs *value* des différentes cases cochées.
- ligne 24 : les chaînes de caractères qui vont alimenter les libellés des cases de *checkboxlist*.
- ligne 25 : associé au champ caché *hidden*. Sa valeur initiale va fixer la valeur du champ caché. Au moment du POST, recevra cette même valeur car l'utilisateur n'a pas la possibilité de la modifier.
- ligne 26 : associé à la balise *submit*. Au moment du POST, reçoit le libellé du bouton. On peut se passer de ce champ.
- les méthodes des lignes 29, 34 et 39 seront commentées un peu plus loin.

9.6 La vue [Form.JSP] – partie Confirmation

La vue [Form.JSP] a une partie " Confirmation des saisies " qui n'a pas encore été vue :

Confirmation des valeurs saisies

champ	valeur
1-textfield	texte
2-password	
3-textarea	ligne1 ligne2
4-select (multiple=false, size=1)	zéro
5-select (multiple=false, size=3)	trois
6-select (multiple=true, size=3)	zéro deux
7-radio	bleu
8-checkbox	true
9-checkboxlist	vélo bus
10-hidden	initial

Son code est le suivant :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. ...
4.     <hr/>
5.     <h2><s:text name="Confirmation.message"/></h2>
6.     <table border="1">
7.         <tr>
8.             <th><s:text name="Confirmation.champ"/></th>
9.             <th><s:text name="Confirmation.valeur"/></th>

```

```

10.     </tr>
11.     <tr>
12.         <td><s:text name="Form.textfield"/></td>
13.         <td><s:property value="textfield"/>
14.     </tr>
15.     <tr>
16.         <td><s:text name="Form.password"/></td>
17.         <td><s:property value="password"/>
18.     </tr>
19.     <tr>
20.         <td><s:text name="Form.textarea"/></td>
21.         <td><s:property value="textarea"/>
22.     </tr>
23.     <tr>
24.         <td><s:text name="Form.select1"/></td>
25.         <td><s:property value="select1"/>
26.     </tr>
27.     <tr>
28.         <td><s:text name="Form.select2"/></td>
29.         <td><s:property value="select2"/>
30.     </tr>
31.     <tr>
32.         <td><s:text name="Form.select3"/></td>
33.         <td><s:property value="select3SelectedValues"/>
34.     </tr>
35.     <tr>
36.         <td><s:text name="Form.radio"/></td>
37.         <td><s:property value="radio"/>
38.     </tr>
39.     <tr>
40.         <td><s:text name="Form.checkbox"/></td>
41.         <td><s:property value="checkbox"/>
42.     </tr>
43.     <tr>
44.         <td><s:text name="Form.checkboxlist"/></td>
45.         <td><s:property value="checkboxlistSelectedValues"/>
46.     </tr>
47.     <tr>
48.         <td><s:text name="Form.hidden"/></td>
49.         <td><s:property value="hidden"/>
50.     </tr>
51. </table>
52. </body>
53. </html>

```

- ligne 13 : affiche la valeur du champ `[Form].textfield`
- ligne 17 : affiche la valeur du champ `[Form].password`
- ligne 21 : affiche la valeur du champ `[Form].textarea`
- ligne 25 : affiche la valeur du champ `[Form].select1`
- ligne 29 : affiche la valeur du champ `[Form].select2`
- ligne 33 : affiche les valeurs du tableau `[Form].select3`. Utilise pour cela, la méthode `[Form].getSelect3SelectedValues`.
- ligne 37 : affiche la valeur du champ `[Form].radio`
- ligne 41 : affiche la valeur du champ `[Form].checkbox`
- ligne 45 : affiche les valeurs du tableau `[Form].checkboxlist`. Utilise pour cela, la méthode `[Form].getCheckboxlistSelectedValues`.
- ligne 49 : affiche la valeur du champ `[Form].hidden`

9.7 L'action [Form] – partie confirmation

Les méthodes `getSelect3SelectedValues` et `getCheckboxlistSelectedValues` citées précédemment, sont définies dans la classe `[Form]` :

```

1. // valeurs sélectionnées dans champ select3
2. public String getSelect3SelectedValues() {
3.     return getValue(select3);
4. }
5.
6. // valeurs sélectionnées dans champ checkboxlist
7. public String getCheckboxlistSelectedValues() {
8.     return getValue(checkboxlist);
9. }
10.
11. // méthode utilitaire
12. public String getValue(String[] values) {

```

```

13. String result = "";
14. for (String value : values) {
15.     result += " " + value;
16. }
17. return result;
18. }

```

Elles se contentent de concaténer les éléments des tableaux qu'elles doivent afficher.

9.8 Les tests

A l'exécution du projet Netbeans, on obtient la page initiale suivante :

The screenshot shows a web browser window with the address bar displaying 'localhost:8084/exemple-07/exemple/Form.action'. The page title is 'Struts 2 - les tags de formulaire'. The form contains several fields: a text field labeled '1-textfield' with the value 'texte'; a password field labeled '2-password'; a text area labeled '3-textarea' with the value 'ligne1\nligne2'; a dropdown menu labeled '4-select (multiple=false, size=1)' with the value 'zéro'; a list box labeled '5-select (multiple=false, size=3)' with the values 'un', 'deux', 'trois'; a list box labeled '6-select (multiple=true, size=3)' with the values 'zéro', 'un', 'deux'; radio buttons labeled '7-radio' with the value 'bleu'; a checkbox labeled '8-checkbox' with the value 'false'; and a checkbox list labeled '9-checkboxlist' with the values 'vélo', 'bus', 'métró'. A 'Valider' button is at the bottom right. To the right of the form is a table titled 'Confirmation des valeurs saisies' with two columns: 'champ' and 'valeur'. The table contains 10 rows of data, including the values entered in the form fields. The table is highlighted with a yellow background.

champ	valeur
1-textfield	texte
2-password	secret
3-textarea	ligne1 ligne2
4-select (multiple=false, size=1)	zéro
5-select (multiple=false, size=3)	trois
6-select (multiple=true, size=3)	zéro deux
7-radio	bleu
8-checkbox	false
9-checkboxlist	vélo bus
10-hidden	initial

Les valeurs affichées en [1] et [2], proviennent des valeurs initiales des champs de l'action [Form]. Prenons quelques exemples :

```
private String textarea = "ligne1\nligne2\n";
```

La valeur du champ `textarea` explique les affichages [1A] et [2A].

```
private String[] select3 = new String[]{"zéro", "deux"};
private String[] select3Values = new String[]{"zéro", "un", "deux", "trois", "quatre"};
```

La valeur du champ `select3Values` explique le contenu de la liste en [1B]. La valeur du champ `select3` explique les éléments sélectionnés en [1B] et affichés en [2B].

Si nous saisissons d'autres valeurs dans le formulaire et que nous validons celui-ci, les valeurs saisies vont être postées dans les champs correspondants de l'action [Form]. Puis, la vue [Form.JSP] va être réaffichée. On est donc dans le même cas que précédemment, sauf que les valeurs initiales ont été remplacées par les valeurs postées. Voici un exemple :

localhost:8084/exemple-07/example/Form.action

Struts 2 - les tags de formulaire

langues

- [Anglais](#)
- [Français](#)

1-textfield:

2-password:

3-textarea:

Struts 2
par l'exemple

4-select (multiple=false, size=1):

5-select (multiple=false, size=3):

zéro
un
deux

6-select (multiple=true, size=3):

zéro
un
deux

7-radio: ☐ bleu ☒ blanc ☐ rouge

☒ 8-checkbox

9-checkboxlist: ☒ voiture ☐ tram ☐ vélo ☒ bus ☒ métro

Confirmation des valeurs saisies

champ	valeur
1-textfield	un autre texte
2-password	secret
3-textarea	Struts 2 par l'exemple
4-select (multiple=false, size=1)	deux
5-select (multiple=false, size=3)	un
6-select (multiple=true, size=3)	un trois
7-radio	blanc
8-checkbox	true
9-checkboxlist	voiture bus métro
10-hidden	initial

9.9 L'action [Form1] et la vue [Form1.JSP]

Le fichier de configuration [example.xml] configure l'action [Form1] suivante :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.     <package name="example" namespace="/example" extends="struts-default">
8.         <action name="Form" class="example.Form">
9.             <result name="success"/>example/Form.JSP</result>
10.        </action>
11.        <action name="Form1" class="example.Form1">
12.            <result name="success"/>example/Form1.JSP</result>
13.        </action>
14.        <action name="Form2" class="example.Form2">
15.            <result name="success"/>example/Form2.JSP</result>
16.        </action>
17.    </package>
18. </struts>

```

Lignes 11-13, l'appel à l'action [Form1] génère la vue [Form1.JSP].

L'action [Form1] est la suivante :

```

1. package example;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4.
5. public class Form1 extends ActionSupport{
6.
7.     // constructeur sans paramètre
8.     public Form1() {

```

```

9.     }
10.    // champs du formulaire
11.    private Data data=new Data();
12.
13.    /**
14.     * @return the data
15.     */
16.    public Data getData() {
17.        return data;
18.    }
19.
20.    /**
21.     * @param data the data to set
22.     */
23.    public void setData(Data data) {
24.        this.data = data;
25.    }
26. }

```

L'action [Form1] est identique à l'action [Form] si ce n'est que sa partie modèle a été reléguée dans une classe annexe, la classe [Data], ligne11. La classe [Data] est la suivante :

```

1. package example;
2.
3. public class Data {
4.
5.     // constructeur sans paramètre
6.     public Data() {
7.     }
8.
9.    // champs du formulaire
10.    private String textfield = "texte";
11.    private String password = "secret";
12.    private String textarea = "ligne1\nligne2\n";
13.    private String select1 = "zéro";
14.    private String[] select1Values = new String[]{"zéro", "un", "deux"};
15.    private String select2 = "trois";
16.    private String[] select2Values = new String[]{"zéro", "un", "deux", "trois", "quatre"};
17.    private String[] select3 = new String[]{"zéro", "deux"};
18.    private String[] select3Values = new String[]{"zéro", "un", "deux", "trois", "quatre"};
19.    private String radio="bleu";
20.    private String[] radioValues = new String[]{"bleu", "blanc", "rouge"};
21.    private Boolean checkbox = false;
22.    private String[] checkboxlist = new String[]{"vélo", "bus"};
23.    private String[] checkboxlistValues = new String[]{"voiture", "tram", "vélo", "bus", "métro"};
24.    private String hidden = "initial";
25.    private String submitText;
26.
27.    // valeurs sélectionnées dans champ select3
28.    public String getSelect3SelectedValues() {
29.        return getValue(select3);
30.    }
31.
32.    // valeurs sélectionnées dans champ checkboxlist
33.    public String getCheckboxlistSelectedValues() {
34.        return getValue(checkboxlist);
35.    }
36.
37.    // méthode utilitaire
38.    public String getValue(String[] values) {
39.        String result = "";
40.        for (String value : values) {
41.            result += " " + value;
42.        }
43.        return result;
44.    }
45.
46.    // getters et setters
47. ....
48.
49. }

```

On retrouve les champs déclarés précédemment dans l'action [Form].

Assez logiquement, la vue [Form1.JSP] est proche de la vue [Form.JSP] :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
4.   <head>
5.     <title><s:text name="Form.titre"/></title>
6.     <s:head/>
7.   </head>
8.
9.   <body background="<s:url value="/ressources/standard.jpg"/>">
10.    <h2><s:text name="Form.message"/></h2>
11.    <h3><s:text name="Form.langues"/></h3>
12.    <ul>
13.      <li>
14.        <s:url id="URL" action="Form">
15.          <s:param name="request_locale">en</s:param>
16.        </s:url>
17.        <s:a href="%{URL}"><s:text name="Form.anglais"/></s:a>
18.      </li>
19.      <li>
20.        <s:url id="URL" action="Form">
21.          <s:param name="request_locale">fr</s:param>
22.        </s:url>
23.        <s:a href="%{URL}"><s:text name="Form.francais"/></s:a>
24.      </li>
25.    </ul>
26.    <s:form name="formulaire">
27.      <s:textfield name="data.textfield" key="Form.textfield" />
28.      <s:password name="data.password" key="Form.password"/>
29.      <s:textarea name="data.textarea" key="Form.textarea" cols="40" rows="5"/>
30.      <s:select name="data.select1" list="data.select1Values" size="1" key="Form.select1"
headerValue="-- select 1 --" headerKey="-1" />
31.      <s:select name="data.select2" size="3" list="data.select2Values" key="Form.select2"/>
32.      <s:select name="data.select3" size="3" list="data.select3Values" key="Form.select3"
multiple="true"/>
33.      <s:radio name="data.radio" list="data.radioValues" key="Form.radio"/>
34.      <s:checkbox name="data.checkbox" key="Form.checkbox"/>
35.      <s:checkboxlist name="data.checkboxlist" list="data.checkboxlistValues"
key="Form.checkboxlist"/>
36.      <s:hidden name="data.hidden" key="Form.hidden"/>
37.      <s:submit key="Form.submitText" name="data.submitText"/>
38.    </s:form>
39.    <hr/>
40.    <h2><s:text name="Confirmation.message"/></h2>
41.    <table border="1">
42.      <tr>
43.        <th><s:text name="Confirmation.champ"/></th>
44.        <th><s:text name="Confirmation.valeur"/></th>
45.      </tr>
46.      <tr>
47.        <td><s:text name="Form.textfield"/></td>
48.        <td><s:property value="data.textfield"/>
49.      </tr>
50.      <tr>
51.        <td><s:text name="Form.password"/></td>
52.        <td><s:property value="data.password"/>
53.      </tr>
54.      <tr>
55.        <td><s:text name="Form.textarea"/></td>
56.        <td><s:property value="data.textarea"/>
57.      </tr>
58.      <tr>
59.        <td><s:text name="Form.select1"/></td>
60.        <td><s:property value="data.select1"/>
61.      </tr>
62.      <tr>
63.        <td><s:text name="Form.select2"/></td>
64.        <td><s:property value="data.select2"/>
65.      </tr>
66.      <tr>
67.        <td><s:text name="Form.select3"/></td>
68.        <td><s:property value="data.select3SelectedValues"/>
69.      </tr>
70.      <tr>
71.        <td><s:text name="Form.radio"/></td>
72.        <td><s:property value="data.radio"/>
73.      </tr>
74.      <tr>
75.        <td><s:text name="Form.checkbox"/></td>

```

```

76.         <td><s:property value="data.checkbox"/>
77.     </tr>
78.     <tr>
79.         <td><s:text name="Form.checkboxlist"/></td>
80.         <td><s:property value="data.checkboxlistSelectedValues"/>
81.     </tr>
82.     <tr>
83.         <td><s:text name="Form.hidden"/></td>
84.         <td><s:property value="data.hidden"/>
85.     </tr>
86. </table>
87. </body>
88. </html>

```

La différence entre les vues [Form] et [Form1] peut être illustrée par la ligne 27 reproduite ci-dessous :

```
<s:textfield name="data.textfield" key="Form.textfield" />
```

Le champ de saisie est lié en lecture et écriture au champ `data.textfield` de l'action [Form1]. On rappelle qu'après exécution, les propriétés de l'action [Form1] sont accessibles à la vue. Ainsi l'attribut `name` précédent va être évalué comme `[Form1].getData().getTextField()`. Il faut donc que les deux méthodes `get` précédentes existent.

9.10 L'action [Form2] et la vue [Form2.JSP]

Le fichier de configuration [example.xml] configure l'action [Form2] suivante :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.     <package name="example" namespace="/example" extends="struts-default">
8.         <action name="Form" class="example.Form">
9.             <result name="success"/>example/Form.JSP</result>
10.        </action>
11.        <action name="Form1" class="example.Form1">
12.            <result name="success"/>example/Form1.JSP</result>
13.        </action>
14.        <action name="Form2" class="example.Form2">
15.            <result name="success"/>example/Form2.JSP</result>
16.        </action>
17.    </package>
18. </struts>

```

Lignes 14-16, l'appel à l'action [Form2] génère la vue [Form2.JSP].

L'action [Form2] est la suivante :

```

1. package example;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4. import com.opensymphony.xwork2.ModelDriven;
5.
6. public class Form2 extends ActionSupport implements ModelDriven {
7.
8.     // constructeur sans paramètre
9.     public Form2() {
10.    }
11.    // modèle de l'action
12.    public Object getModel() {
13.        return new Data();
14.    }
15. }

```

- ligne 6 : l'action [Form2] implémente l'interface [ModelDriven]. Il n'y a qu'une méthode à implémenter, la méthode `getModel` de la ligne 12. Celle-ci rend une instance de la classe [Data] étudiée précédemment.

Le fait que l'action [Form2] implémente l'interface [ModelDriven] a la conséquence suivante : la vue affichée à la suite de l'action [Form2] a un accès direct aux propriétés du modèle de l'action [Form2], celui-ci étant rendu par la méthode `getModel()`. Aussi la vue [Form2.JSP] redevient ce qu'elle était avec l'action [Form] initiale :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
4.   <head>
5.     <title><s:text name="Form.titre"/></title>
6.     <s:head/>
7.   </head>
8.
9.   <body background="<s:url value="/ressources/standard.jpg"/>">
10.    <h2><s:text name="Form.message"/></h2>
11.    <h3><s:text name="Form.langues"/></h3>
12.    <ul>
13.      <li>
14.        <s:url id="URL" action="Form">
15.          <s:param name="request_locale">en</s:param>
16.        </s:url>
17.        <s:a href="%{URL}"><s:text name="Form.anglais"/></s:a>
18.      </li>
19.      <li>
20.        <s:url id="URL" action="Form">
21.          <s:param name="request_locale">fr</s:param>
22.        </s:url>
23.        <s:a href="%{URL}"><s:text name="Form.francais"/></s:a>
24.      </li>
25.    </ul>
26.    <s:form name="formulaire">
27.      <s:textfield name="textfield" key="Form.textfield" />
28.      <s:password name="password" key="Form.password"/>
29.      <s:textarea name="textarea" key="Form.textarea" cols="40" rows="5"/>
30.      <s:select name="select1" list="select1Values" size="1" key="Form.select1" headerValue="--
select 1 -->" headerKey="-1" />
31.      <s:select name="select2" size="3" list="select2Values" key="Form.select2"/>
32.      <s:select name="select3" size="3" list="select3Values" key="Form.select3" multiple="true"/>
33.      <s:radio name="radio" list="radioValues" key="Form.radio"/>
34.      <s:checkbox name="checkbox" key="Form.checkbox"/>
35.      <s:checkboxlist name="checkboxlist" list="checkboxlistValues" key="Form.checkboxlist"/>
36.      <s:hidden name="hidden" key="Form.hidden"/>
37.      <s:submit key="Form.submitText" name="submitText"/>
38.    </s:form>
39.    <hr/>
40.    <h2><s:text name="Confirmation.message"/></h2>
41.    <table border="1">
42.      <tr>
43.        <th><s:text name="Confirmation.champ"/></th>
44.        <th><s:text name="Confirmation.valeur"/></th>
45.      </tr>
46.      <tr>
47.        <td><s:text name="Form.textfield"/></td>
48.        <td><s:property value="textfield"/>
49.      </tr>
50.      <tr>
51.        <td><s:text name="Form.password"/></td>
52.        <td><s:property value="password"/>
53.      </tr>
54.      <tr>
55.        <td><s:text name="Form.textarea"/></td>
56.        <td><s:property value="textarea"/>
57.      </tr>
58.      <tr>
59.        <td><s:text name="Form.select1"/></td>
60.        <td><s:property value="select1"/>
61.      </tr>
62.      <tr>
63.        <td><s:text name="Form.select2"/></td>
64.        <td><s:property value="select2"/>
65.      </tr>
66.      <tr>
67.        <td><s:text name="Form.select3"/></td>
68.        <td><s:property value="select3SelectedValues"/>
69.      </tr>
70.      <tr>
71.        <td><s:text name="Form.radio"/></td>
72.        <td><s:property value="radio"/>
```



```

73.     </tr>
74.     <tr>
75.         <td><s:text name="Form.checkbox"/></td>
76.         <td><s:property value="checkbox"/>
77.     </tr>
78.     <tr>
79.         <td><s:text name="Form.checkboxlist"/></td>
80.         <td><s:property value="checkboxlistSelectedValues"/>
81.     </tr>
82.     <tr>
83.         <td><s:text name="Form.hidden"/></td>
84.         <td><s:property value="hidden"/>
85.     </tr>
86. </table>
87. </body>
88. </html>

```

Ainsi la ligne 27 qui était précédemment :

```
<s:textfield name="data.textfield" key="Form.textfield" />
```

redevient :

```
<s:textfield name="textfield" key="Form.textfield" />
```

L'avantage de mettre le modèle en-dehors de la vue, est que cette architecture délimite plus clairement les fonctionnalités de chaque élément de l'architecture MVC. Le modèle **M** est clairement identifié par une classe. Par ailleurs, le modèle peut pré-exister aux actions. Ainsi, si une application web gère une base de personnes, il est probable que la classe *Personne* existe. Si une action propose un formulaire de saisie pour créer une nouvelle personne, le modèle de cette action est tout trouvé : c'est la classe *Personne*.

Nous avons vu comme il a été simple de passer de l'action [Form] à l'action [Form2] :

- le modèle **M** de la vue **V** a été encapsulé dans une classe à part
- l'action qui génère la vue utilisant ce modèle implémente l'interface **ModelDriven** avec la méthode **getModel** qui rend une instance du modèle **M**. L'action peut être alors vue comme une extension du contrôleur **C** de Struts 2, la classe **FilterDispatcher**.

Le lecteur pourra appliquer cette démarche à toute action décrite par la suite.

10 Exemple 08 - Les balises associées à des listes

L'exemple suivant s'intéresse aux balises qui affichent des listes. Nous les avons déjà rencontrées dans l'exemple précédent : la liste déroulante, les cases à cocher et les boutons radio. Le formulaire de test sera le suivant :

localhost:8084/exemple-08/example/Form.action

Struts 2 - les tags à valeurs multiples

langues

- [Anglais](#)
- [Français](#)

1-select1 (multiple=false, size=1): jaune

2-select2 (multiple=true, size=3):
 vert
 jaune
 rouge

3-select3 (multiple=true, size=3):
 vert(01)
 jaune(02)
 rouge(03)

4-select4 (multiple=true, size=3):
 bleu(3)
 jaune(2)
 vert(1)

5-select5 (multiple=true, size=3):
 bleu(3)
 jaune(2)
 vert(1)

6-checkboxlist1: ☒ vert(01) ☐ jaune(02) ☒ rouge(03) ☐ blanc(04) ☐ noir(05)

7-checkboxlist2: ☒ bleu(3) ☐ jaune(2) ☒ vert(1) ☐ blanc(5) ☐ rouge(4)

8-checkboxlist3: ☒ bleu(3) ☐ jaune(2) ☒ vert(1) ☐ blanc(5) ☐ rouge(4)

9-radio1: ☒ vert(01) ☐ jaune(02) ☐ rouge(03) ☐ blanc(04) ☐ noir(05)

10-radio2: ☐ bleu(3) ☐ jaune(2) ☒ vert(1) ☐ blanc(5) ☐ rouge(4)

11-radio3: ☐ bleu(3) ☐ jaune(2) ☒ vert(1) ☐ blanc(5) ☐ rouge(4)

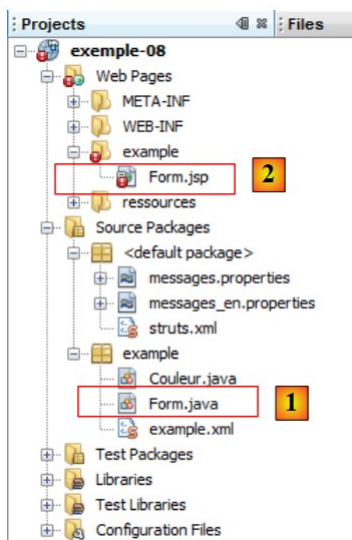
Valider

Confirmation des valeurs saisies

champ	valeur
1-select1 (multiple=false, size=1)	jaune
2-select2 (multiple=true, size=3)	vert rouge
3-select3 (multiple=true, size=3)	01 03
4-select4 (multiple=true, size=3)	1 3
5-select5 (multiple=true, size=3)	1 3
6-checkboxlist1	01 03
7-checkboxlist2	1 3
8-checkboxlist3	1 3
9-radio1	01
10-radio2	1
11-radio3	1

10.1 Le projet Netbeans

Le projet Netbeans est le suivant :



- en [1] l'action et en [2] la vue associée.

Le fichier [exemple/example.xml] configure l'application :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">

```

```

5.
6. <struts>
7.   <package name="example" namespace="/example" extends="struts-default">
8.     <action name="Form" class="example.Form">
9.       <result name="success"/>/example/Form.JSP</result>
10.    </action>
11.  </package>
12. </struts>

```

Les lignes 8-9 montrent qu'un appel à l'URL [/example/Form.action] génère la vue [/example/Form.JSP]. L'action [Form] se chargera donc de construire le modèle de la vue [Form.JSP].

10.2 Le fichier des messages internationalisés

Le fichier [messages.properties] des messages internationalisés est le suivant :

```

1. Form.francais=Fran\u00E7ais
2. Form.anglais=Anglais
3. Form.titre=Struts 2 - les tags \u00E0 valeurs multiples
4. Form.message=Struts 2 - les tags \u00E0 valeurs multiples
5. Form.langues=langues
6. Form.select1=1-select1 (multiple=false, size=1)
7. Form.select2=2-select2 (multiple=true, size=3)
8. Form.select3=3-select3 (multiple=true, size=3)
9. Form.select4=4-select4 (multiple=true, size=3)
10. Form.select5=5-select5 (multiple=true, size=3)
11. Form.checkboxlist1=6-checkboxlist1
12. Form.checkboxlist2=7-checkboxlist2
13. Form.checkboxlist3=8-checkboxlist3
14. Form.radio1=9-radio1
15. Form.radio2=10-radio2
16. Form.radio3=11-radio3
17. Form.submitText=Valider
18. Confirmation.message=Confirmation des valeurs saisies
19. Confirmation.champ=champ
20. Confirmation.valeur=valeur
21. Confirmation.select1=1-select1 (multiple=false, size=1)
22. Confirmation.select2=2-select2 (multiple=false, size=3)
23. Confirmation.select3=3-select3 (multiple=true, size=3)
24. Confirmation.select4=4-select4 (multiple=true, size=3)
25. Confirmation.select5=5-select5 (multiple=true, size=3)
26. Confirmation.checkboxlist1=6-checkboxlist1
27. Confirmation.checkboxlist2=7-checkboxlist2
28. Confirmation.checkboxlist3=8-checkboxlist3
29. Confirmation.radio1=9-radio1
30. Confirmation.radio2=10-radio2
31. Confirmation.radio3=11-radio3

```

La vue [Form.JSP] utilise ces messages.

10.3 La vue [Form.JSP]

La vue [Form.JSP] est la suivante :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
4.   <head>
5.     <title><s:text name="Form.titre"/></title>
6.     <s:head/>
7.   </head>
8.
9.   <body background="<s:url value="/ressources/standard.jpg"/>">
10.    <h2><s:text name="Form.message"/></h2>
11.    <h3><s:text name="Form.langues"/></h3>
12.    <ul>
13.      <li>
14.        <s:url id="URL" action="Form">
15.          <s:param name="request_locale">en</s:param>
16.        </s:url>
17.        <s:a href="%{URL}"><s:text name="Form.anglais"/></s:a>
18.      </li>

```

```

19.     <li>
20.         <s:url id="URL" action="Form">
21.             <s:param name="request_locale">fr</s:param>
22.         </s:url>
23.         <s:a href="%{URL}"><s:text name="Form.francais"/></s:a>
24.     </li>
25. </ul>
26. <s:form name="formulaire">
27.     <s:select name="select1" list="{ 'vert', 'jaune', 'rouge' }" size="1" key="Form.select1"/>
28.     <s:select name="select2" list="{ 'vert', 'jaune', 'rouge' }" size="3" key="Form.select2" multiple="true"/>
29.     <s:select name="select3"
list="{ '01': 'vert (01)', '02': 'jaune (02)', '03': 'rouge (03)', '04': 'blanc (04)', '05': 'noir (05)' }" size="3"
key="Form.select3" multiple="true"/>
30.     <s:select name="select4" list="dico" size="3" key="Form.select4" multiple="true"/>
31.     <s:select name="select5" list="couleurs" listKey="id" listValue="nom" size="3" key="Form.select5"
multiple="true" />
32.     <s:checkboxlist name="checkboxlist1"
list="{ '01': 'vert (01)', '02': 'jaune (02)', '03': 'rouge (03)', '04': 'blanc (04)', '05': 'noir (05)' }"
key="Form.checkboxlist1"/>
33.     <s:checkboxlist name="checkboxlist2" list="dico" key="Form.checkboxlist2"/>
34.     <s:checkboxlist name="checkboxlist3" list="couleurs" listKey="id" listValue="nom" key="Form.checkboxlist3" />
35.     <s:radio name="radio1"
list="{ '01': 'vert (01)', '02': 'jaune (02)', '03': 'rouge (03)', '04': 'blanc (04)', '05': 'noir (05)' }" key="Form.radio1"/>
36.     <s:radio name="radio2" list="dico" key="Form.radio2"/>
37.     <s:radio name="radio3" list="couleurs" listKey="id" listValue="nom" key="Form.radio3" />
38.     <s:submit key="Form.submitText" name="submitText"/>
39. </s:form>
40. <hr/>
41. <h2><s:text name="Confirmation.message"/></h2>
42. <table border="1">
43.     <tr>
44.         <th><s:text name="Confirmation.champ"/></th>
45.         <th><s:text name="Confirmation.valeur"/></th>
46.     </tr>
47.     <tr>
48.         <td><s:text name="Form.select1"/></td>
49.         <td><s:property value="select1"/>
50.     </tr>
51.     <tr>
52.         <td><s:text name="Form.select2"/></td>
53.         <td><s:property value="select2SelectedValues"/>
54.     </tr>
55.     <tr>
56.         <td><s:text name="Form.select3"/></td>
57.         <td><s:property value="select3SelectedValues"/>
58.     </tr>
59.     <tr>
60.         <td><s:text name="Form.select4"/></td>
61.         <td><s:property value="select4SelectedValues"/>
62.     </tr>
63.     <tr>
64.         <td><s:text name="Form.select5"/></td>
65.         <td><s:property value="select5SelectedValues"/>
66.     </tr>
67.     <tr>
68.         <td><s:text name="Form.checkboxlist1"/></td>
69.         <td><s:property value="checkboxlist1SelectedValues"/>
70.     </tr>
71.     <tr>
72.         <td><s:text name="Form.checkboxlist2"/></td>
73.         <td><s:property value="checkboxlist2SelectedValues"/>
74.     </tr>
75.     <tr>
76.         <td><s:text name="Form.checkboxlist3"/></td>
77.         <td><s:property value="checkboxlist3SelectedValues"/>
78.     </tr>
79.     <tr>
80.         <td><s:text name="Form.radio1"/></td>
81.         <td><s:property value="radio1"/>
82.     </tr>
83.     <tr>
84.         <td><s:text name="Form.radio2"/></td>
85.         <td><s:property value="radio2"/>
86.     </tr>
87.     <tr>
88.         <td><s:text name="Form.radio3"/></td>
89.         <td><s:property value="radio3"/>
90.     </tr>
91. </table>
92. </body>
93. </html>

```

Examinons les différentes balises de saisie du formulaire :

- ligne 27 : la balise <s:select> :

```
<s:select name="select1" list="{ 'vert', 'jaune', 'rouge' }" size="1" key="Form.select1"/>
```

Cette balise générera le code Html suivant :

```
1. <select name="select1" size="1" id="Form_select1">
2.     <option value="vert">vert</option>
3.     <option value="jaune" selected="selected">jaune</option>
4.     <option value="rouge">rouge</option>
5. </select>
```

La valeur de l'attribut *list* est un tableau de chaînes de caractères utilisé pour générer les valeurs et les libellés des options. Le modèle associé est le champ *select1* de l'action [Form] suivant :

```
private String select1 = "jaune";
```

Ce champ est associé en lecture et écriture à la balise nommée *select1*. En **écriture**, la valeur de l'option sélectionnée sera injectée dans le champ *select1*. En **lecture**, la valeur du champ *select1* sert à déterminer l'option sélectionnée. Ici, la valeur *jaune* du champ *select1* sert à sélectionner l'option de la balise *select1* qui a la valeur jaune. C'est pourquoi, ligne 3, l'option est sélectionnée.

- ligne 28

```
<s:select name="select2" list=("'vert','jaune','rouge')" size="3" key="Form.select2" multiple="true"/>
```

Il n'y a pas de différence fondamentale avec la balise de la ligne 27, si ce n'est que la sélection est ici multiple. Donc la valeur postée est un tableau de valeurs, celles des options sélectionnées. Le champ *select2* de l'action associée au formulaire est le suivant :

```
private String[] select2 = new String[]{"vert", "rouge"};
```

C'est bien un tableau. Sa valeur initiale sert à sélectionner les options dont les valeurs sont présentes dans le tableau.

- ligne 29

```
<s:select name="select3"
list="#{'01':'vert (01)', '02':'jaune (02)', '03':'rouge (03)', '04':'blanc (04)', '05':'noir (05)'}" size="3"
key="Form.select3" multiple="true"/>
```

Ici, la valeur de l'attribut *list* est un dictionnaire dont les clés servent de valeurs aux options et les valeurs associées servent de libellés aux options. Le code Html généré est le suivant :

```
1. <select name="select3" size="3" id="Form_select3" multiple="multiple">
2.     <option value="01" selected="selected">vert (01)</option>
3.     <option value="02">jaune (02)</option>
4.     <option value="03" selected="selected">rouge (03)</option>
5.     <option value="04">blanc (04)</option>
6.     <option value="05">noir (05)</option>
7. </select>
```

Le champ associé dans l'action est le suivant :

```
private String[] select3 = new String[]{"01", "03"};
```

On a de nouveau un tableau de valeurs car la balise associée est à choix multiple. Les valeurs initiales du tableau expliquent pourquoi les options des lignes 2 et 4 sont initialement sélectionnées.

- ligne 30

```
<s:select name="select4" list="dico" size="3" key="Form.select4" multiple="true"/>
```

Ici, les options de la liste vont être fournies par le modèle de la vue, modèle qui fait ici partie de l'action [Form] :

```
private HashMap<String, String> dico = new HashMap<String, String>();
```

Les clés du dictionnaire *dico* seront les valeurs des options. Les valeurs du dictionnaire *dico* seront les libellés des options. Le champ *select4* de l'action est le suivant :

```
private String[] select4 = new String[]{"1", "3"};
```

C'est un tableau de valeurs qui recevra les valeurs postées par la balise *select4*.

- ligne 31

```
<s:select name="select5" list="couleurs" listKey="id" listValue="nom" size="3" key="Form.select5" multiple="true" />
```

La liste des options est générée par le champ *couleurs* défini par l'attribut *list*. Le champ *couleurs* de l'action [Form] est définie comme suit :

```
private Couleur[] couleurs;
```

La classe [Couleur] est définie comme suit :

```
1. package example;
2.
3. public class Couleur {
4.
5.     // champs
6.     private String id;
7.     private String nom;
8.
9.     // constructeur
10.    public Couleur() {
11.    }
12.
13.    // getters et setters
14.    ...
15. }
```

Elle a deux champs nommés *id* et *nom*. Dans la balise *select5* on a *listKey="id"* *listValue="nom"*. Cela signifie que le champ *id* servira de valeur à une option et le champ *nom* servira de libellé. On rappelle que ce sont les valeurs des options qui sont postées. Elles seront postées au champ *select5* suivant :

```
private String[] select5 = new String[]{"1", "3"};
```

- ligne 32

```
<s:checkboxlist name="checkboxlist1"
list="#{'01':'vert (01)', '02':'jaune (02)', '03':'rouge (03)', '04':'blanc (04)', '05':'noir (05)'}"
key="Form.checkboxlist1"/>
```

Cette balise va générer le code Html suivant :

```
1. <input type="checkbox" name="checkboxlist1" value="01" id="checkboxlist1-1" checked="checked"/>
2. <label for="checkboxlist1-1" class="checkboxLabel">vert (01)</label>
3. <input type="checkbox" name="checkboxlist1" value="02" id="checkboxlist1-2"/>
4. <label for="checkboxlist1-2" class="checkboxLabel">jaune (02)</label>
5. <input type="checkbox" name="checkboxlist1" value="03" id="checkboxlist1-3" checked="checked"/>
6. <label for="checkboxlist1-3" class="checkboxLabel">rouge (03)</label>
7. <input type="checkbox" name="checkboxlist1" value="04" id="checkboxlist1-4"/>
8. <label for="checkboxlist1-4" class="checkboxLabel">blanc (04)</label>
9. <input type="checkbox" name="checkboxlist1" value="05" id="checkboxlist1-5"/>
10. <label for="checkboxlist1-5" class="checkboxLabel">noir (05)</label>
```

Toutes les cases à cocher portent le nom *checkboxlist1*. Comme chacune d'elles peut être cochée, plusieurs paramètres *checkboxlist1* peuvent être postés. Il faudra donc que le champ *checkboxlist1* du modèle soit de type tableau. L'attribut *list* de la ligne 22 est un dictionnaire dont les clés génèrent les attributs *value* des cases à cocher et les valeurs les libellés de ces mêmes cases. Le champ *checkboxlist1* de l'action [Form] est le suivant :

```
private String[] checkboxlist1 = new String[]{"01", "03"};
```

- ligne 33

```
<s:checkboxlist name="checkboxlist2" list="dico" key="Form.checkboxlist2"/>
```

Cette balise fonctionne comme la balise *select4* étudiée précédemment. Le champ associé dans l'action est le suivant :

```
private String[] checkboxlist2 = new String[]{"1", "3"};
```

- ligne 34

```
<s:checkboxlist name="checkboxlist3" list="couleurs" listKey="id" listValue="nom"
key="Form.checkboxlist3" />
```

Cette balise fonctionne comme la balise *select5* étudiée précédemment. Le champ *checkboxlist3* associé dans l'action est le suivant :

```
private String[] checkboxlist3 = new String[]{"1", "3"};
```

- ligne 35

```
<s:radio name="radio1"
list="#{'01':'vert (01)', '02':'jaune (02)', '03':'rouge (03)', '04':'blanc (04)', '05':'noir (05)'}"
key="Form.radio1"/>
```

Cette balise fonctionne dans son principe comme la balise *select3* vue précédemment. Le code Html généré est le suivant :

```
1. <input type="radio" name="radio1" id="Form_radio101" checked="checked" value="01"/><label
for="Form_radio101">vert (01)</label>
2. <input type="radio" name="radio1" id="Form_radio102" value="02"/><label
for="Form_radio102">jaune (02)</label>
3. <input type="radio" name="radio1" id="Form_radio103" value="03"/><label
for="Form_radio103">rouge (03)</label>
4. <input type="radio" name="radio1" id="Form_radio104" value="04"/><label
for="Form_radio104">blanc (04)</label>
5. <input type="radio" name="radio1" id="Form_radio105" value="05"/><label
for="Form_radio105">noir (05)</label>
```

Toutes les balises ont le même nom *radio1*. Un seul bouton radio peut être coché. Il n'y a donc qu'une valeur postée, celle du bouton radio coché. Les clés du dictionnaire *list* servent à générer les attributs *value* des boutons radio alors que les valeurs du dictionnaire servent à générer les libellés associés à ces boutons. Le champ *radio1* associé dans l'action est le suivant :

```
private String radio1="01";
```

- ligne 36

```
<s:radio name="radio2" list="dico" key="Form.radio2"/>
```

Cette balise fonctionne dans son principe comme la balise *select4*. Le champ *radio2* associé dans l'action est le suivant :

```
private String radio2="1";
```

- ligne 37

```
<s:radio name="radio3" list="couleurs" listKey="id" listValue="nom" key="Form.radio3" />
```

Cette balise fonctionne dans son principe comme la balise *select5*. Le champ *radio3* associé dans l'action est le suivant :

```
private String radio3="1";
```

Voilà pour la partie *saisie* du formulaire. Il y a également une partie *confirmation* qui est analogue à celle du formulaire de l'exemple précédent : elle confirme les valeurs postées. Nous ne commenterons que les lignes suivantes :

```
1. <tr>
2. <td><s:text name="Form.select2"/></td>
3. <td><s:property value="select2SelectedValues"/>
4. </tr>
```

qui correspond visuellement à ce qui suit :

```
2-select2 (multiple=true, size=3) vert rouge
```

Le texte dans la cellule de gauche vient du fichier [messages.properties] :

```
Form.select2=2-select2 (multiple=true, size=3)
```

Le texte dans la cellule de droite a été généré par la méthode *getSelect2SelectedValues* de l'action [Form] :

```
1. // valeurs sélectionnées dans listes déroulantes
2. public String getSelect2SelectedValues() {
3.     return getArrayValue(select2);
4. }
5.
6. // méthodes utilitaires
7. public String getArrayValue(String[] values) {
8.     String result = "";
9.     for (String value : values) {
10.         result += " " + value;
11.     }
12.     return result;
13. }
```

La méthode *getSelect2SelectedValues* rend les valeurs du tableau *select2* sous la forme d'une chaîne de caractères.

10.4 L'action [Form]

En commentant la vue [Form.JSP] nous avons présenté les champs de l'action [Form] associée. Le code de l'action [Form] est le suivant :

```
1. package example;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4. import java.util.HashMap;
5.
6. public class Form extends ActionSupport {
7.
8.     // champs du formulaire
9.     private String select1 = "jaune";
10.    private String[] select2 = new String[]{"vert", "rouge"};
11.    private String[] select3 = new String[]{"01", "03"};
12.    private String[] select4 = new String[]{"1", "3"};
13.    private String[] select5 = new String[]{"1", "3"};
14.    private HashMap<String, String> dico = new HashMap<String, String>();
15.    private Couleur[] couleurs;
16.    private String[] checkboxlist1 = new String[]{"01", "03"};
17.    private String[] checkboxlist2 = new String[]{"1", "3"};
18.    private String[] checkboxlist3 = new String[]{"1", "3"};
19.    private String radio1="01";
20.    private String radio2="1";
21.    private String radio3="1";
22.
23.    private String submitText;
24.
25.    // constructeur sans paramètre
26.    public Form() {
27.        // init dictionnaire de couleurs
28.        dico.put("1", "vert(1)");
29.        dico.put("2", "jaune(2)");
30.        dico.put("3", "bleu(3)");
31.        dico.put("4", "rouge(4)");
32.        dico.put("5", "blanc(5)");
33.
34.        // init tableau d'objets Couleur
35.        couleurs = new Couleur[dico.size()];
36.        int i = 0;
37.        for (String key : dico.keySet()) {
38.            couleurs[i] = new Couleur();
39.            couleurs[i].setId(key);
40.            couleurs[i].setNom(dico.get(key));
41.            i++;
42.        }
43.    }
44.
45.    // valeurs sélectionnées dans listes déroulantes
46.    public String getSelect2SelectedValues() {
```



```

47.     return getArrayValue(select2);
48. }
49.
50. public String getSelect3SelectedValues() {
51.     return getArrayValue(select3);
52. }
53.
54. public String getSelect4SelectedValues() {
55.     return getArrayValue(select4);
56. }
57.
58. public String getSelect5SelectedValues() {
59.     return getArrayValue(select5);
60. }
61.
62. public String getCheckboxlist1SelectedValues() {
63.     return getArrayValue(checkboxlist1);
64. }
65.
66. public String getCheckboxlist2SelectedValues() {
67.     return getArrayValue(checkboxlist2);
68. }
69. public String getCheckboxlist3SelectedValues() {
70.     return getArrayValue(checkboxlist3);
71. }
72.
73. // méthodes utilitaires
74. public String getArrayValue(String[] values) {
75.     String result = "";
76.     for (String value : values) {
77.         result += " " + value;
78.     }
79.     return result;
80. }
81.
82. // getters et setters
83. ...
84. }

```

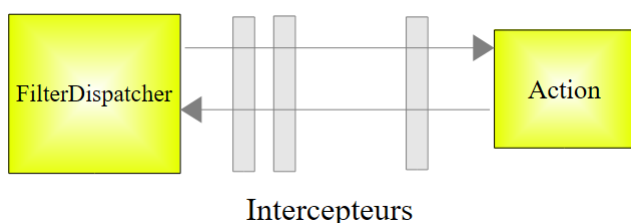
On ne trouve pas de méthode *execute* dans cette action. C'est donc la méthode *execute* de la classe parent *ActionSupport* qui est exécutée. Celle-ci ne fait rien et se contente de rendre la clé de navigation *SUCCESS*.

10.5 Les tests

Le lecteur est invité à tester l'application [exemple-08].

11 Exemple 09 - Conversion et validation des nombres entiers

Nous abordons maintenant une série d'exemples sur la conversion et la validation des paramètres d'un formulaire. Le problème est le suivant. Pour traiter une URL de la forme [http://machine:port/.../Action], le contrôleur [FilterDispatcher] instancie la classe qui implémente l'action demandée et exécute l'une de ses méthodes, par défaut la méthode appelée *execute*. L'appel à cette méthode *execute* traverse une série d'intercepteurs :



La liste des intercepteurs est définie dans le fichier [struts-default.xml] à la racine de l'archive [struts2-core.jar]. La liste des intercepteurs qui y est définie est la suivante :

```

1.         <interceptor-stack name="defaultStack">
2.             <interceptor-ref name="exception"/>
3.             <interceptor-ref name="alias"/>
4.             <interceptor-ref name="servletConfig"/>

```

```

5.      <interceptor-ref name="i18n"/>
6.      <interceptor-ref name="prepare"/>
7.      <interceptor-ref name="chain"/>
8.      <interceptor-ref name="debugging"/>
9.      <interceptor-ref name="scopedModelDriven"/>
10.     <interceptor-ref name="modelDriven"/>
11.     <interceptor-ref name="fileUpload"/>
12.     <interceptor-ref name="checkbox"/>
13.     <interceptor-ref name="multiselect"/>
14.     <interceptor-ref name="staticParams"/>
15.     <interceptor-ref name="actionMappingParams"/>
16.     <interceptor-ref name="params">
17.       <param name="excludeParams">dojo\..*,^struts\..*</param>
18.     </interceptor-ref>
19.     <interceptor-ref name="conversionError"/>
20.     <interceptor-ref name="validation">
21.       <param name="excludeMethods">input,back,cancel,browse</param>
22.     </interceptor-ref>
23.     <interceptor-ref name="workflow">
24.       <param name="excludeMethods">input,back,cancel,browse</param>
25.     </interceptor-ref>
26. </interceptor-stack>

```

Parmi les intercepteurs, il y en a un qui se charge d'injecter dans l'action les valeurs *valeuri* des paramètres *parami* qui accompagnent la requête sous la forme *parami=valeuri*. On sait que *valeuri* sera injectée dans le champ *parami* de l'action via la méthode *setParami* si elle existe. Sinon, aucune injection n'intervient et aucune erreur n'est signalée.

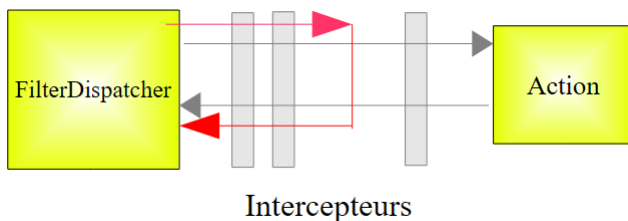
La chaîne *parami=valeuri* est une chaîne de caractères. Jusqu'à maintenant, l'injection de *valeuri* a été faite dans des champs *parami* de type *String* :

```
private String parami ;
```

L'injection de la chaîne *valeuri* comme valeur de la chaîne *parami* ne posait pas de problème. Si *parami* n'est pas de type *String*, alors *valeuri* doit subir une conversion vers le type *Ti* de *parami*. C'est le problème de la **conversion**. Par exemple, on voudra qu'un âge soit entier et on écrira dans l'action :

```
private int age ;
```

Par ailleurs, on peut vouloir délimiter l'âge entre 1 et 150. On a là un problème de **validation**. Le paramètre *parami* peut être converti dans le bon type sans pour autant être valide. Il y a donc deux étapes à passer. Si on revient au schéma du traitement d'une requête :



Deux intercepteurs vont s'occuper respectivement de la **conversion** et de la **validation** des paramètres. Si l'une des étapes échoue, la requête ne poursuit pas son cheminement vers l'action (circuit rouge ci-dessus). Le formulaire à partir duquel ont été postés les paramètres erronés est réaffiché avec des messages d'erreurs.

Les intercepteurs concernés par la conversion et la validation des paramètres sont les intercepteurs *conversionError* et *validation* des lignes 19 et 20 de la liste des intercepteurs présentée précédemment. On notera lignes 20-22 que l'intercepteur *validation* n'est pas appliqué si la méthode appelée est l'une des méthodes *input*, *back*, *cancel*, *browse*. Nous utiliserons cette propriété par la suite.

Nous commençons par étudier la conversion et la validation des nombres entiers. Nous allons passer du temps sur ce premier exemple car la validation fait intervenir de nombreux éléments. Une fois ceux-ci connus, nous irons plus vite sur les exemples qui suivront.

11.1 Le formulaire

localhost:8084/exemple-09/exemple/FormInt.action

Struts 2 - Conversion et validation de nombres entiers

1-Nombre entier positif de deux chiffres:

2-Nombre entier:

3-Nombre entier ≥ -1 :

4-Nombre entier ≤ 10 :

5-Nombre entier dans l'intervalle [1,10]:

6-Nombre entier dans l'intervalle [2,20]:

[Annuler](#)
[Raz modèle](#)

localhost:8084/exemple-09/exemple/FormInt.action

Struts 2 - Conversion et validation de nombres entiers

Tapez un nombre entier positif de deux chiffres

1-Nombre entier positif de deux chiffres:

Tapez un nombre entier

2-Nombre entier:

Tapez un nombre entier ≥ -1

3-Nombre entier ≥ -1 :

Tapez un nombre entier ≤ 10

4-Nombre entier ≤ 10 :

Tapez un nombre entier dans l'intervalle [1,10]

5-Nombre entier dans l'intervalle [1,10]:

Tapez un nombre entier dans l'intervalle [2,20]

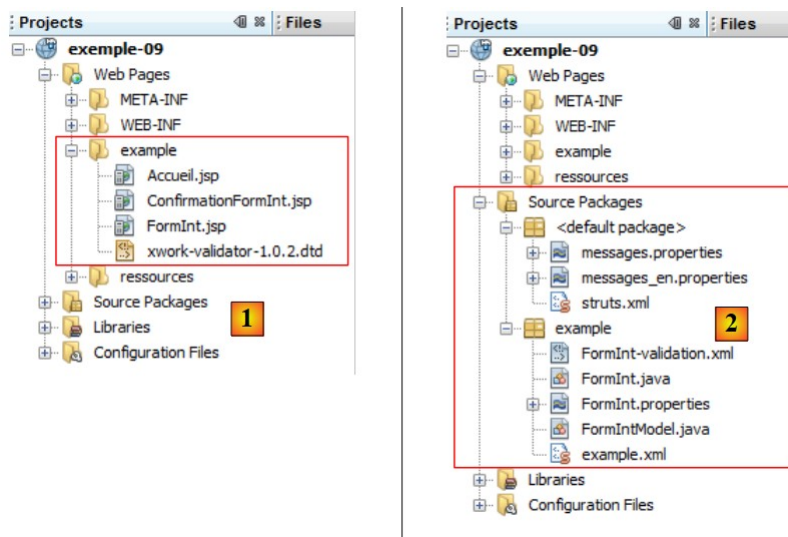
6-Nombre entier dans l'intervalle [2,20]:

[Annuler](#)
[Raz modèle](#)

- en [1], le formulaire de saisie
- en [2], le résultat lorsqu'on valide sans entrer de valeurs

11.2 Le projet Netbeans

Le projet Netbeans est le suivant :



- en [1], les trois vues de l'application
- en [2], les codes source, les fichiers des messages internationalisés, les fichiers de configuration de Struts.

11.3 Configuration de Struts

L'application est configurée par les fichiers [struts.xml] et [example.xml].

Le fichier [struts.xml] est le suivant :

```
1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.     <constant name="struts.custom.i18n.resources" value="messages" />
8.
9.     <include file="example/example.xml"/>
10.
11.     <package name="default" namespace="/" extends="struts-default">
12.         <default-action-ref name="index" />
13.         <action name="index">
14.             <result type="redirectAction">
15.                 <param name="actionName">Accueil</param>
16.                 <param name="namespace">/example</param>
17.             </result>
18.         </action>
19.     </package>
20. </struts>
```

Les lignes 12-18 définissent l'action [/example/Accueil] comme action par défaut lorsque l'utilisateur n'en précise pas.

Le fichier [example.xml] est le suivant :

```
1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.     <package name="example" namespace="/example" extends="struts-default">
8.         <action name="Accueil">
9.             <result name="success">/example/Accueil.JSP</result>
10.        </action>
11.        <action name="FormInt" class="example.FormInt">
12.            <result name="input">/example/FormInt.JSP</result>
13.            <result name="cancel" type="redirect">/example/Accueil.JSP</result>
14.            <result name="success">/example/ConfirmationFormInt.JSP</result>
15.        </action>
16.    </package>
17. </struts>
```

- lignes 8-10 : l'action [Accueil] fait afficher la vue [Accueil.JSP]
- ligne 11 : l'action [FormInt] fait exécuter par défaut la méthode *execute* de la classe [example.FormInt]. Nous verrons que deux autres méthodes seront exécutées, les méthodes *input* et *cancel*. Ces méthodes seront alors précisées dans les paramètres de la requête.
- ligne 12 : la clé *input* fera afficher la vue [FormInt.JSP] (ligne 5). Cette vue est celle du formulaire.
- ligne 13 : la clé *cancel* sera retournée par une méthode *cancel* associé au lien [Annuler]. La vue rendue sera alors la vue [Accueil.JSP] après une redirection (*type=redirect*).
- ligne 14 : la clé *success* est rendue par la méthode *execute* de l'action [FormInt]. Si la requête arrive jusqu'à la méthode *execute*, c'est qu'elle a passé avec succès tous les intercepteurs, notamment ceux qui vérifient la validité des paramètres. La méthode *execute* se contente alors de rendre la clé *success* qui va faire afficher la vue de confirmation [ConfirmationInt.JSP].

11.4 Les fichiers de messages

Le fichier [messages.properties] est le suivant :

```
1. Accueil.titre=Accueil
2. Accueil.message=Struts 2 - Conversions et validations
3. Accueil.FormInt=Saisie de nombres entiers
4. Form.titre=Conversions et validations
5. FormInt.message=Struts 2 - Conversion et validation de nombres entiers
6. Form.submitText=Valider
7. Form.cancelText=Annuler
8. Form.clearModel=Raz mod\u00e9
9. Confirmation.titre=Confirmation
10. Confirmation.message=Confirmation des valeurs saisies
```

```

11. Confirmation.champ=champ
12. Confirmation.valeur=valeur
13. Confirmation.lien=Formulaire de test

```

En plus de ce fichier, les vues utilisent le fichier [FormInt.properties] suivant :

```

1. int1.prompt=1-Nombre entier positif de deux chiffres
2. int1.error=Tapez un nombre entier positif de deux chiffres
3. int2.prompt=2-Nombre entier
4. int2.error=Tapez un nombre entier
5. int3.prompt=3-Nombre entier >=-1
6. int3.error=Tapez un nombre entier >=-1
7. int4.prompt=4-Nombre entier <=10
8. int4.error=Tapez un nombre entier <=10
9. int5.prompt=5-Nombre entier dans l'intervalle [1,10]
10. int5.error=Tapez un nombre entier dans l'intervalle [1,10]
11. int6.prompt=6-Nombre entier dans l'intervalle [2,20]
12. int6.error=Tapez un nombre entier dans l'intervalle [2,20]

```

Le fichier [FormInt.properties] n'est exploité que lorsque l'action qui a généré la vue, est l'action [FormInt]. C'est une façon de segmenter le fichier des messages si celui-ci est trop important. On internationalise les messages de l'action **Action** dans le fichier [Action.properties].

11.5 Les vues et les actions

Nous présentons maintenant les vues et les actions de l'application. D'après la configuration de l'application :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.   <package name="example" namespace="/example" extends="struts-default">
8.     <action name="Accueil">
9.       <result name="success">/example/Accueil.JSP</result>
10.    </action>
11.    <action name="FormInt" class="example.FormInt">
12.      <result name="input">/example/FormInt.JSP</result>
13.      <result name="cancel" type="redirect">/example/Accueil.JSP</result>
14.      <result name="success">/example/ConfirmationFormInt.JSP</result>
15.    </action>
16.  </package>
17. </struts>

```

on voit qu'il y a trois vues [Accueil.JSP, FormInt.JSP, ConfirmationFormInt.JSP] et deux actions [Accueil, FormInt].

11.5.1 Accueil.JSP

La vue [Accueil.JSP] est la suivante :



Son code est le suivant :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
4.   <head>
5.     <title><s:text name="Accueil.titre"/></title>
6.     <s:head/>
7.   </head>
8.
9.   <body background="<s:url value="/ressources/standard.jpg"/>">
10.    <h2><s:text name="Accueil.message"/></h2>
11.    <ul>
12.      <li>
13.        <s:url id="URL" action="FormInt!input"/>
14.        <s:a href="%{URL}"><s:text name="Accueil.FormInt"/></s:a>
15.      </li>
16.    </ul>
17.  </body>
18. </html>

```

Le lien de la ligne 14 génère le code Html suivant :

```
<a href="/exemple-09/example/FormInt!input.action">Saisie de nombres entiers</a>
```

C'est donc un lien vers l'action [FormInt] configurée comme suit dans [example.xml] :

```

1.   <action name="FormInt" class="example.FormInt">
2.     <result name="input"/>example/FormInt.JSP</result>
3.     <result name="cancel" type="redirect"/>example/Accueil.JSP</result>
4.     <result name="success"/>example/ConfirmationFormInt.JSP</result>
5. </action>

```

Un clic sur le lien va donc instancier la classe [example.FormInt] et exécuter la méthode *input* de celle-ci. Comme elle n'existe pas, c'est la méthode *input* de la classe parent *ActionSupport* qui sera exécutée. Celle-ci ne fait rien sinon rendre la clé *input*. C'est donc la vue [/example/FormInt.JSP] qui sera affichée.

Par ailleurs, la méthode *input* est l'une des méthodes ignorées par l'intercepteur de validation :

```

1.   <interceptor-ref name="validation">
2.     <param name="excludeMethods">input,back,cancel,browse</param>
3. </interceptor-ref>

```

Aussi n'y aura-t-il pas de validation de paramètres. C'est important car ici il n'y a pas de paramètres et nous verrons ultérieurement que les règles de validation vont imposer l'existence de six paramètres.

11.5.2 L'action [FormInt]

L'action [FormInt] est associée à la classe [FormInt] suivante :

```

1. package example;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4. import com.opensymphony.xwork2.ModelDriven;
5. import java.util.Map;
6. import org.apache.struts2.interceptor.SessionAware;
7. import org.apache.struts2.interceptor.validation.SkipValidation;
8.
9. public class FormInt extends ActionSupport implements ModelDriven, SessionAware {
10.
11.   // constructeur sans paramètre
12.   public FormInt() {
13.   }
14.
15.   // modèle de l'action
16.   public Object getModel() {
17.     if (session.get("model") == null) {
18.       session.put("model", new FormIntModel());
19.     }
20.     return session.get("model");
21.   }
22.
23.   public String cancel() {
24.     // on nettoie le modèle
25.     ((FormIntModel) getModel()).clearModel();

```

```

26.     // résultat
27.     return "cancel";
28. }
29.
30. @SkipValidation
31. public String clearModel() {
32.     // raz du modèle
33.     ((FormIntModel) getModel()).clearModel();
34.     // résultat
35.     return INPUT;
36. }
37.
38. // SessionAware
39. private Map<String, Object> session;
40.
41. public void setSession(Map<String, Object> session) {
42.     this.session = session;
43. }
44.
45. // validation
46. @Override
47. public void validate() {
48.     // saisie int6 valide ?
49.     if (getFieldErrors().get("int6") == null) {
50.         int int6 = Integer.parseInt(((FormIntModel) getModel()).getInt6());
51.         if (int6 < 2 || int6 > 20) {
52.             addFieldError("int6", getText("int6.error"));
53.         }
54.     }
55. }
56. }

```

Nous commenterons ce code au fur et à mesure des besoins. Pour l'instant :

- ligne 9, la classe [FormInt] implémente deux interfaces :
 - **ModelDriven** qui n'a qu'une méthode, **getModel** de la ligne 16
 - **SessionAware** qui n'a qu'une méthode, **setSession** de la ligne 41
- lignes 16-21 : implémentation de l'interface *ModelDriven*. On rappelle que cette interface permet de déporter le modèle d'une vue dans une classe externe, ici la classe [FormIntModel] suivante :

```

1. package example;
2.
3. public class FormIntModel {
4.
5.     // constructeur sans paramètre
6.     public FormIntModel() {
7.     }
8.
9.     // champs du formulaire
10.    private String int1;
11.    private Integer int2;
12.    private Integer int3;
13.    private Integer int4;
14.    private Integer int5;
15.    private String int6;
16.
17.    // raz modèle
18.    public void clearModel() {
19.        int1=null;
20.        int2=null;
21.        int3=null;
22.        int4=null;
23.        int5=null;
24.        int6=null;
25.    }
26.
27.    // getters et setters
28.    ...
29. }

```

Le modèle [FormIntModel] a six champs correspondant aux six champs de saisie de la vue [FormInt.JSP]. Ce sont ces six champs qui vont recevoir les valeurs postées. Quatre d'entre-eux ont le type *Integer*. Se posera donc pour eux, le problème de la conversion *String* --> *Integer*. La méthode *clearModel* permet de réinitialiser le modèle.

Revenons à la méthode `getModel` de l'action [FormInt] :

```
1. // modèle de l'action
2. public Object getModel() {
3.     if (session.get("model") == null) {
4.         session.put("model", new FormIntModel());
5.     }
6.     return session.get("model");
7. }
```

- lignes 3-5 : le modèle est cherché dans la session. S'il n'y est pas, une instance du modèle est créée et mise dans la session.
- ligne 6 : alors qu'une instance de l'action est créée à chaque nouvelle requête faite à l'action, son modèle lui demeurera dans la session.

Nous voyons que la classe ne définit pas de méthode *input* mais la classe parent en a une qui rend la clé *input*. L'exécution de cette méthode amène l'affichage de la vue [FormInt.JSP] que nous présentons maintenant.

11.5.3 La vue [FormInt.JSP]

La vue [FormInt.JSP] est la suivante :

The image shows two side-by-side browser screenshots of a web application titled "Struts 2 - Conversion et validation de nombres entiers". The browser address bar shows "localhost:8084/exemple-09/exemple/FormInt.action".

The left screenshot, marked with a yellow box containing the number "1", shows a clean form with six input fields and a "Valider" button. The labels for the fields are:

- 1-Nombre entier positif de deux chiffres:
- 2-Nombre entier:
- 3-Nombre entier >=-1:
- 4-Nombre entier <=10:
- 5-Nombre entier dans l'intervalle [1,10]:
- 6-Nombre entier dans l'intervalle [2,20]:

Below the form are links for "Annuler" and "Raz modèle".

The right screenshot, marked with a yellow box containing the number "2", shows the same form after validation. Red error messages are displayed above each input field:

- 1-Tapez un nombre entier positif de deux chiffres
- 2-Tapez un nombre entier
- 3-Tapez un nombre entier >=-1
- 4-Tapez un nombre entier <=10
- 5-Tapez un nombre entier dans l'intervalle [1,10]
- 6-Tapez un nombre entier dans l'intervalle [2,20]

The "Valider" button is still present, and the "Annuler" and "Raz modèle" links are also visible.

- en [1], le formulaire vierge
- en [2], le formulaire après une validation de paramètres erronés.

Son code est le suivant :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
4.     <head>
5.         <title><s:text name="Form.titre"/></title>
6.         <s:head/>
7.     </head>
8.
9.     <body background="<s:url value="/ressources/standard.jpg"/>">
10.        <h2><s:text name="FormInt.message"/></h2>
11.        <s:form name="formulaire" action="FormInt">
12.            <s:textfield name="int1" key="int1.prompt"/>
13.            <s:textfield name="int2" key="int2.prompt"/>
14.            <s:textfield name="int3" key="int3.prompt"/>
15.            <s:textfield name="int4" key="int4.prompt"/>
16.            <s:textfield name="int5" key="int5.prompt"/>
17.            <s:textfield name="int6" key="int6.prompt"/>
18.            <s:submit key="Form.submitText" method="execute"/>
19.        </s:form>
```



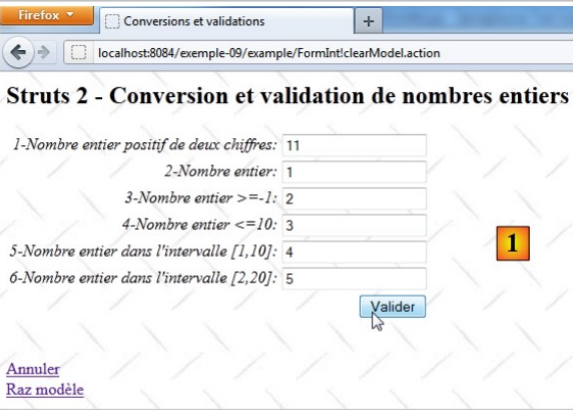
```

20.     <br/>
21.     <s:url id="URL" action="FormInt" method="cancel"/>
22.     <s:a href="%{URL}"><s:text name="Form.cancelText"/></s:a>
23.     <br/>
24.     <s:url id="URL" action="FormInt" method="clearModel"/>
25.     <s:a href="%{URL}"><s:text name="Form.clearModel"/></s:a>
26. </body>
27. </html>

```

- lignes 12-17 : les six champs de saisie correspondant aux six champs du modèle [FormIntModel] de l'action [FormInt]. A l'affichage de la vue, ce sont les attributs *value* des champs de saisie qui sont utilisés pour la valeur affichée par ces champs. En l'absence de l'attribut *value*, alors c'est l'attribut *name* qui est utilisé.
- ligne 12 : le champ de saisie est associé (*name*) au champ *int1* de l'action ou de son modèle si l'action implémente l'interface *ModelDriven*. C'est le cas ici. C'est vrai également pour tous les autres champs.
- ligne 18 : le bouton [Valider] poste les saisies à l'action [FormInt] définie ligne 11. C'est sa méthode *execute* qui sera exécutée.
- lignes 21-22 : le lien [Annuler] exécute la méthode [FormInt.cancel].
- ligne 24-25 : le lien [Raz modèle] exécute la méthode [FormInt.clearModel].

11.5.4 La vue [ConfirmationFormInt.JSP]



Struts 2 - Conversion et validation de nombres entiers

1-Nombre entier positif de deux chiffres: 11

2-Nombre entier: 1

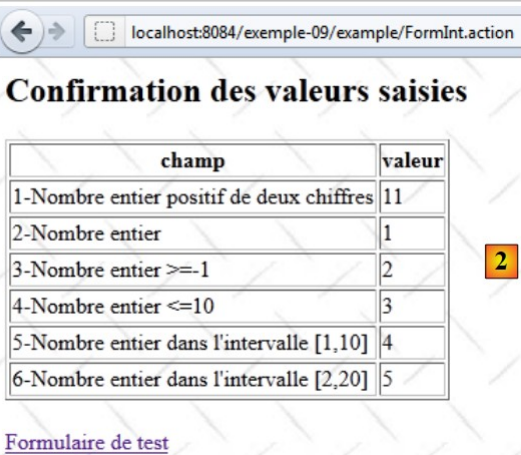
3-Nombre entier >=-1: 2

4-Nombre entier <=10: 3

5-Nombre entier dans l'intervalle [1,10]: 4

6-Nombre entier dans l'intervalle [2,20]: 5

[Annuler](#)
[Raz modèle](#)



Confirmation des valeurs saisies

champ	valeur
1-Nombre entier positif de deux chiffres	11
2-Nombre entier	1
3-Nombre entier >=-1	2
4-Nombre entier <=10	3
5-Nombre entier dans l'intervalle [1,10]	4
6-Nombre entier dans l'intervalle [2,20]	5

[Formulaire de test](#)

Elle est affichée lorsque toutes les saisies du formulaire [FormInt.JSP] sont valides.

- en [1], on poste des valeurs valides
- en [2], la page de confirmation

Le code de la vue [ConfirmationInt.JSP] est le suivant :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
4.   <head>
5.     <title><s:text name="Confirmation.titre"/></title>
6.     <s:head/>
7.   </head>
8.
9.   <body background="<s:url value="/ressources/standard.jpg"/>">
10.    <h2><s:text name="Confirmation.message"/></h2>
11.    <table border="1">
12.      <tr>
13.        <th><s:text name="Confirmation.champ"/></th>
14.        <th><s:text name="Confirmation.valeur"/></th>
15.      </tr>
16.      <tr>
17.        <td><s:text name="int1.prompt"/></td>
18.        <td><s:property value="int1"/></td>
19.      </tr>
20.      <tr>
21.        <td><s:text name="int2.prompt"/></td>
22.        <td><s:property value="int2"/></td>

```

```

23.     </tr>
24.     <tr>
25.         <td><s:text name="int3.prompt"/></td>
26.         <td><s:property value="int3"/></td>
27.     </tr>
28.     <tr>
29.         <td><s:text name="int4.prompt"/></td>
30.         <td><s:property value="int4"/></td>
31.     </tr>
32.     <tr>
33.         <td><s:text name="int5.prompt"/></td>
34.         <td><s:property value="int5"/></td>
35.     </tr>
36.     <tr>
37.         <td><s:text name="int6.prompt"/></td>
38.         <td><s:property value="int6"/></td>
39.     </tr>
40. </table>
41. <br/>
42. <s:url id="URL" action="FormInt" method="input"/>
43. <s:a href="%{URL}"><s:text name="Confirmation.lien"/></s:a>
44. </body>
45. </html>

```

Pour comprendre ce code, il faut rappeler que la vue est affichée après instantiation de la classe [FormInt]. Les champs de celle-ci et de son modèle [FormIntModel] sont donc accessibles à la vue.

- lignes 16-38 : les valeurs des six champs sont affichés
- lignes 42-43 : un lien vers l'action [FormInt]. Le code généré pour ce lien est le suivant :

```
<a href="/exemple-09/example/FormInt!input.action">Formulaire de test</a>
```

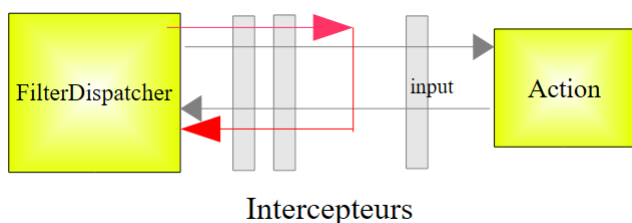
L'URL particulière du lien indique que la méthode *input* de l'action [FormInt] doit traiter la requête. Rappelons la configuration de l'action [FormInt] dans [example.xml] :

```

1.     <action name="FormInt" class="example.FormInt">
2.         <result name="input"/>example/FormInt.JSP</result>
3.         <result name="cancel" type="redirect"/>example/Accueil.JSP</result>
4.         <result name="success"/>example/ConfirmationFormInt.JSP</result>
5.     </action>

```

La méthode *input* de la classe [FormInt] sera celle de sa classe parent *ActionSupport*. L'exécution de la méthode *input* de la classe [FormInt] se fait après exécution des intercepteurs



On sait que l'appel à la méthode *input* est ignorée par l'intercepteur de validation. Il n'y aura donc pas de validation.

La vue [FormInt.JSP] est affichée :

localhost:8084/exemple-09/example/FormInt.action

Confirmation des valeurs saisies

champ	valeur
1-Nombre entier positif de deux chiffres	11
2-Nombre entier	1
3-Nombre entier ≥ -1	2
4-Nombre entier ≤ 10	3
5-Nombre entier dans l'intervalle $[1,10]$	4
6-Nombre entier dans l'intervalle $[2,20]$	5

[Formulaire de test](#)

localhost:8084/exemple-09/example/FormInt!show.action

Struts 2 - Conversion et validation de nombres entiers

1-Nombre entier positif de deux chiffres: 11
 2-Nombre entier: 1
 3-Nombre entier ≥ -1 : 2
 4-Nombre entier ≤ 10 : 3
 5-Nombre entier dans l'intervalle $[1,10]$: 4
 6-Nombre entier dans l'intervalle $[2,20]$: 5

[Annuler](#)
[Raz modèle](#)

En [2], les champs de saisie retrouvent leurs valeurs de saisie. Cela peut paraître normal mais ça ne l'est pas. L'action [FormInt] ayant été appelée, la classe associée [FormInt] a été instanciée. Parce que cette classe implémente l'interface *ModelDriven*, sa méthode *getModel* a été appelée :

```

1. // modèle de l'action
2. public Object getModel() {
3.     if (session.get("model") == null) {
4.         session.put("model", new FormIntModel());
5.     }
6.     return session.get("model");
7. }

```

On voit que le modèle de l'action est récupéré dans la session. Dans l'étape précédente, ce modèle avait été mis à jour par les valeurs postées. On retrouve donc ces valeurs. Si on n'avait pas mis le modèle dans la session, on aurait eu six champs vides dans la vue [FormInt.JSP].

11.5.5 L'action [FormInt!clearModel]

L'action [FormInt!clearModel] est déclenchée par un clic sur le lien [Raz modèle] :

localhost:8084/exemple-09/example/FormInt.action

Struts 2 - Conversion et validation de nombres entiers

1-Nombre entier positif de deux chiffres: 11
 2-Nombre entier: 1
 Tapez un nombre entier ≥ -1
 3-Nombre entier ≥ -1 : x
 4-Nombre entier ≤ 10 : 3
 5-Nombre entier dans l'intervalle $[1,10]$: 4
 6-Nombre entier dans l'intervalle $[2,20]$: 5

[Annuler](#)
[Raz modèle](#)

localhost:8084/exemple-09/example/FormInt!clearModel.action

Struts 2 - Conversion et validation de nombres entiers

1-Nombre entier positif de deux chiffres:
 2-Nombre entier:
 3-Nombre entier ≥ -1 :
 4-Nombre entier ≤ 10 :
 5-Nombre entier dans l'intervalle $[1,10]$:
 6-Nombre entier dans l'intervalle $[2,20]$:

[Annuler](#)
[Raz modèle](#)

- en [1], le formulaire après une saisie erronée
- en [2], le formulaire après un clic sur le lien [Raz modèle].

La méthode [FormInt.clearModel] est la suivante :

```

1. @SkipValidation
2. public String clearModel() {

```

```

3. // raz du modèle
4. ((FormIntModel) getModel()).clearModel();
5. // résultat
6. return INPUT;
7. }

```

- ligne 1 : il n'y a pas de validation à faire. On utilise la notation `@SkipValidation` pour l'indiquer. L'intercepteur de validation ne fera alors pas les validations.
- ligne 4 : la méthode `[FormIntModel].clearModel` est exécutée. Nous l'avons déjà rencontrée. Elle réinitialise à `null` les six champs du modèle.
- ligne 7 : la méthode retourne la clé `input`.

Si on revient à la configuration de l'action `[FormInt]` :

```

1. <action name="FormInt" class="example.FormInt">
2.   <result name="input">/example/FormInt.JSP</result>
3.   <result name="cancel" type="redirect">/example/Accueil.JSP</result>
4.   <result name="success">/example/ConfirmationFormInt.JSP</result>
5. </action>

```

on voit que la clé `input` va faire afficher la vue `[FormInt.JSP]`. Celle-ci affiche les six champs du modèle. Ceux-ci étant à `null`, la vue affiche six champs vides [2].

11.5.6 L'action `[FormInt!cancel]`

L'action `[FormInt!cancel]` est déclenchée par un clic sur le lien `[Annuler]` :

- en [1], le formulaire après une saisie erronée
- en [2], la page d'accueil après un clic sur le lien `[Annuler]`.

La méthode `[FormInt.cancel]` est la suivante :

```

1. public String cancel() {
2.   // on nettoie le modèle
3.   ((FormIntModel) getModel()).clearModel();
4.   // résultat
5.   return "cancel";
6. }

```

- ligne 1 : on notera que la méthode n'est pas précédée de l'annotation `SkipValidation`. Or on ne veut pas faire les validations. La méthode `cancel` fait partie des quatre méthodes `input`, `back`, `cancel`, `browse` ignorées par l'intercepteur de validation aussi l'annotation `SkipValidation` n'est-elle pas nécessaire.
- ligne 3 : elle vide le modèle

- ligne 5 : elle rend la clé *cancel*

Si on revient à la configuration de l'action [FormInt] :

```
1. <action name="FormInt" class="example.FormInt">
2. <result name="input"/>example/FormInt.JSP</result>
3. <result name="cancel" type="redirect"/>example/Accueil.JSP</result>
4. <result name="success"/>example/ConfirmationFormInt.JSP</result>
5. </action>
```

on voit que la clé *cancel* va faire afficher la vue [Accueil.JSP] après une redirection du client. C'est ce que montre la vue [2].

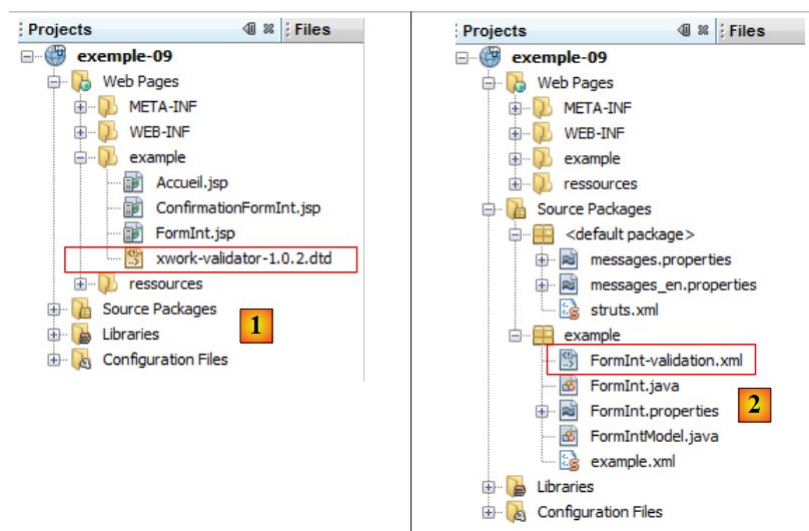
11.6 Le processus de validation

Nous abordons maintenant la validation des six champs de saisie associés aux six champs suivants du modèle :

```
1. // champs du formulaire
2. private String int1;
3. private Integer int2;
4. private Integer int3;
5. private Integer int4;
6. private Integer int5;
7. private String int6;
```

Cette validation a lieu à chaque fois que la classe [FormInt] est instanciée et que la méthode exécutée n'est pas ignorée par l'intercepteur de validation. Elle est pilotée par :

- le fichier [FormInt-validation.xml], s'il existe dans le même dossier que la classe [FormInt]
- la méthode [FormInt.validate] si elle existe.



- en [1] : le fichier [xwork-validator-1.0.2.dtd] nécessaire au processus de validation
- en [2] : le fichier [FormInt-validation.xml] dans le même dossier que la classe [FormInt]

Le fichier [FormInt-validation.xml] est le suivant :

```
1. <!--
2. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
3. EN" "http://www.opensymphony.com/xwork/xwork-validator-1.0.2.dtd">
4. -->
5.
6. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
7. EN" "http://localhost:8084/exemple-09/example/xwork-validator-1.0.2.dtd">
8.
9. <validators>
10.
11. <field name="int1" >
12. <field-validator type="requiredstring" short-circuit="true">
```



```

13.     <message key="intl.error"/>
14. </field-validator>
15. <field-validator type="regex" short-circuit="true">
16.     <param name="expression">^\d{2}$</param>
17.     <param name="trim">true</param>
18.     <message key="intl.error"/>
19. </field-validator>
20. </field>
21.
22. <field name="int2" >
23.     ...
24. </field>
25.
26. ...
27. </validators>

```

- en [3], l'URL de la DTD (Document Type Definition) du fichier de validation. Celle-ci doit être accessible sinon le fichier de validation n'est pas utilisé.
- en [7], l'URL de la DTD utilisée par l'application. Nous avons mis le fichier DTD dans le dossier [exemple] du projet *exemple-09* [1] afin de l'avoir même si on n'a pas d'accès à Internet.
- lignes 11-20 : fixent les conditions de validation du paramètre *int1* associé au champ *int1* du modèle.
La balise nommée *int1* dans le formulaire est la suivante :

```
<s:textfield name="int1" key="intl.prompt" />
```

Le champ *int1* du modèle est déclaré comme suit :

```
private String int1;
```

- lignes 12-14 : vérifient que le paramètre *int1* existe (non *null*) et est de longueur non nulle. Si ce n'est pas le cas, un message d'erreur est associé au champ de saisie. Il est défini dans [FormInt.properties] comme suit :

```
intl.error=Tapez un nombre entier positif de deux chiffres
```

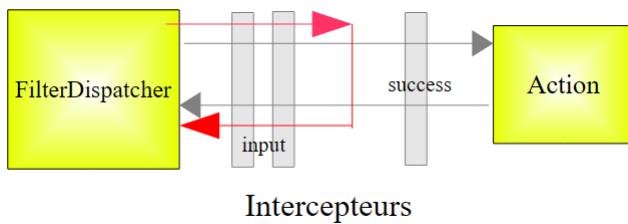
S'il y a erreur, le processus de validation du paramètre *int1* s'arrête (*short-circuit=true*).

- lignes 15-19 : la validité du paramètre *int1* est vérifiée par une expression régulière.
- ligne 16 : l'expression régulière, ici 2 chiffres sans rien ni devant ni derrière.
- ligne 17 : le paramètre *int1* avant d'être comparé à l'expression régulière sera débarrassé des espaces de début et de fin.
- ligne 18 : le message d'erreur éventuel. C'est le même que pour le précédent validateur.

Voyons ce que ça donne :

- en [1], une saisie erronée pour le champ *int1*
- en [2], la page renvoyée :
 - le message d'erreur de clé *intl.error* est présent. Il est en rouge.
 - le libellé du champ erroné est également en rouge.
 - la saisie erronée est réaffichée. Il faut le prévoir car ce n'est pas forcément le comportement par défaut.

Nous avons vu que la validation du formulaire provoque l'exécution de la méthode [FormInt].*execute* si la requête arrive à passer tous les intercepteurs, notamment celui de validation :



- si la requête arrive jusqu'à la méthode *execute* de l'action, celle-ci renvoie la clé *success* au contrôleur comme nous l'avons vu..
- si l'intercepteur de validation arrête la requête parce que les paramètres testés ne sont pas valides, alors la clé *input* est renvoyée au contrôleur.

Comme l'action [FormInt] est configurée comme suit :

```
1. <action name="FormInt" class="example.FormInt">
2.   <result name="input"/>example/FormInt.JSP</result>
3.   <result name="cancel" type="redirect"/>example/Accueil.JSP</result>
4.   <result name="success"/>example/ConfirmationFormInt.JSP</result>
5. </action>
```

lorsqu'il y a une erreur de validation, c'est la vue [FormInt.JSP] qui est affichée, donc le formulaire. Les balises Struts sont faites de telle manière qu'elles affichent les éventuels messages d'erreur qui leur sont attachés. On aura donc la vue [FormInt.JSP] avec les messages d'erreur attachés aux différents champs. C'est ce que montre la vue [2].

Examinons maintenant la validation du champ *int2* déclaré comme suit dans le modèle :

```
private Integer int2;
```

La validation du champ *int2* dans [FormInt-validation.xml] est la suivante :

```
1. <field name="int2" >
2.   <field-validator type="required" short-circuit="true">
3.     <message key="int2.error"/>
4.   </field-validator>
5.   <field-validator type="conversion" short-circuit="true">
6.     <message key="int2.error"/>
7.   </field-validator>
8. </field>
```

- lignes 2-4 : vérifient que le paramètre *int2* existe.
- lignes 5-7 : vérifient que la conversion *String* --> *Integer* est possible
- ligne 3, 6 : le message d'erreur de clé *int2.error* est le suivant :

```
int2.error=Tapez un nombre entier
```

La validation du champ *Integer int3* du modèle dans [FormInt-validation.xml] est la suivante :

```
1. <field name="int3" >
2.   <field-validator type="required" short-circuit="true">
3.     <message key="int3.error"/>
4.   </field-validator>
5.   <field-validator type="conversion" short-circuit="true">
6.     <message key="int2.error"/>
7.   </field-validator>
8.   <field-validator type="int" short-circuit="true">
9.     <param name="min">-1</param>
10.    <message key="int3.error"/>
11.  </field-validator>
12. </field>
```

- lignes 8-11 : vérifient que le champ *int3* est de type entier ≥ -1
- ligne 3, 7 : le message d'erreur de clé *int3.error* est le suivant :

```
int3.error=Tapez un nombre entier >=-1
```

La validation du champ *Integer int4* du modèle dans [FormInt-validation.xml] est la suivante :

```
1. <field name="int4" >
2.   <field-validator type="required" short-circuit="true">
3.     <message key="int4.error"/>
4.   </field-validator>
5.   <field-validator type="conversion" short-circuit="true">
6.     <message key="int2.error"/>
7.   </field-validator>
8.   <field-validator type="int" short-circuit="true">
9.     <param name="max">10</param>
10.    <message key="int4.error"/>
11.  </field-validator>
12. </field>
```

- lignes 8-11 : vérifient qu'il est de type entier <=10
- ligne 3, 7 : le message d'erreur de clé *int4.error* est le suivant :

```
int4.error=Tapez un nombre entier <=10
```

La validation du champ *Integer int5* du modèle dans [FormInt-validation.xml] est la suivante :

```
1. <field name="int5" >
2.   <field-validator type="required" short-circuit="true">
3.     <message key="int5.error"/>
4.   </field-validator>
5.   <field-validator type="conversion" short-circuit="true">
6.     <message key="int2.error"/>
7.   </field-validator>
8.   <field-validator type="int" short-circuit="true">
9.     <param name="min">1</param>
10.    <param name="max">10</param>
11.    <message key="int5.error"/>
12.  </field-validator>
13. </field>
```

- lignes 5-9 : vérifient qu'il est de type entier dans l'intervalle [1, 10].
- ligne 3, 8 : le message d'erreur de clé *int5.error* est le suivant :

```
int5.error=Tapez un nombre entier dans l'intervalle [1,10]
```

La validation du champ *String int6* du modèle dans [FormInt-validation.xml] est la suivante :

```
1. <field name="int6" >
2.   <field-validator type="requiredstring" short-circuit="true">
3.     <message key="int6.error"/>
4.   </field-validator>
5.   <field-validator type="regex" short-circuit="true">
6.     <param name="expression">^\d{1,2}$</param>
7.     <param name="trim">true</param>
8.     <message key="int6.error"/>
9.   </field-validator>
10. </field>
```

- lignes 5-9 : vérifient que *int6* est une chaîne de 2 chiffres.
- ligne 3, 8 : le message d'erreur de clé *int6.error* est le suivant :

```
int6.error=Tapez un nombre entier dans l'intervalle [2,20]
```

La validation précédente ne vérifie pas que le paramètre *int6* est un entier dans l'intervalle [2,20]. Cette vérification est faite dans la méthode [FormInt].*validate* qui est exécutée après que le fichier [FormInt-validation.xml] a été exploitée. Cette méthode est la suivante :

```
1. // validation
2. @Override
3. public void validate() {
4.   // saisie int6 valide ?
5.   if (getFieldErrors().get("int6") == null) {
6.     int int6 = Integer.parseInt(((FormIntModel) getModel()).getInt6());
7.     if (int6 < 2 || int6 > 20) {
```



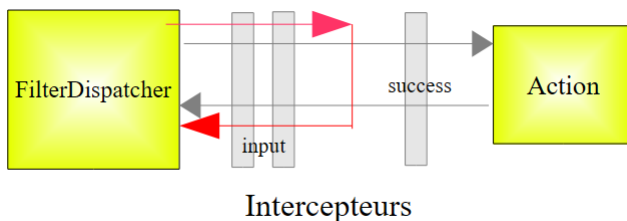
```

8.         addFieldError("int6", getText("int6.error"));
9.     }
10. }
11. }

```

- ligne 5 : on regarde s'il y a des erreurs attachées au champ *int6*. Si oui, on ne va pas plus loin.
- ligne 6 : s'il n'y a pas eu d'erreur, on récupère le champ *String int6* du modèle et on le transforme en entier.
- ligne 7 : on vérifie que l'entier récupéré est dans l'intervalle [2,20].
- ligne 8 : si ce n'est pas le cas, un message d'erreur est attaché au champ *int6*. Ce message d'erreur est cherché dans le fichier des messages avec la clé *int6.error*.

Si au terme de ce processus de validation, il y a des erreurs, l'appel vers la méthode [FormInt].*execute* est interrompu et la clé *input* est rendue au contrôleur Struts.



11.7 Derniers détails

Nous avons vu plusieurs façons de saisir des nombres entiers. Elles ne sont pas toutes équivalentes. Considérons par exemple les champs de saisie *int5* et *int6* :

Dans la vue [FormInt.JSP], ils sont déclarés comme suit :

```

1.     <s:textfield name="int5" key="int5.prompt"/>
2. <s:textfield name="int6" key="int6.prompt"/>

```

Leur modèle est déclaré dans [FormIntModel.java] :

```

1.     private Integer int5;
2.     private String int6;

```

Le champ *int5* est de type *Integer* alors que le champ *int6* est de type *String*. Leurs règles de validation sont différentes :

```

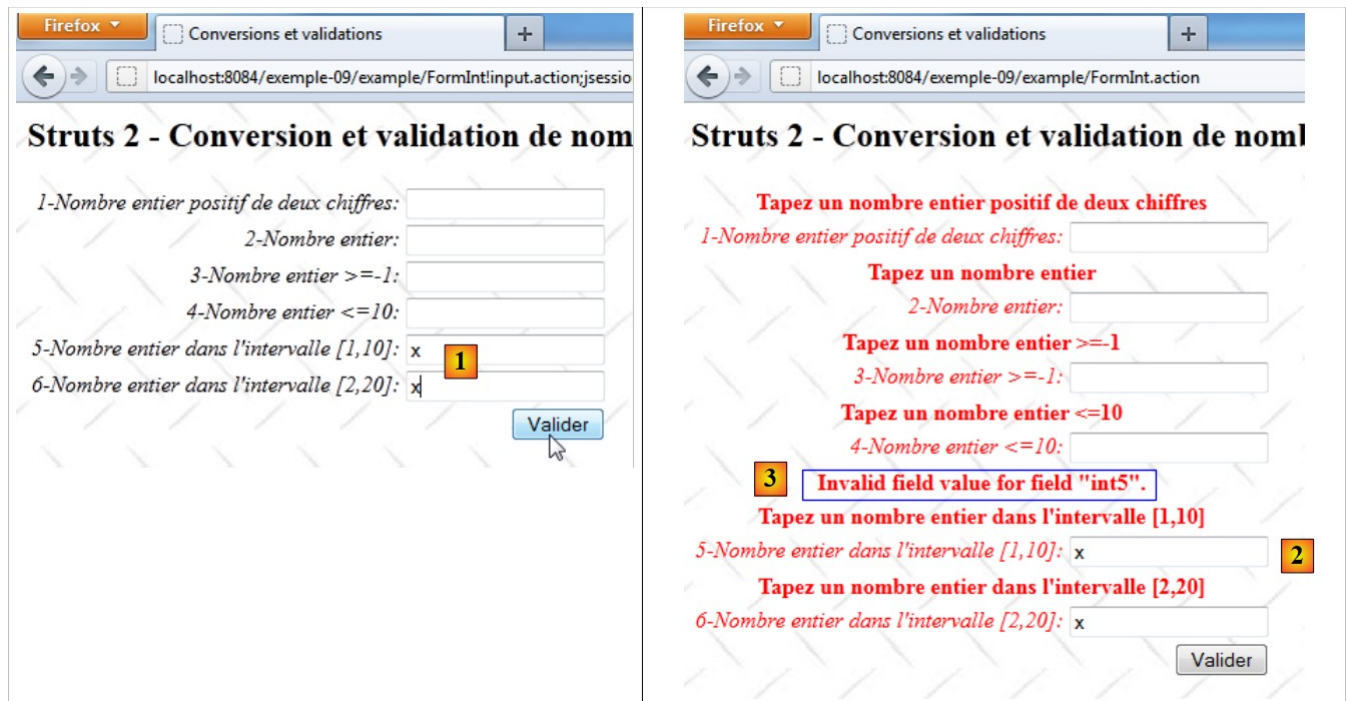
1. <field name="int5" >
2.     <field-validator type="required" short-circuit="true">
3.         <message key="int5.error"/>
4.     </field-validator>
5.     <field-validator type="conversion" short-circuit="true">
6.         <message key="int2.error"/>
7.     </field-validator>
8.     <field-validator type="int" short-circuit="true">
9.         <param name="min">1</param>
10.        <param name="max">10</param>
11.        <message key="int5.error"/>
12.    </field-validator>
13. </field>
14.
15. <field name="int6" >
16.     <field-validator type="requiredstring" short-circuit="true">
17.         <message key="int6.error"/>
18.     </field-validator>
19.     <field-validator type="regex" short-circuit="true">
20.         <param name="expression">^\d{1,2}$</param>
21.         <param name="trim">true</param>
22.         <message key="int6.error"/>
23.     </field-validator>
24. </field>

```

La validation du champ *int6* est complétée par la méthode *validate* de l'action [FormInt] :

```
1. public void validate() {
2.     // saisie int6 valide ?
3.     if (getFieldErrors().get("int6") == null) {
4.         int int6 = Integer.parseInt(((FormIntModel) getModel()).getInt6());
5.         if (int6 < 2 || int6 > 20) {
6.             addFieldError("int6", getText("int6.error"));
7.         }
8.     }
```

Les règles de validation bien qu'exprimées différemment visent toutes les deux à vérifier que le champ saisi est un nombre entier compris dans un intervalle. Le comportement des champs *int5* et *int6* est cependant différent à l'exécution comme le montrent les copies d'écran suivantes :



- en [1], la même saisie incorrecte pour les deux champs
- en [2], la page d'erreurs renvoyée. Les deux champs ont des messages d'erreurs différents.
- en [3] apparaît pour le champ *int5* un message indésirable parce qu'il est en anglais. Il vient de la conversion ratée *String* --> *Integer*. On a par ailleurs une exception dans les logs d'Apache :

```
1. Avertissement: Error setting expression 'int5' with value '[Ljava.lang.String;@1ad405d8'
2. ognl.MethodFailedException: Method "setInt5" failed for object example.FormIntModel@21b63266
   [java.lang.NoSuchMethodException: example.FormIntModel.setInt5([Ljava.lang.String;)]
```

Assez curieusement, Struts a cherché une méthode *FormIntModel.setInt5(String value)* qu'il n'a pas trouvée.

La clé du message indésirable est *xwork.default.invalid.fieldvalue*. Pour le passer en français, il suffit d'associer un texte français à cette clé. On ajoute ainsi au fichier [messages.properties] la ligne suivante :

```
1. ...
2. xwork.default.invalid.fieldvalue=Valeur invalide pour le champ "{0}".
```

11.8 Conclusion

L'étude de cette première application dédiée aux validations de paramètres se termine. Elle fut complexe à expliquer. Nous allons maintenant étudier des applications similaires. Aussi ne commenterons-nous que ce qui change.

12 Exemple 09B – Validation du modèle

L'application qui suit est une variante de la précédente. Précédemment, pour l'action **A** :

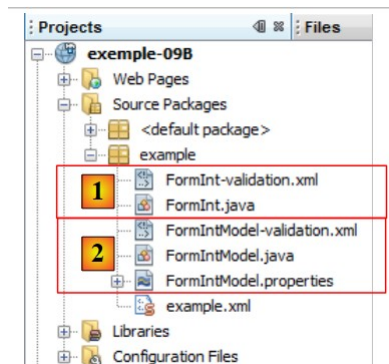
- le fichier des règles de validation s'appelait **A-validation.xml**
- le fichier des messages d'erreurs de la validation s'appelait **A.properties**

Ici, pour l'action **A** de modèle **M** :

- le fichier des règles de validation s'appelle **M-validation.xml**
- le fichier des messages d'erreurs de la validation s'appelle **M.properties**
- le fichier **A-validation.xml** existe toujours mais avec un contenu qui redirige les règles de validation sur le fichier **M-validation.xml**.

Ce sont les seuls changements que nous apporterons à la version précédente.

Le projet Netbeans est le suivant :



- en [1], l'action [FormInt] et son fichier de validation
- en [2], son modèle [FormIntModel], le fichier de validation de ce modèle et les messages associés à ce modèle.

Tout le reste est identique à la version précédente.

Le fichier [FormInt-validation.xml] est le suivant :

```
1. <!--
2. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
3. EN" "http://www.opensymphony.com/xwork/xwork-validator-1.0.2.dtd">
4. -->
5. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
6. EN" "http://localhost:8084/exemple-09B/example/xwork-validator-1.0.2.dtd">
7.
8. <validators>
9.   <field name="model" >
10.     <field-validator type="visitor">
11.       <param name="appendPrefix">false</param>
12.       <message/>
13.     </field-validator>
14.   </field>
15. </validators>
```

On ne cherchera pas à expliquer le contenu de ce fichier. On le prendra tel quel en se souvenant de sa fonctionnalité : déporter les règles de validation sur le modèle. On peut se demander à quoi ça sert. En fait le modèle, ses règles de validation et ses messages peuvent pré-exister et être utilisés par différentes actions. Plutôt que de répéter les mêmes règles de validation et messages pour les différentes actions, il est préférable alors de déléguer ces fonctionnalités au modèle.

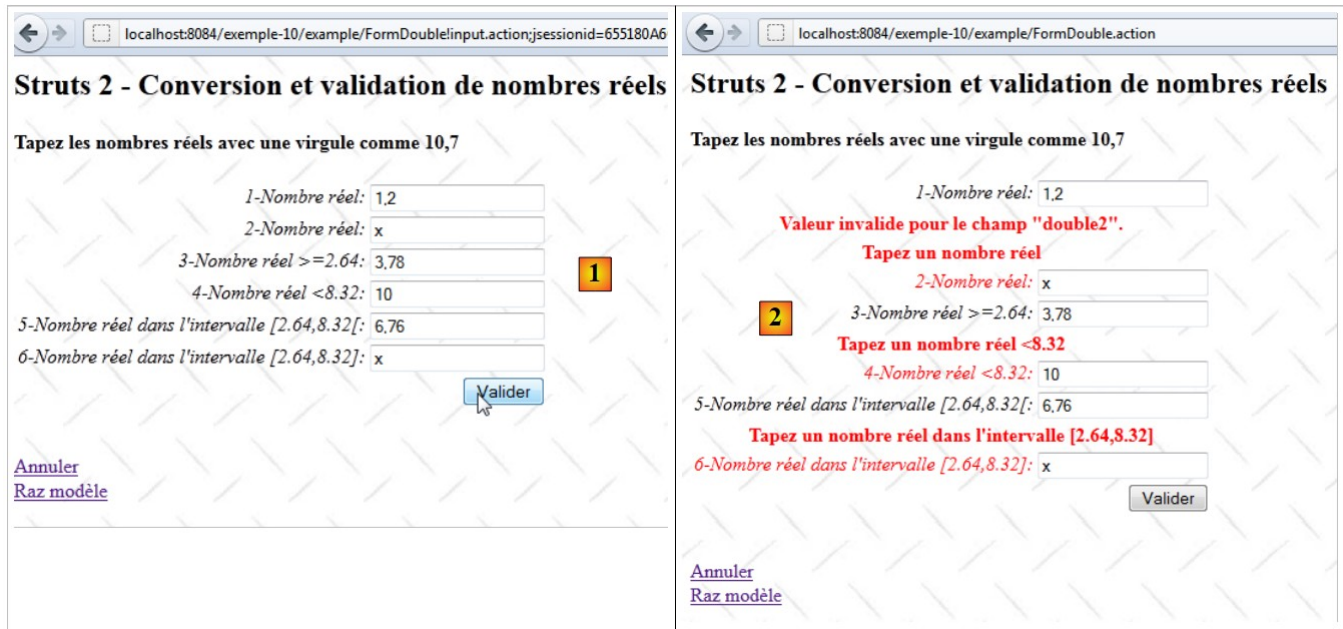
Le fichier [FormIntModel-validation.xml] est identique au fichier [FormInt-validation.xml] de la version précédente.

Le fichier [FormIntModel.properties] est identique au fichier [FormInt.properties] de la version précédente.

Le lecteur est invité à tester cette nouvelle version.

13 Exemple 10 – Conversion et validation des nombres réels

La nouvelle application présente la saisie des nombres réels :

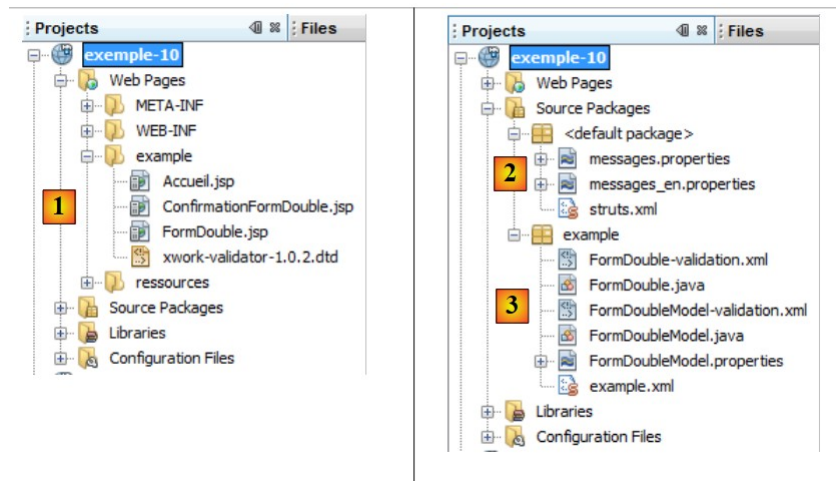


- en [1], le formulaire de saisie
- en [2], la réponse renvoyée

L'application a un fonctionnement similaire à celui de la saisie des nombres entiers aussi ne commenterons-nous que les points qui diffèrent.

13.1 Le projet Netbeans

Le projet Netbeans est le suivant :



- en [1], les vues de l'application
 - [Accueil.JSP] : la page d'accueil
 - [FormDouble.JSP] : le formulaire de saisie
 - [ConfirmationDouble.JSP] : la page de confirmation
- en [2], le fichier des messages [messages.properties] et le fichier de configuration principal de Struts
- en [3] :
 - [FormDouble.java] : l'action qui affiche et traite le formulaire
 - [FormDouble-validation.xml] : les règles de validation de l'action [FormDouble]. Ce fichier délègue ces validation au modèle selon la méthode qui vient d'être vue.
 - [FormDoubleModel] : le modèle de l'action [FormDouble]
 - [FormDoubleModel-validation.xml] : les règles de validation du modèle

- [FormDoubleModel.properties] : le fichier des messages du modèle
- [example.xml] : fichier de configuration secondaire de Struts

13.2 La configuration du projet

Le projet est principalement configuré par le fichier [example.xml] suivant :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.   <package name="example" namespace="/example" extends="struts-default">
8.     <action name="Accueil">
9.       <result name="success">/example/Accueil.JSP</result>
10.    </action>
11.    <action name="FormDouble" class="example.FormDouble">
12.      <result name="input">/example/FormDouble.JSP</result>
13.      <result name="cancel" type="redirect">/example/Accueil.JSP</result>
14.      <result name="success">/example/ConfirmationFormDouble.JSP</result>
15.    </action>
16.  </package>
17. </struts>

```

Il est analogue à celui qui a été étudié pour la saisie des nombres entiers.

13.3 Les fichiers des messages

Le fichier [messages.properties] est le suivant :

```

1. Accueil.titre=Accueil
2. Accueil.message=Struts 2 - Conversions et validations
3. Accueil.FormDouble=Saisie de nombres r\u00e9els
4. Form.titre=Conversions et validations
5. FormDouble.message=Struts 2 - Conversion et validation de nombres r\u00e9els
6. FormDouble.conseil=Tapez les nombres r\u00e9els avec une virgule comme 10,7
7. Form.submitText=Valider
8. Form.cancelText=Annuler
9. Form.clearModel=Raz mod\u00e8le
10. Confirmation.titre=Confirmation
11. Confirmation.message=Confirmation des valeurs saisies
12. Confirmation.champ=champ
13. Confirmation.valeur=valeur
14. Confirmation.lien=Formulaire de test
15. xwork.default.invalid.fieldvalue=Valeur invalide pour le champ "{0}".

```

Le fichier [FormDoubleModel.properties] est le suivant :

```

1. double.format={0,number}
2. double1.prompt=1-Nombre r\u00e9el
3. double1.error=Tapez un nombre r\u00e9el
4. double2.prompt=2-Nombre r\u00e9el
5. double2.error=Tapez un nombre r\u00e9el
6. double3.prompt=3-Nombre r\u00e9el >=2.64
7. double3.error=Tapez un nombre r\u00e9el >=2.64
8. double4.prompt=4-Nombre r\u00e9el <8.32
9. double4.error=Tapez un nombre r\u00e9el <8.32
10. double5.prompt=5-Nombre r\u00e9el dans l'intervalle [2.64,8.32[
11. double5.error=Tapez un nombre r\u00e9el dans l'intervalle [2.64,8.32[
12. double6.prompt=6-Nombre r\u00e9el dans l'intervalle [2.64,8.32]
13. double6.error=Tapez un nombre r\u00e9el dans l'intervalle [2.64,8.32]

```

La ligne 1 joue un rôle important. Nous y reviendrons.

13.4 Le formulaire de saisie

La vue [FormDouble.JSP] est la suivante :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
4.   <head>
5.     <title><s:text name="Form.titre"/></title>
6.     <s:head/>
7.   </head>
8.
9.   <body background="<s:url value="/ressources/standard.jpg"/>">
10.    <h2><s:text name="FormDouble.message"/></h2>
11.    <h4><s:text name="FormDouble.conseil"/></h4>
12.    <s:form name="formulaire" action="FormDouble">
13.      <s:textfield name="double1" key="double1.prompt"/>
14.      <s:textfield name="double2" key="double2.prompt" value="%{#parameters['double2']!=null ?
#parameters['double2'] : double2==null ? '' :getText('double.format',{double2})}"/>
15.      <s:textfield name="double3" key="double3.prompt" value="%{#parameters['double3']!=null ?
#parameters['double3'] : double3==null ? '' :getText('double.format',{double3})}"/>
16.      <s:textfield name="double4" key="double4.prompt" value="%{#parameters['double4']!=null ?
#parameters['double4'] : double4==null ? '' :getText('double.format',{double4})}"/>
17.      <s:textfield name="double5" key="double5.prompt" value="%{#parameters['double5']!=null ?
#parameters['double5'] : double5==null ? '' :getText('double.format',{double5})}"/>
18.      <s:textfield name="double6" key="double6.prompt"/>
19.      <s:submit key="Form.submitText" method="execute"/>
20.    </s:form>
21.    <br/>
22.    <s:url id="URL" action="FormDouble" method="cancel"/>
23.    <s:a href="%{URL}"><s:text name="Form.cancelText"/></s:a>
24.    <br/>
25.    <s:url id="URL" action="FormDouble" method="clearModel"/>
26.    <s:a href="%{URL}"><s:text name="Form.clearModel"/></s:a>
27.  </body>
28. </html>
```

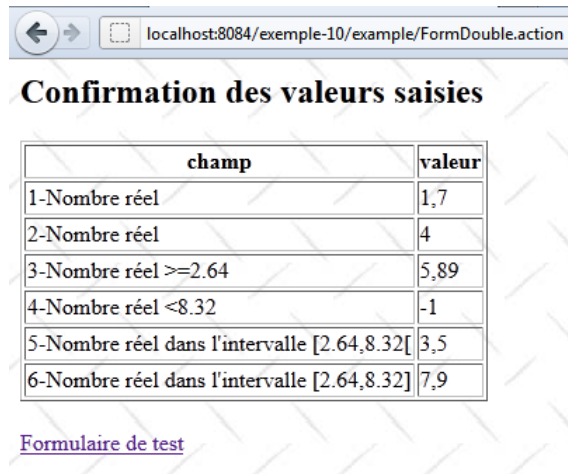
Les lignes 13 à 18 sont les six zones de saisie des nombres réels. Les champs *double2* à *double5* ont un attribut *value* complexe. Normalement les six champs de saisie devraient être les suivants :

```
1. <s:textfield name="double1" key="double1.prompt"/>
2. <s:textfield name="double2" key="double2.prompt"/>
3. <s:textfield name="double3" key="double3.prompt"/>
4. <s:textfield name="double4" key="double4.prompt"/>
5. <s:textfield name="double5" key="double5.prompt"/>
6. <s:textfield name="double6" key="double6.prompt"/>
```

Pour résoudre certains problèmes rencontrés lors des tests, il a fallu complexifier les choses. Pour l'instant, le lecteur peut ignorer cette complexité. Nous l'expliquerons ultérieurement.

13.5 La vue de confirmation

La vue de confirmation [ConfirmationFormDouble.JSP] est la suivante :



Son code est le suivant :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
4.   <head>
5.     <title><s:text name="Confirmation.titre"/></title>
6.     <s:head/>
7.   </head>
8.
9.   <body background="<s:url value="/ressources/standard.jpg"/>">
10.    <h2><s:text name="Confirmation.message"/></h2>
11.    <table border="1">
12.      <tr>
13.        <th><s:text name="Confirmation.champ"/></th>
14.        <th><s:text name="Confirmation.valeur"/></th>
15.      </tr>
16.      <tr>
17.        <td><s:text name="double1.prompt"/></td>
18.        <td><s:text name="double1"/></td>
19.      </tr>
20.      <tr>
21.        <td><s:text name="double2.prompt"/></td>
22.        <td>
23.          <s:text name="double.format">
24.            <s:param value="double2"/>
25.          </s:text>
26.        </td>
27.      </tr>
28.      <tr>
29.        <td><s:text name="double3.prompt"/></td>
30.        <td>
31.          <s:text name="double.format">
32.            <s:param value="double3"/>
33.          </s:text>
34.        </td>
35.      </tr>
36.      <tr>
37.        <td><s:text name="double4.prompt"/></td>
38.        <td>
39.          <s:text name="double.format">
40.            <s:param value="double4"/>
41.          </s:text>
42.        </td>
43.      </tr>
44.      <tr>
45.        <td><s:text name="double5.prompt"/></td>
46.        <td>
47.          <s:text name="double.format">
48.            <s:param value="double5"/>
49.          </s:text>
50.        </td>
51.      </tr>

```



```

52.     <tr>
53.         <td><s:text name="double6.prompt"/></td>
54.         <td><s:text name="double6"/></td>
55.     </tr>
56. </table>
57. <br/>
58. <s:url id="URL" action="FormDouble!input"/>
59. <s:a href="%{URL}"><s:text name="Confirmation.lien"/></s:a>
60. </body>
61. </html>

```

La ve [ConfirmationFormDouble.JSP] se contente d'afficher le modèle [FormDoubleModel]. Les lignes 47-49 montrent l'affichage d'un nombre réel. On veut que cet affichage tienne compte de la *localisation* de l'application. Selon celle-ci, un nombre réel ne s'affichera pas de la même façon en France (10,7) et en Grande-Bretagne (10.7). Pour cela, on utilise la balise `<s:text>` qu'on avait utilisée jusque là pour l'internationalisation de l'application. Cette balise sert donc également pour la localisation.

Ici, la clé de message utilisée est *double.format*. Cette clé est trouvée dans le fichier [FormDoubleModel.properties] :

```
double.format={0,number}
```

La valeur associée à la clé n'est pas ici un message, mais un format d'affichage :

- **0** est un paramètre représentant la valeur à afficher. Ici ce sera le champ *double5* du modèle.
- **number** représente le format numérique. Il sera adapté à chaque pays. Sans ce format, le nombre 10.7 sera toujours affiché 10.7 quelque soit le pays.

La balise

```

1.     <s:text name="double.format">
2.         <s:param value="double5"/>
3. </s:text>

```

Fait afficher le message `{0, number}` où le paramètre *double5* (ligne 2) viendra remplacer le paramètre *0* du format. Ainsi, le modèle *double5* sera affiché au format numérique localisé.

13.6 Le modèle [FormDoubleModel]

Les champs *double1* à *double6* du formulaire [FormDouble.JSP] sont injectés dans le modèle [FormDoubleModel] suivant :

```

1. package example;
2.
3. public class FormDoubleModel {
4.
5.     // constructeur sans paramètre
6.     public FormDoubleModel() {
7.     }
8.     // champs
9.     private String double1;
10.    private Double double2;
11.    private Double double3;
12.    private Double double4;
13.    private Double double5;
14.    private String double6;
15.
16.    // raz modèle
17.    public void clearModel() {
18.        double1 = null;
19.        double2 = null;
20.        double3 = null;
21.        double4 = null;
22.        double5 = null;
23.        double6 = null;
24.    }
25.
26.    // getters et setters
27.    ...
28. }

```

- les champs *double1* et *double6* sont de type *String*
- les autres champs sont de type *Double*

13.7 La validation du modèle

La validation du modèle est contrôlée par deux fichiers : [FormDouble-validation.xml] et [FormDoubleModel-validation.xml].

Le fichier [FormDouble-validation.xml] délègue les validations à [FormDoubleModel-validation.xml] :

```
1. <!--
2. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
3. EN" "http://www.opensymphony.com/xwork/xwork-validator-1.0.2.dtd">
4. -->
5. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
6. EN" "http://localhost:8084/exemple-10/example/xwork-validator-1.0.2.dtd">
7.
8. <validators>
9.   <field name="model" >
10.     <field-validator type="visitor">
11.       <param name="appendPrefix">false</param>
12.       <message/>
13.     </field-validator>
14.   </field>
15. </validators>
```

Nous avons déjà rencontré ce fichier.

Le fichier [FormDoubleModel-validation.xml] contient les règles de validation suivantes :

```
1. <!--
2. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
3. EN" "http://www.opensymphony.com/xwork/xwork-validator-1.0.2.dtd">
4. -->
5.
6. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
7. EN" "http://localhost:8084/exemple-10/example/xwork-validator-1.0.2.dtd">
8.
9.
10. <validators>
11.
12.   <field name="double1" >
13.     <field-validator type="requiredstring" short-circuit="true">
14.       <message key="double1.error"/>
15.     </field-validator>
16.     <field-validator type="regex" short-circuit="true">
17.       <param name="expression">^[+|-]*\s*\d+(,\d+)*$</param>
18.       <param name="trim">true</param>
19.       <message key="double1.error"/>
20.     </field-validator>
21.   </field>
22.
23.   <field name="double2" >
24.     <field-validator type="required" short-circuit="true">
25.       <message key="double2.error"/>
26.     </field-validator>
27.     <field-validator type="conversion" short-circuit="true">
28.       <message key="double2.error"/>
29.     </field-validator>
30.   </field>
31.
32.   <field name="double3" >
33.     <field-validator type="required" short-circuit="true">
34.       <message key="double3.error"/>
35.     </field-validator>
36.     <field-validator type="conversion" short-circuit="true">
37.       <message key="double3.error"/>
38.     </field-validator>
39.     <field-validator type="double" short-circuit="true">
40.       <param name="minInclusive">2.64</param>
41.       <message key="double3.error"/>
42.     </field-validator>
43.   </field>
44.
45.   <field name="double4" >
46.     <field-validator type="required" short-circuit="true">
```

```

47.     <message key="double4.error"/>
48. </field-validator>
49. <field-validator type="conversion" short-circuit="true">
50.     <message key="double4.error"/>
51. </field-validator>
52. <field-validator type="double" short-circuit="true">
53.     <param name="maxExclusive">8.32</param>
54.     <message key="double4.error"/>
55. </field-validator>
56. </field>
57.
58. <field name="double5" >
59.     <field-validator type="required" short-circuit="true">
60.         <message key="double5.error"/>
61.     </field-validator>
62.     <field-validator type="conversion" short-circuit="true">
63.         <message key="double5.error"/>
64.     </field-validator>
65.     <field-validator type="double" short-circuit="true">
66.         <param name="minInclusive">2.64</param>
67.         <param name="maxExclusive">8.32</param>
68.         <message key="double5.error"/>
69.     </field-validator>
70. </field>
71.
72. <field name="double6" >
73.     <field-validator type="requiredstring" short-circuit="true">
74.         <message key="double6.error"/>
75.     </field-validator>
76.     <field-validator type="regex" short-circuit="true">
77.         <param name="expression">^[+|-]*\s*\d+ (, \d+)*$</param>
78.         <param name="trim">true</param>
79.         <message key="double6.error"/>
80.     </field-validator>
81. </field>
82. </validators>

```

- la règle des lignes 12-21 vérifie que le champ de saisie *double1* suit le modèle d'une expression régulière représentant un nombre réel.
- la règle des lignes 23-30 vérifie que le champ de saisie *double2* peut être converti en un réel double.
- la règle des lignes 32-43 vérifie que le champ de saisie *double3* peut être converti en un réel double ≥ 2.64 . On notera qu'il faut utiliser la notation anglo-saxonne des réels.
- la règle des lignes 45-56 vérifie que le champ de saisie *double4* peut être converti en un réel double < 8.32 .
- la règle des lignes 58-70 vérifie que le champ de saisie *double5* peut être converti en un réel double dans l'intervalle $[2.64, 8.32[$.
- la règle des lignes 72-80 vérifie que le champ *double6* suit le modèle d'une expression régulière représentant un nombre réel.

Une fois le fichier [FormDoubleModel-validation.xml] exploité par l'intercepteur de validation, celui-ci fait exécuter la méthode *validate* de l'action [FormDouble] si elle existe. Nous allons la présenter en même temps que la totalité de l'action.

13.8 L'action [FormDouble]

L'action [FormDouble] est la suivante :

```

1. package example;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4. import com.opensymphony.xwork2.ModelDriven;
5. import java.util.Map;
6. import org.apache.struts2.interceptor.SessionAware;
7. import org.apache.struts2.interceptor.validation.SkipValidation;
8.
9. public class FormDouble extends ActionSupport implements ModelDriven, SessionAware {
10.
11.     // constructeur sans paramètre
12.     public FormDouble() {
13.     }
14.
15.     // modèle de l'action
16.     public Object getModel() {
17.         if (session.get("model") == null) {
18.             session.put("model", new FormDoubleModel());

```

```

19.     }
20.     return session.get("model");
21. }
22.
23. @SkipValidation
24. public String clearModel() {
25.     // raz du modèle
26.     ((FormDoubleModel) getModel()).clearModel();
27.     // résultat
28.     return INPUT;
29. }
30.
31. public String cancel() {
32.     // on nettoie le modèle
33.     ((FormDoubleModel) getModel()).clearModel();
34.     // résultat
35.     return "cancel";
36. }
37. // SessionAware
38. Map<String, Object> session;
39.
40. public void setSession(Map<String, Object> session) {
41.     this.session = session;
42. }
43.
44. // validation
45. @Override
46. public void validate() {
47.     // saisie double6 valide ?
48.     if (getFieldErrors().get("double6") == null) {
49.         // on remplace la virgule par le point dans la chaîne double6
50.         String strDouble6 = ((FormDoubleModel) getModel()).getDouble6().replace(',', '.');
51.         // String --> double
52.         double double6 = Double.parseDouble(strDouble6);
53.         // vérification
54.         if (double6 < 2.64 || double6 > 8.32) {
55.             addFieldError("double6", getText("double6.error"));
56.         }
57.     }
58. }
59. }

```

L'action [FormDouble] est bâtie sur le même modèle que l'action [FormInt]. Nous ne commenterons que la méthode *validate*. On rappelle que la méthode *validate* est exécutée après exploitation du fichier de validation [FormDoubleModel-validation.xml] et avant l'exécution de la méthode *execute*.

- ligne 48 : s'il y a déjà des erreurs sur le champ *double6*, on ne fait rien de plus.
- ligne 50 : on a eu une chaîne de la forme 45,67. Elle a été stockée dans le champ *double6* du modèle. On remplace la virgule par le point pour avoir 45.67.
- ligne 52 : la chaîne 45.67 est transformée en réel double. Ca doit marcher puisque la chaîne *double6* respecte le format d'un nombre réel.
- ligne 54 : on vérifie que le réel double obtenu est dans l'intervalle [2.64, 8.32].
- ligne 55 : si ce n'est pas le cas, le message d'erreur de clé *double6.error* est attaché au champ *double6*. Ce message sera trouvé dans le fichier [FormDoubleModel.properties]. Il sera affiché quand le formulaire erroné sera réaffiché.

13.9 Derniers détails

Maintenant, retour à la complexité des champs de saisie du formulaire [FormDouble.JSP]. On va raisonner sur le champ *double2*. Le raisonnement s'étend aux champs *double3* à *double5* qui ont un modèle de type *Double*. Pour les champs *double1* et *double6* qui ont un modèle de type *String*, il n'y a pas de problème.

Le champ de saisie *double2* est le suivant :

```

<s:textfield name="double2" key="double2.prompt" value="%{#parameters['double2']!=null ?
#parameters['double2'] : double2==null ? '' :getText('double.format',{double2})}"/>

```

Partons de la balise la plus simple :

```

<s:textfield name="double2" key="double2.prompt"/>

```

et regardons ce qui se passe :

Struts 2 - Conversion et validation de nombres réels

Tapez les nombres réels avec une virgule comme 10,7

1-Nombre réel: 6,77
2-Nombre réel: 4,32
3-Nombre réel ≥ 2.64 : 3,78
4-Nombre réel < 8.32 : 4,78
5-Nombre réel dans l'intervalle [2.64,8.32[: 5,06
6-Nombre réel dans l'intervalle [2.64,8.32]: 8,32

Valider

Confirmation des valeurs saisies

champ	valeur
1-Nombre réel	6,77
2-Nombre réel	4,32
3-Nombre réel ≥ 2.64	3,78
4-Nombre réel < 8.32	4,78
5-Nombre réel dans l'intervalle [2.64,8.32[	5,06
6-Nombre réel dans l'intervalle [2.64,8.32]	8,32

[Formulaire de test](#)

- en [1], on valide le nombre *double2* correct. Sur la copie d'écran, ça ne se voit pas très bien mais on tape le nombre avec une virgule.
- en [2], la vue de confirmation. La saisie *double2* a passé les tests de validation. Le nombre est affiché avec une virgule.

Confirmation des valeurs saisies

champ	valeur
1-Nombre réel	6,77
2-Nombre réel	4,32
3-Nombre réel ≥ 2.64	3,78
4-Nombre réel < 8.32	4,78
5-Nombre réel dans l'intervalle [2.64,8.32[	5,06
6-Nombre réel dans l'intervalle [2.64,8.32]	8,32

[Formulaire de test](#)

Struts 2 - Conversion et validation de nombres réels

Tapez les nombres réels avec une virgule comme 10,7

1-Nombre réel: 6,77
2-Nombre réel: 4,32
3-Nombre réel ≥ 2.64 : 3,78
4-Nombre réel < 8.32 : 4,78
5-Nombre réel dans l'intervalle [2.64,8.32[: 5,06
6-Nombre réel dans l'intervalle [2.64,8.32]: 8,32

Valider

[Annuler](#)
[Raz modèle](#)

- en [3], on revient au formulaire
- en [4], le formulaire. Ce que montre mal la copie d'écran c'est que nombre *double2* qui était initialement 4,32 est devenu 4.32 avec un point décimal.

- en [5], on revalide le formulaire sans rien changer
- en [6], une erreur est signalée sur le champ *double2*.

Le problème est le suivant :

- initialement dans [1], la chaîne de saisie "4,32" a été transformée avec succès en nombre réel 4.32. Ce qui veut dire que l'opération *String* --> *Double* a réussi et que donc dans ce sens Struts tient compte de la locale, ici la France.
- le nouveau formulaire [4] affiche dans le champ *double2* la valeur du réel 4.32. Comme on n'a pas localisé l'affichage, il l'affiche par défaut à l'anglo-saxonne, c.a.d. avec un point décimal. Donc dans le sens *Double* --> *String*, Struts ne tient plus compte de la locale sinon il aurait affiché 4,32 avec une virgule.

C'est pour le moins incohérent. Mais qu'à cela ne tienne, on va localiser l'affichage du nombre 4.32. La balise de saisie devient la suivante :

```
<s:textfield name="double2" key="double2.prompt" value="%{getText('double.format',{double2})}"/>
```

L'attribut *value* précise la valeur à afficher dans le champ *double2*. Celle-ci est celle d'une expression OGNL. Rappelons la définition de la clé *double.format* dans le fichier [FormDoubleModel.properties] :

```
double.format={0,number}
```

La méthode *getText('clé')* permet d'avoir le message associé à une clé. Celui-ci est cherché dans le fichier correspondant à la locale courante. Ainsi si celle-ci était **es** (Espagne), la clé *double.format* aurait été cherchée dans le fichier [FormDoubleModel_es.properties].

La méthode *getText('clé', {param0, param1, ...})* permet de retrouver un message paramétré. Le message

```
{0, number}
```

est un message paramétré par le paramètre 0. Celui-ci est un paramètre positionnel. La méthode *getText('double.format', {double2})* va affecter le nombre *double2* au paramètre 0. Au final, on demande la valeur de *double2* au format numérique localisé. En France, le nombre 4.56 sera localisé en chaîne "4,56".

Après cette transformation, on refait les tests.

Struts 2 - Conversion et validation de nombre

Tapez les nombres réels avec une virgule comme 10,7

1-Nombre réel:

2-Nombre réel: double.format **1**

3-Nombre réel ≥ 2.64 :

4-Nombre réel < 8.32 :

5-Nombre réel dans l'intervalle $[2.64, 8.32[$:

6-Nombre réel dans l'intervalle $[2.64, 8.32]$:

Dès l'affichage initial du formulaire, on a une anomalie en [1]. Revenons à la balise :

```
<s:textfield name="double2" key="double2.prompt" value="%{getText('double.format',{double2})}"/>
```

A l'affichage initial, la valeur du modèle `double2` est `null`, une valeur non numérique. On fait évoluer la balise de la façon suivante :

```
<s:textfield name="double2" key="double2.prompt" value="%{double2==null ? '' : getText('double.format',{double2})}"/>
```

Cette fois-ci, on teste si `double2==null`. Si oui, on affiche une chaîne vide.

Ce changement fait, nous reprenons les tests :

Struts 2 - Conversion et validation de nombre

Tapez les nombres réels avec une virgule comme 10,7

1-Nombre réel:

2-Nombre réel:

3-Nombre réel ≥ 2.64 : **1**

4-Nombre réel < 8.32 :

5-Nombre réel dans l'intervalle $[2.64, 8.32[$:

6-Nombre réel dans l'intervalle $[2.64, 8.32]$:

localhost:8084/exemple-10/example/FormDouble.action

Confirmation des valeurs saisies

champ	valeur
1-Nombre réel	1
2-Nombre réel	4,67
3-Nombre réel ≥ 2.64	3
4-Nombre réel < 8.32	4
5-Nombre réel dans l'intervalle $[2.64, 8.32[$	5
6-Nombre réel dans l'intervalle $[2.64, 8.32]$	6

[Formulaire de test](#)

2

localhost:8084/exemple-10/exemple/FormDoubleInput.action

Struts 2 - Conversion et validation de nombre

Tapez les nombres réels avec une virgule comme 10,7

1-Nombre réel: 1 3

2-Nombre réel: 4,67

3-Nombre réel ≥ 2.64 : 3

4-Nombre réel < 8.32 : 4

5-Nombre réel dans l'intervalle [2.64,8.32[: 5

6-Nombre réel dans l'intervalle [2.64,8.32]: 6

Valider

localhost:8084/exemple-10/exemple/FormDouble.action

Confirmation des valeurs saisies

champ	valeur
1-Nombre réel	1
2-Nombre réel	4,67
3-Nombre réel ≥ 2.64	3
4-Nombre réel < 8.32	4
5-Nombre réel dans l'intervalle [2.64,8.32[	5
6-Nombre réel dans l'intervalle [2.64,8.32]	6

[Formulaire de test](#)

Les écrans [1] à [4] montrent que le problème que l'on cherchait à résoudre est résolu :

- en [1], on saisit 4,67
- en [2], ce nombre a été accepté
- en [3], il est réaffiché comme 4,67 avec la virgule ce qui est confirmé par [4].

Malheureusement, les problèmes ne s'arrêtent pas là. Regardons la séquence suivante :

Struts 2 - Conversion et validation de nombre

Tapez les nombres réels avec une virgule comme 10,7

1-Nombre réel: 1

2-Nombre réel: 4,67x

3-Nombre réel ≥ 2.64 : 3 5

4-Nombre réel < 8.32 : 4

5-Nombre réel dans l'intervalle [2.64,8.32[: 5

6-Nombre réel dans l'intervalle [2.64,8.32]: 6

Valider

Tapez les nombres réels avec une virgule comme 10,7

1-Nombre réel: 1

Valeur invalide pour le champ "double2".

Tapez un nombre réel

2-Nombre réel: 4,67 6

3-Nombre réel ≥ 2.64 : 3

4-Nombre réel < 8.32 : 4

5-Nombre réel dans l'intervalle [2.64,8.32[: 5

6-Nombre réel dans l'intervalle [2.64,8.32]: 6

Valider

- en [5], on ajoute un caractère pour invalider *double2*
- en [6], les tests de validation ont fait leur travail et l'erreur est signalée. Seulement la chaîne affichée en [6] n'est pas la chaîne erronée mais l'actuelle valeur du modèle *double2*. On a perdu la valeur saisie.

Lorsqu'on passe de [5] à [6] la requête ne va pas jusqu'à son terme. Elle est arrêtée par l'intercepteur de validation. En [6] on a affiché la valeur du modèle *double2* qui n'a pas reçu de nouvelle valeur à cause de cet arrêt. C'est donc sa précédente valeur qui est affichée alors qu'il aurait fallu afficher la chaîne saisie. Les paramètres d'une requête sont disponibles à une vue via la notation **#parameters['param']**. On fait évoluer le champ de saisie *double2* de la façon suivante :

```
<s:textfield name="double2" key="double2.prompt" value="%{#parameters['double2']!=null ? #parameters['double2'] : double2==null ? '' : getText('double.format',{double2})}"/>
```

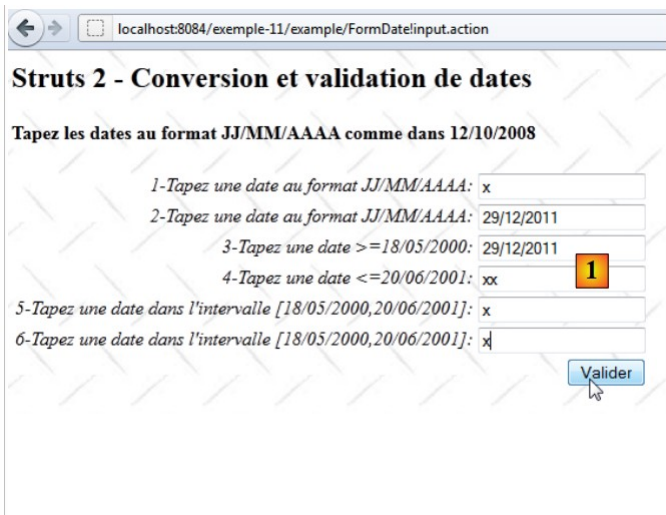
La valeur affichée du champ *double2* est calculée comme suit : si le paramètre 'double2' existe alors on l'affiche sinon on affiche le modèle *double2*. Le lecteur est invité à tester que cette nouvelle mouture de la balise résoud les problèmes rencontrés.

13.10 Conclusion

Il est étonnant que saisir des nombres réels avec contrôle de validité soit aussi compliqué... Peut-être ai-je raté quelque chose dans la documentation ?

14 Exemple 11 – Conversion et validation de dates

La nouvelle application présente la saisie de dates :



Struts 2 - Conversion et validation de dates

Tapez les dates au format JJ/MM/AAAA comme dans 12/10/2008

1-Tapez une date au format JJ/MM/AAAA: x

2-Tapez une date au format JJ/MM/AAAA: 29/12/2011

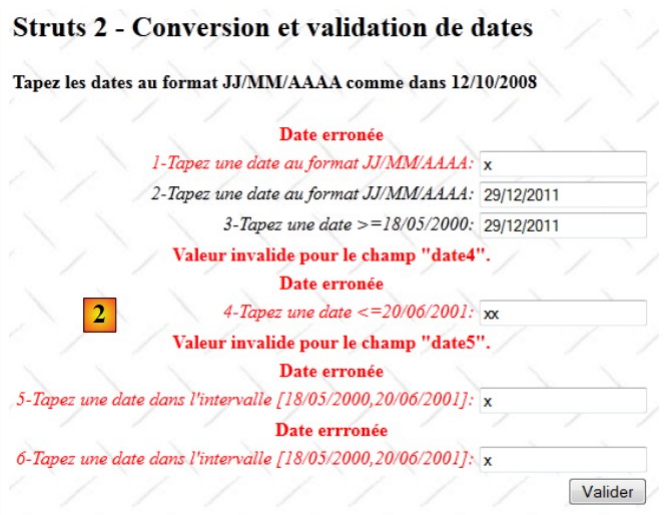
3-Tapez une date >=18/05/2000: 29/12/2011

4-Tapez une date <=20/06/2001: xx

5-Tapez une date dans l'intervalle [18/05/2000,20/06/2001]: x

6-Tapez une date dans l'intervalle [18/05/2000,20/06/2001]: x

Valider



Struts 2 - Conversion et validation de dates

Tapez les dates au format JJ/MM/AAAA comme dans 12/10/2008

Date erronée

1-Tapez une date au format JJ/MM/AAAA: x

2-Tapez une date au format JJ/MM/AAAA: 29/12/2011

3-Tapez une date >=18/05/2000: 29/12/2011

Valeur invalide pour le champ "date4".

Date erronée

4-Tapez une date <=20/06/2001: xx

Valeur invalide pour le champ "date5".

Date erronée

5-Tapez une date dans l'intervalle [18/05/2000,20/06/2001]: x

Date erronée

6-Tapez une date dans l'intervalle [18/05/2000,20/06/2001]: x

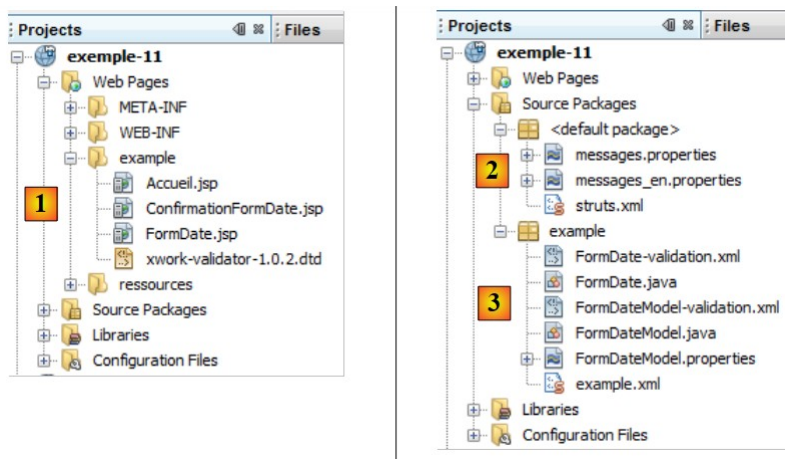
Valider

- en [1], le formulaire de saisie
- en [2], la réponse renvoyée

L'application a un fonctionnement similaire à celui de la saisie des nombres entiers aussi ne commenterons-nous que les points qui diffèrent.

14.1 Le projet Netbeans

Le projet Netbeans est le suivant :



- en [1], les vues de l'application
 - [Accueil.JSP] : la page d'accueil
 - [FormDate.JSP] : le formulaire de saisie
 - [ConfirmationFormDate.JSP] : la page de confirmation
- en [2], le fichier des messages [messages.properties] et le fichier de configuration principal de Struts
- en [3] :
 - [FormDate.java] : l'action qui affiche et traite le formulaire

- [FormDate-validation.xml] : les règles de validation de l'action [FormDate]. Ce fichier délègue ces validations au modèle.
- [FormDateModel] : le modèle de l'action [FormDate]
- [FormDateModel-validation.xml] : les règles de validation du modèle
- [FormDateModel.properties] : le fichier des messages du modèle
- [example.xml] : fichier de configuration secondaire de Struts

14.2 La configuration du projet

Le projet est principalement configuré par le fichier [example.xml] suivant :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.     <package name="example" namespace="/example" extends="struts-default">
8.         <action name="Accueil">
9.             <result name="success">/example/Accueil.JSP</result>
10.        </action>
11.        <action name="FormDate" class="example.FormDate">
12.            <result name="input">/example/FormDate.JSP</result>
13.            <result name="cancel" type="redirect">/example/Accueil.JSP</result>
14.            <result name="success">/example/ConfirmationFormDate.JSP</result>
15.        </action>
16.    </package>
17. </struts>

```

Il est analogue à celui qui a été étudié pour la saisie des nombres entiers.

14.3 Les fichiers des messages

Le fichier [messages.properties] est le suivant :

```

1. Accueil.titre=Accueil
2. Accueil.message=Struts 2 - Conversions et validations
3. Accueil.FormDate=Saisie de dates
4. Form.titre=Conversions et validations
5. FormDate.message=Struts 2 - Conversion et validation de dates
6. FormDate.conseil=Tapez les dates au format JJ/MM/AAAA comme dans 12/10/2008
7. Form.submitText=Valider
8. Form.cancelText=Annuler
9. Form.clearModel=Raz mod\u00e8le
10. Confirmation.titre=Confirmation
11. Confirmation.message=Confirmation des valeurs saisies
12. Confirmation.champ=champ
13. Confirmation.valeur=valeur
14. Confirmation.lien=Formulaire de test
15. xwork.default.invalid.fieldvalue=Valeur invalide pour le champ "{0}".

```

Le fichier [FormDateModel.properties] est le suivant :

```

1. date.format={0,date,dd/MM/yyyy}
2. date1.prompt=1-Tapez une date au format JJ/MM/AAAA
3. date1.error=Date erron\u00e9e
4. date2.prompt=2-Tapez une date au format JJ/MM/AAAA
5. date2.error=Date erron\u00e9e
6. date3.prompt=3-Tapez une date >=18/05/2000
7. date3.error=Date erron\u00e9e
8. date4.prompt=4-Tapez une date <=20/06/2001
9. date4.error=Date erron\u00e9e
10. date5.prompt=5-Tapez une date dans l'intervalle [18/05/2000,20/06/2001]
11. date5.error=Date erron\u00e9e
12. date6.prompt=6-Tapez une date dans l'intervalle [18/05/2000,20/06/2001]
13. date6.error=Date erron\u00e9e

```

La ligne 1 joue un rôle important. Nous y reviendrons.

14.4 Le formulaire de saisie

La vue [FormDate.JSP] est la suivante :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
4.   <head>
5.     <title><s:text name="Form.titre"/></title>
6.   </head>
7.   </head>
8.
9.   <body background="<s:url value="/ressources/standard.jpg"/>">
10.    <h2><s:text name="FormDate.message"/></h2>
11.    <h4><s:text name="FormDate.conseil"/></h4>
12.    <s:form name="formulaire" action="FormDate">
13.      <s:textfield name="date1" key="date1.prompt"/>
14.      <s:textfield name="date2" key="date2.prompt" value="%{#parameters['date2']!=null ?
#parameters['date2'] : date2==null ? '' :getText('date.format',{date2})}"/>
15.      <s:textfield name="date3" key="date3.prompt" value="%{#parameters['date3']!=null ?
#parameters['date3'] : date3==null ? '' :getText('date.format',{date3})}"/>
16.      <s:textfield name="date4" key="date4.prompt" value="%{#parameters['date4']!=null ?
#parameters['date4'] : date4==null ? '' :getText('date.format',{date4})}"/>
17.      <s:textfield name="date5" key="date5.prompt" value="%{#parameters['date5']!=null ?
#parameters['date5'] : date5==null ? '' :getText('date.format',{date5})}"/>
18.      <s:textfield name="date6" key="date6.prompt"/>
19.      <s:submit key="Form.submitText" method="execute"/>
20.    </s:form>
21.    <br/>
22.    <s:url id="URL" action="FormDate" method="cancel"/>
23.    <s:a href="%{URL}"><s:text name="Form.cancelText"/></s:a>
24.    <br/>
25.    <s:url id="URL" action="FormDate" method="clearModel"/>
26.    <s:a href="%{URL}"><s:text name="Form.clearModel"/></s:a>
27.  </body>
28. </html>
```

Les lignes 13 à 18 sont les six zones de saisie de dates. Les champs *date2* à *date5* ont un attribut *value* complexe, le même que pour la saisie des nombres réels. Les mêmes difficultés ayant été rencontrées, on les a résolues de la même façon.

14.5 La vue de confirmation

La vue de confirmation [ConfirmationFormDate.JSP] est la suivante :

Confirmation des valeurs saisies

champ	valeur
1-Tapez une date au format JJ/MM/AAAA	13/10/1954
2-Tapez une date au format JJ/MM/AAAA	29/12/2011
3-Tapez une date >=18/05/2000	29/12/2011
4-Tapez une date <=20/06/2001	10/10/2000
5-Tapez une date dans l'intervalle [18/05/2000,20/06/2001]	01/02/2001
6-Tapez une date dans l'intervalle [18/05/2000,20/06/2001]	28/02/2001

[Formulaire de test](#)

Son code est le suivant :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
```

```

4.  <head>
5.    <title><s:text name="Confirmation.titre"/></title>
6.    <s:head/>
7.  </head>
8.
9.  <body background="<s:url value="/ressources/standard.jpg"/>">
10.    <h2><s:text name="Confirmation.message"/></h2>
11.    <table border="1">
12.      <tr>
13.        <th><s:text name="Confirmation.champ"/></th>
14.        <th><s:text name="Confirmation.valeur"/></th>
15.      </tr>
16.      <tr>
17.        <td><s:text name="date1.prompt"/></td>
18.        <td><s:text name="date1"/></td>
19.      </tr>
20.      <tr>
21.        <td><s:text name="date2.prompt"/></td>
22.        <td>
23.          <s:text name="date.format">
24.            <s:param value="date2"/>
25.          </s:text>
26.        </td>
27.      </tr>
28.      <tr>
29.        <td><s:text name="date3.prompt"/></td>
30.        <td>
31.          <s:text name="date.format">
32.            <s:param value="date3"/>
33.          </s:text>
34.        </td>
35.      </tr>
36.      <tr>
37.        <td><s:text name="date4.prompt"/></td>
38.        <td>
39.          <s:text name="date.format">
40.            <s:param value="date4"/>
41.          </s:text>
42.        </td>
43.      </tr>
44.      <tr>
45.        <td><s:text name="date5.prompt"/></td>
46.        <td>
47.          <s:text name="date.format">
48.            <s:param value="date5"/>
49.          </s:text>
50.        </td>
51.      </tr>
52.      <tr>
53.        <td><s:text name="date6.prompt"/></td>
54.        <td><s:text name="date6"/></td>
55.      </tr>
56.    </table>
57.    <br/>
58.    <s:url id="URL" action="FormDate!input"/>
59.    <s:a href="%{URL}"><s:text name="Confirmation.lien"/></s:a>
60.  </body>
61. </html>

```

La vue [ConfirmationFormDate.JSP] se contente d'afficher le modèle [FormDateModel]. Les lignes 47-49 montrent l'affichage d'une date. On veut que cet affichage tienne compte d'un format de date. Pour cela, on utilise la balise `<s:text>` qu'on avait utilisée jusque là pour l'internationalisation de l'application.

Ici, la clé de message utilisée est `date.format`. Cette clé est trouvée dans le fichier [FormDateModel.properties] :

```
|date.format={0,date,dd/MM/yyyy}
```

La valeur associée à la clé n'est pas ici un message, mais un format d'affichage :

- **0** est un paramètre représentant la valeur à afficher. Ici ce sera le champ `date5` du modèle.
- **date** représente une date. Précédemment, on avait *number* pour un nombre.
- **dd/MM/yyyy** représente le format de la date. Ici, on le veut sous la forme jj/mm/aaaa. Le format Java qui convient est dd/MM/yyyy (d : day, M : Month, y : year)

La balise

```

1. <s:text name="date.format">
2.     <s:param value="date5"/>
3. </s:text>

```

fait afficher le message {0, date, dd/MM/yyyy} où le paramètre *date5* (ligne 2) viendra remplacer le paramètre 0 du format. Ainsi, le modèle *date5* sera affiché au format jj/mm/aaaa.

14.6 Le modèle [FormDateModel]

Les champs *date1* à *date6* du formulaire [FormDate.JSP] sont injectés dans le modèle [FormDateModel] suivant :

```

1. package example;
2.
3. import java.util.Date;
4.
5. public class FormDateModel {
6.
7.     // constructeur sans paramètre
8.     public FormDateModel() {
9.     }
10.    // champs
11.    private String date1;
12.    private Date date2 = new Date();
13.    private Date date3 = new Date();
14.    private Date date4;
15.    private Date date5;
16.    private String date6;
17.
18.    // raz modèle
19.    public void clearModel() {
20.        date1 = null;
21.        date2 = null;
22.        date3 = null;
23.        date4 = null;
24.        date5 = null;
25.        date6 = null;
26.    }
27.
28.    // getters et setters
29.    ...
30. }

```

- les champs *date1* et *date6* sont de type *String*
- les autres champs sont de type *Date*

14.7 La validation du modèle

La validation du modèle est contrôlée par deux fichiers : [FormDate-validation.xml] et [FormDateModel-validation.xml].

Le fichier [FormDate-validation.xml] délègue les validations à [FormDateModel-validation.xml] :

```

1. <!--
2. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
3. EN" "http://www.opensymphony.com/xwork/xwork-validator-1.0.2.dtd">
4. -->
5. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
6. EN" "http://localhost:8084/exemple-10/example/xwork-validator-1.0.2.dtd">
7.
8. <validators>
9.     <field name="model" >
10.         <field-validator type="visitor">
11.             <param name="appendPrefix">false</param>
12.             <message/>
13.         </field-validator>
14.     </field>
15. </validators>

```

Nous avons déjà rencontré ce fichier.

Le fichier [FormDateModel-validation.xml] contient les règles de validation suivantes :

```
1. <!--
2. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
3. EN" "http://www.opensymphony.com/xwork/xwork-validator-1.0.2.dtd">
4. -->
5.
6. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
7. EN" "http://localhost:8084/exemple-11/example/xwork-validator-1.0.2.dtd">
8.
9. <validators>
10.
11.   <field name="date1" >
12.     <field-validator type="requiredstring" short-circuit="true">
13.       <message key="date1.error"/>
14.     </field-validator>
15.     <field-validator type="regex" short-circuit="true">
16.       <param name="expression">^\d{2}/\d{2}/\d{4}$</param>
17.       <param name="trim">true</param>
18.       <message key="date1.error"/>
19.     </field-validator>
20.   </field>
21.
22.   <field name="date2" >
23.     <field-validator type="required" short-circuit="true">
24.       <message key="date2.error"/>
25.     </field-validator>
26.     <field-validator type="conversion" short-circuit="true">
27.       <message key="date2.error"/>
28.     </field-validator>
29.   </field>
30.
31.   <field name="date3" >
32.     <field-validator type="required" short-circuit="true">
33.       <message key="date2.error"/>
34.     </field-validator>
35.     <field-validator type="conversion" short-circuit="true">
36.       <message key="date3.error"/>
37.     </field-validator>
38.     <field-validator type="date" short-circuit="true">
39.       <param name="min">18/05/2000</param>
40.       <message key="date3.error"/>
41.     </field-validator>
42.   </field>
43.
44.   <field name="date4" >
45.     <field-validator type="required" short-circuit="true">
46.       <message key="date2.error"/>
47.     </field-validator>
48.     <field-validator type="conversion" short-circuit="true">
49.       <message key="date4.error"/>
50.     </field-validator>
51.     <field-validator type="date" short-circuit="true">
52.       <param name="max">20/06/2001</param>
53.       <message key="date4.error"/>
54.     </field-validator>
55.   </field>
56.
57.   <field name="date5" >
58.     <field-validator type="required" short-circuit="true">
59.       <message key="date2.error"/>
60.     </field-validator>
61.     <field-validator type="conversion" short-circuit="true">
62.       <message key="date5.error"/>
63.     </field-validator>
64.     <field-validator type="date" short-circuit="true">
65.       <param name="min">18/05/2000</param>
66.       <param name="max">20/06/2001</param>
67.       <message key="date5.error"/>
68.     </field-validator>
69.   </field>
70.
71.   <field name="date6" >
72.     <field-validator type="requiredstring" short-circuit="true">
73.       <message key="date6.error"/>
74.     </field-validator>
75.     <field-validator type="regex" short-circuit="true">
```



```

76.     <param name="expression">^\d{2}/\d{2}/\d{4}$</param>
77.     <param name="trim">true</param>
78.     <message key="date6.error"/>
79. </field-validator>
80. </field>
81.
82. </validators>

```

- la règle des lignes 11-20 vérifie que le champ de saisie *date1* suit le modèle d'une expression régulière représentant une date de la forme jj/mm/aaaa. On notera qu'on vérifie que la chaîne a la forme d'une date mais qu'on ne vérifie pas que c'est une date valide. Ainsi 29/02/2011 a la forme d'une date mais n'est pas une date valide.
- la règle des lignes 22-29 vérifie que le champ de saisie *date2* est une date valide.
- la règle des lignes 31-42 vérifie que le champ de saisie *date3* est une date valide $\geq 18/05/2000$
- la règle des lignes 44-55 vérifie que le champ de saisie *date4* est une date valide $\leq 20/06/2001$.
- la règle des lignes 57-69 vérifie que le champ de saisie *date5* est une date valide $\leq 20/06/2001$ et $\geq 18/05/2000$.
- la règle des lignes 71-80 vérifie que le champ *date6* a la forme d'une date jj/mm/aaaa.

Une fois le fichier [FormDateModel-validation.xml] exploité par l'intercepteur de validation, celui-ci fait exécuter la méthode *validate* de l'action [FormDate] si elle existe. Nous allons la présenter en même temps que la totalité de l'action.

14.8 L'action [FormDate]

L'action [FormDate] est la suivante :

```

1. package example;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4. import com.opensymphony.xwork2.ModelDriven;
5. import java.text.SimpleDateFormat;
6. import java.util.Date;
7. import java.util.Map;
8. import org.apache.struts2.interceptor.SessionAware;
9. import org.apache.struts2.interceptor.validation.SkipValidation;
10.
11. public class FormDate extends ActionSupport implements ModelDriven, SessionAware {
12.
13.     // constructeur sans paramètre
14.     public FormDate() {
15.     }
16.
17.     // modèle de l'action
18.     public Object getModel() {
19.         if (session.get("model") == null) {
20.             session.put("model", new FormDateModel());
21.         }
22.         return session.get("model");
23.     }
24.
25.     @SkipValidation
26.     public String clearModel() {
27.         // raz du modèle
28.         ((FormDateModel) getModel()).clearModel();
29.         // résultat
30.         return INPUT;
31.     }
32.
33.     public String cancel() {
34.         // on nettoie le modèle
35.         ((FormDateModel) getModel()).clearModel();
36.         // résultat
37.         return "cancel";
38.     }
39.     // SessionAware
40.     Map<String, Object> session;
41.
42.     public void setSession(Map<String, Object> session) {
43.         this.session = session;
44.     }
45.
46.     // validation
47.     @Override

```

```

48. public void validate() {
49.     // formatage des dates
50.     SimpleDateFormat formateurDate = new SimpleDateFormat("dd/MM/yyyy");
51.     formateurDate.setLenient(false);
52.     // saisie date1 valide ?
53.     if (getFieldErrors().get("date1") == null) {
54.         // vérification validité date
55.         try {
56.             formateurDate.parse(((FormDateModel) getModel()).getDate1());
57.         } catch (Exception e) {
58.             addFieldError("date1", getText("date1.error"));
59.         }
60.     }
61.     // saisie date6 valide ?
62.     if (getFieldErrors().get("date6") == null) {
63.         Date d = null;
64.         try {
65.             // vérification validité date
66.             d = formateurDate.parse(((FormDateModel) getModel()).getDate6());
67.             // vérification bornes
68.             if (d.after(formateurDate.parse("20/06/2001")) ||
69.                 d.before(formateurDate.parse("18/05/2000"))) {
70.                 addFieldError("date6", getText("date6.error"));
71.                 return;
72.             }
73.         } catch (Exception e) {
74.             addFieldError("date6", getText("date6.error"));
75.             return;
76.         }
77.     }
78. }

```

L'action [FormDate] est bâtie sur le même modèle que l'action [FormInt]. Nous ne commenterons que la méthode *validate*. On rappelle que la méthode *validate* est exécutée après exploitation du fichier de validation [FormDateModel-validation.xml] et avant l'exécution de la méthode *execute*.

- la méthode *validate* complète la validation des dates *date1* et *date6*. Le fichier de validation [FormDateModel-validation.xml] avait vérifié que les chaînes saisies avaient la forme d'une date. On vérifie maintenant que ce sont bien des dates valides. On rappelle également que *date1* et *date6* étaient les deux seuls champs du modèle à être de type *String*.
- ligne 50 : on crée un format de date jj/mm/aaaa afin de vérifier que les chaînes de ce type sont bien des dates valides.
- ligne 51 : *setLenient(false)* permet d'éviter que la date invalide 32/01/2012 soit interprétée comme la date valide 01/02/2012.
- ligne 53 : on ne valide *date1* que si les précédentes validations n'ont pas eu d'erreurs sur ce champ.
- ligne 56 : on vérifie que *date1* est une date valide au format jj/mm/aaaa.
- ligne 58 : si ce n'est pas le cas, on associe un message d'erreur au champ *date1*.
- ligne 62 : on ne valide *date6* que si ce champ a passé les précédentes validations.
- ligne 66 : on vérifie que *date6* est une date valide
- ligne 68 : si *date6* est <18/05/2000 ou *date6* > 20/06/2001 alors *date6* est incorrect.

14.9 Derniers détails

Le formulaire précédent présente des failles comme le montre l'exemple suivant :

Struts 2 - Conversion et validation de dates

Tapez les dates au format JJ/MM/AAAA comme dans 12/10/2008

1-Tapez une date au format JJ/MM/AAAA: 13/10/1954
2-Tapez une date au format JJ/MM/AAAA: 30/12/2011xxxxx **1**
3-Tapez une date >=18/05/2000: 30/12/2011
4-Tapez une date <=20/06/2001: 20/06/2001
5-Tapez une date dans l'intervalle [18/05/2000,20/06/2001]: 20/06/2001
6-Tapez une date dans l'intervalle [18/05/2000,20/06/2001]: 28/02/2001

[Annuler](#)
[Raz modèle](#)

Confirmation des valeurs saisies

champ	valeur
1-Tapez une date au format JJ/MM/AAAA	13/10/1954
2-Tapez une date au format JJ/MM/AAAA	30/12/2011
3-Tapez une date >=18/05/2000	30/12/2011
4-Tapez une date <=20/06/2001	20/06/2001
5-Tapez une date dans l'intervalle [18/05/2000,20/06/2001]	20/06/2001
6-Tapez une date dans l'intervalle [18/05/2000,20/06/2001]	28/02/2001

[Formulaire de test](#)

- en [1], on rentre une date invalide
- en [2], elle a été acceptée. Seuls les premiers caractères jj/mm/aaaa ont été pris en compte.

Les champs *date2* à *date5* qui sont liés à des modèles de type *Date* ont tous ce problème. Là encore, peut-être ai-je raté quelque chose dans la documentation. Les champs *date1* et *date6* qui sont liés à des modèles de type *String* n'ont pas ce problème.

15 Exemple 12 – Conversions et validations diverses

La nouvelle application présente la saisie de divers éléments disposant de validateurs Struts :

Struts 2 - Conversions et validations de types divers

1-Tapez une adresse électronique (x@y.z): serge.tahe@nowhere.com
2-Tapez une url (http://www.ibm.com): http://istia.univ-angers.fr
3-Tapez une chaîne de 5 caractères: 12345

[Annuler](#)
[Raz modèle](#)

1

Confirmation des valeurs saisies

champ	valeur
1-Tapez une adresse électronique (x@y.z)	serge.tahe@nowhere.com
2-Tapez une url (http://www.ibm.com)	http://istia.univ-angers.fr
3-Tapez une chaîne de 5 caractères	12345

[Formulaire de test](#)

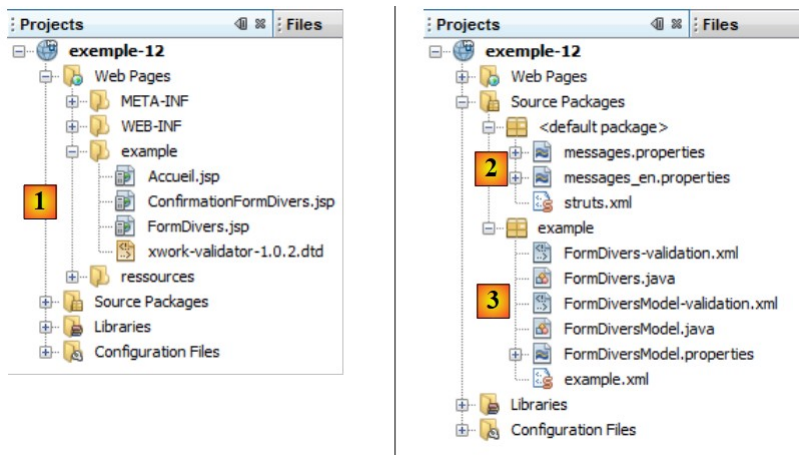
2

- en [1], le formulaire de saisie
- en [2], la confirmation des saisies

L'application a un fonctionnement similaire aux précédentes aussi ne commenterons-nous que les points qui diffèrent.

15.1 Le projet Netbeans

Le projet Netbeans est le suivant :



- en [1], les vues de l'application
 - [Accueil.JSP] : la page d'accueil
 - [FormDivers.JSP] : le formulaire de saisie
 - [ConfirmationFormDivers.JSP] : la page de confirmation
- en [2], le fichier des messages [messages.properties] et le fichier de configuration principal de Struts
- en [3] :
 - [FormDivers.java] : l'action qui affiche et traite le formulaire
 - [FormDivers-validation.xml] : les règles de validation de l'action [FormDivers]. Ce fichier délègue ces validations au modèle.
 - [FormDiversModel] : le modèle de l'action [FormDivers]
 - [FormDiversModel-validation.xml] : les règles de validation du modèle
 - [FormDiversModel.properties] : le fichier des messages du modèle
 - [example.xml] : fichier de configuration secondaire de Struts

15.2 La configuration du projet

Le projet est principalement configuré par le fichier [example.xml] suivant :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.   <package name="example" namespace="/example" extends="struts-default">
8.     <action name="Accueil">
9.       <result name="success"/>example/Accueil.JSP</result>
10.    </action>
11.    <action name="FormDivers" class="example.FormDivers">
12.      <result name="input"/>example/FormDivers.JSP</result>
13.      <result name="cancel" type="redirect"/>example/Accueil.JSP</result>
14.      <result name="success"/>example/ConfirmationFormDivers.JSP</result>
15.    </action>
16.  </package>
17. </struts>

```

Il est analogue à celui qui a été étudié dans les version précédentes.

15.3 Les fichiers des messages

Le fichier [messages.properties] est le suivant :

```

1. Accueil.titre=Accueil
2. Accueil.message=Struts 2 - Conversions et validations
3. Accueil.FormDivers=Saisies diverses (email, URL, chaine de caract\u00e8res avec contr\u00f4le du
   nombre de caract\u00e8res)
4. Form.titre=Conversions et validations
5. FormDivers.message=Struts 2 - Conversions et validations de types divers

```

```

6. Form.submitText=Valider
7. Form.cancelText=Annuler
8. Form.clearModel=Raz mod\u00e8le
9. Confirmation.titre=Confirmation
10. Confirmation.message=Confirmation des valeurs saisies
11. Confirmation.champ=champ
12. Confirmation.valeur=valeur
13. Confirmation.lien=Formulaire de test
14. xwork.default.invalid.fieldvalue=Valeur invalide pour le champ "{0}".

```

Le fichier [FormDiversModel.properties] est le suivant :

```

1. email.prompt=1-Tapez une adresse \u00E9lectronique (x@y.z)
2. email.error=Format invalide
3. URL.prompt=2-Tapez une URL (http://www.ibm.com)
4. URL.error=Format invalide
5. chaine.prompt=3-Tapez une chaine de 5 caract\u00E8res
6. chaine.error=Format invalide

```

15.4 Le formulaire de saisie

La vue [FormDivers.JSP] est la suivante :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
4.   <head>
5.     <title><s:text name="Form.titre"/></title>
6.     <s:head/>
7.   </head>
8.
9.   <body background="<s:url value="/ressources/standard.jpg"/>">
10.    <h2><s:text name="FormDivers.message"/></h2>
11.    <s:form name="formulaire" action="FormDivers">
12.      <s:textfield name="email" key="email.prompt" size="30"/>
13.      <s:textfield name="URL1" key="URL.prompt" size="30"/>
14.      <s:textfield name="chaine" key="chaine.prompt" size="10"/>
15.      <s:submit key="Form.submitText" method="execute"/>
16.    </s:form>
17.    <br/>
18.    <s:url id="URL" action="FormDivers" method="cancel"/>
19.    <s:a href="%{URL}"><s:text name="Form.cancelText"/></s:a>
20.    <br/>
21.    <s:url id="URL" action="FormDivers" method="clearModel"/>
22.    <s:a href="%{URL}"><s:text name="Form.clearModel"/></s:a>
23.  </body>
24. </html>

```

Les lignes 12 à 14 sont les trois zones de saisie.

15.5 La vue de confirmation

La vue de confirmation [ConfirmationFormDivers.JSP] est la suivante :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
4.   <head>
5.     <title><s:text name="Confirmation.titre"/></title>
6.     <s:head/>
7.   </head>
8.
9.   <body background="<s:url value="/ressources/standard.jpg"/>">
10.    <h2><s:text name="Confirmation.message"/></h2>
11.    <table border="1">
12.      <tr>
13.        <th><s:text name="Confirmation.champ"/></th>
14.        <th><s:text name="Confirmation.valeur"/></th>
15.      </tr>
16.      <tr>

```

```

17.     <td><s:text name="email.prompt"/></td>
18.     <td><s:text name="email"/></td>
19. </tr>
20. <tr>
21.     <td><s:text name="URL.prompt"/></td>
22.     <td><s:text name="URL1"/></td>
23. </tr>
24. <tr>
25.     <td><s:text name="chaine.prompt"/></td>
26.     <td><s:text name="chaine"/></td>
27. </tr>
28. </table>
29. <br/>
30. <s:url id="URL" action="FormDivers!input"/>
31. <s:a href="%{URL}"><s:text name="Confirmation.lien"/></s:a>
32. </body>
33. </html>

```

15.6 Le modèle [FormDiversModel]

Les champs de saisie du formulaire [FormDivers.JSP] sont injectés dans le modèle [FormDiversModel] suivant :

```

1. package example;
2.
3. public class FormDiversModel {
4.
5.     // constructeur sans paramètre
6.     public FormDiversModel() {
7.     }
8.
9.     // champs
10.    private String email;
11.    private String URL1 ;
12.    private String chaine;
13.
14.    // raz modèle
15.    public void clearModel() {
16.        email = null;
17.        URL1 = null;
18.        chaine = null;
19.    }
20.
21.    // getters et setters
22.    ...
23. }

```

15.7 La validation du modèle

La validation du modèle est contrôlée par deux fichiers : [FormDivers-validation.xml] et [FormDiversModel-validation.xml].

Le fichier [FormDivers-validation.xml] délègue les validations au fichier [FormDiversModel-validation.xml] suivant :

```

1. <!--
2. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
3. EN" "http://www.opensymphony.com/xwork/xwork-validator-1.0.2.dtd">
4. -->
5. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
6. EN" "http://localhost:8084/exemple-10/example/xwork-validator-1.0.2.dtd">
7.
8. <validators>
9.     <field name="model" >
10.         <field-validator type="visitor">
11.             <param name="appendPrefix">false</param>
12.             <message/>
13.         </field-validator>
14.     </field>
15. </validators>

```

Nous avons déjà rencontré ce fichier.

Le fichier [FormDateModel-validation.xml] contient les règles de validation suivantes :

```
1. <!--
2. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
3. EN" "http://www.opensymphony.com/xwork/xwork-validator-1.0.2.dtd">
4. -->
5.
6. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//
7. EN" "http://localhost:8084/exemple-12/example/xwork-validator-1.0.2.dtd">
8.
9. <validators>
10.
11.   <field name="email" >
12.     <field-validator type="requiredstring" short-circuit="true">
13.       <message key="email.error"/>
14.     </field-validator>
15.     <field-validator type="email" short-circuit="true">
16.       <message key="email.error"/>
17.     </field-validator>
18.   </field>
19.
20.   <field name="URL1" >
21.     <field-validator type="requiredstring" short-circuit="true">
22.       <message key="URL.error"/>
23.     </field-validator>
24.     <field-validator type="URL" short-circuit="true">
25.       <message key="URL.error"/>
26.     </field-validator>
27.   </field>
28.
29.   <field name="chaine" >
30.     <field-validator type="requiredstring" short-circuit="true">
31.       <message key="chaine.error"/>
32.     </field-validator>
33.     <field-validator type="stringlength" short-circuit="true">
34.       <param name="minLength">5</param>
35.       <param name="maxLength">5</param>
36.       <message key="chaine.error"/>
37.     </field-validator>
38.   </field>
39.
40. </validators>
```

- lignes 11-18 : vérifient la validité du champ *email*
- lignes 15-17 : vérifient que la chaîne *email* est une adresse électronique valide
- lignes 20-27 : vérifient la validité du champ *URL1*
- lignes 24-26 : vérifient que la chaîne *URL1* est une URL valide
- lignes 29-38 : vérifient la validité du champ *chaine*
- lignes 33-37 : vérifient que la chaîne *chaine* a 5 caractères exactement.

15.8 L'action [FormDivers]

L'action [FormDivers] est la suivante :

```
1. package example;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4. import com.opensymphony.xwork2.ModelDriven;
5. import java.util.Map;
6. import org.apache.struts2.interceptor.SessionAware;
7. import org.apache.struts2.interceptor.validation.SkipValidation;
8.
9. public class FormDivers extends ActionSupport implements ModelDriven, SessionAware {
10.
11.   // constructeur sans paramètre
12.   public FormDivers() {
13.   }
14.
15.   // modèle de l'action
16.   public Object getModel() {
17.     if (session.get("model") == null) {
18.       session.put("model", new FormDiversModel());
19.     }
20.   }
21. }
```



```

20.     return session.get("model");
21. }
22.
23. @SkipValidation
24. public String clearModel() {
25.     // raz du modèle
26.     ((FormDiversModel) getModel()).clearModel();
27.     // résultat
28.     return INPUT;
29. }
30.
31. public String cancel() {
32.     // on nettoie le modèle
33.     ((FormDiversModel) getModel()).clearModel();
34.     // résultat
35.     return "cancel";
36. }
37.
38. // SessionAware
39. Map<String, Object> session;
40.
41. public void setSession(Map<String, Object> session) {
42.     this.session = session;
43. }
44. }

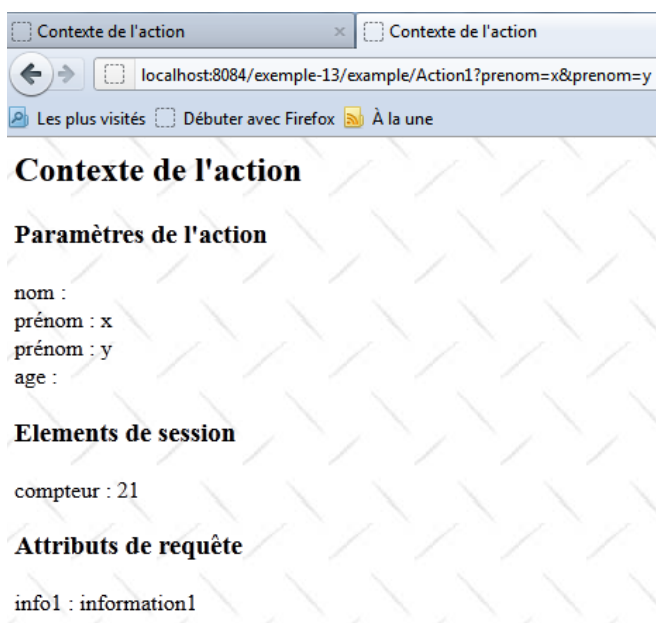
```

L'action [FormDivers] est bâtie sur le même modèle que les actions étudiées précédemment. Ici, il n'y a simplement pas de méthode *validate* pour compléter la validation faite par le fichier [FormDiversModel-validation.xml].

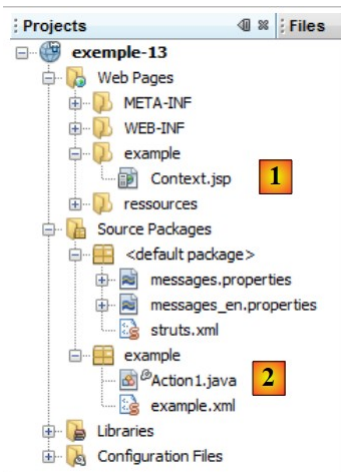
16 Exemple 13 - le contexte d'une action

Cette application vise à montrer qu'une action a accès :

- aux paramètres de la requête
- aux attributs de la requête
- aux attributs de la session de l'utilisateur



16.1 Le projet Netbeans



Le projet Netbeans est le suivant :

- en [1], la vue [Context.JSP]
- en [2], l'action [Action1.java] et le fichier de configuration Struts [example.xml]

16.2 Configuration

La configuration du projet est faite dans [example.xml] :

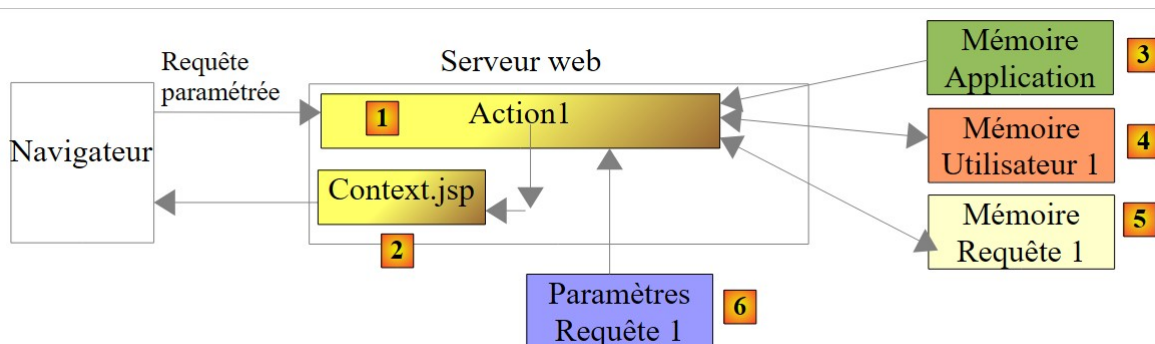
```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.   <package name="example" namespace="/example" extends="struts-default">
8.     <action name="Action1" class="example.Action1">
9.       <result name="success"/>/example/Context.JSP</result>
10.    </action>
11.  </package>
12.</struts>

```

- ligne 8 : la demande de l'URL [/example/Action1] va provoquer l'instanciation de la classe [example.Action]. Comme aucune méthode n'est précisée, ce sera la méthode *execute* qui sera exécutée.
- ligne 9 : une seule clé est acceptée. La clé *success* conduit à l'affichage de la vue [Context.JSP].

L'architecture simplifiée de traitement d'une requête va être la suivante :



La requête va être traitée par deux éléments de l'application web : l'action [Action1] [1] et la vue [Context.JSP] [2]. Ces deux éléments ont accès à des données de différentes natures :

- des données de portée *Application* [3], c.a.d. des données accessibles à toutes les requêtes de tous les utilisateurs. Elles sont quasiment tout le temps en lecture seule. On trouve souvent dans ces données, la configuration initiale de l'application. Ici [Action1] et [Context.JSP] ont accès à ces données.
- des données de portée *Session* [4], c.a.d. des données accessibles à toutes les requêtes d'un même utilisateur. Elles sont en lecture / écriture. Ici [Action1] utilisera la session en lecture / écriture, alors que [Context.JSP] l'utilisera en lecture.
- des données de portée *Requête* [5], accessibles à tous les éléments qui traitent la requête. Ici [Action1] mettra une donnée dans cette mémoire et [Context.JSP] la récupèrera. Les données de portée Requête permettent à un élément N de transmettre de l'information à l'élément N+1.
- les paramètres de la requête [6] envoyés par le client. Ils sont utilisés en lecture seule par les éléments qui traitent la requête.

16.3 L'action [Action1]

Le code de la classe [Action1] est la suivante :

```

1. package example;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4. import java.util.Map;
5. import java.util.Set;
6. import org.apache.struts2.interceptor.ParameterAware;
7. import org.apache.struts2.interceptor.RequestAware;
8. import org.apache.struts2.interceptor.SessionAware;
9.
10. public class Action1 extends ActionSupport implements SessionAware, RequestAware, ParameterAware {
11.
12.     // constructeur sans paramètre
13.     public Action1() {
14.     }
15.     // Session, Request, Parametres
16.     Map<String, Object> session;
17.     Map<String, Object> request;
18.     Map<String, String[]> parameters;
19.
20.     @Override
21.     public String execute() {
22.         // liste des paramètres
23.         System.out.println("Paramètres...");
24.         Set<String> clés = parameters.keySet();
25.         for (String clé : clés) {
26.             for (String valeur : parameters.get(clé)) {
27.                 System.out.println(String.format("[%s,%s]", clé, valeur));
28.             }
29.         }
30.         // session
31.         System.out.println("Session...");
32.         if (session.get("compteur") == null) {
33.             session.put("compteur", new Integer(0));
34.         }
35.         Integer compteur = (Integer) session.get("compteur");
36.         compteur = compteur + 1;
37.         session.put("compteur", compteur);
38.         System.out.println(String.format("compteur=%s", compteur));
39.         // requête
40.         request.put("info1", "information1");
41.         // affichage page JSP
42.         return SUCCESS;
43.     }
44.
45.     // session
46.     public void setSession(Map<String, Object> session) {
47.         this.session = session;
48.     }
49.
50.     // requête
51.     public void setRequest(Map<String, Object> request) {
52.         this.request = request;
53.     }
54.
55.     // paramètres
56.     public void setParameters(Map<String, String[]> parameters) {
57.         this.parameters = parameters;

```

```
58. }  
59. }
```

- ligne 10 : la classe implémente les interfaces suivantes
 - **SessionAware** : pour avoir accès au dictionnaire des attributs de la session (ligne 16). Cette interface n'a qu'une méthode, celle de la ligne 46.
 - **RequestAware** : pour avoir accès au dictionnaire des attributs de la requête (ligne 17). Cette interface n'a qu'une méthode, celle de la ligne 51.
 - **ParameterAware** : pour avoir accès au dictionnaire des paramètres de la requête (ligne 18). On remarquera qu'à une clé (le nom du paramètre) correspond un tableau de valeurs. Ceci est nécessaire pour prendre en compte les zones de saisie qui postent plusieurs valeurs comme par exemple une liste à sélection multiple. L'interface **ParameterAware** n'a qu'une méthode, celle de la ligne 56.
- ligne 21 : la méthode *execute* qui est exécutée lorsqu'on demande l'action [Action1]. Lorsqu'elle s'exécute, les intercepteurs ont fait leur travail :
 - la méthode **setParameters** (ligne 56) a été appelée et le dictionnaire **parameters** de la ligne 18 contient tous les paramètres de la requête.
 - la méthode **setSession** (ligne 46) a été appelée et le dictionnaire **session** de la ligne 16 contient tous les attributs de la session.
 - la méthode **setRequest** (ligne 51) a été appelée et le dictionnaire **request** de la ligne 17 contient tous les attributs de la requête.
- lignes 31-38 : on écrit la valeur associée à la clé *compteur* dans la session
- lignes 32-34 : la clé *compteur* est cherchée dans la session. Si elle ne s'y trouve pas, on l'y met associée à la valeur entière 0.
- lignes 35-37 : la clé *compteur* est cherchée dans la session, sa valeur est incrémentée puis la clé est remise dans la session.
- ligne 38 : la valeur associée à la clé *compteur* est affichée. L'incrément étant fait à chaque requête sur l'action [Action1], on devrait voir la valeur du compteur augmenter au fil des requêtes.
- ligne 40 : on insère dans le dictionnaire des attributs de la requête un attribut de clé *info1* et de valeur *information1*. Les attributs d'une requête sont différents de ses paramètres. Les paramètres sont envoyés par le client de l'application web. Les attributs de la requête eux permettent la communication entre les différents éléments de l'application web qui la traitent. Ainsi, après l'exécution de [Action1], la vue [Context.JSP] va être affichée. Nous allons voir qu'elle est capable de récupérer les attributs de la requête.
- ligne 42 : la méthode *execute* rend la clé *succes*.

16.4 Le fichier des messages

Le fichier [messages.properties] est le suivant :

```
1. Context.titre=Contexte de l'action  
2. Context.message=Contexte de l'action  
3. Context.parameters=Paramètres de l'action  
4. Context.session=Elements de session  
5. Context.request=Attributs de requête
```

16.5 La vue [Context.JSP]

La vue [Context.JSP] a pour rôle d'afficher :

- certains paramètres de la requête
- la valeur de la clé *compteur* dans la session
- la valeur de la clé *info1* dans la requête

Son code est le suivant :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>  
2. <%@ taglib prefix="s" uri="/struts-tags" %>  
3. <html>  
4.   <head>  
5.     <title><s:text name="Context.titre"/></title>  
6.     <s:head/>  
7.   </head>  
8.  
9.   <body background="<s:url value="/ressources/standard.jpg"/>">  
10.    <h2><s:text name="Context.message"/></h2>  
11.    <h3><s:text name="Context.parameters"/></h3>  
12.    <s:iterator value="#parameters['nom']" var="nom">
```

```

13.     nom : <s:property value="nom"/><br/>
14.     </s:iterator>
15.     <s:iterator value="#parameters['prenom']" var="prenom">
16.         prenom : <s:property value="prenom"/><br/>
17.     </s:iterator>
18.     <s:iterator value="#parameters['age']" var="age">
19.         âge : <s:property value="age"/><br/>
20.     </s:iterator>
21.     <h3><s:text name="Context.session"/></h3>
22.     compteur : <s:property value="#session['compteur']"/>
23.     <h3><s:text name="Context.request"/></h3>
24.     info1 : <s:property value="#request['info1']"/>
25. </body>
26. </html>

```

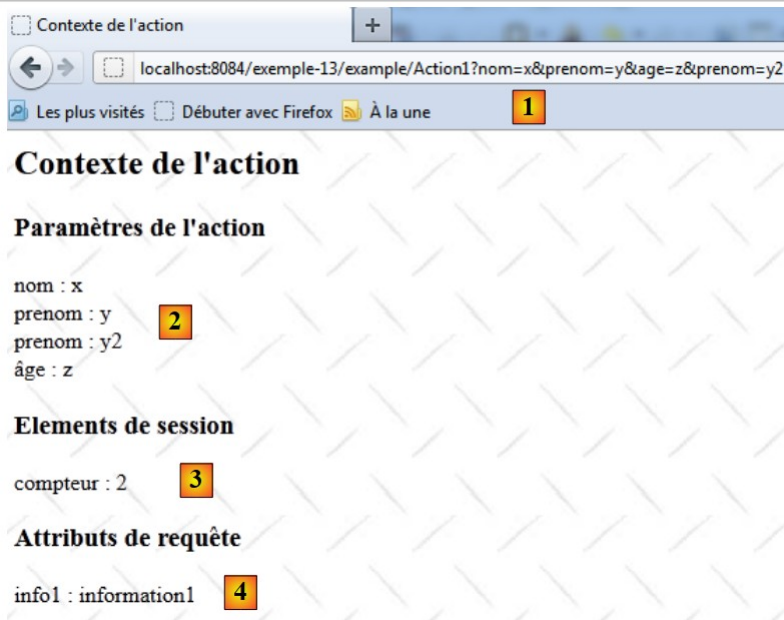
- lignes 12-14 : affichent toutes les valeurs associées au paramètre *nom*
- lignes 15-17 : affichent toutes les valeurs associées au paramètre *prenom*
- lignes 18-20 : affichent toutes les valeurs associées au paramètre *age*
- ligne 22 : affiche la valeur associée à la clé *compteur* dans la session
- ligne 24 : affiche la valeur associée à la clé *info1* dans la requête

16.6 Les tests



- en [1], *Action1* est demandée sans paramètres
- en [2], [Context.JSP] n'a pas trouvé de paramètres
- en [3], [Context.JSP] a trouvé la clé *compteur* dans la session
- en [4], [Context.JSP] a trouvé la clé *info1* dans la requête

Faisons un autre test :



- en [1], *Action1* est demandée avec des paramètres
- en [2], [Context.JSP] affiche ces paramètres
- en [3], [Context.JSP] a trouvé la clé *compteur* dans la session. Le compteur a bien été incrémenté de 1 montrant par là qu'il y bien eu mémorisation entre les deux requêtes.
- en [4], [Context.JSP] a trouvé la clé *info1* dans la requête

On se rappelle que la méthode [Action1.execute] écrivait sur la console du serveur web. Voilà un exemple :

```
1. Paramètres...
2. [prenom,y]
3. [prenom,y2]
4. [age,z]
5. [nom,x]
6. Session...
7. compteur=2
```

16.7 Conclusion

On se rappellera les points suivants :

- pour mémoriser des informations à partager par toutes les requêtes de tous les utilisateurs, on utilisera la mémoire de l'application. Nous allons en montrer un exemple bientôt.
- pour mémoriser des informations à partager par toutes les requêtes d'un même utilisateur, on utilisera la session de celui-ci.
- pour mémoriser des informations à partager par tous les éléments traitant une requête, on utilisera la mémoire de la requête.

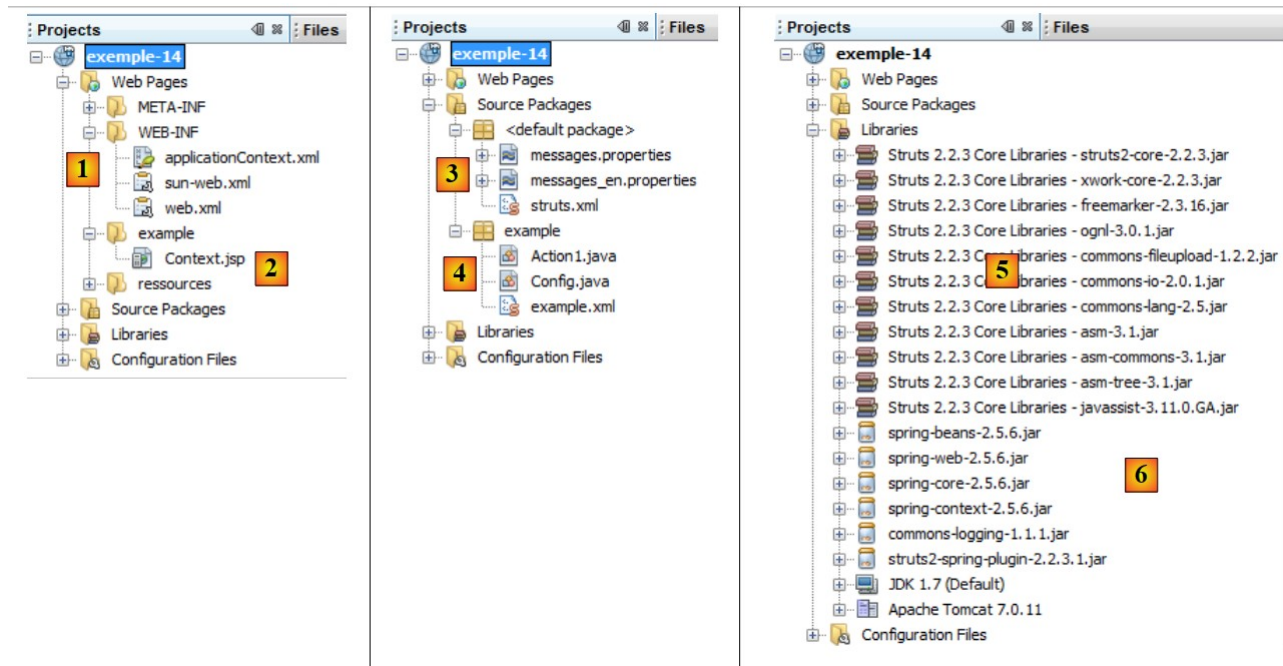
17 Exemple 15 – Intégration Struts 2 / Spring

Dans l'exemple précédent, nous n'avions pas de données de portée *Application* partagées par toutes les requêtes de tous les utilisateurs. Nous en montrons un exemple ici. Pour le mettre en oeuvre, nous utilisons le framework Spring [<http://www.springsource.org/>]. Spring est un outil de très grande valeur. Nous n'en montrons dans cet exemple qu'une infime partie. Un exemple ultérieur l'utilisera plus intensivement.

Le but de cette application est d'afficher des données de portée *Application*.

17.1 Le projet Netbeans

Le projet Netbeans de l'application est le suivant :



- en [1] :
 - [web.xml] qui configure l'application web va évoluer vis à vis de ce qu'il était dans les versions précédentes
 - [applicationContext.xml] est le fichier de configuration de Spring
- en [2] : la vue [Context.jsp] qui affichera les données de portée Application
- en [3] :
 - le fichier des messages [messages.properties]
 - le fichier de configuration principal de Struts [struts.xml]. Va évoluer vis à vis des applications précédentes.
- en [4] :
 - l'action Struts [Action1.java]
 - la classe [Config.java] qui va mémoriser les données de portée application
 - le fichier de configuration secondaire de Struts [example.xml]
- en [5] : la bibliothèque Struts 2
- en [6] :
 - les archives nécessaires à Spring [spring-core, spring-context, spring-beans, spring-web, commons-logging]
 - l'archive du plugin Spring pour Struts 2. Permet l'intégration de Spring avec Struts 2 [struts2-spring-plugin]

Vis à vis des versions précédentes :

- il faut rajouter des archives au projet
- modifier les fichiers [web.xml] et [struts.xml]

17.2 Configuration

17.2.1 Le fichier [web.xml]

Le fichier [web.xml] évolue comme suit :

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <web-app id="WebApp_9" version="2.4" xmlns="http://java.sun.com/xml/ns/j2ee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee http://java.sun.com/xml/ns/j2ee/web-
  app_2_4.xsd">
```



```

3. <display-name>Exemple 14</display-name>
4. <filter>
5.   <filter-name>struts2</filter-name>
6.   <filter-class>org.apache.struts2.dispatcher.FilterDispatcher</filter-class>
7. </filter>
8. <filter-mapping>
9.   <filter-name>struts2</filter-name>
10.  <URL-pattern>/*</URL-pattern>
11. </filter-mapping>
12. <listener>
13.   <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
14. </listener>
15. </web-app>

```

La modification a lieu lignes 12-14. Un *listener* est ajouté à l'application web. Lorsque l'application web va être lancée, la classe implémentant ce *listener* va être instanciée. C'est une classe de Spring. Cette classe va exploiter le fichier [WEB-INF/applicationContext.xml]. Celui-ci est le suivant :

```

1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
3.   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.   xmlns:tx="http://www.springframework.org/schema/tx"
5.   xsi:schemaLocation="http://www.springframework.org/schema/beans
6.     http://www.springframework.org/schema/beans/spring-beans-2.0.xsd
7.     http://www.springframework.org/schema/tx http://www.springframework.org/schema/tx/spring-
8.     tx-2.0.xsd">
9.
10.  <!-- configuration des beans de portée application -->
11.  <bean id="config" class="example.Config" >
12.    <property name="nbMaxUsers" value="10"/>
13.  </bean>
14.
15. </beans>

```

- le fichier de configuration de Spring a la balise racine `<beans>` (lignes 2 et 11). On pourrait traduire *beans* par objets. Spring va instancier tous les objet (*bean*) trouvés dans ce fichier de configuration. Il ne le fait qu'une fois, lorsque le *listener* Spring est lancé au démarrage de l'application web.
- lignes 7-9 : définissent un bean nommé *config* (id). Le bean config est associé à la classe (*class*) [example.Config]. Spring va instancier cette classe.
- ligne 8 : définit une propriété de la classe [example.Config]. Spring va appeler la méthode [example.Config].setNbMaxUsers(10). Il faut donc que la méthode *setNbMaxUsers* existe dans la classe. Celle-ci est la suivante :

```

1. package example;
2.
3. public class Config {
4.
5.   private int nbMaxUsers;
6.
7.   public int getNbMaxUsers() {
8.     return nbMaxUsers;
9.   }
10.
11.   public void setNbMaxUsers(int nbMaxUsers) {
12.     this.nbMaxUsers = nbMaxUsers;
13.   }
14.
15. }

```

Cette classe ne définit qu'un champ *nbMaxUsers* avec ses *get* et *set*. Si on a bien suivi ce qui a été dit, au démarrage de l'application web, une instance de la classe [Config] est instanciée avec la valeur 10 pour son champ *nbMaxUsers*. Nous n'avons défini qu'un unique champ mais c'est suffisant pour notre démonstration. Nous voulons montrer que ce champ, qui pourrait représenter un nombre maximal d'utilisateurs, est une donnée de portée *Application* accessible à toutes les actions. C'est ce que nous montrerons avec l'action [Action1].

17.2.2 Le fichier [struts.xml]

Le fichier de configuration principal de Struts est le suivant :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC

```

```

3.    "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.    "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.   <!-- internationalisation -->
8.   <constant name="struts.custom.i18n.resources" value="messages" />
9.   <!-- intégration Spring -->
10.  <constant name="struts.objectFactory.spring.autoWire" value="name" />
11.
12.  <include file="example/example.xml"/>
13.
14.  <package name="default" namespace="/" extends="struts-default">
15.    <default-action-ref name="index" />
16.    <action name="index">
17.      <result type="redirectAction">
18.        <param name="actionName">Action1</param>
19.        <param name="namespace">/example</param>
20.      </result>
21.    </action>
22.  </package>
23.
24. </struts>

```

Seule la ligne 10 est différente des versions précédentes de ce fichier. Elle définit une constante Struts. Les objets instanciés par Spring peuvent être injectés dans d'autres objets. C'est le fondement même de Spring. Ici une référence sur l'objet de type [Config] créé par Spring va pouvoir être injecté dans des objets Struts. C'est là qu'intervient le plugin Spring pour Struts 2 qui assure l'intégration de Spring par Struts.

La ligne 10 indique que l'injection d'objets Spring dans un objet Struts se fera de façon automatique (*autowire*) par nom (*value*). Revenons au fichier de configuration [applicationContext.xml] :

```

1. <!-- configuration des beans de portée application -->
2. <bean id="config" class="example.Config" >
3.   <property name="nbMaxUsers" value="10"/>
4. </bean>

```

Ce que Struts 2 appelle le nom du bean est en fait ici l'*id*. Donc ici, le bean de type [Config] instancié a pour nom *config*.

L'action [Action1.java] est la suivante :

```

1. package example;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4. ...
5. public class Action1 extends ActionSupport implements SessionAware, RequestAware, ParameterAware {
6.
7.   // constructeur sans paramètre
8.   public Action1() {
9.   }
10.
11.   // Session, Request, Paramètres
12.   private Map<String, Object> session;
13.   private Map<String, Object> request;
14.   private Map<String, String[]> parameters;
15.   private Config config;
16.
17.   @Override
18.   public String execute() {
19.     ....
20.     // requête
21.     request.put("nbMaxUsers", config.getNbMaxUsers());
22.
23.     // affichage page JSP
24.     return SUCCESS;
25.   }
26. ...
27.   public Config getConfig() {
28.     return config;
29.   }
30.
31.   public void setConfig(Config config) {
32.     this.config = config;
33.   }
34. }

```

L'action [Action1] est identique à ce qu'elle était dans l'application précédente aux détails près suivants :

- ligne 15 : un champ *config* a été ajouté. Parce qu'il porte le nom d'un bean instancié par Spring, ce champ recevra automatiquement une référence sur cet objet. Ainsi l'action a accès à la configuration de l'application ou plus généralement aux données de portée *Application*.
- ligne 31 : Springinstanciera le champ *config* via sa méthode *set*. Elle doit donc être présente.
- ligne 21 : l'action accède aux informations de portée *Application*. On met le nombre maximal d'utilisateurs *nbMaxUsers* dans la requête afin que la vue [Context.JSP] puisse l'afficher.

17.2.3 Le fichier [example.xml]

Le fichier secondaire de configuration de Struts est identique à celui de la version précédente :

```
1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.   <package name="example" namespace="/example" extends="struts-default">
8.     <action name="Action1" class="example.Action1">
9.       <result name="success"/>example/Context.JSP</result>
10.    </action>
11.  </package>
12. </struts>
```

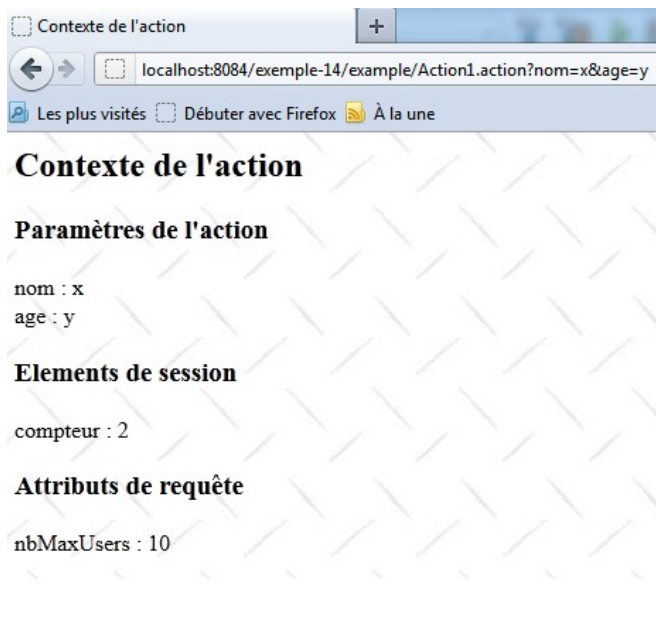
17.3 La vue [Context.JSP]

La vue [Context.JSP] est la suivante :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <html>
4.   <head>
5.     <title><s:text name="Context.titre"/></title>
6.     <s:head/>
7.   </head>
8.
9.   <body background="<s:url value="/ressources/standard.jpg"/>">
10.    <h2><s:text name="Context.message"/></h2>
11.    <h3><s:text name="Context.parameters"/></h3>
12.    nom : <s:property value="#parameters['nom']"/><br/>
13.    age : <s:property value="#parameters['age']"/>
14.    <h3><s:text name="Context.session"/></h3>
15.    compteur : <s:property value="#session['compteur']"/>
16.    <h3><s:text name="Context.request"/></h3>
17.    nbMaxUsers : <s:property value="#request['nbMaxUsers']"/>
18.  </body>
19. </html>
```

17.4 Les tests

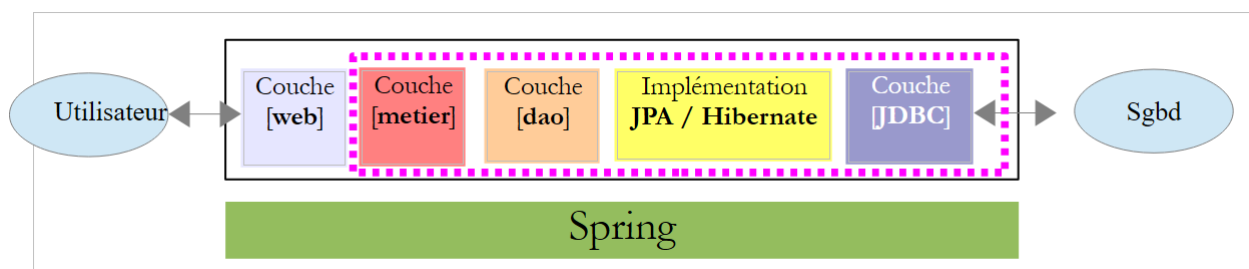
Un exemple d'exécution est le suivant :



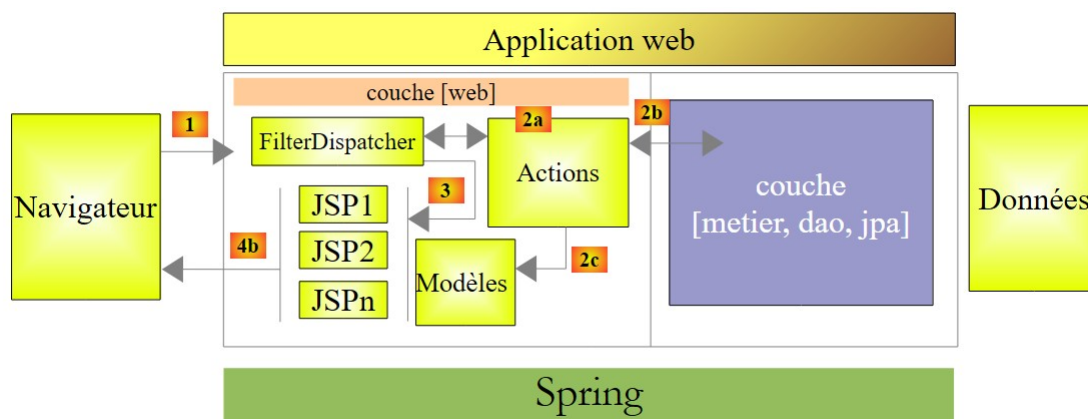
18 Etude de cas : Struts 2 / Tiles / Spring / Hibernate / MySQL

Nous allons terminer notre apprentissage de Struts par une étude de cas. Afin d'être réaliste, l'exemple étudié va être nettement plus complexe que ceux étudiés précédemment. Pour les débutants, il est sans doute préférable d'approfondir les bases de Struts 2 avec des applications personnelles avant d'aborder cette étude de cas.

L'application va utiliser une architecture à couches :



L'ensemble des couches [metier], [dao], [jpa/hibernate] nous sera fourni sous la forme d'une archive jar dont nous détaillerons les fonctionnalités. L'intégration des couches sera assurée par Spring. La couche [web] sera implémentée par Struts 2.



18.1 Le problème

On se propose d'écrire une application web permettant d'établir le bulletin de salaire des assistantes maternelles employées par la "Maison de la petite enfance" d'une commune.

Calcul du salaire des assistantes maternelles

[Faire la simulation](#)
[Effacer la simulation](#)
[Voir les simulations](#)
[Enregistrer la simulation](#)
[Terminer la session](#)

Employé: Marie Jouveinal ▼

Heures travaillées: 150,00

Jours travaillés: 20

Informations Employé

Nom	Prénom	Adresse
Jouveinal	Marie	5 rue des oiseaux
Ville	Code Postal	Indice
St Corentin	49203	2

Cotisations Sociales

Csgd	Csgd	Retraite	Sécu
3,49 %	6,15 %	7,88 %	9,39 %

Indemnités

Salaire horaire	Entretien/Jour	Repas/Jour	Congés payés
2,10 €	2,10 €	3,10 €	15,00 €

Salaire

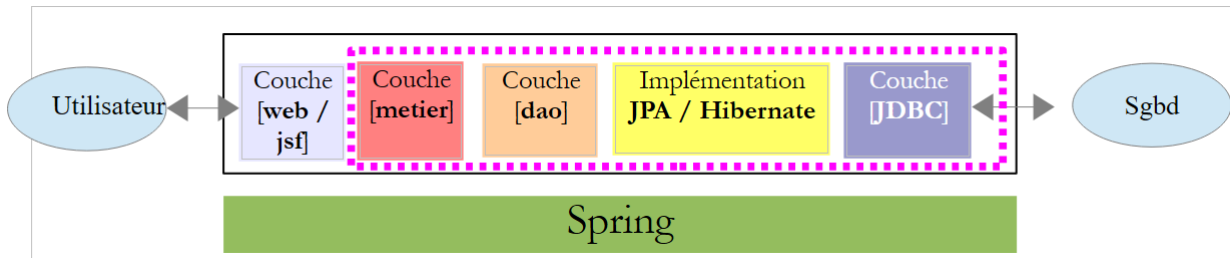
Salaire de base	Cotisations sociales	Indemnités d'entretien	Indemnités de repas
362,25 €	97,48 €	42,00 €	62,00 €

Salaire net 368,77 €

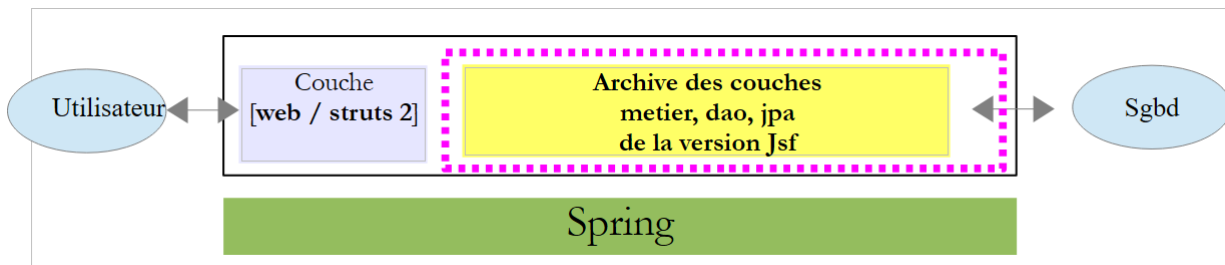
Cette étude de cas est présentée dans le document :

Introduction à Java EE 5 disponible à l'URL [<http://tahe.developpez.com/java/javae5>]

Dans ce document, l'étude de cas est implémentée avec l'architecture multi-couches suivante :



La couche [web] est implémentée à l'aide du framework JSF (Java Server Faces). Nous allons prendre cette même architecture en implémentant la couche [web] avec Struts 2. Pour montrer l'intérêt des architectures en couches, nous allons utiliser l'archive jar des couches [metier, dao, jpa] de la version JSF et la connecter à une couche [web / struts2] :



Nous présenterons les éléments suivants des couches [metier, dao, jpa] :

- la couche [web] s'adresse à l'interface de la couche [métier]. Nous présenterons cette interface.
- la couche [jpa] accède à une base de données. Nous la présenterons.
- la couche [jpa] transforme les lignes des tables de la base de données en entités Jpa manipulées par toutes les couches de l'application. Nous les présenterons.
- les couches [metier, dao, jpa] sont instanciées par Spring. Nous présenterons le fichier de configuration qui réalise cette instantiation et cette intégration.

18.2 La base de données

Nous utiliserons la base de données MySQL [dbpam_hibernate] suivante :



- en [1], la base a trois tables :
 - [employees] : une table qui enregistre les employés d'une crèche
 - [cotisations] : une table qui enregistre des taux de cotisations sociales
 - [indemnites] : une table qui enregistre des informations permettant de calculer la paie des employés

Table [employees]

Nom du Champ	Type de Ch...	Taille	Précision	Non Nul	Défaut	ID	
ID	BIGINT	20	0	☒	Nul	VERSION	clé primaire de type autoincrément
PRENOM	VARCHAR	20	0	☒		PRENOM	n° de version de l'enregistrement
SS	VARCHAR	15	0	☒		NOM	prénom de l'employée
ADRESSE	VARCHAR	50	0	☒		ADRESSE	son nom
CP	VARCHAR	5	0	☒		CP	son adresse
VILLE	VARCHAR	30	0	☒		VILLE	son code postal
NOM	VARCHAR	30	0	☒		INDEMNITE_ID	sa ville
VERSION	INTEGER	11	0	☒			clé étrangère sur INDEMNITES(ID)
INDEMNITE_ID	BIGINT	20	0	☒			

- en [2], la table des employés et en [3], la signification de ses champs

Le contenu de la table pourrait être le suivant :

ID	PRENOM	SS	ADRESSE	CP	VILLE	NOM	VERSION	INDEMNITE_ID
5	Marie	254104940426058	5 rue des oiseaux	49203	St Corentin	Jouveinal	1	7
6	Justine	260124402111742	La Brûlerie	49014	St Marcel	Laverti	1	8

Table [cotisations]

Nom du Champ	Type de Champ	Taille	Précision	Non Nul	Défaut	ID	
ID	BIGINT	20	0	☒	Nul	VERSION	clé primaire de type autoincrément
SECU	DOUBLE	22	31	☒		SECU	n° de version de l'enregistrement
RETRAITE	DOUBLE	22	31	☒		RETRAITE	taux (pourcentage) de cotisation pour la sécurité sociale
CSGD	DOUBLE	22	31	☒		CSGD	taux de cotisation pour la retraite
CSGRDS	DOUBLE	22	31	☒		CSGRDS	taux de cotisation pour la contribution sociale généralisée déductible
VERSION	INTEGER	11	0	☒			taux de cotisation pour la contribution sociale généralisée et la contribution au remboursement de la dette sociale

- en [4], la table des cotisations et en [5], la signification de ses champs

Le contenu de la table pourrait être le suivant :

ID	SECU	RETRAITE	CSGD	CSGRDS	VERSION
3	9.39	7.88	6.15	3.49	1

Table [indemnites]

Nom du Champ	Type de Cha...	Taille	Précision	Non Nul	Défaut	ID	
ID	BIGINT	20	0	☒	Nul	VERSION	clé primaire de type autoincrément
ENTRETIEN_JOUR	DOUBLE	22	31	☒		BASE_HEURE	n° de version de l'enregistrement
REPAS_JOUR	DOUBLE	22	31	☒		ENTRETIEN_JOUR	coût en euro d'une heure de garde
INDICE	INTEGER	11	0	☒		REPAS_JOUR	indemnité en euro par jour de garde
INDEMNITES_CP	DOUBLE	22	31	☒		INDEMNITES_CP	indemnité de repas en euro par jour de garde
BASE_HEURE	DOUBLE	22	31	☒			indemnités de congés payés. C'est un pourcentage à appliquer au salaire de base.
VERSION	INTEGER	11	0	☒			

- en [6], la table des indemnités et en [7], la signification de ses champs

Le contenu de la table pourrait être le suivant :

ID	ENTRETIEN_JOUR	REPAS_JOUR	INDICE	INDEMNITES_CP	BASE_HEURE	VERSION
7	2	3	1	12	1.93	1
8	2,1	3,1	2	15	2,1	1

L'exportation de la structure de la base vers un fichier SQL donne le résultat suivant :

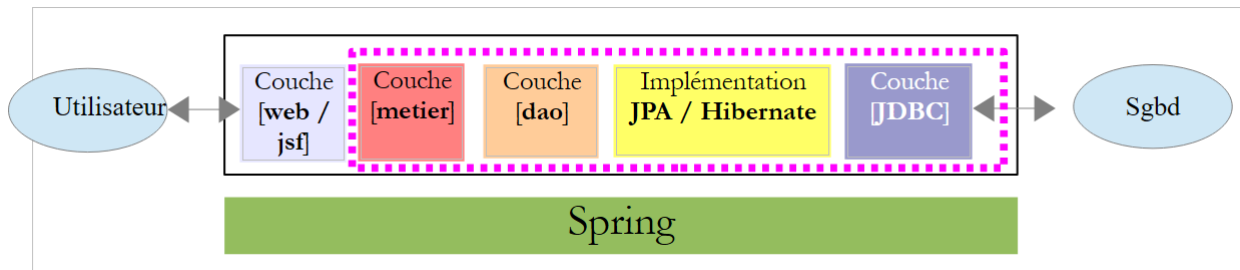
```

1. #
2. # Structure for the `cotisations` table :
3. #
4.
5. CREATE TABLE `cotisations` (
6.   `ID` bigint(20) NOT NULL auto_increment,
7.   `SECU` double NOT NULL,
8.   `RETRAITE` double NOT NULL,
9.   `CSGD` double NOT NULL,
10.  `CSGRDS` double NOT NULL,
11.  `VERSION` int(11) NOT NULL,
12.  PRIMARY KEY (`ID`)
13.) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=latin1;
14.
15. #
16. # Structure for the `indemnites` table :
17. #
18.
19. CREATE TABLE `indemnites` (
20.   `ID` bigint(20) NOT NULL auto_increment,
21.   `ENTRETIEN_JOUR` double NOT NULL,
22.   `REPAS_JOUR` double NOT NULL,
23.   `INDICE` int(11) NOT NULL,
24.   `INDEMNITES_CP` double NOT NULL,
25.   `BASE_HEURE` double NOT NULL,
26.   `VERSION` int(11) NOT NULL,
27.   PRIMARY KEY (`ID`),
28.   UNIQUE KEY `INDICE` (`INDICE`)
29.) ENGINE=InnoDB AUTO_INCREMENT=9 DEFAULT CHARSET=latin1;
30.
31. #
32. # Structure for the `employes` table :
33. #
34.
35. CREATE TABLE `employes` (
36.   `ID` bigint(20) NOT NULL auto_increment,
37.   `PRENOM` varchar(20) NOT NULL,
38.   `SS` varchar(15) NOT NULL,
39.   `ADRESSE` varchar(50) NOT NULL,
40.   `CP` varchar(5) NOT NULL,
41.   `VILLE` varchar(30) NOT NULL,
42.   `NOM` varchar(30) NOT NULL,
43.   `VERSION` int(11) NOT NULL,
44.   `INDEMNITE_ID` bigint(20) NOT NULL,
45.   PRIMARY KEY (`ID`),
46.   UNIQUE KEY `SS` (`SS`),
47.   KEY `FK_EMPLOYES_INDEMNITE_ID` (`INDEMNITE_ID`),
48.   CONSTRAINT `FK_EMPLOYES_INDEMNITE_ID` FOREIGN KEY (`INDEMNITE_ID`) REFERENCES `indemnites`
   (`ID`)
49.) ENGINE=InnoDB AUTO_INCREMENT=7 DEFAULT CHARSET=latin1;

```

18.3 Les entités JPA

Dans l'architecture suivante



La couche [jpa] joue le rôle de pont entre les objets manipulés par la couche [dao] et les lignes des tables de la base de données manipulées par le pilote jdbc. Les lignes des tables lues dans la base de données sont transformées en objets appelés entités Jpa. Inversement en écriture, les entités Jpa sont transformées en lignes dans les tables. Ces entités sont manipulées par toutes les couches et notamment par la couche web. Il nous faut donc les connaître :

L'entité [Employee] représente une ligne de la table [Employees]

Nom du Champ	Type de Ch...	Taille	Précision	Non Nul	Défaut
ID	BIGINT	20	0	☒	Nul
PRENOM	VARCHAR	20	0	☒	
SS	VARCHAR	15	0	☒	
ADRESSE	VARCHAR	50	0	☒	
CP	VARCHAR	5	0	☒	
VILLE	VARCHAR	30	0	☒	
NOM	VARCHAR	30	0	☒	
VERSION	INTEGER	11	0	☒	
INDEMNITE_ID	BIGINT	20	0	☒	

ID	clé primaire de type autoincrément
VERSION	n° de version de l'enregistrement
PRENOM	prénom de l'employée
NOM	son nom
ADRESSE	son adresse
CP	son code postal
VILLE	sa ville
INDEMNITE_ID	clé étrangère sur INDEMNITES(ID)

L'entité [Employee] est la suivante :

```

1. package jpa;
2.
3. ...
4.
5. @Entity
6. @Table(name="EMPLOYES")
7. public class Employee implements Serializable {
8.
9.     @Id
10.    @GeneratedValue(strategy = GenerationType.AUTO)
11.    private Long id;
12.    @Version
13.    @Column(name="VERSION", nullable=false)
14.    private int version;
15.    @Column(name="SS", nullable=false, unique=true, length=15)
16.    private String SS;
17.    @Column(name="NOM", nullable=false, length=30)
18.    private String nom;
19.    @Column(name="PRENOM", nullable=false, length=20)
20.    private String prenom;
21.    @Column(name="ADRESSE", nullable=false, length=50)
22.    private String adresse;
23.    @Column(name="VILLE", nullable=false, length=30)
24.    private String ville;
25.    @Column(name="CP", nullable=false, length=5)
26.    private String codePostal;
27.    @ManyToOne
28.    @JoinColumn(name="INDEMNITE_ID", nullable=false)
29.    private Indemnite indemnite;
30.
31.
32.    public Employee() {
33.    }
34.
35.    public Employee(String SS, String nom, String prenom, String adresse, String ville, String
        codePostal, Indemnite indemnite){
36.        setSS(SS);
  
```

```

37.     setNom(nom);
38.     setPrenom(prenom);
39.     setAdresse(adresse);
40.     setVille(ville);
41.     setCodePostal(codePostal);
42.     setIndemnité(indemnité);
43. }
44.
45. @Override
46. public String toString() {
47.     return "jpa.Empley[id=" + getId()
48.         + ",version="+getVersion()
49.         + ",SS="+getSS()
50.         + ",nom="+getNom()
51.         + ",prenom="+getPrenom()
52.         + ",adresse="+getAdresse()
53.         + ",ville="+getVille()
54.         + ",code postal="+getCodePostal()
55.         + ",indice="+getIndemnité().getIndice()
56.         + "]";
57. }
58.
59. // getters et setters
60. ...
61. }

```

On ignorera les annotations `@` destinées à la couche [Jpa]. Les différents champs de la classe reflètent les différentes colonnes de la table [EMPLOYES]. Le champ `indemnités` (ligne 29) reflète le fait que la table [EMPLOYES] a une clé étrangère sur la table [INDEMNITES]. Lorsqu'on manipule un employé, on manipule aussi ses indemnités.

L'entité [Indemnité] est l'expression objet d'une ligne de la table [INDEMNITES] :

Nom du Champ	Type de Cha...	Taille	Précision	Non Nul	Défaut
ID	BIGINT	20	0	☒	Nul
ENTRETIEN_JOUR	DOUBLE	22	31	☒	
REPAS_JOUR	DOUBLE	22	31	☒	
INDICE	INTEGER	11	0	☒	
INDEMNITES_CP	DOUBLE	22	31	☒	
BASE_HEURE	DOUBLE	22	31	☒	
VERSION	INTEGER	11	0	☒	

ID	clé primaire de type autoincrément
VERSION	n° de version de l'enregistrement
BASE_HEURE	coût en euro d'une heure de garde
ENTRETIEN_JOUR	indemnité en euro par jour de garde
REPAS_JOUR	indemnité de repas en euro par jour de garde
INDEMNITES_CP	indemnités de congés payés. C'est un pourcentage à appliquer au salaire de base.

L'entité [Indemnité] est la suivante :

```

1. package jpa;
2.
3. ...
4.
5. @Entity
6. @Table(name="INDEMNITES")
7. public class Indemnité implements Serializable {
8.
9.     @Id
10.    @GeneratedValue(strategy = GenerationType.AUTO)
11.    private Long id;
12.    @Version
13.    @Column(name="VERSION", nullable=false)
14.    private int version;
15.    @Column(name="INDICE", nullable=false, unique=true)
16.    private int indice;
17.    @Column(name="BASE_HEURE", nullable=false)
18.    private double baseHeure;
19.    @Column(name="ENTRETIEN_JOUR", nullable=false)
20.    private double entretienJour;
21.    @Column(name="REPAS_JOUR", nullable=false)
22.    private double repasJour;
23.    @Column(name="INDEMNITES_CP", nullable=false)
24.    private double indemnitesCP;
25.
26.    public Indemnité() {
27.    }
28.

```

```

29. public Indemnité(int indice, double baseHeure, double entretienJour, double repasJour, double
    indemnitésCP) {
30.     setIndice(indice);
31.     setBaseHeure(baseHeure);
32.     setEntretienJour(entretienJour);
33.     setRepasJour(repasJour);
34.     setIndemnitésCP(indemnitésCP);
35. }
36.
37. @Override
38. public String toString() {
39.     return "jpa.Indemnité[id=" + getId()
40.         + ",version="+getVersion()
41.         + ",indice="+getIndice()
42.         + ",base heure="+getBaseHeure()
43.         + ",entretien jour="+getEntretienJour()
44.         + ",repas jour="+getRepasJour()
45.         + ",indemnités CP="+getIndemnitésCP()
46.         + "]";
47. }
48.
49. // getters et setters
50. ....
51. }

```

Les différents champs de la classe reflètent les différentes colonnes de la table [INDEMNITES].

L'entité [Cotisation] est l'expression objet d'une ligne de la table [COTISATIONS] :

Nom du Champ	Type de Champ	Taille	Précision	Non Nul	Défaut	
ID	BIGINT	20	0	☒	Nul	ID clé primaire de type autoincrément
SECURITE	DOUBLE	22	31	☒		VERSION n° de version de l'enregistrement
RETRAITE	DOUBLE	22	31	☒		SECURITE taux (pourcentage) de cotisation pour la sécurité sociale
CSGRD	DOUBLE	22	31	☒		RETRAITE taux de cotisation pour la retraite
CSGRDS	DOUBLE	22	31	☒		CSGRD taux de cotisation pour la contribution sociale généralisée déductible
VERSION	INTEGER	11	0	☒		CSGRDS taux de cotisation pour la contribution sociale généralisée et la contribution au remboursement de la dette sociale

L'entité [Cotisation] est la suivante :

```

1. package jpa;
2.
3. ...
4.
5. @Entity
6. @Table(name="COTISATIONS")
7. public class Cotisation implements Serializable {
8.
9.     @Id
10.    @GeneratedValue(strategy = GenerationType.AUTO)
11.    private Long id;
12.    @Version
13.    @Column(name="VERSION", nullable=false)
14.    private int version;
15.    @Column(name="CSGRDS", nullable=false)
16.    private double csgrds;
17.    @Column(name="CSGD", nullable=false)
18.    private double csgd;
19.    @Column(name="SECURITE", nullable=false)
20.    private double securite;
21.    @Column(name="RETRAITE", nullable=false)
22.    private double retraite;
23.
24.    public Cotisation() {
25.    }
26.
27.    public Cotisation(double csgrds, double csgd, double securite, double retraite){
28.        setCsgrds(csgrds);

```

```

29.     setCsgd(csgd);
30.     setSecu(secu);
31.     setRetraite(retraite);
32. }
33.
34. @Override
35. public String toString() {
36.     return "jpa.Cotisation[id=" + getId() + ",version=" + getVersion()+",csgrds="+getCsgrds()+" +
37.         ",csgd="+getCsgd()+",secu="+getSecu()+",retraite="+getRetraite()+"]";
38. }
39.
40. // getters et setters
41. ...
42. }

```

Les différents champs de la classe reflètent les différentes colonnes de la table [INDEMNITES].

18.4 Mode de calcul du salaire d'une assistante maternelle

L'application web que nous allons écrire va nous permettre de calculer le salaire d'une employée à partir de trois informations :

- l'indice de l'employée
- le nombre de jours travaillés
- le nombre d'heures travaillées

Voici une copie d'écran de calcul d'un salaire :

Calcul du salaire des assistantes maternelles

[Faire la simulation](#)
[Effacer la simulation](#)
[Voir les simulations](#)
[Enregistrer la simulation](#)
[Terminer la session](#)

Employé: Marie Jouveinal
 Heures travaillées: 150,00
 Jours travaillés: 20

Informations Employé

Nom	Prénom	Adresse
Jouveinal	Marie	5 rue des oiseaux
Ville	Code Postal	Indice
St Corentin	49203	2

Cotisations Sociales

Csgd	Csgd	Retraite	Sécu
3,49 %	6,15 %	7,88 %	9,39 %

Indemnités

Salaire horaire	Entretien/Jour	Repas/Jour	Congés payés
2,10 €	2,10 €	3,10 €	15,00 €

Salaire

Salaire de base	Cotisations sociales	Indemnités d'entretien	Indemnités de repas
362,25 €	97,48 €	42,00 €	62,00 €

Salaire net 368,77 €

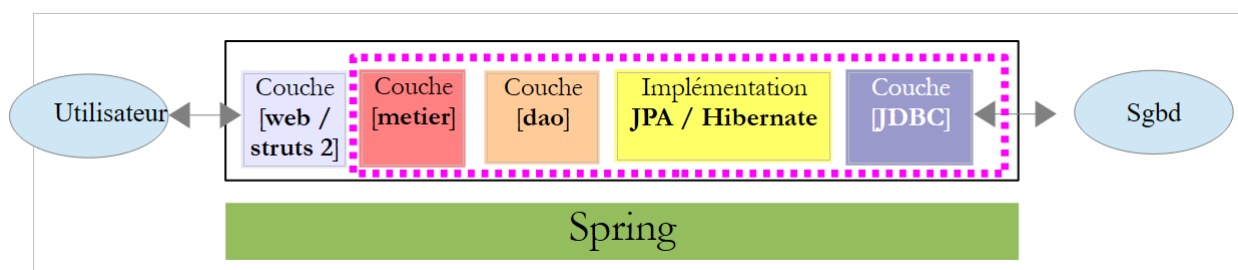
Nous présentons maintenant le mode de calcul du salaire mensuel d'une assistante maternelle. Il ne prétend pas être celui utilisé dans la réalité. Nous prenons pour exemple, le salaire de Mme Marie Jouveinal qui a travaillé 150 h sur 20 jours pendant le mois à payer.

Les éléments suivants sont pris en compte :	[TOTALHEURES]: total des heures travaillées dans le mois	[TOTALHEURES]=150 [TOTALJOURS]= 20
	[TOTALJOURS]: total des jours travaillés dans le mois	
Le salaire de base de l'assistante maternelle est donné par la formule suivante :	$[SALAIREBASE] = ([TOTALHEURES] * [BASEHEURE]) * (1 + [INDEMNITESCP] / 100)$	$[SALAIREBASE] = (150 * [2.1]) * (1 + 0.15) = 362,25$
Un certain nombre de cotisations sociales doivent être prélevées sur ce salaire de base :	Contribution sociale généralisée et contribution au remboursement de la dette sociale : $[SALAIREBASE] * [CSGRDS / 100]$	CSGRDS : 12,64 CSGD : 22,28
	Contribution sociale généralisée déductible : $[SALAIREBASE] * [CSGD / 100]$	Sécurité sociale : 34,02 Retraite : 28,55
	Sécurité sociale, veuvage, vieillesse :	

Les éléments suivants sont pris en compte :	[TOTALHEURES]: total des heures travaillées dans le mois	[TOTALHEURES]=150 [TOTALJOURS]= 20
	[TOTALJOURS]: total des jours travaillés dans le mois	
	[SALAIREBASE]*[SECU/100]	
	Retraite Complémentaire + AGPF + Assurance Chômage : [SALAIREBASE]*[RETRAITE/100]	
Total des cotisations sociales :	[COTISATIONSSOCIALES]=[SALAIREBASE]* (CSGRDS+CSGD+SECU+RETRAITE) /100	[COTISATIONSSOCIALES]=97,48
Par ailleurs, l'assistante maternelle a droit, chaque jour travaillé, à une indemnité d'entretien ainsi qu'à une indemnité de repas. A ce titre elle reçoit les indemnités suivantes :	[Indemnités]=[TOTALJOURS]* (ENTRETIENJOUR+REPASJOUR)	[INDEMNITES]=104
Au final, le salaire net à payer à l'assistante maternelle est le suivant :	[SALAIREBASE]-[COTISATIONSSOCIALES]+[INDEMNITES]	[salaire NET]=368,77

18.5 L'interface de la couche [métier]

Revenons sur l'architecture de l'application que nous construisons :



La couche [web / struts 2] communique avec l'interface de la couche [métier]. Celle-ci est la suivante :

```

1. package metier;
2.
3. import java.util.List;
4. import jpa.Employe;
5.
6. public interface IMetier {
7.     // obtenir la feuille de salaire
8.     FeuilleSalaire calculerFeuilleSalaire(String SS, double nbHeuresTravaillées, int
nbJoursTravaillés );
9.     // liste des employés
10.    List<Employe> findAllEmployes();
11. }

```

- ligne 8 : la méthode qui nous permettra de calculer le salaire d'un employé
- ligne 10 : la méthode qui nous permettra de remplir le combo des employés

La méthode *calculerFeuilleSalaire* rend une instance de la classe [FeuilleSalaire] suivante :

```

1. package metier;
2.
3. import java.io.Serializable;
4. import jpa.Cotisation;
5. import jpa.Employe;
6.

```



```

7. public class FeuilleSalaire implements Serializable {
8.     // champs privés
9.
10.    private Employe employe;
11.    private Cotisation cotisation;
12.    private ElementsSalaire elementsSalaire;
13.
14.    // constructeurs
15.    public FeuilleSalaire() {
16.    }
17.
18.    public FeuilleSalaire(Employe employe, Cotisation cotisation,
19.        ElementsSalaire elementsSalaire) {
20.        setEmploye(employe);
21.        setCotisation(cotisation);
22.        setElementsSalaire(elementsSalaire);
23.    }
24.
25.    // toString
26.    @Override
27.    public String toString() {
28.        return "[" + employe + "," + cotisation + ","
29.            + elementsSalaire + "];"
30.    }
31.
32.    // getters et setters
33.    ...
34. }

```

La feuille de salaire encapsule les informations suivantes :

- ligne 10 : des informations sur l'employé dont on calcule le salaire
- ligne 11 : les différents taux de cotisations
- ligne 12 : des éléments du salaire

La classe [ElementsSalaire] est la suivante :

```

1. package metier;
2.
3. import java.io.Serializable;
4.
5. public class ElementsSalaire implements Serializable{
6.
7.     // champs privés
8.     private double salaireBase;
9.     private double cotisationsSociales;
10.    private double indemnitesEntretien;
11.    private double indemnitesRepas;
12.    private double salaireNet;
13.
14.    // constructeurs
15.    public ElementsSalaire() {
16.    }
17.
18.
19.    public ElementsSalaire(double salaireBase, double cotisationsSociales,
20.        double indemnitesEntretien, double indemnitesRepas,
21.        double salaireNet) {
22.        setSalaireBase(salaireBase);
23.        setCotisationsSociales(cotisationsSociales);
24.        setIndemnitesEntretien(indemnitesEntretien);
25.        setIndemnitesRepas(indemnitesRepas);
26.        setSalaireNet(salaireNet);
27.    }
28.
29.    // toString
30.    @Override
31.    public String toString() {
32.        return "[salaire base=" + salaireBase + ",cotisations sociales=" + cotisationsSociales +
33.            ",indemnités d'entretien="
34.            + indemnitesEntretien + ",indemnités de repas=" + indemnitesRepas + ",salaire net="
35.            + salaireNet + "];"
36.    }
37.
38.    // getters et setters
39.    ...

```

- lignes 8-12 : les éléments du salaire

18.6 Le fichier de configuration de Spring

L'intégration des couches [métier, dao, jpa] est assurée par le fichier de configuration Spring suivant :

```

1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3.     xmlns:tx="http://www.springframework.org/schema/tx"
4.     xsi:schemaLocation="http://www.springframework.org/schema/beans
   http://www.springframework.org/schema/beans/spring-beans-2.0.xsd
   http://www.springframework.org/schema/tx http://www.springframework.org/schema/tx/spring-
   tx-2.0.xsd">
5.
6.     <!-- couches applicatives -->
7.
8.     <!-- métier -->
9.     <bean id="metier" class="metier.Metier">
10.         <property name="employeDao" ref="employeDao"/>
11.         <property name="cotisationDao" ref="cotisationDao"/>
12.     </bean>
13.     <!-- dao -->
14.     <bean id="employeDao" class="dao.EmployeDao" />
15.     <bean id="indemnitéDao" class="dao.IndemnitéDao" />
16.     <bean id="cotisationDao" class="dao.CotisationDao" />
17.
18.     <!-- configuration JPA -->
19.     <bean id="entityManagerFactory"
   class="org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean">
20.         <property name="dataSource" ref="dataSource" />
21.         <property name="jpaVendorAdapter">
22.             <bean class="org.springframework.orm.jpa.vendor.HibernateJpaVendorAdapter">
23.                 <property name="databasePlatform" value="org.hibernate.dialect.MySQL5InnoDBDialect" />
24.             </bean>
25.         </property>
26.         <property name="loadTimeWeaver">
27.             <bean class="org.springframework.instrument.classloading.InstrumentationLoadTimeWeaver" />
28.         </property>
29.     </bean>
30.
31.     <!-- la source de données DBCP -->
32.     <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource" destroy-method="close">
33.         <property name="driverClassName" value="com.mysql.jdbc.Driver" />
34.         <property name="URL" value="jdbc:mysql://localhost:3306/dbpam_hibernate" />
35.         <property name="username" value="root" />
36.         <property name="password" value="" />
37.     </bean>
38.
39.     <!-- le gestionnaire de transactions -->
40.     <tx:annotation-driven transaction-manager="txManager" />
41.     <bean id="txManager" class="org.springframework.orm.jpa.JpaTransactionManager">
42.         <property name="entityManagerFactory" ref="entityManagerFactory" />
43.     </bean>
44.
45.     <!-- traduction des exceptions -->
46.     <bean class="org.springframework.dao.annotation.PersistenceExceptionTranslationPostProcessor" />
47.
48.     <!-- persistence -->
49.     <bean class="org.springframework.orm.jpa.support.PersistenceAnnotationBeanPostProcessor" />
50.
51. </beans>

```

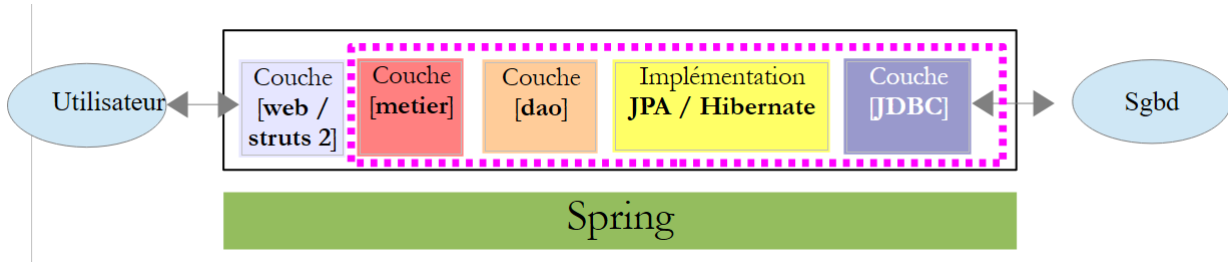
Nous n'essaierons pas d'expliquer cette configuration. Elle est nécessaire pour l'instanciation et l'intégration des couches [métier, dao, jpa]. Notre application web qui va s'appuyer sur ces couches devra donc reprendre cette configuration. On notera que les lignes 32-37 configurent les caractéristiques Jdbc de la base de données. Le lecteur qui voudrait changer de base de données doit modifier ces lignes.

Pour plus d'informations sur cette configuration, on lira le document **Introduction à Java EE 5** disponible à l'URL [<http://tahe.developpez.com/java/javacee/>].

19 Etude de cas – version 1

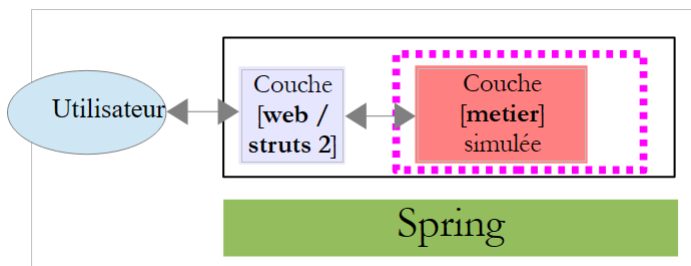
19.1 La couche [metier] simulée

Revenons sur l'architecture de l'application que nous construisons :



Nous disposons de l'archive des couches [metier, dao, jpa] et nous avons présenté les éléments de ces couches que la couche [web] devait connaître. Nous sommes prêts pour écrire celle-ci à l'aide du framework Struts.

Pour simplifier les tests de notre application en cours de développement, nous allons créer une couche métier simulée qui respectera l'interface de la couche [metier]. L'architecture va devenir la suivante :



Nous allons développer la couche [web] avec la couche [métier] simulée. Les tests seront plus simples à faire car il n'y a plus de base de données dans l'architecture. Grâce à Spring et à l'utilisation d'interfaces, remplacer ultérieurement la couche [metier] simulée par la véritable architecture [metier, dao, jpa] aura un impact nul sur le code de la couche [web / struts2]. La couche [web / struts2] que nous allons développer maintenant pourra être utilisée telle quelle.

La couche [metier] simulée que nous allons utiliser est la suivante :

```
1. package metier;
2.
3. import java.util.ArrayList;
4. import java.util.HashMap;
5. import java.util.List;
6. import java.util.Map;
7. import jpa.Cotisation;
8. import jpa.Emloyee;
9. import jpa.Indemnité;
10.
11. public class MetierSimule implements IMetier {
12.
13.     // liste des employes
14.     private Map<String, Emloyee> hashEmployes = new HashMap<String, Emloyee>();
15.     private List<Emloyee> listEmployes;
16.
17.     // obtenir la feuille de salaire
18.     public FeuilleSalaire calculerFeuilleSalaire(String SS,
19.         double nbHeuresTravaillées, int nbJoursTravaillés) {
20.         // on récupère l'employé de n° SS
21.         Emloyee e = hashEmployes.get(SS);
22.         // on rend une feuille de salaire fictive
23.         return new FeuilleSalaire(e, new Cotisation(3.49, 6.15, 9.39, 7.88), new ElementsSalaire(100,
24.             100, 100, 100));
```

```

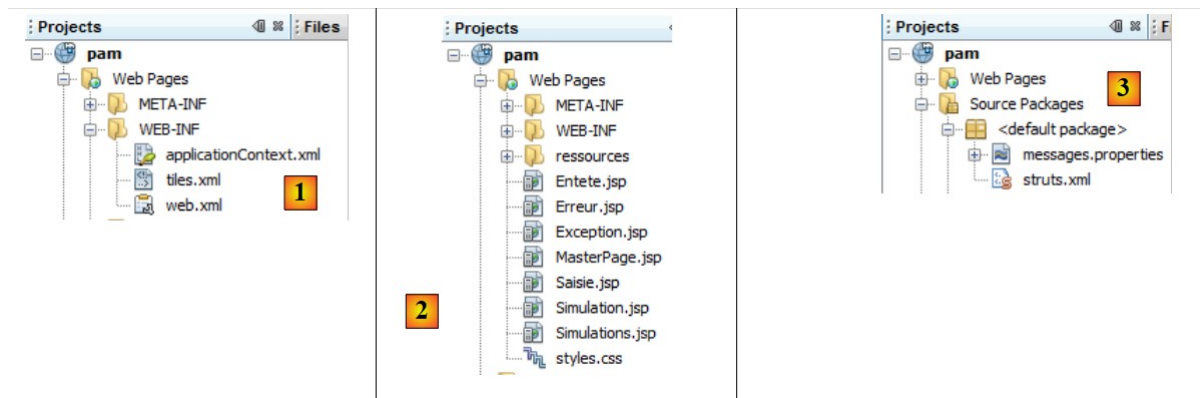
24. }
25.
26. // liste des employés
27. public List<Employe> findAllEmployes() {
28.     if (listEmployes == null) {
29.         // on crée une liste de deux employés
30.         listEmployes = new ArrayList<Employe>();
31.         listEmployes.add(new Employe("254104940426058", "Jouveinal", "Marie", "5 rue des oiseaux",
32. "St Corentin", "49203", new Indemnité(2, 2.1, 2.1, 3.1, 15)));
33.         listEmployes.add(new Employe("260124402111742", "Laverti", "Justine", "La br  lerie", "St
34. Marcel", "49014", new Indemnité(1, 1.93, 2, 3, 12)));
35.         // dictionnaire des employ  s
36.         for (Employe e : listEmployes) {
37.             hashEmployes.put(e.getSS(), e);
38.         }
39.         // on rend la liste des employ  s
40.         return listEmployes;
41.     }
42. }

```

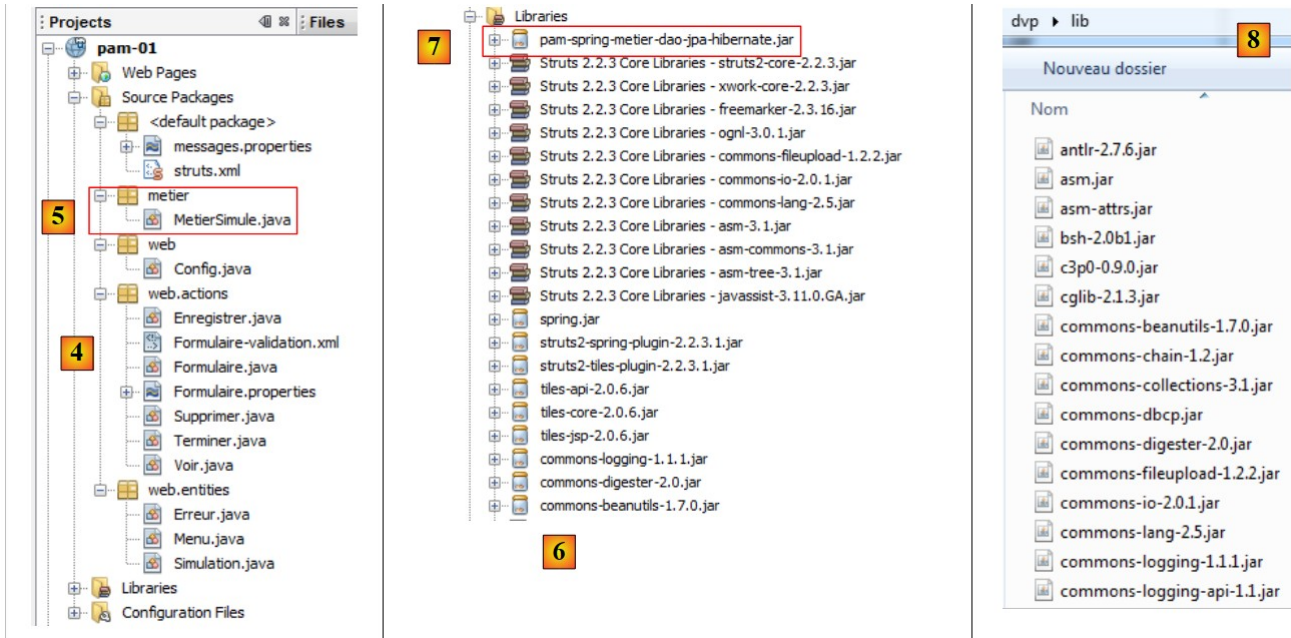
- ligne 11 : la classe [MetierSimule] impl  mente l'interface [IMetier] que la v  ritable couche [metier] impl  mente.
- ligne 14 : un dictionnaire des employ  s index  s par leur n   INSEE
- ligne 15 : la liste des employ  s
- lignes 27-39 : impl  mentation de la m  thode *findAllEmployes* de l'interface [IMetier].
- lignes 30-33 : cr  ation d'une liste de deux employ  s
- lignes 34-36 : cr  ation du dictionnaire des employ  s index   par le n   INSEE
- lignes 18-24 : impl  mentation de la m  thode *calculerSalaire* de l'interface [IMetier]. Ici on rend une feuille de salaire fictive.

19.2 Le projet Netbeans

Le projet Netbeans est le suivant :



- en [1] :
 - [applicationContext.xml] est le fichier de configuration de Spring
 - [tiles.xml] est le fichier de configuration d'un framework appel   **Tiles**.
 - [web.xml] est le fichier de configuration de l'application web
- en [2] : les diff  rentes vues de l'application
- en [3] :
 - [messages.properties] : le fichier des messages
 - [struts.xml] : le fichier de configuration de Struts



- en [4] : les codes source de l'application. Les actions Struts sont dans le package [web.actions].
- en [5] : la couche [metier] simulée
- en [6] : les archives utilisées. On trouve les archives des différents outils utilisés : Spring, Tiles, Struts 2, le plugin d'intégration Struts 2 / Spring, le plugin d'intégration Struts 2 / Tiles.
- en [7] : l'archive de la couche [metier, dao, jpa] véritable. Elle nous permet d'avoir accès aux entités Jpa, à l'interface [IMetier], aux classes [FeuilleSalaire] et [ElementsSalaire]. Tous ces éléments sont en effet utilisés par notre classe [MetierSimule].

19.3 Configuration du projet

Le projet est configuré par différents fichiers :

- [web.xml] qui configure l'application web
- [struts.xml] qui configure le framework Struts
- [applicationContext.xml] qui configure le framework Spring
- [tiles.xml] qui configure le framework Tiles

19.3.1 Configuration de l'application web

Le fichier [web.xml] est le suivant :

```

1. <?xml version="1.0" encoding="UTF-8"?>
2. <web-app id="pam_struts_01" version="2.4" xmlns="http://java.sun.com/xml/ns/j2ee"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee http://java.sun.com/xml/ns/j2ee/web-
   app_2_4.xsd">
3.   <display-name>Pam</display-name>
4.   <!-- Tiles -->
5.   <context-param>
6.     <param-name> org.apache.tiles.impl.BasicTilesContainer.DEFINITIONS_CONFIG </param-name>
7.     <param-value>/WEB-INF/tiles.xml</param-value>
8.   </context-param>
9.   <listener>
10.    <listener-class>org.apache.struts2.tiles.StrutsTilesListener</listener-class>
11.  </listener>
12.  <!-- Struts 2 -->
13.  <filter>
14.    <filter-name>struts2</filter-name>
15.    <filter-class>org.apache.struts2.dispatcher.FilterDispatcher</filter-class>
16.  </filter>
17.  <filter-mapping>

```

```

18.     <filter-name>struts2</filter-name>
19.     <URL-pattern>/*</URL-pattern>
20. </filter-mapping>
21.     <!-- Spring -->
22. <listener>
23.     <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>
24. </listener>
25. </web-app>

```

- lignes 13-20 : configurent le filtre Struts 2 – déjà vu
- lignes 22-24 : configurent le listener de Spring – déjà vu
- lignes 9-11 : configurent le listener **Tiles**. La classe [org.apache.struts2.tiles.StrutsTilesListener] va être instanciée au démarrage de l'application web. Elle va alors exploiter son fichier de configuration. Celui-ci est défini par les lignes 5-8. Le fichier de configuration de Tiles est donc le fichier [WEB-INF/tiles.xml].

Au final, au démarrage de l'application Struts, trois classes sont instanciées :

- l'une pour le filtre Struts 2. C'est elle qui assure le C de MVC.
- une autre pour le listener Spring. Spring va exploiter le fichier [applicationContext.xml] pour instancier les couches [métier, dao, jpa] de l'application. Spring va également instancier, comme dans un exemple précédent, une classe [Config] qui contiendra les données de portée Application. Enfin Spring injectera dans chaque action Struts qui en a besoin, une référence sur cette unique instance [Config].
- une autre pour le listener Tiles. Ce framework va assurer la gestion des vues. Nous y reviendrons bientôt.

19.3.2 Configuration du framework Struts

Le framework Struts est configuré par le fichier [struts.xml] suivant :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.     "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.     "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.     <!-- internationalisation -->
8.     <constant name="struts.custom.i18n.resources" value="messages" />
9.     <!-- intégration Spring -->
10.    <constant name="struts.objectFactory.spring.autoWire" value="name" />
11.
12.
13.    <!-- actions Struts /Tiles -->
14.    <package name="default" namespace="/" extends="tiles-default">
15.        <!-- action par défaut -->
16.        <default-action-ref name="index" />
17.        <action name="index">
18.            <result type="redirectAction">
19.                <param name="actionName">Formulaire</param>
20.                <param name="namespace"/></param>
21.            </result>
22.        </action>
23.        <!-- action Formulaire -->
24.        <action name="Formulaire" class="web.actions.Formulaire" method="input">
25.            <result name="success" type="tiles">saisie</result>
26.            <result name="exception" type="tiles">exception</result>
27.        </action>
28.        <!-- action FaireSimulation -->
29.        <action name="FaireSimulation" class="web.actions.Formulaire" method="calculSalaire">
30.            <result name="success" type="tiles">simulation</result>
31.            <result name="exception" type="tiles">exception</result>
32.            <result name="input" type="tiles">saisie</result>
33.        </action>
34.        <!-- action EnregistrerSimulation -->
35.        <action name="EnregistrerSimulation" class="web.actions.Enregistrer" method="execute">
36.            <result name="error" type="tiles">erreur</result>
37.            <result name="simulations" type="tiles">simulations</result>
38.        </action>
39.        <!-- action RetourFormulaire -->
40.        <action name="RetourFormulaire" >
41.            <result type="redirectAction">
42.                <param name="actionName">Formulaire</param>
43.                <param name="namespace"/></param>
44.            </result>
45.        </action>

```

```

46. <!-- action VoirSimulations -->
47. <action name="VoirSimulations" class="web.actions.Voir">
48.   <result name="success" type="tiles">simulations</result>
49. </action>
50. <!-- action RetirerSimulation -->
51. <action name="SupprimerSimulation" class="web.actions.Supprimer" method="execute">
52.   <result name="erreur" type="tiles">erreur</result>
53.   <result name="simulations" type="tiles">simulations</result>
54. </action>
55. <!-- action TerminerSession -->
56. <action name="TerminerSession" class="web.actions.Terminer" method="execute">
57.   <result name="success" type="redirectAction">
58.     <param name="actionName">Formulaire</param>
59.     <param name="namespace">/</param>
60.   </result>
61. </action>
62. </package>
63.
64. </struts>

```

Nous commenterons les différentes actions Struts au fur et à mesure où nous les étudierons. Pour l'instant, on peut noter les points suivants :

- ligne 8 : définit le fichier des messages
- ligne 10 : définit le mode d'injection des beans Spring dans les actions Struts. L'injection se fait selon le nom du bean. Il faut que le champ de l'action Struts qui doit être initialisé par Spring porte le nom du bean à injecter.
- ligne 25 : définit la vue à afficher pour la clé de navigation *success* de l'action [Formulaire]. On voit que le résultat `<result>` a un attribut *type='tiles'* que nous ne connaissons pas. Nous connaissons le type *redirect* qui permet de rediriger le client vers une vue. Ici la vue de type *tiles* est gérée par le framework *Tiles*. Le type *tiles* est défini dans le fichier [struts-plugin.xml] de l'archive [struts2-tiles-plugin-2.2.3.1.jar] :

```

1. <struts>
2.   <package name="tiles-default" extends="struts-default">
3.     <result-types>
4.       <result-type name="tiles" class="org.apache.struts2.views.tiles.TilesResult"/>
5.     </result-types>
6.   </package>
7. </struts>

```

- lignes 3-5 : la définition du type de résultat *tiles*.
- ligne 2 : ce type est défini dans le package [tiles-default] qui étend le package [struts-default].
- ligne 14 : définit le package [default] dans lequel seront toutes les actions de l'application. Pour profiter de la définition du type de vue *tiles*, le package étend [tiles-default].

19.3.3 Configuration du framework Spring

Le framework Spring est configuré par le fichier [WEB-INF/applicationContext.xml] suivant :

```

1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
3.   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.   xmlns:tx="http://www.springframework.org/schema/tx"
5.   xsi:schemaLocation="http://www.springframework.org/schema/beans
6.     http://www.springframework.org/schema/beans/spring-beans-2.0.xsd
7.     http://www.springframework.org/schema/tx http://www.springframework.org/schema/tx/spring-
8.     tx-2.0.xsd">
9.   <!-- couches applicatives -->
10.  <!-- web -->
11.  <bean id="config" class="web.Config" init-method="init">
12.    <property name="metier" ref="metier"/>
13.  </bean>
14.  <!-- métier -->
15.  <bean id="metier" class="metier.MetierSimule"/>
16. </beans>

```

- ligne 13 : la couche [metier] simulée instanciée par la classe [metier.MetierSimule]

- lignes 9-11 : configurent un bean nommé *config*. Comme dans un exemple précédemment étudié, ce bean servira à encapsuler les informations de portée *Application*. La classe associée à ce bean est la classe [Config] suivante :

```

1. package web;
2.
3. import java.util.List;
4. import jpa.Emloye;
5. import metier.IMetier;
6.
7. public class Config {
8.
9.     // couche métier initialisée par Spring
10.    private IMetier metier;
11.    // liste des employés
12.    private List<Emloye> employes;
13.    // erreurs
14.    private Exception initException;
15.
16.    // constructeur
17.    public Config() {
18.    }
19.
20.    // méthode Spring d'initialisation de l'objet
21.    public void init() {
22.        // on demande la liste des employés
23.        try {
24.            employes = metier.findAllEmployes();
25.        } catch (Exception ex) {
26.            initException = ex;
27.        }
28.    }
29.
30.    // getters et setters
31.    ...
32. }

```

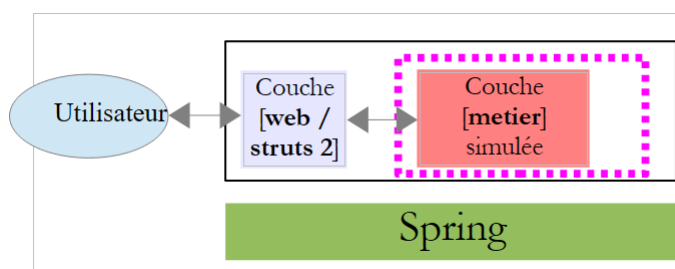
Revenons à la configuration du bean *config* :

```

1. <bean id="config" class="web.Config" init-method="init">
2.     <property name="metier" ref="metier"/>
3. </bean>
4. <!-- métier -->
5. <bean id="metier" class="metier.Metier">
6.     ...
7. </bean>

```

Ligne 2, on peut voir que le bean *metier* de la ligne 5, est injecté (ref) dans le champ nommé *metier* (name) de l'objet [Config]. Le bean *metier* est une référence sur la couche [metier] :



Pour dialoguer avec la couche [metier], toutes les actions Struts de la couche [web] auront besoin d'une référence sur celle-ci. On peut dire que la référence sur la couche [metier] est une donnée de portée *Application*. Toutes les requêtes de tous les utilisateurs en auront besoin. C'est pourquoi, nous mettons cette référence dans l'objet [Config]. Par ailleurs, ligne 1, la configuration du bean *config* a un attribut *init-method*. Cet attribut désigne la méthode du bean à exécuter après instantiation du bean. Ici, on indique qu'après instantiation de la classe [web.Config], on doit exécuter sa méthode *init*. Celle-ci est la suivante :

```

1. // couche métier initialisée par Spring
2. private IMetier metier;
3. // liste des employés
4. private List<Emloye> employes;
5. // erreurs
6. private Exception initException;
7.

```

```

8. // constructeur
9. public Config() {
10. }
11.
12. // méthode Spring d'initialisation de l'objet
13. public void init() {
14. // on demande la liste des employés
15. try {
16.     employes = metier.findAllEmployes();
17. } catch (Exception ex) {
18.     initException = ex;
19. }
20. }

```

Lorsque la méthode *init* est exécutée, le champ *metier* de la classe a été instancié par Spring. La méthode *init* a donc accès à la couche *metier* d'interface [IMetier] (ligne 2) :

```

1. package metier;
2.
3. import java.util.List;
4. import jpa.Employe;
5.
6. public interface IMetier {
7. // obtenir la feuille de salaire
8. FeuilleSalaire calculerFeuilleSalaire(String SS, double nbHeuresTravaillées, int
nbJoursTravaillés );
9. // liste des employés
10. List<Employe> findAllEmployes();
11. }

```

- ligne 8 : la méthode permet de calculer le salaire d'un employé
- ligne 10 : la méthode permet d'obtenir la liste des employés

On voit que la méthode *init* demande la liste des employés à la couche [métier]. Cette liste est enregistrée dans le champ de la ligne 4. S'il se produit une exception, celle-ci est mémorisée dans le champ de la ligne 6.

Pour conclure, l'objet unique [Config] contient :

- une référence sur la couche [métier]
- la liste des employés

19.4 Génération des vues Tiles

On l'a vu dans le fichier de configuration de Struts, les vues vont être générées par le framework *Tiles*. Nous n'allons expliquer que le strict nécessaire utile à l'écriture de notre application.

Tiles permet de générer des vues à partir d'une page maître. Celle-ci appelée ici [MasterPage.JSP] sera l'assemblage des fragments JSP suivants :

Entete.JSP

Saisie.JSP

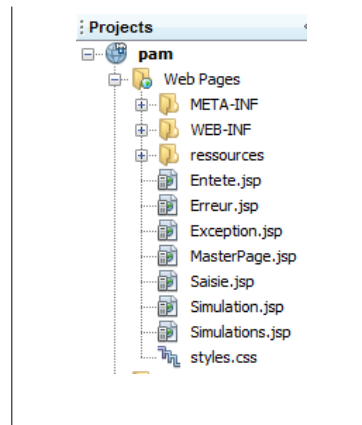
Simulation.JSP

Simulations.JSP

Exception.JSP

Erreur.JSP

Ces fragments JSP sont définis dans le projet Netbeans :



Le framework *Tiles* nous permet de définir quels fragments seront insérés dans la page maître.

La page maître [MasterPage.JSP] est la suivante :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <%@ taglib uri="http://tiles.apache.org/tags-tiles" prefix="tiles" %>
4. <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
5.   "http://www.w3.org/TR/html4/loose.dtd">
6.
7. <html>
8.   <head>
9.     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
10.    <link href="styles.css" rel="stylesheet" type="text/css"/>
11.    <title>
12.      <tiles:insertAttribute name="titre" ignore="true" />
13.    </title>
14.    <s:head/>
15.  </head>
16.  <body background="<s:url value="/ressources/standard.jpg"/>">
17.    <tiles:insertAttribute name="entete" />
18.    <hr/>
19.    <tiles:insertAttribute name="saisie" />
20.    <tiles:insertAttribute name="simulation" />
21.    <tiles:insertAttribute name="exception" />
22.    <tiles:insertAttribute name="erreur" />
23.    <tiles:insertAttribute name="simulations" />
24.  </body>
25. </html>

```

La page maître est un conteneur de fragments JSP. Ici, elle est l'assemblage de six fragments, ceux des lignes 17 à 23. A la génération, il peut y avoir de 0 à 6 fragments assemblés dans la page maître. Cette génération est gouvernée par le fichier [WEB-INF/tiles.xml] qui définit les vues *Tiles* :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2.
3. <!DOCTYPE tiles-definitions PUBLIC "-//Apache Software Foundation//DTD Tiles Configuration
4.   2.0//EN"
5.   "http://tiles.apache.org/dtds/tiles-config_2_0.dtd">
6. <tiles-definitions>
7.
8.   <!-- la page maître -->
9.   <definition name="masterPage" template="/MasterPage.JSP">
10.     <put-attribute name="entete" value="/Entete.JSP"/>
11.     <put-attribute name="titre" value="Pam"/>
12.     <put-attribute name="saisie" value=""/>
13.     <put-attribute name="simulation" value=""/>
14.     <put-attribute name="simulations" value=""/>
15.     <put-attribute name="exception" value=""/>
16.     <put-attribute name="erreur" value=""/>
17.   </definition>
18.
19.   <!-- la vue saisie -->
20.   <definition name="saisie" extends="masterPage">
21.     <put-attribute name="saisie" value="/Saisie.JSP"/>

```

```

22. </definition>
23.
24. <!-- la vue simulation -->
25. <definition name="simulation" extends="saisie">
26.   <put-attribute name="simulation" value="/Simulation.JSP"/>
27. </definition>
28.
29. <!-- la vue simulations -->
30. <definition name="simulations" extends="masterPage">
31.   <put-attribute name="simulations" value="/Simulations.JSP"/>
32. </definition>
33.
34. <!-- la vue exception -->
35. <definition name="exception" extends="masterPage">
36.   <put-attribute name="exception" value="/Exception.JSP"/>
37. </definition>
38.
39. <!-- la vue erreur -->
40. <definition name="erreur" extends="masterPage">
41.   <put-attribute name="erreur" value="/Erreur.JSP"/>
42. </definition>
43. </tiles-definitions>

```

- le fichier ci-dessus définit six vues *Tiles* nommées : **masterPage** (ligne 9), **saisie** (ligne 20), **simulation** (ligne 25), **simulations** (ligne 30), **exception** (ligne 35), **erreur** (ligne 40).
- lignes 9-17 : définissent une vue appelée *masterPage* (*name*) et associée à la page maître [MasterPage.JSP] (*template*). On a vu que cette page JSP définissait six sous-vues. Une vue *Tiles* associée à la page maître doit indiquer le fragment JSP associé à chacune des six sous-vues. On voit que certaines sous-vues reçoivent pour valeur (*value*) la chaîne vide. Ces sous-vues ne seront pas incluses dans la page maître [MasterPage.JSP]. La vue *Tiles* nommée *masterPage* est donc constituée du seul sous-fragment [Entete.JSP].
- lignes 20-22 : définissent une vue appelée *saisie* (*name*) et qui étend (*extends*) la vue nommée *masterPage* vue précédemment. Cela signifie qu'elle reprend toutes les définitions de la vue *masterPage*. Sa définition est équivalente à la suivante :

```

1. <definition name="saisie" template="/MasterPage.JSP">
2.   <put-attribute name="entete" value="/Entete.JSP"/>
3.   <put-attribute name="titre" value="Pam"/>
4.   <put-attribute name="saisie" value=""/>
5.   <put-attribute name="simulation" value=""/>
6.   <put-attribute name="simulations" value=""/>
7.   <put-attribute name="exception" value=""/>
8.   <put-attribute name="erreur" value=""/>
9.   <put-attribute name="saisie" value="/Saisie.JSP"/>
10. </definition>

```

On voit qu'elle est associée à la page JSP [MasterPage.JSP] et qu'à ce titre elle doit définir les six sous-vues de cette page. On voit que la définition de la ligne 9 annule celle de la ligne 4. La vue *Tiles* nommée *saisie* est donc constituée des fragments JSP [Entete.JSP, Saisie.JSP]

Si nous continuons ce raisonnement, nous obtenons le tableau suivant :

vue Tiles	pages JSP
masterPage	Entete.JSP
saisie	Entete.JSP, Saisie.JSP
simulation	Entete.JSP, Saisie.JSP, Simulation.JSP
simulations	Entete.JSP, Simulations.JSP
exception	Entete.JSP, Exception.JSP
erreur	Entete.JSP, Erreur.JSP

19.5 Les fichiers de messages

L'application a été internationalisée. On trouve les messages dans les fichiers [messages.properties] et [Formulaire.properties].

Le fichier [messages.properties] est le suivant :

```

1. Pam.titre=Calcul du salaire des assistantes maternelles
2. Pam.Erreurs.titre=Les erreurs suivantes se sont produites :

```

```

3. Pam.Erreurs.classe=Exception
4. Pam.Erreurs.message=Message
5. Pam.Erreur.libelle=L'erreur suivante s'est produite
6. Pam.Saisie.Heures.libell\u00e9=Heures travaill\u00e9es
7. Pam.Saisie.Jours.libell\u00e9=Jours travaill\u00e9s
8. Pam.Saisie.employ\u00e9=Employ\u00e9
9. Pam.BtnSalaire.libell\u00e9=Salaire
10. Pam.BtnEffacer.libell\u00e9=Effacer
11. Simulation.Infos.employe=Informations Employ\u00e9
12. Simulation.Employe.nom=Nom
13. Simulation.Employe.prenom=Pr\u00e9nom
14. Simulation.Employe.adresse=Adresse
15. Simulation.Employe.indice=Indice
16. Simulation.Employe.ville=Ville
17. Simulation.Employe.codePostal=Code Postal
18. Simulation.Infos.cotisations=Cotisations Sociales
19. Simulation.Cotisations.csgrds=CsgRds
20. Simulation.Cotisations.csgrds=Csgd
21. Simulation.Cotisations.retraite=Retraite
22. Simulation.Cotisations.secu=S\u00e9cu
23. Form.Infos.indemnites=Indemnit\u00e9s
24. Simulation.Indemnites.salaireHoraire=Salaire horaire
25. Simulation.Indemnites.entretienJour=Entretien/Jour
26. Simulation.Indemnites.repasJour=Repas/Jour
27. Simulation.Indemnites.cong\u00e9sPay\u00e9s=Cong\u00e9s pay\u00e9s
28. Simulation.Infos.Salaire=Salaire
29. Simulation.Salaire.salaireBase=Salaire de base
30. Simulation.Salaire.cotisationsSociales=Cotisations sociales
31. Simulation.Salaire.entretien=Indemnit\u00e9s d'entretien
32. Simulation.Salaire.repas=Indemnit\u00e9s de repas
33. Simulation.salaireNet=Salaire net
34. # formats
35. Format.heure = {0,time}
36. Format.nombre = {0,number,#0.0##}
37. Format.pourcent = {0,number,##0.00' %'}
38. Format.monnaie={0,number,##0.00' \u20ac'}
39. # liste des simulations
40. Pam.Simulations.titre=Liste des simulations
41. Pam.Simulations.num=Num\u00e9ro
42. Pam.Simulations.nom=Nom
43. Pam.Simulations.prenom=Pr\u00e9nom
44. Pam.Simulations.heures=Heures
45. Pam.Simulations.jours=Jours
46. Pam.Simulations.salairebase=Salaire de base
47. Pam.Simulations.indemnites=Indemnites
48. Pam.Simulations.cotisationsSociales=Cotisations
49. Pam.Simulations.salairenet=Salaire
50. Pam.SimulationsVides.titre=La liste des simulations est vide
51. # menu
52. Menu.FaireSimulation=Faire la simulation
53. Menu.EffacerSimulation=Effacer la simulation
54. Menu.VoirSimulations=Voir les simulations
55. Menu.RetourFormulaire=Retour au formulaire de navigation
56. Menu.EnregistrerSimulation=Enregistrer la simulation
57. Menu.TerminerSession=Terminer la session
58. # msg d'erreur
59. Erreur.sessionexpir\u00e9e=La session a expir\u00e9
60. Erreur.numSimulation=N\u00b0 de simulation incorrect
61. # erreur de conversion
62. xwork.default.invalid.fieldvalue=Valeur invalide pour le champ "{0}".

```

Le fichier [Formulaire.properties] est le suivant :

```

1. # pour que les doubles soient au format local
2. double.format={0,number,#0.00##}
3. # msg d'erreur
4. joursTravaill\u00e9s.error=Tapez un nombre entier compris entre 1 et 31
5. heuresTravaill\u00e9es.error=Tapez un nombre r\u00e9el entre 0 et 300

```

19.6 La feuille de style

Les vues Tiles utilisent la feuille de style [styles.css] suivante :

```

1. .libelle{
2.     background-color: #ccffff;

```

```

3.  font-family: 'Times New Roman',Times,serif;
4.  font-size: 14px;
5.  font-weight: bold;;
6.  padding-right: 5px;
7.  padding-left: 5px;
8.  padding-bottom: 5px;
9.  padding-top: 5px;
10. }
11.
12.
13. .info{
14.  background-color: #99cc00;;
15.  padding-right: 5px;
16.  padding-left: 5px;
17.  padding-bottom: 5px;
18.  padding-top: 5px;
19. }
20.
21. .titreInfos{
22.  background-color: #ffcc00
23.  }

```

19.7 La vue initiale

Pour étudier l'application, nous allons la présenter à partir des différentes actions de l'utilisateur. Nous étudierons à chaque fois, l'action *Struts* qui exécute cette action et la vue *Tiles* qui est envoyée en réponse.

Dans [struts.xml] nous avons les actions suivantes :

```

1.  <!-- action par défaut -->
2.  <default-action-ref name="index" />
3.  <action name="index">
4.    <result type="redirectAction">
5.      <param name="actionName">Formulaire!input</param>
6.      <param name="namespace">/</param>
7.    </result>
8.  </action>
9.  <!-- action Formulaire -->
10. <action name="Formulaire" class="web.actions.Formulaire">
11.   <result name="success" type="tiles">saisie</result>
12.   <result name="exception" type="tiles">exception</result>
13.   <result name="input" type="tiles">saisie</result>
14.   <result name="simulation" type="tiles">simulation</result>
15. </action>

```

- lignes 2-8 : l'action par défaut de l'application est [Formulaire!input].

La classe [Formulaire] est la suivante :

```

1.  package web.actions;
2.
3.  ...
4.  public class Formulaire extends ActionSupport implements Preparable, SessionAware {
5.
6.    // configuration initialisée par Spring
7.    private Config config;
8.    // liste des employés
9.    private List<Employe> employes;
10.   // liste des erreurs
11.   private List<Erreur> erreurs;
12.   // feuille de salaire
13.   private FeuilleSalaire feuilleSalaire;
14.   // saisies
15.   private String comboEmployesValue;
16.   private Double heuresTravaillees;
17.   private Integer joursTravaillees;
18.   // session
19.   private Map<String, Object> session;
20.   // menu
21.   private Menu menu;
22.
23.   @Override
24.   public void prepare() throws Exception {

```

```

25.     ...
26. }
27.
28. @Override
29. public String input() {
30.     ....
31. }
32.
33. // calcul du salaire
34. public String calculSalaire() {
35.     ...
36. }
37. }
38.
39. @Override
40. public void validate() {
41.     ...
42. }
43.
44. @Override
45. public void setSession(Map<String, Object> map) {
46.     session = map;
47. }
48.
49. // getters et setters
50. ...
51. }

```

- ligne 4 : l'action [Formulaire] implémente l'interface *Preparable*. Celle-ci n'a qu'une méthode, la méthode *prepare* de la ligne 24. Cette méthode est exécutée une fois avant toute méthode de l'action. Elle sert généralement à initialiser le modèle de l'action.

Le modèle de l'action est formé des lignes 6-21 :

- ligne 7 : le champ *config* est initialisé par Spring comme il a été expliqué. Il donne accès aux données de portée application :
 - une référence sur la couche [métier]
 - une référence sur la liste des employés.
 - une référence sur l'exception qui a pu éventuellement se produire lors de l'instanciation de l'objet [Config]
- ligne 9 : une liste des employés. Celle-ci va alimenter le combo des employés dans le fragment [Saisie.JSP].
- ligne 11 : une liste d'erreurs. Celle-ci va alimenter le fragment [Erreur.JSP].
- ligne 21 : la liste des options du menu du fragment [Entete.JSP]

En [1], les liens du menu affiché sont contrôlés par le champ *menu* de l'action [Formulaire].

La méthode *prepare* est exécutée avant la méthode *input*. C'est la suivante :

```

1. @Override
2. public void prepare() throws Exception {
3.     // erreur de configuration ?
4.     Exception initException = config.getInitException();
5.     if (initException != null) {

```



```

6.     erreurs = new ArrayList<Erreur>();
7.     Throwable th = initException;
8.     while (th != null) {
9.         erreurs.add(new Erreur(th.getClass().getName(), th.getMessage()));
10.        th = th.getCause();
11.    }
12. } else {
13.     employes = config.getEmployes();
14. }
15. }

```

- ligne 4 : on récupère l'exception dans l'objet [Config] instancié par Spring
- ligne 5 : s'il y a eu exception dans l'instanciation de l'objet [Config] alors on initialise la liste d'erreurs de la ligne 11. La classe [Erreur] est la suivante :

```

1. package web.entities;
2.
3. import java.io.Serializable;
4.
5. public class Erreur implements Serializable{
6.
7.     public Erreur() {
8.     }
9.
10.    // champs
11.    private String classe;
12.    private String message;
13.
14.    // constructeur
15.    public Erreur(String classe, String message){
16.        this.setClasse(classe);
17.        this.message=message;
18.    }
19.
20.    // getters et setters
21. ...
22. }

```

La classe sert à mémoriser la pile des exceptions :

- ligne 11 : la classe de l'exception
- ligne 12 : le message de l'exception

Revenons à la méthode *prepare* :

- ligne 13 : la liste des employés de l'objet [Config] est mémorisée dans le champ *employes* de l'action.

Une fois la méthode *prepare* exécutée, la méthode *input* va l'être à son tour. C'est la suivante :

```

1. @Override
2. public String input() {
3.     if (erreurs == null) {
4.         // menu
5.         menu = new Menu(true, false, false, true, false, true);
6.         return SUCCESS;
7.     } else {
8.         // menu
9.         menu = new Menu(false, false, false, false, false, false);
10.        return "exception";
11.    }
12. }

```

La méthode *input* se contente de positionner la liste des options de menu à afficher. La classe [Menu] est la suivante :

```

1. package web.entities;
2.
3. import java.io.Serializable;
4.
5. public class Menu implements Serializable {
6.    // éléments du menu
7.
8.    private boolean faireSimulation;
9.    private boolean effacerSimulation;

```

```

10. private boolean enregistrerSimulation;
11. private boolean voirSimulations;
12. private boolean retourFormulaire;
13. private boolean terminerSession;
14.
15. public Menu() {
16. }
17.
18. public Menu(boolean faireSimulation, boolean effacerSimulation, boolean enregistrerSimulation,
    boolean voirSimulations, boolean retourFormulaire, boolean terminerSession) {
19.     this.faireSimulation = faireSimulation;
20.     this.effacerSimulation = effacerSimulation;
21.     this.enregistrerSimulation = enregistrerSimulation;
22.     this.voirSimulations = voirSimulations;
23.     this.retourFormulaire = retourFormulaire;
24.     this.terminerSession = terminerSession;
25. }
26.
27. // getters et setters
28. ...
29. }

```

- lignes 8-13 : il y a 6 liens possibles dans le menu
- lignes 18-25 : le constructeur de la classe permet de fixer les liens qui doivent être affichés et ceux qui ne le doivent pas.

Les liens du menu sont affichés dans [Entete.JSP], fragment JSP présent dans toutes les vues *Tiles*. Chaque action aura un champ *menu* pour contrôler l'affichage du menu de [Entete.JSP].

Revenons à la méthode *input* :

```

1. @Override
2. public String input() {
3.     if (erreurs == null) {
4.         // menu
5.         menu = new Menu(true, false, false, true, false, true);
6.         return SUCCESS;
7.     } else {
8.         // menu
9.         menu = new Menu(false, false, false, false, false, false);
10.        return "exception";
11.    }
12. }

```

- lignes 3-6 : si la liste des erreurs est vide, le menu [Faire la simulation, Voir les simulations, Terminer la session] sera affiché et la clé *input* retournée.
- lignes 9-10 : si la liste des erreurs n'est pas vide, le menu sera vide et la clé *exception* sera retournée.

Revenons à la configuration de l'action [Formulaire] dans [struts.xml] :

```

1. <!-- action Formulaire -->
2. <action name="Formulaire" class="web.actions.Formulaire">
3.     <result name="success" type="tiles">saisie</result>
4.     <result name="exception" type="tiles">exception</result>
5.     <result name="input" type="tiles">saisie</result>
6.     <result name="simulation" type="tiles">simulation</result>
7. </action>

```

- ligne 5 : la clé *input* affiche la vue Tiles nommée *saisie*
- ligne 4 : la clé *exception* affiche la vue Tiles nommée *exception*

Commençons par la vue Tiles nommée *saisie*. Elle est composée des fragments JSP [Entete.JSP] et [Saisie.JSP].

Le fragment [Entete.JSP] est le suivant :

Calcul du salaire des assistantes maternelles

[Faire la simulation](#)
[Voir les simulations](#)
[Terminer la session](#)

Son code est le suivant :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3.
4. <table>
5.   <tr>
6.     <td><h1><s:text name="Pam.titre"/></h1></td>
7.     <td>
8.       <s:if test="menu.faireSimulation">
9.         |<a href="javascript:doSimulation()"><s:text name="Menu.FaireSimulation"/></a><br/>
10.      </s:if>
11.      <s:if test="menu.effacerSimulation">
12.        |<a href="<s:url action='Formulaire!input'/"><s:text name="Menu.EffacerSimulation"/></a><br/>
13.      </s:if>
14.      <s:if test="menu.voirSimulations">
15.        |<a href="<s:url action='VoirSimulations'/"><s:text name="Menu.VoirSimulations"/></a><br/>
16.      </s:if>
17.      <s:if test="menu.retourFormulaire">
18.        |<a href="<s:url action='RetourFormulaire'/"><s:text name="Menu.RetourFormulaire"/></a><br/>
19.      </s:if>
20.      <s:if test="menu.enregistrerSimulation">
21.        |<a href="<s:url action='EnregistrerSimulation'/"><s:text name="Menu.EnregistrerSimulation"/></a><br/>
22.      </s:if>
23.      <s:if test="menu.terminerSession">
24.        |<a href="<s:url action='TerminerSession'/"><s:text name="Menu.TerminerSession"/></a><br/>
25.      </s:if>
26.    </td>
27.  </tr>
28. </table>
```

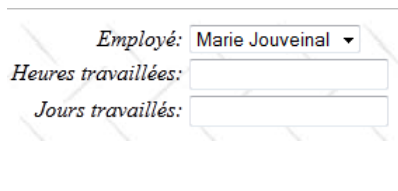
- lignes 8-25 : affichage des six liens du menu [Faire la simulation (lignes 8-10), Effacer la simulation (lignes 11-13), Voir les simulations (lignes 14-16), Retour au formulaire (lignes 17-19), Enregistrer la simulation (lignes 20-22), Terminer la session (lignes 23-25).
- lignes 8, 11, 14, 17, 20, 23 : l'affichage des liens est contrôlé par le champ *menu* de l'action courante.

On remarquera que le fragment [Entete.JSP] affiche une table Html (lignes 4-28) mais n'est pas une page Html complète. Il ne faut pas oublier ici que toutes les vues de l'application s'insèrent dans la page maître [MasterPage.JSP] suivante :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <%@ taglib uri="http://tiles.apache.org/tags-tiles" prefix="tiles" %>
4. <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
5.   "http://www.w3.org/TR/html4/loose.dtd">
6.
7. <html>
8.   <head>
9.     <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
10.    <link href="styles.css" rel="stylesheet" type="text/css"/>
11.    <title>
12.      <tiles:insertAttribute name="titre" ignore="true" />
13.    </title>
14.    <s:head/>
15.  </head>
16.  <body background="<s:url value='//ressources/standard.jpg'/">>
17.    <tiles:insertAttribute name="entete" />
18.    <hr/>
19.    <tiles:insertAttribute name="saisie" />
20.    <tiles:insertAttribute name="simulation" />
21.    <tiles:insertAttribute name="exception" />
22.    <tiles:insertAttribute name="erreur" />
23.    <tiles:insertAttribute name="simulations" />
24.  </body>
25. </html>
```

Le fragment [Entete.JSP] s'insère en ligne 17, à l'intérieur d'une page Html régulière.

Le fragment [Saisie.JSP] s'insère ligne 19. C'est la vue suivante :



Le code du fragment [Saisie.JSP] est le suivant :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3. <%@ taglib uri="http://tiles.apache.org/tags-tiles" prefix="tiles" %>
4. <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
5.   "http://www.w3.org/TR/html4/loose.dtd">
6.
7.
8. <script language="javascript" type="text/javascript">
9.   function doSimulation() {
10.    // on poste le formulaire
11.    document.forms['Saisie'].elements['action'].name='action:Formulaire!calculSalaire'
12.    document.forms['Saisie'].submit();
13.  }
14. </script>
15.
16. <!-- saisie des informations -->
17. <s:form name="Saisie" id="Saisie">
18.   <s:select name="comboEmployesValue" list="employees" listKey="SS" listValue="prenom+ ' ' +nom"
19.     key="Pam.Saisie.employé"/>
20.   <s:textfield name="heuresTravaillees" key="Pam.Saisie.Heures.libellé" value="%
21.     {#parameters['heuresTravaillees']!=null ? #parameters['heuresTravaillees'] :
22.     heuresTravaillees==null ? ' ' : getText('double.format',{heuresTravaillees})}"/>
23.   <s:textfield name="joursTravailles" key="Pam.Saisie.Jours.libellé" value="%
24.     {#parameters['joursTravailles']!=null ? #parameters['joursTravailles'] : joursTravailles==null ?
25.     ' ' : joursTravailles}"/>
26.   <input type="hidden" name="action"/>
27. </s:form>
```

- ligne 17 : le formulaire n'a pas d'attribut *action*. Par défaut, on a *action='Formulaire'*.
- ligne 18 : affichage du combo des employés. Le contenu du combo (attribut *list*) est fourni par le champ *employees* de l'action courante. L'attribut *value* des options sera le n° SS des employés (attribut *listKey*). Le libellé affiché pour chaque option sera le prénom et le nom de l'employé (attribut *listValue*). Le n° SS de l'employé sélectionné dans le combo sera posté au champ [Formulaire].*comboEmployesvalue* (attribut *name*).
- ligne 19 : champ de saisie des heures travaillées. La valeur affichée (attribut *value*) est celle du champ *heuresTravaillees* de l'action [Formulaire] au format suivant (Formulaire.properties) :

```
{double.format={0,number,#0.00##}}
```

La valeur sera postée au champ [Formulaire].*heuresTravaillees* (attribut *name*).

- ligne 20 : champ de saisie des jours travaillés. La valeur affichée (attribut *value*) est celle du champ *joursTravailles* de l'action [Formulaire].
La valeur sera postée au champ [Formulaire].*joursTravailles* (attribut *name*).

Au final, la vue Tiles *saisie* affichée au démarrage lorsqu'il n'y a pas d'erreurs est la suivante :

Calcul du salaire des assistantes maternelles

[Faire la simulation](#)[Voir les simulations](#)[Terminer la session](#)

Employé:

Heures travaillées:

Jours travaillés:

Revenons à la configuration de l'action [Formulaire] :

```
1. <!-- action Formulaire -->
2. <action name="Formulaire" class="web.actions.Formulaire">
3.   <result name="success" type="tiles">saisie</result>
4.   <result name="exception" type="tiles">exception</result>
5.   <result name="input" type="tiles">saisie</result>
6.   <result name="simulation" type="tiles">simulation</result>
7. </action>
```

On a vu que l'action [Formulaire].*input* pouvait également rendre la clé *exception* de la ligne 4. Dans ce cas, c'est la vue Tiles nommée *exception* qui est affichée. Celle-ci est composée des fragments [Entete.JSP] et [Exception.JSP]. Nous avons déjà présenté le fragment [Entete.JSP]. Le fragment [Exception.JSP] est le suivant :

Les erreurs suivantes se sont produites :

Exception	Message
org.springframework.transaction.CannotCreateTransactionException	Could not open JPA EntityManager for transaction; nested exception is javax.persistence.PersistenceException: org.hibernate.exception.GenericJDBCException: Cannot open connection
javax.persistence.PersistenceException	org.hibernate.exception.GenericJDBCException: Cannot open connection
org.hibernate.exception.GenericJDBCException	Cannot open connection
org.apache.commons.dbcp.SQLNestedException	Cannot create PoolableConnectionFactory (Communications link failure Last packet sent to the server was 0 ms ago.)
com.mysql.jdbc.exceptions.jdbc4.CommunicationsException	Communications link failure Last packet sent to the server was 0 ms ago.
java.net.ConnectException	Connection refused: connect

Ceci est la page de démarrage de la version 2 de l'application lorsque le Sgbd n'a pas été lancé . Le code JSP du fragment [Erreur.JSP] est le suivant :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3.
4. <h2><s:text name="Pam.Erreurs.titre"/></h2>
5. <table>
6.   <tr class="titreInfos">
7.     <th><s:text name="Pam.Erreurs.classe"/></th>
8.     <th><s:text name="Pam.Erreurs.message"/></th>
9.   </tr>
10.   <s:iterator value="erreurs">
11.     <tr>
12.       <td class="libelle"><s:property value="classe"/></td>
13.       <td class="info"><s:property value="message"/></td>
14.     </tr>
```

```
15. </s:iterator>
16. </table>
```

- lignes 10-14 : un itérateur sur la collection *List<Erreur> erreurs* de l'action [Formulaire]. On se rappelle qu'en cas d'erreur on y avait stocké une pile d'exceptions.

19.8 Faire une simulation

Une fois la vue de démarrage obtenue, on peut faire un calcul de salaire via le lien [Faire une simulation].

19.8.1 Validation des saisies

Considérons la séquence suivante :

- en [1], une saisie erronée
- en [2], la réponse envoyée.

Considérons la configuration de l'action [Formulaire] dans [struts.xml] :

```
1. <!-- action Formulaire -->
2. <action name="Formulaire" class="web.actions.Formulaire">
3.   <result name="success" type="tiles">saisie</result>
4.   <result name="exception" type="tiles">exception</result>
5.   <result name="input" type="tiles">saisie</result>
6.   <result name="simulation" type="tiles">simulation</result>
7. </action>
```

On sait qu'en cas d'erreur de validation, l'intercepteur de validation renvoie la clé *input*. C'est donc la vue Tiles *saisie* qui est renvoyée. Le processus de validation fait que les champs erronés sont accompagnés de messages d'erreur.

La validation de l'action [Formulaire] est assurée par le fichier [Formulaire-validation.xml] suivant :

```
1. <!DOCTYPE validators PUBLIC "-//OpenSymphony Group//XWork Validator 1.0.2//EN"
   "http://www.opensymphony.com/xwork/xwork-validator-1.0.2.dtd">
2.
3.
4. <validators>
5.
6.   <field name="heuresTravaillees" >
7.     <field-validator type="required" short-circuit="true">
8.       <message key="heuresTravaillées.error"/>
9.     </field-validator>
10.
11.    <field-validator type="conversion" short-circuit="true">
12.      <message key="heuresTravaillées.error"/>
13.    </field-validator>
14.
15.    <field-validator type="double" short-circuit="true">
16.      <param name="minInclusive">0</param>
17.      <param name="maxInclusive">300</param>
18.      <message key="heuresTravaillées.error"/>
19.    </field-validator>
```

```

20. </field>
21.
22. <field name="joursTravaillés" >
23.   <field-validator type="required" short-circuit="true">
24.     <message key="joursTravaillés.error"/>
25.   </field-validator>
26.
27.   <field-validator type="conversion" short-circuit="true">
28.     <message key="joursTravaillés.error"/>
29.   </field-validator>
30.
31.   <field-validator type="int" short-circuit="true">
32.     <param name="min">0</param>
33.     <param name="max">31</param>
34.     <message key="joursTravaillés.error"/>
35.   </field-validator>
36. </field>
37.
38. </validators>

```

- les lignes 6-20 vérifient que le champ *heuresTravaillées* est un nombre réel dans l'intervalle [0,300].
- les lignes 22-36 vérifient que le champ *joursTravaillés* est un nombre entier dans l'intervalle [0,31].

Revenons à la configuration de l'action [Formulaire] dans [struts.xml] :

```

8.   <!-- action Formulaire -->
9.   <action name="Formulaire" class="web.actions.Formulaire">
10.     ...
11.     <result name="input" type="tiles">saisie</result>
12. </action>

```

On sait qu'en cas d'erreur de validation, l'intercepteur de validation renvoie la clé *input*. C'est donc la vue Tiles *saisie* qui est renvoyée. Rappelons que celle-ci est composée des fragments [Entete.JSP] et [Saisie.JSP] où [Entete.JSP] comporte un titre et un choix d'options et [Saisie.JSP] le formulaire de saisie. En cas d'erreurs de saisie, le processus de validation fait que les champs erronés sont accompagnés de messages d'erreur et affichent de plus leur valeur erronée. Le fragment [Entete.JSP] n'intervient en rien dans le processus de validation. Regardons son code :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3.
4. <table>
5.   <tr>
6.     <td><h1><s:text name="Pam.titre"/></h1></td>
7.     <td>
8.       <s:if test="menu.faireSimulation">
9.         |<a href="javascript:doSimulation()"><s:text name="Menu.FaireSimulation"/></a><br/>
10.      </s:if>
11.      <s:if test="menu.effacerSimulation">
12.        |<a href="<s:url action="Formulaire!input"/>"><s:text name="Menu.EffacerSimulation"/></a><br/>
13.      </s:if>
14.      <s:if test="menu.voirSimulations">
15.        |<a href="<s:url action="VoirSimulations"/>"><s:text name="Menu.VoirSimulations"/></a><br/>
16.      </s:if>
17.      <s:if test="menu.retourFormulaire">
18.        |<a href="<s:url action="RetourFormulaire"/>"><s:text name="Menu.RetourFormulaire"/></a><br/>
19.      </s:if>
20.      <s:if test="menu.enregistrerSimulation">
21.        |<a href="<s:url action="EnregistrerSimulation"/>"><s:text name="Menu.EnregistrerSimulation"/></a><br/>
22.      </s:if>
23.      <s:if test="menu.terminerSession">
24.        |<a href="<s:url action="TerminerSession"/>"><s:text name="Menu.TerminerSession"/></a><br/>
25.      </s:if>
26.    </td>
27.   </tr>
28. </table>

```

Les six liens sont configurés par le champ *menu* du modèle (lignes 8,11, 14, 17, 20, 23). Lorsqu'il y a une erreur, ce modèle n'est pas mis à jour par l'action, et on retrouve alors une page sans menu. Pour remédier à ce problème, la classe [Formulaire] a la méthode *valider* suivante :

```

1. package web.actions;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4. ...
5. public class Formulaire extends ActionSupport implements Preparable, SessionAware {
6.
7.   ...

```



```

8.  // menu
9.  private Menu menu;
10.
11.  @Override
12.  public void prepare() throws Exception {
13.      ...
14.  }
15.
16.  @Override
17.  public String input() {
18.      ...
19.  }
20.
21.  // calcul du salaire
22.  public String calculSalaire() {
23.      ...
24.  }
25.
26.  @Override
27.  public void validate() {
28.      // des erreurs ?
29.      if (!getFieldErrors().isEmpty()) {
30.          // menu
31.          menu = new Menu(true, false, false, true, false, true);
32.      }
33.  }
34.
35.  // getters et setters
36.  ...
37. }

```

- ligne 27 : on sait que lorsqu'elle est présente, la méthode *validate* est exécutée par le processus de validation. On en profite pour mettre à jour le menu de la ligne 4 qui fait partie du modèle du fragment [Entete.JSP].
- lignes 29-32 : s'il y a eu des erreurs de validation, alors on positionne le menu pour réafficher la vue Tiles *saisie*. S'il n'y a pas eu d'erreurs de validation, alors on ne fait rien. C'est la méthode *calculSalaire* qui est alors chargée de créer le modèle de la vue à afficher.

19.8.2 Le calcul du salaire

Revenons au code JSP de l'entête :

```

1.  <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2.  <%@ taglib prefix="s" uri="/struts-tags" %>
3.
4.  <table>
5.      <tr>
6.          <td><h1><s:text name="Pam.titre"/></h1></td>
7.          <td>
8.              <s:if test="menu.faireSimulation">
9.                  <a href="javascript:doSimulation()"><s:text name="Menu.FaireSimulation"/></a><br/>
10.             </s:if>
11.             ...
12.          </td>
13.      </tr>
14.  </table>

```

- ligne 9 : lorsque l'utilisateur clique sur le lien [Faire la simulation], la fonction Javascript *doSimulation* est exécutée. Celle-ci est défini dans le fragment [Saisie.JSP] :

```

1.  <script language="javascript" type="text/javascript">
2.      function doSimulation() {
3.          // on poste le formulaire
4.          document.forms['Saisie'].elements['action'].name='action:Formulaire!calculSalaire'
5.          document.forms['Saisie'].submit();
6.      }
7.  </script>
8.
9.  <!-- saisie des informations -->
10. <s:form name="Saisie" id="Saisie">
11.     <s:select name="comboEmployesValue" list="employes" listKey="SS" listValue="prenom+ ' ' +nom"
        key="Pam.Saisie.employé"/>

```

```

12. <s:textfield name="heuresTravaillees" key="Pam.Saisie.Heures.libellé" value="%
    {#parameters['heuresTravaillees']!=null ? #parameters['heuresTravaillees'] :
    heuresTravaillees==null ? '' : getText('double.format',{heuresTravaillees})}"/>
13. <s:textfield name="joursTravailles" key="Pam.Saisie.Jours.libellé" value="%
    {#parameters['joursTravailles']!=null ? #parameters['joursTravailles'] : joursTravailles==null ?
    '' : joursTravailles}"/>
14. <input type="hidden" name="action"/>
15. </s:form>

```

- ligne 14 : un champ caché nommé *action* sera posté à l'action [Formulaire]. Ce champ va nous permettre de préciser l'action et la méthode qui doivent être exécutées au POST du formulaire. On se rappelle peut-être des premiers exemples, que celles-ci peuvent être précisées dans un paramètre nommé **action:Action!méthode**. Peu importe la valeur de ce paramètre. Il suffit qu'il soit présent.
- lignes 2-6 : la fonction Javascript qui est exécutée lorsque l'utilisateur clique sur le lien [Faire la simulation] du fragment [Entete.JSP].
- ligne 4 : on change l'attribut *name* du champ caché *action*. On fait en sorte qu'il soit de la forme **action:Action!méthode** attendue par Struts.
- ligne 5 : le formulaire nommé *Saisie* de la ligne 5 est posté. Du coup est postée la chaîne de paramètres suivante :

```
comboEmployesValue=SS1&heuresTravaillees=xx&joursTravailles=yy&action:Formulaire!calculSalaire
```

```

SS1 : n° INSEE de l'employé sélectionné dans le combo
heuresTravaillees : nombre d'heures travaillées
joursTravailles : nombre de jours travaillés
action : Formulaire!calculSalaire : les éléments ci-dessus seront postés à l'action [Formulaire] puis la
méthode calculSalaire de cette action sera exécutée.

```

La méthode [Formulaire].*calculSalaire* est la suivante :

```

1. // calcul du salaire
2. public String calculSalaire() {
3.     try {
4.         // calcul salaire
5.         feuilleSalaire = config.getMetier().calculerFeuilleSalaire(comboEmployesValue,
        heuresTravaillees, joursTravailles);
6.         // on met la simulation dans la session
7.         session.put("simulation", new Simulation(0, "" + heuresTravaillees, "" + joursTravailles,
        feuilleSalaire));
8.         // menu
9.         menu = new Menu(true, true, true, true, false, true);
10.        // fini
11.        return "simulation";
12.    } catch (Throwable th) {
13.        ...
14.    }
15. }

```

- ligne 5 : le calcul de la feuille de salaire est demandée à la couche [métier]
- ligne 7 : on met un objet de type *Simulation* dans la session de l'utilisateur. En effet, on peut en avoir besoin lors d'une requête ultérieure. La classe [Simulation] est la suivante :

```

1. package web.entities;
2.
3. import java.io.Serializable;
4. import metier.FeuilleSalaire;
5.
6. public class Simulation implements Serializable{
7.
8.     public Simulation() {
9.     }
10.
11.    // champs d'une simulation
12.    private Integer num;
13.    private FeuilleSalaire feuilleSalaire;
14.    private String heuresTravaillées;
15.    private String joursTravaillés;
16.
17.    // constructeur
18.    public Simulation(Integer num,String heuresTravaillées, String joursTravaillés, FeuilleSalaire
        feuilleSalaire){
19.        this.setNum(num);
20.        this.setFeuilleSalaire(feuilleSalaire);

```

```

21.     this.setHeuresTravaillées(heuresTravaillées);
22.     this.setJoursTravaillés(joursTravaillés);
23. }
24.
25. public double getIndemnités() {
26.     return feuilleSalaire.getElementsSalaire().getIndemnitésEntretien() +
        feuilleSalaire.getElementsSalaire().getIndemnitésRepas();
27. }
28.
29. // getters et setters
30. ...
31. }

```

- ligne 12 : le n° de la simulation. Est incrémenté à chaque nouvelle simulation enregistrée.
- ligne 13 : la feuille de salaire de l'employé
- ligne 14 : son nombre d'heures travaillées
- ligne 15 : son nombre de jours travaillés
- ligne 25 : la méthode *getIndemnités* rend le total des indemnités de l'employé

Nous verrons que la classe [Simulation] est le modèle du fragment [Simulations.JSP] qui présente toutes les simulations faites.

Retour à la méthode [Formulaire].*calculSalaire* :

```

1. // calcul du salaire
2. public String calculSalaire() {
3.     try {
4.         // calcul salaire
5.         feuilleSalaire = config.getMetier().calculerFeuilleSalaire(comboEmployesValue,
        heuresTravaillees, joursTravaillés);
6.         // on met la simulation dans la session
7.         session.put("simulation", new Simulation(0, "" + heuresTravaillees, "" + joursTravaillés,
        feuilleSalaire));
8.         // menu
9.         menu = new Menu(true, true, true, true, false, true);
10.        // fini
11.        return "simulation";
12.    } catch (Throwable th) {
13.        ...
14.    }

```

- ligne 9 : mise à jour du menu
- ligne 11 : renvoi de la clé de navigation *simulation*.

Retour à la configuration de l'action [Formulaire] :

```

1.     <!-- action Formulaire -->
2.     <action name="Formulaire" class="web.actions.Formulaire">
3.         ...
4.         <result name="simulation" type="tiles">simulation</result>
5.     </action>

```

La ligne 4, montre que la clé de navigation *simulation* fait afficher la vue Tiles nommée *simulation*. Celle-ci est composée des fragments JSP suivants : [Entete, Saisie, Simulation].

La vue rendue est la suivante :

Calcul du salaire des assistantes maternelles

[Faire la simulation](#)
[Effacer la simulation](#)
[Voir les simulations](#)
[Enregistrer la simulation](#)
[Terminer la session](#)

1

Employé: Marie Jouveinal
Heures travaillées: 150
Jours travaillés: 20

2

Informations Employé

Nom	Prénom	Adresse
Jouveinal	Marie	5 rue des oiseaux
Ville	Code Postal	Indice
St Corentin	49203	2

3

Cotisations Sociales

Csgd	Csgd	Retraite	Sécu
3,49 %	6,15 %	7,88 %	9,39 %

Indemnités

Salaire horaire	Entretien/Jour	Repas/Jour	Congés payés
2,10 €	2,10 €	3,10 €	15,00 €

Salaire

Salaire de base	Cotisations sociales	Indemnités d'entretien	Indemnités de repas
100,00 €	100,00 €	100,00 €	100,00 €

Salaire net 100,00 €

- en [1], le fragment [Entete.JSP]
- en [2], le fragment [Saisie.JSP]
- en [3], le fragment [Simulation.JSP]. On rappelle que la feuille de salaire présentée est la feuille de salaire fictive rendue par la couche [metier].

Les deux premiers fragments ont déjà été présentés. Le fragment [Simulation.JSP] est le suivant :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3.
4. <hr/>
5. <!-- informations Employé -->
6. <span class="titreInfos">
7.   <s:text name="Simulation.Infos.employe"/>
8. </span>
9. <br/><br/>
10. <table>
11.   <!-- ligne 1 -->
12.   <tr>
13.     <th class="libelle">
14.       <s:text name="Simulation.Employe.nom"/>
15.     </th>
16.     <th class="libelle">
17.       <s:text name="Simulation.Employe.prenom"/>
18.     </th>
```

```

19.     <th class="libelle">
20.         <s:text name="Simulation.Employe.adresse"/>
21.     </th>
22. </tr>
23. <!-- ligne 2 -->
24. <tr>
25.     <td class="info">
26.         <s:property value="feuilleSalaire.employe.nom"/>
27.     </td>
28.     <td class="info">
29.         <s:property value="feuilleSalaire.employe.prenom"/>
30.     </td>
31.     <td class="info">
32.         <s:property value="feuilleSalaire.employe.adresse"/>
33.     </td>
34. </table>
35. <table>
36.     <!-- ligne 1 -->
37.     <tr>
38.         <th class="libelle"><s:text name="Simulation.Employe.ville"/></th>
39.         <th class="libelle">
40.             <s:text name="Simulation.Employe.codePostal"/>
41.         </th>
42.         <th class="libelle">
43.             <s:text name="Simulation.Employe.indice"/>
44.         </th>
45.     </tr>
46.     <!-- ligne 2 -->
47.     <tr>
48.         <td class="info">
49.             <s:property value="feuilleSalaire.employe.ville"/>
50.         </td>
51.         <td class="info">
52.             <s:property value="feuilleSalaire.employe.codePostal"/>
53.         </td>
54.         <td class="info">
55.             <s:property value="feuilleSalaire.employe.indemnite.indice"/>
56.         </td>
57.     </table>
58. <!-- informations Cotisations -->
59. <br/>
60. <span class="titreInfos">
61.     <s:text name="Simulation.Infos.cotisations"/>
62. </span>
63.
64. <br/><br/>
65. <table>
66.     <!-- ligne 1 -->
67.     <tr>
68.         <th class="libelle">
69.             <s:text name="Simulation.Cotisations.csgrds"/>
70.         </th>
71.         <th class="libelle">
72.             <s:text name="Simulation.Cotisations.csgrds"/>
73.         </th>
74.         <th class="libelle">
75.             <s:text name="Simulation.Cotisations.retraite"/>
76.         </th>
77.         <th class="libelle">
78.             <s:text name="Simulation.Cotisations.secu"/>
79.         </th>
80.     </tr>
81.     <!-- ligne 2 -->
82.     <tr>
83.         <td class="info">
84.             <s:text name="Format.pourcent">
85.                 <s:param value="feuilleSalaire.cotisation.csgrds"/>
86.             </s:text>
87.         </td>
88.         <td class="info">
89.             <s:text name="Format.pourcent">
90.                 <s:param value="feuilleSalaire.cotisation.csgrd"/>
91.             </s:text>
92.         </td>
93.         <td class="info">
94.             <s:text name="Format.pourcent">
95.                 <s:param value="feuilleSalaire.cotisation.retraite"/>
96.             </s:text>

```

```

97.     </td>
98.     <td class="info">
99.         <s:text name="Format.pourcent">
100.             <s:param value="feuilleSalaire.cotisation.secu"/>
101.         </s:text>
102.     </td>
103. </table>
104. <!-- informations Indemnités -->
105. <br/>
106. <span class="titreInfos">
107.     <s:text name="Form.Infos.indemnites"/>
108. </span>
109. <br/><br/>
110. <table>
111.     <!-- ligne 1 -->
112.     <tr>
113.         <th class="libelle">
114.             <s:text name="Simulation.Indemnites.salaireHoraire"/>
115.         </th>
116.         <th class="libelle">
117.             <s:text name="Simulation.Indemnites.entretienJour"/>
118.         </th>
119.         <th class="libelle">
120.             <s:text name="Simulation.Indemnites.repasJour"/>
121.         </th>
122.         <th class="libelle">
123.             <s:text name="Simulation.Indemnites.congésPayés"/>
124.         </th>
125.     </tr>
126.     <!-- ligne 2 -->
127.     <tr>
128.         <td class="info">
129.             <s:text name="Format.monnaie">
130.                 <s:param value="feuilleSalaire.employe.indemnite.baseHeure"/>
131.             </s:text>
132.         </td>
133.         <td class="info">
134.             <s:text name="Format.monnaie">
135.                 <s:param value="feuilleSalaire.employe.indemnite.entretienJour"/>
136.             </s:text>
137.         </td>
138.         <td class="info">
139.             <s:text name="Format.monnaie">
140.                 <s:param value="feuilleSalaire.employe.indemnite.repasJour"/>
141.             </s:text>
142.         </td>
143.         <td class="info">
144.             <s:text name="Format.monnaie">
145.                 <s:param value="feuilleSalaire.employe.indemnite.indemnitesCP"/>
146.             </s:text>
147.         </td>
148.     </tr>
149. </table>
150. <!-- informations Salaire -->
151. <br/>
152. <span class="titreInfos">
153.     <s:text name="Simulation.Infos.Salaire"/>
154. </span>
155. <br/><br/>
156. <table>
157.     <!-- ligne 1 -->
158.     <tr>
159.         <th class="libelle">
160.             <s:text name="Simulation.Salaire.salaireBase"/>
161.         </th>
162.         <th class="libelle">
163.             <s:text name="Simulation.Salaire.cotisationsSociales"/>
164.         </th>
165.         <th class="libelle">
166.             <s:text name="Simulation.Salaire.entretien"/>
167.         </th>
168.         <th class="libelle">
169.             <s:text name="Simulation.Salaire.repas"/>
170.         </th>
171.     </tr>
172.     <!-- ligne 2 -->

```

```

175.     <tr>
176.         <td class="info">
177.             <s:text name="Format.monnaie">
178.                 <s:param value="feuilleSalaire.elementsSalaire.salaireBase"/>
179.             </s:text>
180.         </td>
181.         <td class="info">
182.             <s:text name="Format.monnaie">
183.                 <s:param value="feuilleSalaire.elementsSalaire.cotisations Sociales"/>
184.             </s:text>
185.         </td>
186.         <td class="info">
187.             <s:text name="Format.monnaie">
188.                 <s:param value="feuilleSalaire.elementsSalaire.indemnitesEntretien"/>
189.             </s:text>
190.         </td>
191.         <td class="info">
192.             <s:text name="Format.monnaie">
193.                 <s:param value="feuilleSalaire.elementsSalaire.indemnitesRepas"/>
194.             </s:text>
195.         </td>
196.     </tr>
197. </table>
198. <!-- Salaire net-->
199. <br/>
200. <table>
201.     <tr>
202.         <td class="libelle">
203.             <s:text name="Simulation.salaireNet"/>
204.         <td></td>
205.         <td class="info">
206.             <s:text name="Format.monnaie">
207.                 <s:param value="feuilleSalaire.elementsSalaire.salaireNet"/>
208.             </s:text>
209.         </td>
210.     </tr>
211. </table>

```

C'est long ... mais c'est fonctionnellement simple. Ce fragment affiche les différentes propriétés du champ [Formulaire].*feuilleSalaire* qui représente la feuille de salaire de l'employé.

Retour à la méthode [Formulaire].*calculSalaire* :

```

1. // calcul du salaire
2. public String calculSalaire() {
3.     try {
4.         ...
5.         return "simulation";
6.     } catch (Throwable th) {
7.         erreurs = new ArrayList<Erreur>();
8.         while (th != null) {
9.             erreurs.add(new Erreur(th.getClass().getName(), th.getMessage()));
10.            th = th.getCause();
11.        }
12.        // menu
13.        menu = new Menu(false, false, false, false, true, true);
14.        return "exception";
15.    }
16. }

```

La calcul du salaire peut mal se passer. Ce serait le cas notamment si le lien avec le Sgbd se cassait. Dans ce cas, on gère l'exception qui se produit. Nous avons déjà rencontré ce cas lors de l'étude de la méthode [Formulaire].*input*.

- lignes 7-11 : on crée une liste d'objets *Erreur* à partir de la pile des exceptions
- ligne 13 : on fixe le menu
- ligne 14 : on rend la clé *exception*.

La clé *exception* va faire afficher la vue Tiles *exception* :

```

1. <!-- action Formulaire -->
2. <action name="Formulaire" class="web.actions.Formulaire">
3.     <result name="exception" type="tiles">exception</result>
4.     ...
5. </action>

```


Cette vue Tiles a déjà été présentée. Elle a l'allure suivante :

Les erreurs suivantes se sont produites :

Exception	Message
org.springframework.transaction.CannotCreateTransactionException	Could not open JPA EntityManager for transaction; nested exception is javax.persistence.PersistenceException: org.hibernate.exception.GenericJDBCException: Cannot open connection
javax.persistence.PersistenceException	org.hibernate.exception.GenericJDBCException: Cannot open connection
org.hibernate.exception.GenericJDBCException	Cannot open connection
org.apache.commons.dbcp.SQLNestedException	Cannot create PoolableConnectionFactory (Communications link failure Last packet sent to the server was 0 ms ago.)
com.mysql.jdbc.exceptions.jdbc4.CommunicationsException	Communications link failure Last packet sent to the server was 0 ms ago.
java.net.ConnectException	Connection refused: connect

19.9 Enregistrer une simulation

Après avoir fait une simulation, l'utilisateur peut vouloir l'enregistrer dans la session.

Pam

localhost:8084/pam/Formulaire.action

Les plus visités Débuter avec Firefox À la une

Calcul du salaire des assistantes maternelles

[Faire la simulation](#)
[Effacer la simulation](#)
[Voir les simulations](#)
1 [Enregistrer la simulation](#)
[Terminer la session](#)

Employé: Marie Jouveinal

Heures travaillées: 150

Jours travaillés: 20

Informations Employé

Nom	Prénom	Adresse
Jouveinal	Marie	5 rue des oiseaux
Ville	Code Postal	Indice
St Corentin	49203	2

localhost:8084/pam-01/EnregistrerSimulation.action

Les plus visités Débuter avec Firefox À la une

Calcul du salaire des assistantes maternelles

[Retour au formulaire de navigation](#) [Terminer la session](#)

Liste des simulations 2

Numéro	Nom	Prénom	Heures	Jours	Salaire de base	Indemnités	Cotisations	Salaire	
1	Jouveinal	Marie	150.0	20	100,00 €	200,00 €	100,00 €	100,00 €	Retirer

- en [1], on enregistre la simulation
- en [2], la réponse qui présente la liste des simulations déjà faites à laquelle est ajoutée la nouvelle simulation

Le lien [Enregistrer la simulation] se trouve dans le fragment [Entete.JSP] :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3.
4. <table>
5.   <tr>
6.     <td><h1><s:text name="Pam.titre"/></h1></td>
7.     <td>
8.       ...
9.       <s:if test="menu.enregistrerSimulation">
10.        |<a href="<s:url action="EnregistrerSimulation"/>"><s:text
    name="Menu.EnregistrerSimulation"/></a><br/>
11.        </s:if>
12.        ...
13.      </td>
14.    </tr>
15.  </table>

```

On voit qu'un clic sur le lien provoque l'exécution de l'action [EnregistrerSimulation]. Celle-ci est configurée dans le fichier [struts.xml] de la façon suivante :

```

1. <!-- action EnregistrerSimulation -->
2. <action name="EnregistrerSimulation" class="web.actions.Enregistrer" method="execute">
3.   <result name="error" type="tiles">erreur</result>
4.   <result name="simulations" type="tiles">simulations</result>
5. </action>

```

- ligne 1 : l'action [EnregistrerSimulation] est associée à la classe [Enregistrer] et à sa méthode *execute*.

La classe [Enregistrer] est la suivante :

```

1. package web.actions;
2.
3. ...
4. public class Enregistrer extends ActionSupport implements SessionAware {
5.
6.   // session
7.   private Map<String, Object> session;
8.   // menu
9.   private Menu menu;
10.
11.   @Override
12.   public void setSession(Map<String, Object> session) {
13.     this.session = session;
14.   }
15.
16.   // exécution de l'action
17.   public String execute() {
18.     // on récupère la dernière simulation dans la session
19.     Simulation simulation = (Simulation) session.get("simulation");
20.     if (simulation == null) {
21.       return ERROR;

```

```

22.     }
23.     ...
24. }
25.
26. // getters et setters
27. ...
28. }

```

- ligne 4 : parce que l'action doit avoir accès à la session, elle implémente l'interface *SessionAware*.
- ligne 7 : la session
- ligne 9 : le menu

Lorsque l'action [Enregistrer] a été instanciée, sa méthode *execute* est exécutée. On rappelle que son travail est de mettre dans la session la dernière simulation. Celle-ci sera ajoutée à la liste de simulations déjà faites, elle-aussi conservée dans la session.

- ligne 19 : on récupère dans la session la dernière simulation qui y a été placée.
- lignes 20-22 : si on ne la trouve pas, alors c'est que probablement la session a expiré. En effet, celle-ci ne dure qu'un certain temps qu'on peut fixer dans le fichier [web.xml] qui configure l'application.
- ligne 21 : on rend la clé *error*.

Retour à la configuration de l'action [EnregistrerSimulation] :

```

1. <!-- action EnregistrerSimulation -->
2. <action name="EnregistrerSimulation" class="web.actions.Enregistrer" method="execute">
3.   <result name="error" type="tiles">erreur</result>
4.   <result name="simulations" type="tiles">simulations</result>
5. </action>

```

On voit que la clé *error* (ligne 3) amène l'affichage de la vue Tiles nommée *erreur*. Celle-ci est composée des fragments [Entete.JSP] et [Erreur.JSP] et à l'aspect suivant :

Calcul du salaire des assistantes maternelles

L'erreur suivante s'est produite

La session a expiré

Le fragment [Erreur.JSP] est le suivant :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3.
4. <h2><s:text name="Pam.Erreur.libelle"/></h2>
5. <h4><s:text name="Erreur.sessionexpiree"/></h4>

```

Retour à la méthode [Enregistrer].*execute* :

```

1. // exécution de l'action
2. public String execute() {
3.     // on récupère la dernière simulation dans la session
4.     Simulation simulation = (Simulation) session.get("simulation");
5.     if (simulation == null) {
6.         return ERROR;
7.     }
8.     // on récupère le N° de la dernière simulation
9.     Integer numDerniereSimulation = (Integer) session.get("numDerniereSimulation");
10.    if (numDerniereSimulation == null) {
11.        numDerniereSimulation = 0;
12.    }
13.    // on l'incrémente
14.    numDerniereSimulation++;
15.    // on la remet le nouveau n° dans la session

```

```

16. session.put("numDerniereSimulation", numDerniereSimulation);
17. // on récupère la liste des simulations
18. List<Simulation> simulations = (List<Simulation>) session.get("simulations");
19. if (simulations == null) {
20.     simulations = new ArrayList<Simulation>();
21.     session.put("simulations", simulations);
22. }
23. // on lui ajoute la simulation courante
24. simulation.setNum(numDerniereSimulation);
25. simulations.add(simulation);
26. // on affiche la liste des simulations
27. menu = new Menu(false, false, false, false, true, true);
28. return "simulations";
29. }

```

- lignes 9-16 : les différentes simulations sont numérotées à partir de 1. Le dernier n° attribué est mis dans la session sous la clé *numDerniereSimulation*. Le code des lignes 9-16 consiste à récupérer cette clé et à incrémenter la valeur qui lui est associée.
- lignes 18-22 : la liste des simulations est conservée dans la session associée à la clé *simulations*. Les lignes 18-22 consistent à retrouver cette liste si elle existe ou à la créer si elle n'existe pas.
- lignes 24-25 : une fois la liste des simulations obtenues, on lui ajoute la simulation courante (ligne 25). Auparavant, la simulation courante a reçu un numéro (ligne 24).
- ligne 27 : on fixe le menu à afficher
- ligne 28 : on rend la clé de navigation *simulations*.

Retour à la configuration de l'action [EnregistrerSimulation] dans [struts.xml] :

```

1. <!-- action EnregistrerSimulation -->
2. <action name="EnregistrerSimulation" class="web.actions.Enregistrer" method="execute">
3.     <result name="error" type="tiles">erreur</result>
4.     <result name="simulations" type="tiles">simulations</result>
5. </action>

```

Ligne 4, la clé *simulations* provoque l'affichage de la vue Tiles nommée *simulations*. Cette vue est composée des fragments [Entete.JSP] et [Simulations.JSP]. La vue affichée est la suivante :

1

Calcul du salaire des assistantes maternelles

[Retour au formulaire de navigation](#) [Terminer la session](#)

Liste des simulations

2

Numéro	Nom	Prénom	Heures	Jours	Salaire de base	Indemnités	Cotisations	Salaire
1	Jouveinal	Marie	150.0	20	100,00 €	200,00 €	100,00 €	100,00 € Retirer

- en [1], le fragment [Entete.JSP] que nous connaissons bien maintenant.
- en [2], le fragment [Simulations.JSP]

Le fragment [Simulations.JSP] est le suivant :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3.
4. <!-- liste de simulations vide -->
5. <s:if test="#session['simulations']==null || #session['simulations'].size()==0">
6.     <h2><s:text name="Pam.SimulationsVides.titre"/></h2>
7. </s:if>
8. <!-- liste de simulations non vide -->
9. <s:if test="#session['simulations'].size()!=0">
10.     <h2><s:text name="Pam.Simulations.titre"/></h2>
11.     <table>
12.         <tr class="titreInfos">

```

```

13. <th><s:text name="Pam.Simulations.num"/></th>
14. <th><s:text name="Pam.Simulations.nom"/></th>
15. <th><s:text name="Pam.Simulations.prenom"/></th>
16. <th><s:text name="Pam.Simulations.heures"/></th>
17. <th><s:text name="Pam.Simulations.jours"/></th>
18. <th><s:text name="Pam.Simulations.salairebase"/></th>
19. <th><s:text name="Pam.Simulations.indemnites"/></th>
20. <th><s:text name="Pam.Simulations.cotisations sociales"/></th>
21. <th><s:text name="Pam.Simulations.salairenet"/></th>
22. </tr>
23. <s:iterator value="#session['simulations']">
24.   <s:url action="SupprimerSimulation" var="URL">
25.     <s:param name="id" value="num"/>
26.   </s:url>
27.   <tr>
28.     <td class="libelle"><s:property value="num"/></td>
29.     <td class="info"><s:property value="feuilleSalaire.employe.nom"/></td>
30.     <td class="info"><s:property value="feuilleSalaire.employe.prenom"/></td>
31.     <td class="info"><s:property value="heuresTravaillées"/></td>
32.     <td class="info"><s:property value="joursTravaillés"/></td>
33.     <td class="info">
34.       <s:text name="Format.monnaie">
35.         <s:param value="feuilleSalaire.elementsSalaire.salaireBase"/>
36.       </s:text>
37.     </td>
38.     <td class="info">
39.       <s:text name="Format.monnaie">
40.         <s:param value="indemnites"/>
41.       </s:text>
42.     </td>
43.     <td class="info">
44.       <s:text name="Format.monnaie">
45.         <s:param value="feuilleSalaire.elementsSalaire.cotisations Sociales"/>
46.       </s:text>
47.     </td>
48.     <td class="info">
49.       <s:text name="Format.monnaie">
50.         <s:param value="feuilleSalaire.elementsSalaire.salaireNet"/>
51.       </s:text>
52.     </td>
53.     <td class="info"><a href="#URL">Retirer</a></td>
54.   </tr>
55. </s:iterator>
56. </table>
57. </s:if>

```

- lignes 5-7 : s'il n'y a pas de simulations dans la session, alors on affiche la vue suivante :

Calcul du salaire des assistantes maternelles

[Retour au formulaire de navigation](#)
[Terminer la session](#)

La liste des simulations est vide

- lignes 13-21 : affichage des entêtes des colonnes du tableau

Numéro	Nom	Prénom	Heures	Jours	Salaire de base	Indemnités	Cotisations	Salaire
1	Louisa	Maria	150.0	20	262.25 €	104.00 €	07.48 €	269.77 €

- lignes 23-55 : itérateur sur la liste des simulations trouvée dans la session
- lignes 24-26 : création d'une URL nommé *URL* (attribut *id*). Le lien Html généré par cette URL est le suivant :

```
<a href="/pam/SupprimerSimulation.action?id=1">Retirer</a>
```

On voit que le lien cible l'action [SupprimerSimulation] avec le paramètre *id* qui représente le n° de la simulation à retirer de la liste.

- lignes 28-54 : à chaque itération sur la liste des simulations, on affiche les propriétés de la simulation courante.

Liste des simulations

Numéro	Nom	Prénom	Heures	Jours	Salaire de base	Indemnités	Cotisations	Salaire	
1	Jouveinal	Marie	150.0	20	100,00 €	200,00 €	100,00 €	100,00 €	Retirer

19.10 Retirer une simulation

L'utilisateur peut vouloir enlever une simulation de la liste des simulations :

Calcul du salaire des assistantes maternelles [Retour au formulaire de navigation](#) [Terminer la session](#)

1

Liste des simulations

Numéro	Nom	Prénom	Heures	Jours	Salaire de base	Indemnités	Cotisations	Salaire	
1	Jouveinal	Marie	150.0	20	100,00 €	200,00 €	100,00 €	100,00 €	Retirer
2	Laverti	Justine	100.0	30	100,00 €	200,00 €	100,00 €	100,00 €	Retirer

Calcul du salaire des assistantes maternelles [Retour au formulaire de navigation](#) [Terminer la session](#)

2

Liste des simulations

Numéro	Nom	Prénom	Heures	Jours	Salaire de base	Indemnités	Cotisations	Salaire	
2	Laverti	Justine	100.0	30	100,00 €	200,00 €	100,00 €	100,00 €	Retirer

- en [1], on supprime la simulation n° 1
- en [2], la simulation n° 1 a été supprimée

Le lien [Retirer] se trouve dans le fragment [Simulations.SP] que nous avons déjà étudié :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3.
4. <!-- liste de simulations vide -->
5. <s:if test="#session['simulations']==null || #session['simulations'].size()==0">
6.   <h2><s:text name="Pam.SimulationsVides.titre"/></h2>
7. </s:if>
8. <!-- liste de simulations non vide -->
9. <s:if test="#session['simulations'].size()!=0">
10.  <h2><s:text name="Pam.Simulations.titre"/></h2>
11.  <table>
12.    <tr class="titreInfos">
13.      ...
14.    </tr>
15.    <s:iterator value="#session['simulations']">
16.      <s:url action="SupprimerSimulation" var="URL">
17.        <s:param name="id" value="num"/>
18.      </s:url>
19.      <tr>
20.        ...
21.        <td class="info">
22.          <s:text name="Format.monnaie">
23.            <s:param value="feuilleSalaire.elementsSalaire.salaireNet"/>
24.          </s:text>
25.        </td>
26.        <td class="info"><a href="#URL">Retirer</a></td>
27.      </tr>
28.    </s:iterator>
29.  </table>
30. </s:if>

```

- lignes 16-18 : génèrent le lien Html

```
<a href="/pam-01/SupprimerSimulation.action?id=num">Retirer</a>
```

où *num* est le n° de la simulation à retirer.

L'action [SupprimerSimulation] est définie comme suit dans le fichier [struts.xml] :

```

1. <?xml version="1.0" encoding="UTF-8" ?>
2. <!DOCTYPE struts PUBLIC
3.   "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
4.   "http://struts.apache.org/dtds/struts-2.0.dtd">
5.
6. <struts>
7.   <!-- internationalisation -->
8.   <constant name="struts.custom.i18n.resources" value="messages" />
9.   <!-- intégration Spring -->
10.  <constant name="struts.objectFactory.spring.autoWire" value="name" />
11.
12.
13. <!-- actions Struts /Tiles -->
14. <package name="default" namespace="/" extends="tiles-default">
15.   ...
16.   <!-- action RetirerSimulation -->
17.   <action name="SupprimerSimulation" class="web.actions.Supprimer">
18.     <result name="erreur" type="tiles">erreur</result>
19.     <result name="simulations" type="tiles">simulations</result>
20.   </action>
21.   ...
22. </package>
23.
24. <!-- Add packages here -->
25.
26. </struts>

```

- ligne 16 : l'action [SupprimerSimulation] est associée à la classe [Supprimer]. Parce qu'aucune méthode n'est précisée, c'est sa méthode *execute* qui sera exécutée. La classe [Supprimer] est la suivante :


```

1. package web.actions;
2.
3. ...
4. public class Supprimer extends ActionSupport implements SessionAware {
5.
6.     // session
7.     private Map<String, Object> session;
8.     // id de la simulation à supprimer
9.     private String id;
10.    // menu
11.    private Menu menu;
12.
13.    @Override
14.    public void setSession(Map<String, Object> session) {
15.        this.session = session;
16.    }
17.
18.    // exécution de l'action
19.    public String execute() {
20.        // on récupère les simulations dans la session
21.        List<Simulation> simulations = (List<Simulation>) session.get("simulations");
22.        if (simulations == null) {
23.            // cas anormal - la session a du expirer
24.            menu = new Menu(false, false, false, false, true, true);
25.            return "erreur";
26.        }
27.        // test de id
28.        int num = 0;
29.        boolean erreur = false;
30.        try {
31.            num = Integer.parseInt(id);
32.            erreur = num <= 0;
33.        } catch (NumberFormatException ex) {
34.            // anormal
35.            erreur = true;
36.        }
37.        // erreur ?
38.        if (erreur) {
39.            menu = new Menu(false, false, false, false, true, true);
40.            return "erreur";
41.        }
42.        // on recherche la simulation à supprimer
43.        for (int i = 0; i < simulations.size(); i++) {
44.            if (num == simulations.get(i).getNum()) {
45.                simulations.remove(i);
46.                break;
47.            }
48.        }
49.        // on affiche la liste des simulations
50.        menu = new Menu(false, false, false, false, true, true);
51.        return "simulations";
52.    }
53.
54.    // getters et setters
55.    ...
56. }

```

- ligne 4 : l'action [Supprimer] implémente l'interface [SessionAware] pour avoir accès à la session.
- ligne 7 : la session
- ligne 9 : le n° de la simulation à supprimer. On se rappelle en effet qu'on arrive à l'instanciation de la classe [Supprimer] par l'URL Html :

```
<a href="/pam-01/SupprimerSimulation.action?id=num">Retirer</a>
```

où *num* est le n° de la simulation à retirer. Ce numéro sera stocké dans le champ *id* de la ligne 9.

- ligne 11 : le menu pour la vue qui sera affichée en réponse à la requête
- ligne 19 : la méthode *execute* qui va générer la réponse à la requête.
- ligne 21 : on récupère la liste des simulations déjà faites dans la session
- lignes 22-26 : ne pas récupérer cette liste dans la session veut probablement dire que la session a expiré. On a déjà rencontré ce cas. On rend la clé *erreur* qui fait afficher la vue Tiles *erreur* :

```

1. <!-- action RetirerSimulation -->
2.     <action name="SupprimerSimulation" class="web.actions.Supprimer">
3.         <result name="erreur" type="tiles">erreur</result>
4.         ...

```

```
5. </action>
```

La vue Tiles *erreur* a été présentée paragraphe 19.9.

- lignes 28-36 : on vérifie que la chaîne *id* de la ligne 9 représente bien un nombre entier >0.
- lignes 38-40 : si ce n'est pas le cas, on rend de nouveau la clé *erreur* qui fera afficher la vue Tiles *erreur*
- lignes 43-48 : la simulation à retirer est cherchée dans la liste des simulations. Si elle est trouvée, elle est supprimée.
- ligne 50 : on met le menu à jour pour la vue Tiles *simulations*.
- ligne 51 : on rend la clé *simulations*. Celle-ci va faire afficher la vue Tiles *simulations* :

```
1. <!-- action RetirerSimulation -->
2.     <action name="SupprimerSimulation" class="web.actions.Supprimer">
3.         ...
4.         <result name="simulations" type="tiles">simulations</result>
5.     </action>
```

La vue Tiles *simulations* a été présentée paragraphe 19.9.

19.11 Revenir au formulaire

A partir de la vue Tiles *simulations*, l'utilisateur peut revenir au formulaire :

Calcul du salaire des assistantes maternelles

[Retour au formulaire de simulation](#)
[Terminer la session](#) **1**

Liste des simulations

Numéro	Nom	Prénom	Heures	Jours	Salaire de base	Indemnités	Cotisations	Salaire	
1	Jouveinal	Marie	150.0	20	100,00 €	200,00 €	100,00 €	100,00 €	Retirer

Calcul du salaire des assistantes maternelles

[Faire la simulation](#)
[Voir les simulations](#)
[Terminer la session](#)

Employé: Marie Jouveinal ▼

Heures travaillées:

Jours travaillés: **2**

- en [1], on clique sur le lien de retour au formulaire
- en [2], on retrouve un formulaire vide

Le lien [Retour au formulaire de simulation] est défini dans le fragment [Entete.JSP] :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3.
4. <table>
5.     <tr>
6.         <td><h1><s:text name="Pam.titre"/></h1></td>
7.         <td>
8.             ...
9.             <s:if test="menu.retourFormulaire">
10.                |<a href="<s:url action="RetourFormulaire"/>"><s:text
11.                name="Menu.RetourFormulaire"/></a><br/>
12.            </s:if>
13.            ...
```

```

13.     </td>
14.   </tr>
15. </table>

```

- ligne 10 : le lien pointe sur l'action [RetourFormulaire]. Celle-ci est définie de la façon suivante dans le fichier [struts.xml] :

```

1.   <!-- action RetourFormulaire -->
2.   <action name="RetourFormulaire" >
3.     <result type="redirectAction">
4.       <param name="actionName">Formulaire!input</param>
5.       <param name="namespace"></param>
6.     </result>
7. </action>

```

On voit que cette action n'est associée à aucune classe. Elle se contente de rediriger le navigateur client vers l'action [/Formulaire!input]. On est alors dans le même cas que lors de l'affichage de la vue initiale expliquée au paragraphe 19.7. On retrouve donc cette vue initiale [2].

19.12 Voir la liste des simulations

A partir des vues Tiles *simulation* ou *saisie*, l'utilisateur peut demander à voir les simulations :

Calcul du salaire des assistantes maternelles

Employé: Marie Jouveinal ▼

Heures travaillées:

Jours travaillés:

[Faire la simulation](#)

[Voir les simulations](#) **1**

[Terminer la session](#)

Calcul du salaire des assistantes maternelles

[Retour au formulaire de simulation](#)

[Terminer la session](#)

Liste des simulations **2**

Numéro	Nom	Prénom	Heures	Jours	Salaire de base	Indemnités	Cotisations	Salaire
1	Jouveinal	Marie	150.0	20	100,00 €	200,00 €	100,00 €	100,00 € Retirer
2	Laverti	Justine	100.0	25	100,00 €	200,00 €	100,00 €	100,00 € Retirer

- en [1], on clique sur le lien [Voir les simulations]
- en [2], on retrouve la liste des simulations

Le lien [Voir les simulations] est défini dans le fragment [Entete.JSP] :

```

1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3.
4. <table>
5.   <tr>
6.     <td><h1><s:text name="Pam.titre"/></h1></td>
7.     <td>
8.       ...
9.       <s:if test="menu.voirSimulations">
10.        |<a href="<s:url action="VoirSimulations"/>"><s:text
11.        name="Menu.VoirSimulations"/></a><br/>
12.      </s:if>
13.      ...

```

```

13.     </td>
14. </tr>
15. </table>

```

- ligne 10 : le lien [Voir les simulations] appelle l'action [VoirSimulations]. Celle-ci est définie de la façon suivante dans le fichier [struts.xml] :

```

1.     <!-- action VoirSimulations -->
2.     <action name="VoirSimulations" class="web.actions.Voir">
3.         <result name="success" type="tiles">simulations</result>
4.     </action>

```

L'action [VoirSimulations] est associée à la classe [Voir] sans précision de méthode. C'est donc la méthode [Voir].*execute* qui sera exécutée. La classe [Voir] est la suivante :

```

1. package web.actions;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4. import web.entities.Menu;
5.
6. public class Voir extends ActionSupport{
7.     // menu
8.     private Menu menu=new Menu(false,false,false,false,true,true);
9.     // getters et setters
10.
11.     public Menu getMenu() {
12.         return menu;
13.     }
14.
15.     public void setMenu(Menu menu) {
16.         this.menu = menu;
17.     }
18.
19. }

```

L'action [Voir] ne fait qu'une chose : positionner le menu pour la vue Tiles *simulations* (ligne 8). Il n'y a pas de méthode *execute*. C'est donc celle de la classe parent [ActionSupport] qui sera exécutée. On sait qu'elle ne fait rien si ce n'est rendre la clé *success*.

Retour à l'action dans [struts.xml] :

```

1.     <!-- action VoirSimulations -->
2.     <action name="VoirSimulations" class="web.actions.Voir">
3.         <result name="success" type="tiles">simulations</result>
4.     </action>

```

Ligne 3, on voit que la clé *success* conduit à l'affichage de la vue Tiles *simulations*. Celle-ci a été décrite page 161.

19.13 Effacer la simulation courante

A partir de la vues Tiles *simulation*, l'utilisateur peut demander à effacer la simulation courante :

Calcul du salaire des assistantes maternelles

[Faire la simulation](#)
[Effacer la simulation](#)
[Voir les simulations](#) **1**
[Enregistrer la simulation](#)
[Terminer la session](#)

Employé: Marie Jouveinal ▼
Heures travaillées: 10
Jours travaillés: 5

Informations Employé

Nom	Prénom	Adresse
Jouveinal	Marie	5 rue des oiseaux
Ville	Code Postal	Indice
St Corentin	49203	2

Cotisations Sociales

Calcul du salaire des assistantes maternelles

[Faire la simulation](#)
[Voir les simulations](#)
[Terminer la session](#)

Employé: Marie Jouveinal ▼
Heures travaillées: 10
Jours travaillés: 5

2

- en [1], on efface la simulation courante
- en [2], on retrouve le formulaire de saisie vide

Le lien [Effacer la simulation] est défini dans le fragment [Entete.JSP] :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3.
4. <table>
5.   <tr>
6.     <td><h1><s:text name="Pam.titre"/></h1></td>
7.     <td>
8.       ...
9.       <s:if test="menu.effacerSimulation">
10.        |<a href="<s:url action="Formulaire!input"/>"><s:text name="Menu.EffacerSimulation"/></a><br/>
11.      </s:if>
12.      ...
13.    </td>
14.  </tr>
15. </table>
```

Ligne 10, on voit que le lien [Effacer la simulation] déclenche l'action [Formulaire!input]. On sait que cette action amène à la vue initiale [2].

19.14 Terminer la session courante

A partir de toutes les vues Tiles, l'utilisateur peut demander à terminer la session :

Calcul du salaire des assistantes maternelles

[Retour au formulaire de simulation](#)[Terminer la session](#)**1**

Liste des simulations

Numéro	Nom	Prénom	Heures	Jours	Salaire de base	Indemnités	Cotisations	Salaire	
1	Jouveinal	Marie	150.0	20	100,00 €	200,00 €	100,00 €	100,00 €	Retirer
2	Laverti	Justine	100.0	25	100,00 €	200,00 €	100,00 €	100,00 €	Retirer

Calcul du salaire des assistantes maternelles

[Faire la simulation](#)[Voir les simulations](#)[Terminer la session](#)

Employé: Marie Jouveinal ▼

2Heures travaillées: Jours travaillés:

Calcul du salaire des assistantes maternelles

[Retour au formulaire de simulation](#)[Terminer la session](#)**La liste des simulations est vide****3**

- en [1], on part de la vue des simulations et on termine la session
- en [2], on retrouve le formulaire de saisie vide. On demande à voir les simulations.
- en [3], la liste des simulations est désormais vide.

Le lien [Terminer la session] est défini dans le fragment [Entete.JSP] :

```
1. <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2. <%@ taglib prefix="s" uri="/struts-tags" %>
3.
4. <table>
5.   <tr>
6.     <td><h1><s:text name="Pam.titre"/></h1></td>
7.     <td>
8.       ...
9.       <s:if test="menu.terminerSession">
10.        |<a href="<s:url action="TerminerSession"/>"><s:text
    name="Menu.TerminerSession"/></a><br/>
11.      </s:if>
12.    </td>
13.  </tr>
14. </table>
```

Ligne 10, on voit que le lien [Terminer la session] déclenche l'action [TerminerSession]. Celle-ci est définie de la façon suivante dans le fichier [struts.xml] :

```
1. <action name="TerminerSession" class="web.actions.Terminer">
2.   <result name="success" type="redirectAction">
3.     <param name="actionName">Formulaire!input</param>
4.     <param name="namespace"></param>
5.   </result>
```

6. </action>

- ligne 1 : on voit que la classe [Terminer] va être instanciée et sa méthode *execute* exécutée.
- lignes 2-5 : après exécution de la méthode [Terminer].*execute*, il y aura redirection sur la vue initiale de saisie. C'est ce qui explique l'écran n° 2.

La classe [Terminer] est la suivante :

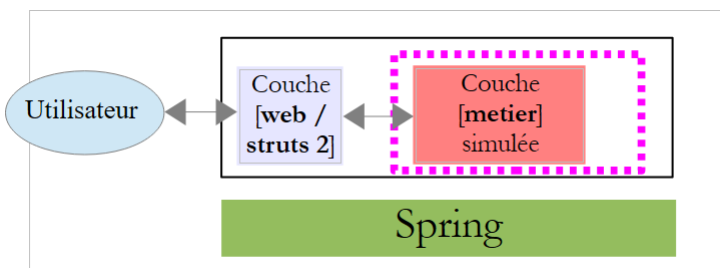
```
1. package web.actions;
2.
3. import com.opensymphony.xwork2.ActionSupport;
4. import java.util.Map;
5. import org.apache.struts2.interceptor.SessionAware;
6.
7. public class Terminer extends ActionSupport implements SessionAware {
8.
9.     // session
10.    private Map<String, Object> session;
11.
12.    @Override
13.    public String execute() {
14.        // abandon de la session courante
15.        session.clear();
16.        return SUCCESS;
17.    }
18.
19.    @Override
20.    public void setSession(Map<String, Object> session) {
21.        this.session = session;
22.    }
23. }
```

Le rôle de l'action [Terminer] est de vider la session courante de ses attributs.

- ligne 7 : l'action [Terminer] implémente l'interface [SessionAware] pour avoir accès à la session.
- ligne 10 : le dictionnaire de la session
- ligne 13 : la méthode *execute* qui est exécutée
- ligne 15 : elle vide le dictionnaire de la session. Du coup, la liste des simulations qui est en session va disparaître. C'est ce qui explique l'écran n° 3.
- ligne 16 : elle rend la clé *success*, qui on l'a vu, va afficher la vue Tiles *saisie* [2].

19.15 Conclusion

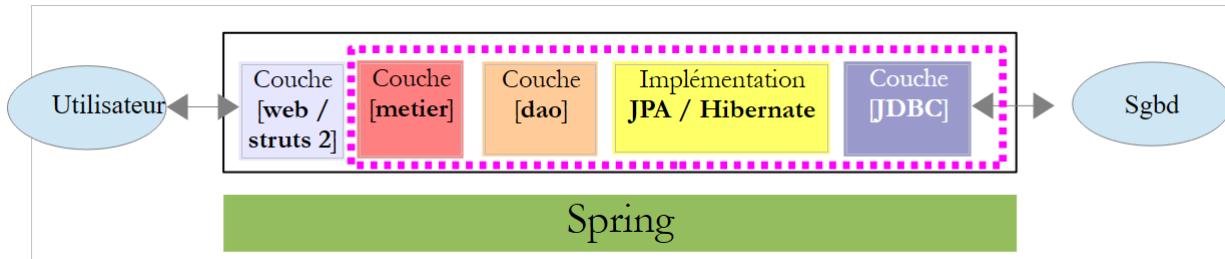
Nous avons entièrement commenté la version 1 de notre étude de cas qui travaille avec une couche [métier] simulée :



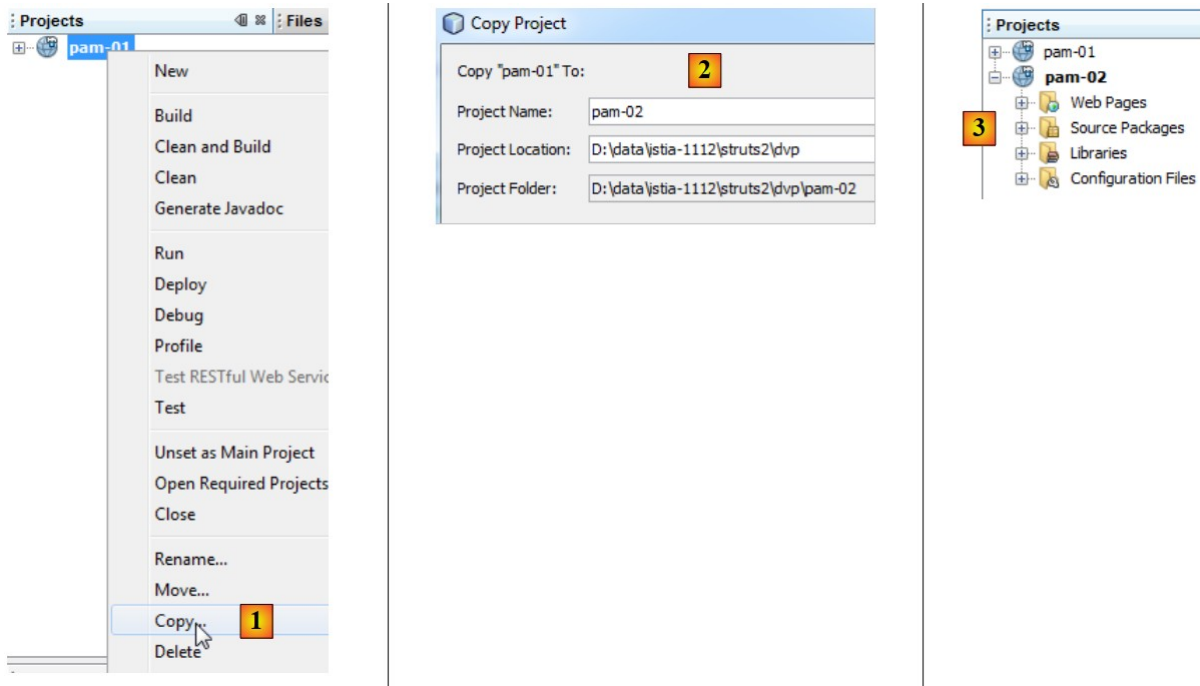
Il nous reste plus qu'à " brancher " la vraie couche métier sur la couche [web].

20 Etude de cas – version 2

Nous présentons maintenant la version finale de notre application :



Le nouveau projet Netbeans [pam-02] est obtenue par recopie du projet [pam-01] :



- en [1], on copie le nouveau projet
- en [2], on nomme le nouveau projet [pam-02] et on précise son dossier
- en [3], le nouveau projet [pam-02]

Pour "brancher" la vraie couche [metier] sur la couche web que nous avons construite, il faut faire trois choses :

1. supprimer la couche [metier] simulée que nous avons créée
2. configurer Spring pour qu'il instancie la vraie couche [metier] qui se trouve dans l'archive [pam-spring-metier-dao-jpa-hibernate.jar].
3. ajouter au projet toutes les archives nécessaires (Spring, Hibernate, Jpa, pilote Jdbc de MySQL).

Le fichier de configuration de Spring [WEB-INF/applicationContext.xml] devient le suivant :

```

1. <?xml version="1.0" encoding="UTF-8"?>
2. <beans xmlns="http://www.springframework.org/schema/beans"
   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3.     xmlns:tx="http://www.springframework.org/schema/tx"
4.     xsi:schemaLocation="http://www.springframework.org/schema/beans
   http://www.springframework.org/schema/beans/spring-beans-2.0.xsd
   http://www.springframework.org/schema/tx http://www.springframework.org/schema/tx/spring-
   tx-2.0.xsd">
5.
6.     <!-- couches applicatives -->
7.
8.     <!-- web -->
9.     <bean id="config" class="web.Config" init-method="init">
10.         <property name="metier" ref="metier"/>
11.     </bean>

```

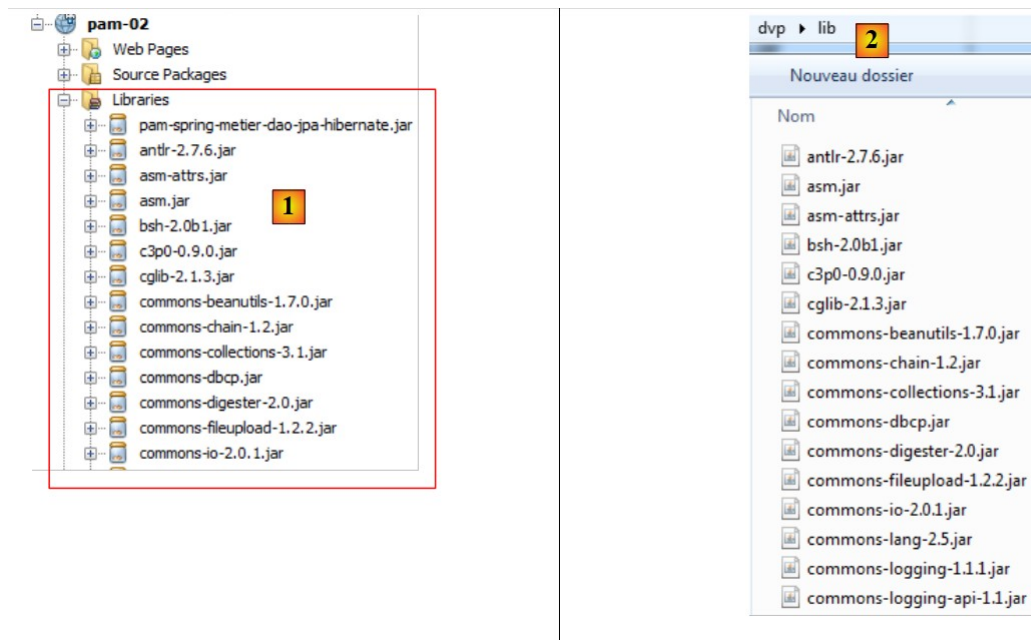
```

12. <!-- métier -->
13. <bean id="metier" class="metier.Metier">
14.   <property name="employeDao" ref="employeDao"/>
15.   <property name="cotisationDao" ref="cotisationDao"/>
16. </bean>
17. <!-- dao -->
18. <bean id="employeDao" class="dao.EmployeDao" />
19. <bean id="indemnitéDao" class="dao.IndemnitéDao" />
20. <bean id="cotisationDao" class="dao.CotisationDao" />
21.
22. <!-- configuration JPA -->
23. <bean id="entityManagerFactory"
class="org.springframework.orm.jpa.LocalContainerEntityManagerFactoryBean">
24.   <property name="dataSource" ref="dataSource" />
25.   <property name="jpaVendorAdapter">
26.     <bean class="org.springframework.orm.jpa.vendor.HibernateJpaVendorAdapter">
27.       <property name="databasePlatform" value="org.hibernate.dialect.MySQL5InnoDBDialect" />
28.     </bean>
29.   </property>
30.   <property name="loadTimeWeaver">
31.     <bean class="org.springframework.instrument.classloading.InstrumentationLoadTimeWeaver" />
32.   </property>
33. </bean>
34.
35. <!-- la source de données DBCP -->
36. <bean id="dataSource" class="org.apache.commons.dbcp.BasicDataSource" destroy-method="close">
37.   <property name="driverClassName" value="com.mysql.jdbc.Driver" />
38.   <property name="URL" value="jdbc:mysql://localhost:3306/dbpam_hibernate" />
39.   <property name="username" value="root" />
40.   <property name="password" value="" />
41. </bean>
42.
43. <!-- le gestionnaire de transactions -->
44. <tx:annotation-driven transaction-manager="txManager" />
45. <bean id="txManager" class="org.springframework.orm.jpa.JpaTransactionManager">
46.   <property name="entityManagerFactory" ref="entityManagerFactory" />
47. </bean>
48.
49. <!-- traduction des exceptions -->
50. <bean class="org.springframework.dao.annotation.PersistenceExceptionTranslationPostProcessor" />
51.
52. <!-- persistence -->
53. <bean class="org.springframework.orm.jpa.support.PersistenceAnnotationBeanPostProcessor" />
54.
55. </beans>

```

Les lignes 36-41 configurent l'accès à la base de données.

L'ajout des nouvelles bibliothèques (Spring, Hibernate, Jpa, pilote Jdbc de MySQL) se fait à partir du dossier [lib] [2]. On prend l'intégralité du dossier. Il y a plusieurs dizaines de fichiers .jar à ajouter [1] :



Il a été assez difficile de constituer cette bibliothèque car ces frameworks utilisent parfois les mêmes archives. Il faut alors supprimer les doublons. Ces fichiers ont été rassemblés dans le dossier [lib] [2] dans l'archive des exemples de ce document afin que le lecteur n'ait pas à reconstituer lui-même cette bibliothèque.

C'est tout. La nouvelle application [pam-02] va désormais travailler avec le SGBD. Voici une copie d'écran d'un calcul de salaire :

Calcul du salaire des assistantes maternelles

[Faire la simulation](#)
[Effacer la simulation](#)
[Voir les simulations](#)
[Enregistrer la simulation](#)
[Terminer la session](#)

Employé: Marie Jouveinal
Heures travaillées: 150
Jours travaillés: 20

Informations Employé

Nom	Prénom	Adresse
Jouveinal	Marie	5 rue des oiseaux
Ville	Code Postal	Indice
St Corentin	49203	2

Cotisations Sociales

Csgd	Csgd	Retraite	Sécu
3,49 %	6,15 %	7,88 %	9,39 %

Indemnités

Salaire horaire	Entretien/Jour	Repas/Jour	Congés payés
2,10 €	2,10 €	3,10 €	15,00 €

Salaire

Salaire de base	Cotisations sociales	Indemnités d'entretien	Indemnités de repas
362,25 €	97,48 €	42,00 €	62,00 €
Salaire net	368,77 €		

Cette fois-ci, le salaire calculé est le vrai salaire et non le salaire fictif de la version 1. Le lecteur est invité à tester la nouvelle application.

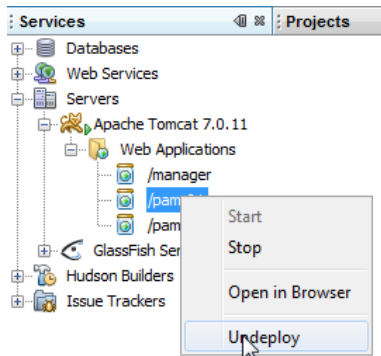
21 Conclusion

Les bases du framework Struts 2 ont été étudiées et une application réunissant plusieurs technologies a été présentée. Le lecteur de niveau débutant ou intermédiaire trouvera dans ce document matière à se former et à progresser. Reste le besoin d'un livre de référence. Je conseille de nouveau l'ouvrage :

" **Struts 2 in Action** " écrit par Donald Brown - Chad Michael Davis – Scott Stanlick aux éditions Manning.

Je voudrais terminer en présentant des problèmes que j'ai rencontrés lors de l'écriture des exemples de ce document. J'ai déjà signalé des comportements inattendus de Struts 2, notamment lors des saisies de nombres et de dates. Voici quelques autres problèmes rencontrés :

Déchargeons l'application [pam-01] du serveur Tomcat :



La console d'Apache affiche alors les logs suivants :

```
1. janv. 05, 2012 11:18:58 AM org.springframework.context.support.AbstractApplicationContext doClose
2. Infos: Closing org.springframework.web.context.support.XmlWebApplicationContext@57488b9c: display
   name [Root WebApplicationContext]; startup date [Thu Jan 05 11:18:45 CET 2012]; root of context
   hierarchy
3. janv. 05, 2012 11:18:58 AM org.springframework.beans.factory.support.DefaultSingletonBeanRegistry
   destroySingletons
4. Infos: Destroying singletons in
   org.springframework.beans.factory.support.DefaultListableBeanFactory@78ae9d8d: defining beans
   [config,metier]; root of factory hierarchy
5. janv. 05, 2012 11:18:58 AM org.apache.tiles.access.TilesAccess setContainer
6. Infos: Removing TilesContext for context: org.apache.catalina.core.ApplicationContextFacade
7. janv. 05, 2012 11:19:00 AM org.apache.catalina.startup.HostConfig checkResources
8. Infos: Repli (undeploy) de l'application web ayant pour chemin de contexte /pam-01
```

Le déchargement s'est bien passé.

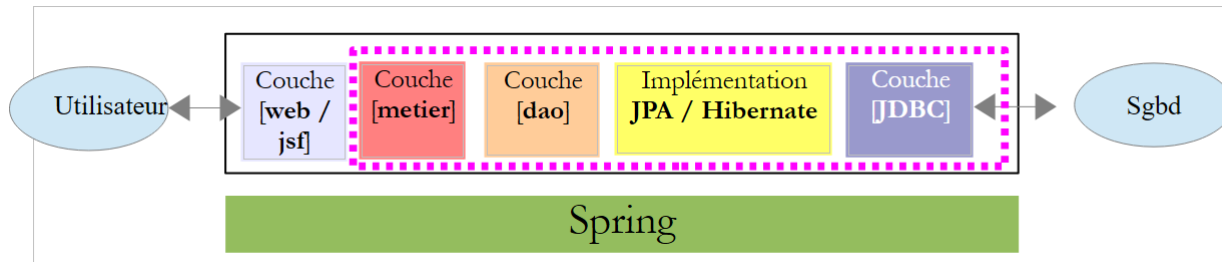
Déchargeons de la même façon [pam-02]. Les logs sont alors les suivants :

```
1. janv. 05, 2012 11:21:07 AM org.springframework.context.support.AbstractApplicationContext doClose
2. Infos: Closing org.springframework.web.context.support.XmlWebApplicationContext@5d9131fa: display
   name [Root WebApplicationContext]; startup date [Thu Jan 05 11:11:10 CET 2012]; root of context
   hierarchy
3. janv. 05, 2012 11:21:07 AM org.springframework.beans.factory.support.DefaultSingletonBeanRegistry
   destroySingletons
4. Infos: Destroying singletons in
   org.springframework.beans.factory.support.DefaultListableBeanFactory@7b66d238: defining beans
   [config,metier,employeeDao,indemnityDao,cotisationDao,entityManagerFactory,dataSource,org.springfra
   mework.aop.config.internalAutoProxyCreator,org.springframework.transaction.annotation.AnnotationTr
   ansactionAttributeSource#0,org.springframework.transaction.interceptor.TransactionInterceptor#0,or
   g.springframework.transaction.config.internalTransactionAdvisor,txManager,org.springframework.dao.
   annotation.PersistenceExceptionTranslationPostProcessor#0,org.springframework.orm.jpa.support.Pers
   istenceAnnotationBeanPostProcessor#0]; root of factory hierarchy
5. janv. 05, 2012 11:21:07 AM org.springframework.orm.jpa.AbstractEntityManagerFactoryBean destroy
6. Infos: Closing JPA EntityManagerFactory for persistence unit 'pam-PU'
7. janv. 05, 2012 11:21:07 AM org.hibernate.impl.SessionFactoryImpl close
8. Infos: closing
9. janv. 05, 2012 11:21:07 AM org.apache.tiles.access.TilesAccess setContainer
10. Infos: Removing TilesContext for context: org.apache.catalina.core.ApplicationContextFacade
11. janv. 05, 2012 11:21:07 AM org.apache.catalina.loader.WebappClassLoader clearReferencesJdbc
12. Grave: The web application [/pam-02] registered the JDBC driver [com.mysql.jdbc.Driver] but failed
   to unregister it when the web application was stopped. To prevent a memory leak, the JDBC Driver
   has been forcibly unregistered.
13. janv. 05, 2012 11:21:07 AM org.apache.catalina.loader.WebappClassLoader clearReferencesThreads
14. Grave: The web application [/pam-02] appears to have started a thread named [Timer-0] but has
   failed to stop it. This is very likely to create a memory leak.
15. janv. 05, 2012 11:21:07 AM org.apache.catalina.loader.WebappClassLoader clearReferencesThreads
16. Grave: The web application [/pam-02] appears to have started a thread named [MySQL Statement
   Cancellation Timer] but has failed to stop it. This is very likely to create a memory leak.
17. janv. 05, 2012 11:21:08 AM org.apache.catalina.startup.HostConfig checkResources
18. Infos: Repli (undeploy) de l'application web ayant pour chemin de contexte /pam-02
```

Lignes 12, 14, 16, des erreurs graves sont signalées. Deux d'entre elles signalent une possibilité de fuite mémoire. Effectivement, l'application [pam-02] présente des fuites mémoire. Au bout d'un moment, l'application n'est plus utilisable. Il faut alors fermer Netbeans et redéployer l'application.

Les erreurs se réfèrent toutes à des threads. L'application [pam-02] n'en utilise pas par elle-même. Il faut donc soupçonner les frameworks utilisés Struts 2, Spring 2.5, Tiles. On peut raisonnablement exclure Tiles qui probablement ne gère pas de threads. Restent Struts et Spring.

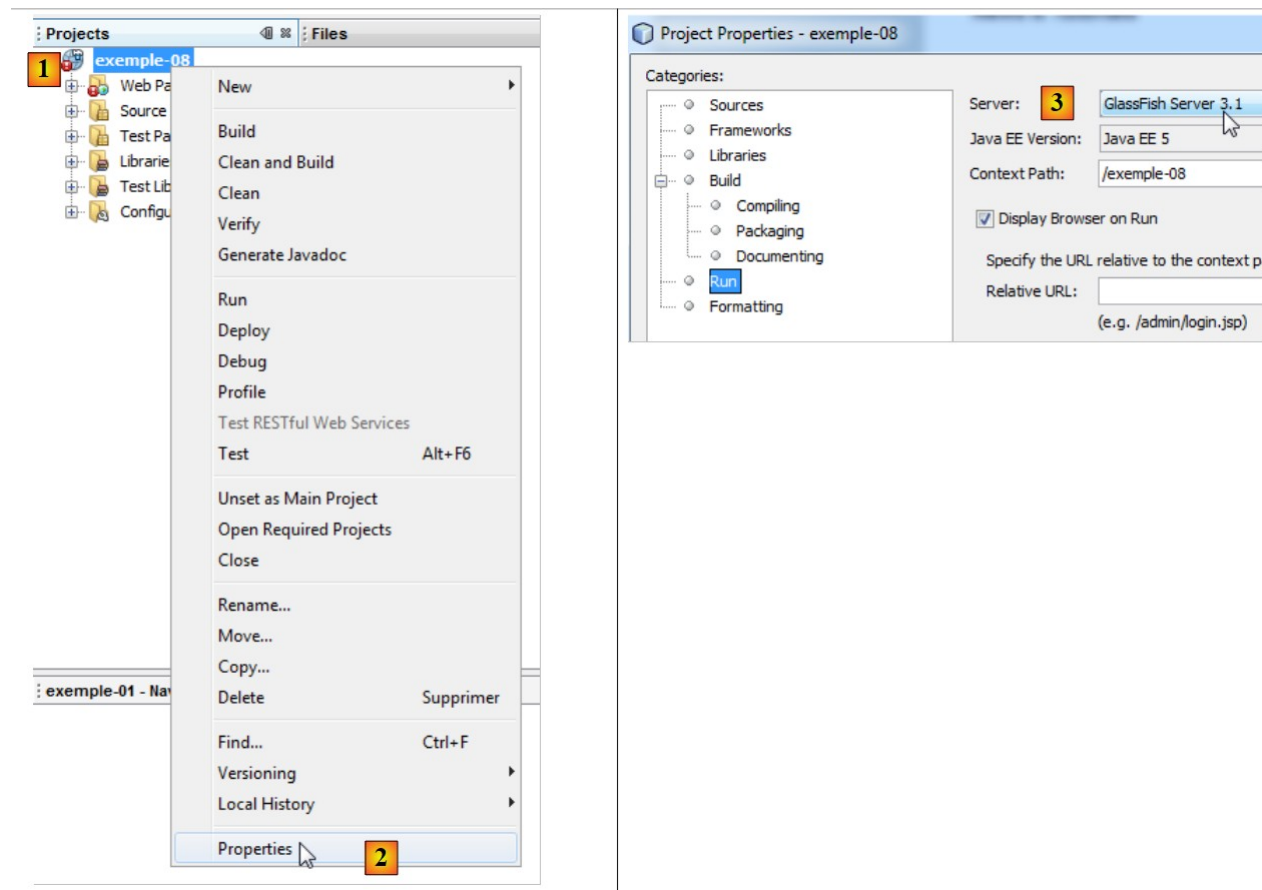
On fait alors une autre expérience :



On implémente la couche [web] avec le framework JSF au lieu de Struts 2. Rien d'autre ne change. On a alors les mêmes messages d'erreur qu'avec Struts 2.

Donc le fautif semble être Spring, le pool de connexions **dbcp** utilisé ici par Spring, ou bien moi-même si je n'ai pas utilisé Spring à bon escient. J'ai pu oublier une configuration de Spring qui permettrait de nettoyer les threads créés, j'ai pu ... La solution à ce problème de fuite mémoire reste donc à trouver.

Autre expérience. L'IDE Netbeans s'installe avec les serveurs Tomcat et Glassfish. Essayons d'exécuter sur Glassfish une application créée dans ce document :



- en [2] les propriétés du projet [exemple-08] [1]
- en [3], le choix du serveur web

Voici la page récupérée dans le navigateur :



La portabilité attendue n'est pas au rendez-vous. Il y a ici deux erreurs :

- en [1], l'internationalisation n'a pas marché
- en [2], on a une exception

Pour la plupart des applications développées dans ce document, on a ces mêmes deux erreurs. Je n'ai pas cherché à résoudre ce nouveau problème.